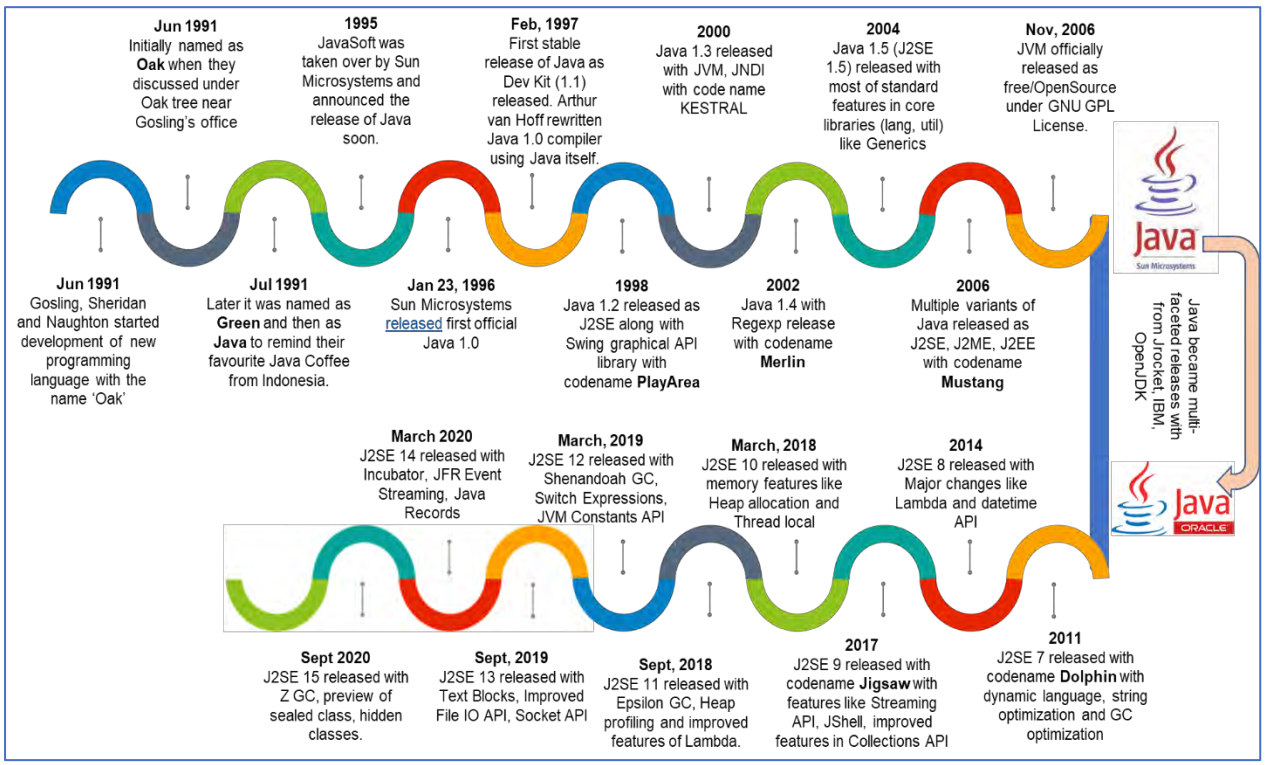
Chapter – 1: Introduction to Java

History of Java



The Java "White Paper" Buzzwords

1) Simple
2) Object-Oriented
3) Distributed
4) Robust
5) Secure
6) Architecture-Neutral
7) Portable
8) Interpreted
9) High-Performance
10) Multithreaded
11) Dynamic

Simple

- The syntax for Java is, indeed, a cleaned-up version of C++ syntax.
- There is no need for header files, pointer arithmetic (or even a pointer syntax), structures, unions, operator overloading, virtual base classes, and so on.
- Another aspect of being simple is being small. One of the goals of Java is to enable the construction of software that can run stand-alone on small machines.

- The size of the basic interpreter and class support is about 40K; the basic standard libraries and thread support (essentially a self-contained microkernel) add another 175K.

Object-Oriented

- Object-oriented design is a programming technique that focuses on the data -objects and on the interfaces to those objects.
- The major difference between Java and C++ lies in multiple inheritance, which Java has replaced with a simpler concept of interfaces. Java has a richer capacity for runtime introspection than C++.

Distributed

- Java has an extensive library of routines for coping with TCP/IP protocols like HTTP and FTP.
- Java applications can open and access objects across the Net via URLs with the same ease as when accessing a local file system.

Robust

- Java is intended for writing programs that must be reliable in a variety of ways.
- Java puts a lot of ***emphasis on early checking for possible problems***, later dynamic (runtime) checking, and eliminating situations that are error-prone.
- The single biggest difference between Java and C/C++ is that Java has a pointer model that eliminates the possibility of overwriting memory and corrupting data.

Secure

- Java enables the construction of virus-free, tamper-free systems.
- Java browser plug-ins no longer trust remote code unless it is digitally signed and users have agreed to its execution.

Architecture-Neutral

- The compiler generates an architecture-neutral object file format. The compiled code is executable on many processors, given the presence of the Java runtime system.
- The Java compiler does this by generating bytecode instructions which have nothing to do with a particular computer architecture. Rather, they are designed to be both easy to interpret on any machine and easy to translate into native machine code on the fly.
- Virtual machines have the option of translating the most frequently executed bytecode sequences into machine code—a process ***called just-in-time compilation***.

Portable

- Unlike C and C++, there are no "implementation-dependent" aspects of the specification. The sizes of the primitive data types are specified, as is the behavior of arithmetic on them.
- The libraries that are a part of the system define portable interfaces. For example, there is an abstract Window class and implementations of it for UNIX, Windows, and the Macintosh.
- The Java libraries do a great job of letting you work in a platform-independent manner. You can work with files, regular expressions, XML, dates and times, databases, network connections, threads, and so on, without worrying about the underlying operating system. Not only are your programs portable, but the Java APIs are often of higher quality than the native ones.

Interpreted

- The Java interpreter can execute Java bytecodes directly on any machine to which the interpreter has been ported.
- Since linking is a more incremental and lightweight process, the development process can be much more rapid and exploratory.

High-Performance

- While the performance of interpreted bytecodes is usually more than adequate, there are situations where higher performance is required. The bytecodes can be translated on the fly (at runtime) into machine code for the particular CPU the application is running on.
- A just-in-time compiler can monitor which code is executed frequently and optimize just that code for speed. A more sophisticated optimization is the elimination (or "inlining") of function calls. The just-in-time compiler knows which classes have been loaded. It can use inlining when, based upon the currently loaded collection of classes, a particular function is never overridden, and it can undo that optimization later if necessary.

Multithreaded

- The benefits of multithreading are better interactive responsiveness and real-time behavior.
- Concurrent programming is never easy, but Java has done a very good job making it manageable.

Dynamic

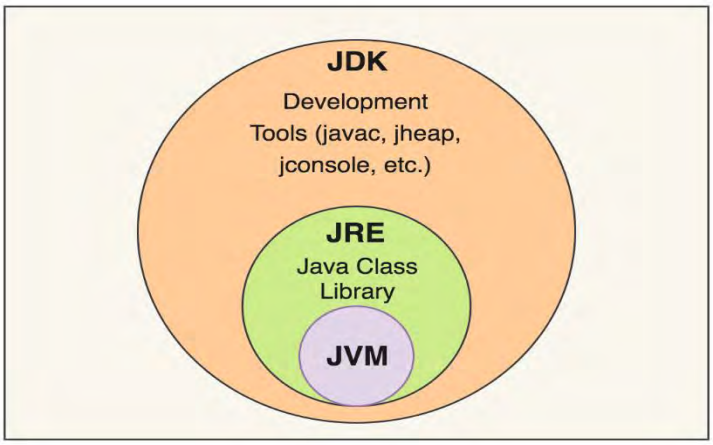
- In a number of ways, Java is a more dynamic language than C or C++. It was designed to adapt to an evolving environment.
- Libraries can freely add new methods and instance variables without any effect on their clients.
- In Java, finding out runtime type information is straightforward.

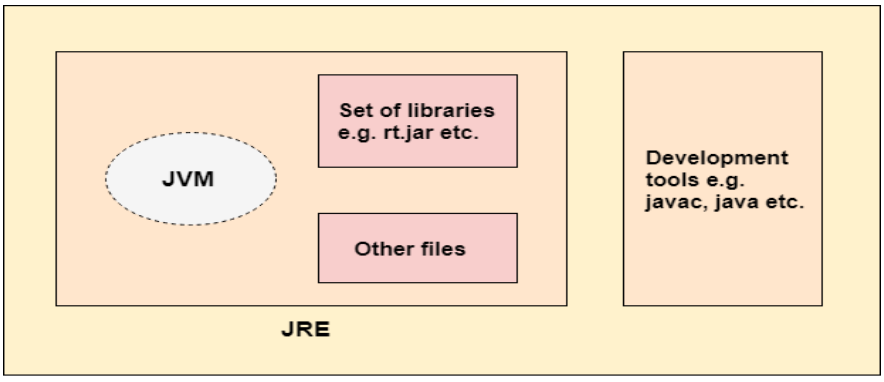
Common Misconceptions about Java

- ***Java is an extension of HTML:*** Java is a programming language; HTML is a way to describe the structure of a web page. They have nothing in common except that there are HTML extensions for placing Java applets on a web page.
- ***I use XML, so I don't need Java***: Java is a programming language; XML is a way to describe data. You can process XML data with any programming language, but the Java API contains excellent support for XML processing. In addition, many important XML tools are implemented in Java.
- ***Java is an easy programming language to learn:*** No programming language as powerful as Java is easy. You always have to distinguish between how easy it is to write toy programs and how hard it is to do serious work.
- ***Java will become a universal programming language for all platforms.*** This is possible in theory. Anything that happens in a browser is controlled by JavaScript. Windows programs are written in C++ or C#. Java has the edge in server-side programming and in cross-platform client applications.

- ***Java is interpreted, so it is too slow for serious applications.*** In the early days of Java, the language was interpreted. Nowadays, the Java virtual machine uses a just-in-time compiler. The "hot spots" of your code will run just as fast in Java as they would in C++, and in some cases even faster.
- ***All Java programs run inside a web page.*** All Java applets run inside a web browser. That is the definition of an applet—a Java program running inside a browser. But most Java programs are standalone applications that run outside of a web browser. In fact, many Java programs run on web servers and produce the code for web pages.
- ***JavaScript is a simpler version of Java.*** JavaScript, a scripting language that can be used inside web pages, was invented by Netscape and originally called LiveScript. JavaScript has a syntax that is reminiscent of Java, and the languages' names sound similar, but otherwise they are unrelated.

Java Jargon





JDK

- ***Java Development Kit (JDK)*** is a software development environment used for developing Java applications and applets. It includes the Java Runtime Environment (JRE), an interpreter/loader (Java), a compiler (javac), an archiver (jar), a documentation generator (Javadoc), and other tools needed in Java development.
- ***Java Runtime Environment:*** JRE (Java Runtime Environment) is an installation package that provides an environment to only run (not develop) the java program (or application) onto your machine. JRE is only used by those who only want to run Java programs that are end-users of your system.

- ***Java Virtual Machine (JVM)*** is an engine that provides runtime environment to drive the Java Code or applications. It converts Java bytecode into machines language. JVM is a part of Java Runtime Environment (JRE).

- ***What is Java Bytecode?***
  - Byte Code can be defined as an intermediate code generated by the compiler after the compilation of source code (JAVA Program). As soon as a java program is compiled, java bytecode is generated. In more clear terms, java bytecode is the machine code in the form of a .class file. With the help of java bytecode, we achieve platform independence in java.

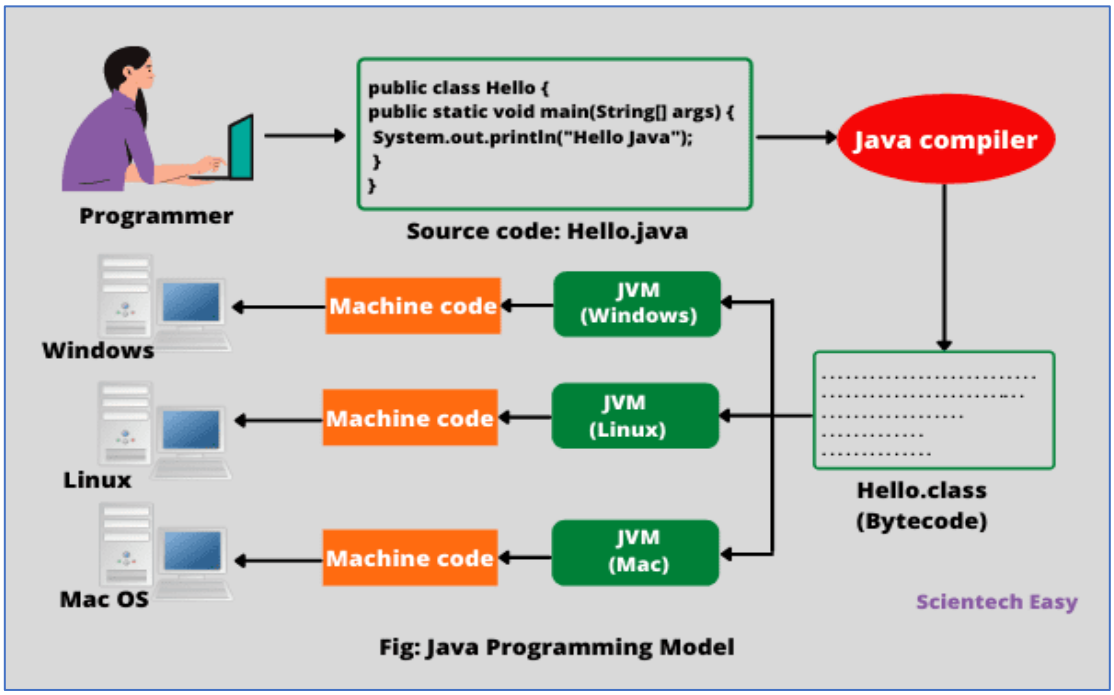
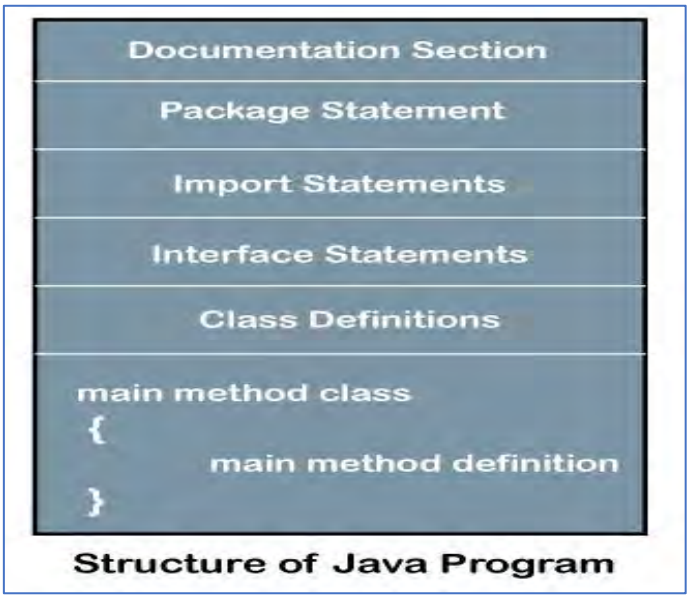


Fig: Java Programming Model

Java Program Structure



Structure of Java Program

Documentation Section

- It is optional to write in your Java program.
- It contains basic information about the class, variables, methods, author's name, program name, version, company name, date of creation, etc.
- The compiler does not execute of contents of the documentation section and it ignores the statements written in it.
- We use comments to write in it and comments can be single-line, multi-line, and documentation comments.
- Example single-line comment:

  ```
  // This example for writing single line comment.
  ```
- Example multi-line comment:

  ```
  /* It is an example for writing Multi-line comment */
  ```
- Example documentation comment:

  ```
  /** It is an example for documentation comment */
  ```

Package Statements

- It is optional to write in your Java program.
- Note that we cannot use more than one package statement in our Java program.
- It should be mentioned before any Interface and class definition.
- ***"package"*** keyword is used for declaring the package name in the program.
- ***For example,***

  ```
  1. package exam;              // where exam is name of package.
  2. package root.exam;         // root is the parent directory of exam.
  ```

Import Statements

- It contains predefined classes, methods, interfaces.
- ***"import"*** keyword is used to import predefined classes or interfaces.
- Keyword "import" imports the classes defined in different packages.
- It is written between classes declaration and package statements.
- ***For example,***

  ```
  import java.util.Scanner;     // Import only the Scanner class present in util package.
  import java.util.*;    // Import all classes of java.util package.
  ```

Class Definition

- Without a class, we cannot create a Java program.
- At least one class must be present in java program.
- You can create more than one class.
- For defining a class, we use ***"class"*** keyword and after that, write the name of the class.
- Example for defining a class:

  ```
  class addTwoNumber{           // addTwoNumber is name of class.
                    // body of class
  }  // end of class.
  ```

Interface Statements

- It is an optional statement.
- "interface" keyword is used to create interface.
- Interface contains methods declaration and constants.
- ***Example for defining an interface:***

```
interface record {     // record is the name of interface.
        void getData ();
        void calculateData (); // getData and calculateData are methods.
        }
```

Main Method Class

- Most important section of Java program and without this Java program would not be able to execute.
- It is written inside a class and inside main () method, we call methods and instantiate classes.
- ***Example for defining the structure of main method:***

```
public class student {      // student is name of class.
        public static void main (String [] args) {      // structure of main method

        }// end of main function.
} // end of class
```

```
1  // Documentation Section
2  /* An Example of a Java program for the addition of two numbers and reading numbers from user. */
3  //firstExample.java
4  import java.util.Scanner;
5  class firstExample {
6      public void addTwoNumbers(int a, int b){
7          int c= a + b;
8          System.out.print("Sum of two numbers is: " + c);
9      }
10     public static void main (String []args) {
11         Scanner sc=new Scanner(System.in);
12
13         System.out.print("Enter the first number: ");
14         int a =sc.nextInt();
15         System.out.print("Enter the second number: ");
16         int b =sc.nextInt();
17
18         firstExample obj=new firstExample();
19         obj.addTwoNumbers(a,b);
20     } // end of main class
21 } // end of class
```

Chapter – 2: Java Programming Basics

Comments in Java

The most common form is a //. Use this for a comment that runs from the // to the end of the line.

```
System.out.println("We will not use 'Hello, World!'"); // is this too cute?
```

When longer comments are needed, you can mark each line with a //, or you can use the /* and */ comment delimiters that let you block off a longer comment.

Finally, a third kind of comment is used to generate documentation automatically. This comment uses a /** to start and a */ to end. You can see this type

Listing 3.1 FirstSample/FirstSample.java

```
1  /**
2   * This is the first sample program in Core Java Chapter 3
3   * @version 1.01 1997-03-22
4   * @author Gary Cornell
5   */
6  public class FirstSample
7  {
8     public static void main(String[] args)
9     {
10       System.out.println("We will not use 'Hello, World!'");
11    }
12 }
```

Java keywords

In the Java programming language, a keyword is any one of 67 reserved words that have a predefined meaning in the language. ***Of these 67 keywords***, *16 of them are only contextually reserved*, and can sometimes be used as an identifier, unlike standard reserved words. The keywords ***const*** and ***goto*** are reserved, even they are not currently in use. Currently they are no longer supported in Java.

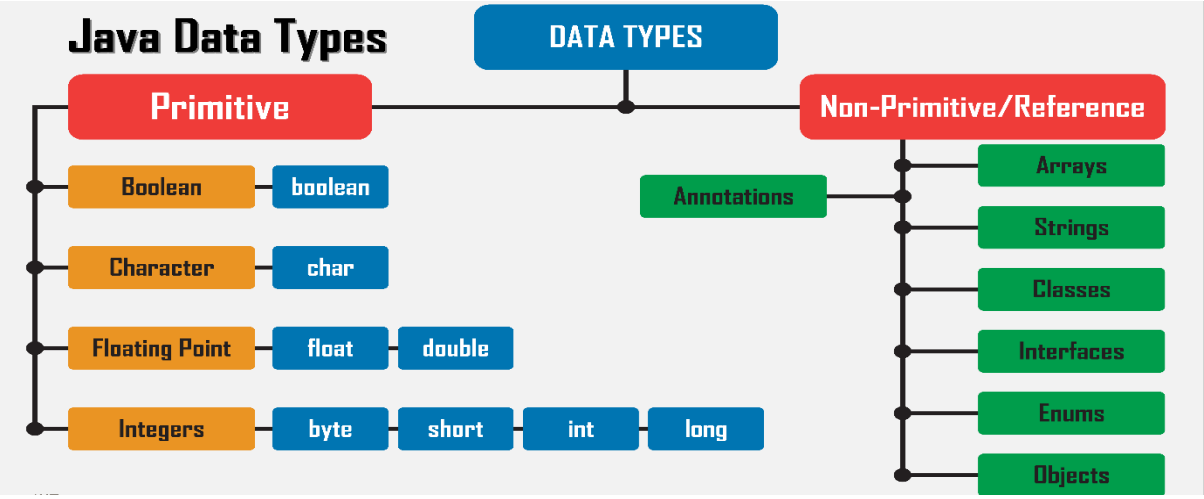
1. abstract
2. assert
3. boolean
4. break
5. byte
6. case
7. catch
8. char
9. class
10. continue
11. default
12. do
13. double
14. else
15. enum
16. extends
17. final
18. finally
19. float
20. for
21. if
22. implements
23. import
24. instanceof
25. int
26. interface
27. long
28. native
29. new
30. package
31. private
32. protected
33. public
34. return
35. short
36. static
37. strictfp
38. super
39. switch
40. synchronized
41. this
42. throw
43. throws
44. transient
45. try
46. void
47. volatile
48. while

Java Identifiers

In Java, there are several points to remember about identifiers. They are as follows −

- All identifiers should begin with a letter (A to Z or a to z), currency character ($) or an underscore (_).
- After the first character, identifiers can have any combination of characters.
- A key word cannot be used as an identifier.
- Most importantly, identifiers are case sensitive.
- Examples of legal identifiers: age, $salary, _value, __1_value.
- Examples of illegal identifiers: 123abc, -salary.

Java's Primitive Data Types



Data Type	Size(bits)	Range		Default
Keyword		Minimum	Maximum	
boolean	1 bit	false	true	false
byte	8 bits	-128	127	0
short	16 bits	-32,768	32,767	0
char	16 bits	'\u0000' (0)	'\uFFFF' (65535)	'\u0000'
int	32 bits	-2,147,483,648	2,147,483,647	0
long	64 bits	-2^{63}	$2^{63}-1$	0
float	32 bits	32-bit IEEE 754 Floating Point $\sim 1.4e^{-045}$	$\sim 3.4e^{+038}$	0.0
double	64 bits	64-bit IEEE 754 Floating Point $\sim 4.9e^{-324}$	$\sim 1.8e^{+308}$	0.0

Variables and Constants

```
double salary;
int vacationDays;
long earthPopulation;
boolean done;
```

Notice the semicolon at the end of each declaration. The semicolon is necessary because a declaration is a complete Java statement, and all Java statements end in semicolons.

A variable name must begin with a letter and must be a sequence of letters or digits. Note that the terms "letter" and "digit" are much broader in Java than in most languages. A letter is defined as 'A'–'Z', 'a'–'z', '_', '$', or *any* Unicode character that denotes a letter in a language. For example, German users can use umlauts such as 'ä' in variable names; Greek speakers could use a π. Similarly, digits are '0'–'9' and *any* Unicode characters that denote a digit in a language. Symbols like '+' or '©' cannot be used inside variable names, nor can spaces. *All* characters in the name of a variable are significant and *case is also significant*. The length of a variable name is essentially unlimited.

After you declare a variable, you must explicitly initialize it by means of an assignment statement—you can never use the value of an uninitialized variable. For example, the Java compiler flags the following sequence of statements as an error:

```
int vacationDays;
System.out.println(vacationDays); // ERROR--variable not initialized
```

You assign to a previously declared variable by using the variable name on the left, an equal sign (=), and then some Java expression with an appropriate value on the right.

```
int vacationDays;
vacationDays = 12;
```

You can both declare and initialize a variable on the same line. For example:

```
int vacationDays = 12;
```

Finally, in Java you can put declarations anywhere in your code. For example, the following is valid code in Java:

```
double salary = 65000.0;
System.out.println(salary);
int vacationDays = 12; // OK to declare a variable here
```

In Java, it is considered good style to declare variables as closely as possible to the point where they are first used.

Dynamic Initialization: Java allows variables to be initialized dynamically, using any expression valid at the time the variable is declared.

```
// Demonstrate dynamic initialization.
class DynInit {
  public static void main(String args[]) {
    double a = 3.0, b = 4.0;

    // c is dynamically initialized
    double c = Math.sqrt(a * a + b * b);

    System.out.println("Hypotenuse is " + c);
  }
}
```

Scope and Lifetime of Variables

In programming, scope of variable defines how a specific variable is accessible within the program or across classes. In programming, a variable can be declared and defined inside a class, method, or block. It defines the scope of the variable i.e., the visibility or accessibility of a variable. Variable declared inside a block or method are not visible to outside.

The lifetime of a variable refers to how long the variable exists before it is destroyed. Destroying variables refers to deallocating the memory that was allotted to the variables when declaring it. There are three types of variables:

1. ***Instance variables***
2. ***Class variables***
3. ***Local variables***

Instance Variables: A variable which is declared inside a class and outside all the methods and blocks is an instance variable. They are known as instance variables because every instance of the class (object) contains a copy of these variables. The general scope of an instance variable is throughout the class except in static methods. The lifetime of an instance variable is until the object stays in memory. Instance variables are non-static variables and are declared in a class outside of any method, constructor, or block.

- As instance variables are declared in a class, these variables are created when an object of the class is created and destroyed when the object is destroyed.
- Unlike local variables, we may use access specifiers for instance variables. If we do not specify any access specifier, then the default access specifier will be used.
- Initialization of an instance variable is not mandatory. Its default value is 0.
- Instance variables can be accessed only by creating objects.

Class Variables: A variable which is declared inside a class, outside all the blocks and is marked static is known as a class variable. The general scope of a class variable is throughout the class and the lifetime of a class variable is until the end of the program or as long as the class is loaded in memory. ***Static variables are also known as class variables.***

- These variables are declared similarly as instance variables. The difference is that ***static variables are declared using the static keyword within a class outside of any method, constructor or block.***
- Unlike instance variables, we can only have one copy of a static variable per class, irrespective of how many objects we create.

- Static variables are created at the start of program execution and destroyed automatically when execution ends.
- Initialization of a static variable is not mandatory. ***Its default value is 0.***
- If we access a static variable like an instance variable (through an object), the compiler will show a warning message, which won't halt the program. The compiler will replace the object name with the class name automatically.
- If we access a static variable without the class name, the compiler will automatically append the class name.

Local Variables: All other variables which are not instance and class variables are treated as local variables including the parameters in a method. Scope of a local variable is within the block in which it is declared and the lifetime of a local variable is until the control leaves the block in which it is declared.

- These variables are created when the block is entered, or the function is called and destroyed after exiting from the block or when the call returns from the function.
- The scope of these variables exists only within the block in which the variables are declared, i.e., we can access these variables only within that block.
- Initialization of the local variable is mandatory before using it in the defined scope.

Variable Type	Scope	Lifetime
Instance variable	Throughout the class except in static methods	Until the object is available in the memory
Class variable	Throughout the class	Until the end of the program
Local variable	Within the block in which it is declared	Until the control leaves the block in which it is declared

Example-1: WAP java program to illustrate the concept of Variables.

```
1  class Variables{
2      static int age; // static or class variables
3      float salary=1098; /// Instance variables
4
5      int sum(){
6         int x=9,y=10; // local variables
7         return x+y;
8          }
9      public static void main(String[] args){
10         Variables var = new Variables();
11         int z=var.sum();
12         System.out.println("Sum of Local Variables: " + z);
13         System.out.println("Instance Variables: " + var.salary);
14         System.out.println("Static Variables: " + age); //accessing static variables
15     }
16 }
```

Java Operators

Operator Type	Category	Precedence
Unary	postfix	*expr++ expr--*
	prefix	*++expr --expr +expr -expr ~ !*
Arithmetic	multiplicative	* / %
	additive	+ -
Shift	shift	<< >> >>>
Relational	comparison	< > <= >= instanceof
	equality	== !=
Bitwise	bitwise AND	&
	bitwise exclusive OR	^
	bitwise inclusive OR	|
Logical	logical AND	&&
	logical OR	||
Ternary	ternary	? :
Assignment	assignment	= += -= *= /= %= &= ^= |= <<= >>= >>>=

Example-2: WAP java program to add two numbers.

```
1  import java.util.*;
2  public class Table {
3      public static void main(String[] args) {
4          // read inputs
5          Scanner in = new Scanner(System.in);
6          System.out.println("Enter first number:  ");
7          int n1=in.nextInt();
8          System.out.println("Enter second number:  ");
9          int n2=in.nextInt();
10
11         System.out.println("Sum of two number: " + (n1+n2));
12     }
13 }
```

Example-3: WAP java program to illustrate the concept of Assignment operators.

```
class Main {
  public static void main(String[] args) {

    // create variables
    int a = 4;
    int var;

    // assign value using =
    var = a;
    System.out.println("var using =: " + var);

    // assign value using =+
    var += a;
    System.out.println("var using +=: " + var);

    // assign value using =*
    var *= a;
    System.out.println("var using *=: " + var);
  }
}
```

Example-4: WAP java program to illustrate the concept of Shift operators.

```
public class OperatorExample{
    public static void main(String args[]){
        System.out.println(10<<2);//10*2^2=10*4=40
        System.out.println(10<<3);//10*2^3=10*8=80
        System.out.println(20<<2);//20*2^2=20*4=80
        System.out.println(15<<4);//15*2^4=15*16=240
        System.out.println(10>>2);//10/2^2=10/4=2
        System.out.println(20>>2);//20/2^2=20/4=5
        System.out.println(20>>3);//20/2^3=20/8=2
    }
}
```

Example-5: WAP java program to illustrate the concept of Relational operators.

```java
class Main {
  public static void main(String[] args) {

    // create variables
    int a = 7, b = 11;

    // value of a and b
    System.out.println("a is " + a + " and b is " + b);

    // == operator
    System.out.println(a == b);  // false

    // != operator
    System.out.println(a != b);  // true

    // > operator
    System.out.println(a > b);  // false

    // < operator
    System.out.println(a < b);  // true

    // >= operator
    System.out.println(a >= b);  // false

    // <= operator
    System.out.println(a <= b);  // true
  }
}
```

Example-6: WAP java program to illustrate the concept of Logical operators.

```java
class Main {
  public static void main(String[] args) {

    // && operator
    System.out.println((5 > 3) && (8 > 5));  // true
    System.out.println((5 > 3) && (8 < 5));  // false

    // || operator
    System.out.println((5 < 3) || (8 > 5));  // true
    System.out.println((5 > 3) || (8 < 5));  // true
    System.out.println((5 < 3) || (8 < 5));  // false

    // ! operator
    System.out.println(!(5 == 3));  // true
    System.out.println(!(5 > 3));  // false
  }
}
```

Type Casting

When one type of data is assigned to another type of variable, an *automatic type conversion* will take place if the following two conditions are met:

- The two types are compatible.
- The destination type is larger than the source type.

When these two conditions are met, a *widening conversion* takes place. For example, the **int** type is always large enough to hold all valid **byte** values, so no explicit cast statement is required.

For widening conversions, the numeric types, including integer and floating-point types, are compatible with each other. However, there are no automatic conversions from the numeric types to **char** or **boolean**. Also, **char** and **boolean** are not compatible with each other.

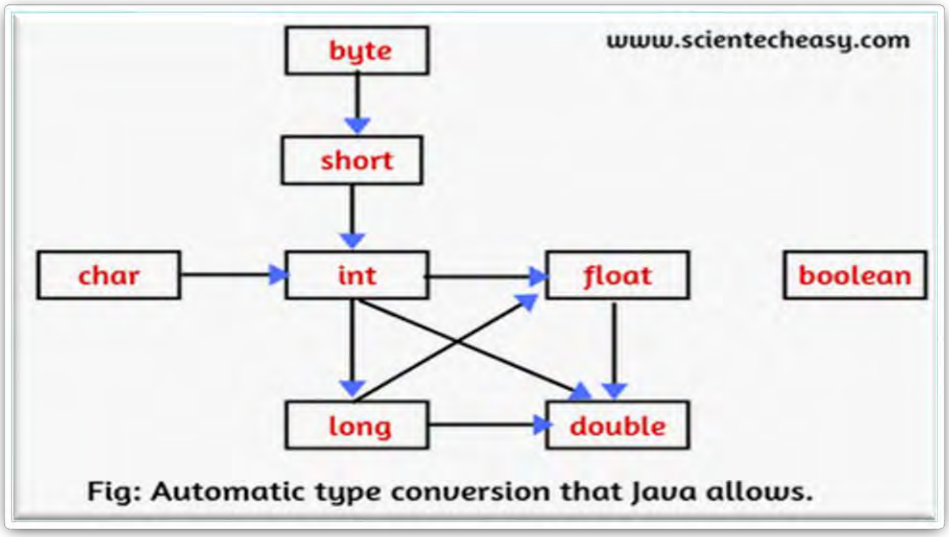


Fig: Automatic type conversion that Java allows.

As we know that automatic conversion is very helpful and safe, but it cannot fulfill all needs.

For example, if you will assign a double value to an int variable, this conversion cannot be performed automatically because an int is smaller than a double.

In this case, we must use explicit type casting to create a conversion between two incompatible types. This kind of conversion is also known as **narrowing conversion in java**.

The conversion of a higher data type into a lower data type is called narrowing conversion.

Syntax

```
(type_name) expression;
```

where type_name specify a standard data type. The expression may be a variable, constant, or an expression.

double d = 57.17;
int i = (int)d; // Explicit casting from long to int data type

Java - Strings Class

In Java programming language, strings are treated as objects. The Java platform provides the String class to create and manipulate strings.
In Java, string is basically an object that represents sequence of char values. An array of characters works same as Java string. For example:

char[] ch={'j','a','v','a','t','p','o','i','n','t'};
String s=new String(ch);
is same as:
String s="javatpoint";

How to create a string object?
There are two ways to create String object:

1. ***By string literal***
2. ***By new keyword***

String Literal: Java String literal is created by using double quotes. For Example:

String s="welcome";

Each time you create a string literal, the JVM checks the "string constant pool" first. If the string already exists in the pool, a reference to the pooled instance is returned. If the string doesn't exist in the pool, a new string instance is created and placed in the pool. For example:
String s1="Welcome";
String s2="Welcome";//It doesn't create a new instance

In the above example, only one object will be created. Firstly, JVM will not find any string object with the value "Welcome" in string constant pool that is why it will create a new object. After that it will find the string with the value "Welcome" in the pool, it will not create a new object but will return the reference to the same instance.
Note: String objects are stored in a special memory area known as the "string constant pool".

By new keyword

String s=new String("Welcome");

In such case, JVM will create a new string object in normal (non-pool) heap memory, and the literal "Welcome" will be placed in the string constant pool. The variable s will refer to the object in a heap (non-pool).

Example-7: WAP java program to illustrate the concept of Basic String.

```
public class StringExample{
    public static void main(String args[]){
        String s1="java";//creating string by Java string literal
        char ch[]={'s','t','r','i','n','g','s'};
        String s2=new String(ch);//converting char array to string
        String s3=new String("example");//creating Java string by new keyword
        System.out.println(s1);
        System.out.println(s2);
        System.out.println(s3);
    }
}
```

Java Arrays

An array is a data structure that stores a collection of values of the same type. You access each individual value through an integer *index*. For example, if `a` is an array of integers, then `a[i]` is the ith integer in the array.

Declare an array variable by specifying the array type—which is the element type followed by `[]`—and the array variable name. For example, here is the declaration of an array `a` of integers:

```
int[] a;
```

However, this statement only declares the variable `a`. It does not yet initialize `a` with an actual array. Use the `new` operator to create the array.

```
int[] a = new int[100]; // or var a = new int[100];
```

This statement declares and initializes an array of 100 integers.

The array length need not be a constant: `new int[n]` creates an array of length `n`.

Syntax

```
dataType[] arrayRefVar; // preferred way.
or
dataType arrayRefVar[]; // works but not preferred way.
```

Note – The style **dataType[] arrayRefVar** is preferred. The style **dataType arrayRefVar[]** comes from the C/C++ language and was adopted in Java to accommodate C/C++ programmers.

Example

The following code snippets are examples of this syntax –

```
double[] myList;   // preferred way.
or
double myList[];   // works but not preferred way.
```

```
arrayRefVar = new dataType[arraySize];
```

The above statement does two things –

- It creates an array using new dataType[arraySize].
- It assigns the reference of the newly created array to the variable arrayRefVar.

Declaring an array variable, creating an array, and assigning the reference of the array to the variable can be combined in one statement, as shown below –

```
dataType[] arrayRefVar = new dataType[arraySize];
```

Alternatively you can create arrays as follows –

```
dataType[] arrayRefVar = {value0, value1, ..., valuek};
```

For example, the following two declarations are equivalent:

```
int al[] = new int[3];
int[] a2 = new int[3];
```

The following declarations are also equivalent:

```
char twod1[][] = new char[3][4];
char[][] twod2 = new char[3][4];
```

Example-7: WAP java program to find sum of array elements and also find the largest element of Array.

```
1  public class TestArray {
2      `public static void main(String[] args) {
3       double[] myList = {1.9, 2.9, 3.4, 3.5};
4       // Print all the array elements
5       for (int i = 0; i < myList.length; i++) {
6            System.out.println(myList[i] + " "); }
7
8        // Summing all elements
9       double total = 0;
10      for (int i = 0; i < myList.length; i++) {
11           total += myList[i];}
12      System.out.println("Total is " + total);
13
14      // Finding the largest element
15      double max = myList[0];
16      for (int i = 1; i < myList.length; i++) {
17           if (myList[i] > max) max = myList[i];}
18      System.out.println("Max is " + max);
19     }
20 }
```

Java Control Statements

Java supports two selection statements: if and switch. These statements allow you to control the flow of your program's execution based upon conditions known only during run time.

```
if (condition) statement1;
else statement2;
```

Here, each *statement* may be a single statement or a compound statement enclosed in curly braces (that is, a *block*). The *condition* is any expression that returns a **boolean** value. The **else** clause is optional.

The **if** works like this: If the *condition* is true, then *statement1* is executed. Otherwise, *statement2* (if it exists) is executed. In no case will both statements be executed. For example, consider the following:

```
int a, b;
//...
if(a < b) a = 0;
else b = 0;
```

Nested ifs

A *nested* **if** is an **if** statement that is the target of another **if** or **else**. Nested **if**s are very common in programming. When you nest **if**s, the main thing to remember is that an **else** statement always refers to the nearest **if** statement that is within the same block as the **else** and that is not already associated with an **else**. Here is an example:

```
if(i == 10) {
  if(j < 20) a = b;
  if(k > 100) c = d; // this if is
  else a = c;        // associated with this else
}
else a = d;          // this else refers to if(i == 10)
```

The if-else-if Ladder

A common programming construct that is based upon a sequence of nested **if**s is the *if-else-if* ladder. It looks like this:

```
if(condition)
  statement;
else if(condition)
  statement;
else if(condition)
  statement;
.
.
.
else
  statement;
```

Example-8: WAP java program to illustrate the concept of If-else statement.

```
class IfElse {
    public static void main(String args[]) {
        int month = 4; // April
        String season;

        if(month == 12 || month == 1 || month == 2)
            season = "Winter";
        else if(month == 3 || month == 4 || month == 5)
            season = "Spring";
        else if(month == 6 || month == 7 || month == 8)
            season = "Summer";
        else if(month == 9 || month == 10 || month == 11)
            season = "Autumn";
        else
            season = "Bogus Month";

        System.out.println("April is in the " + season + ".");
    }
}
```

switch

The **switch** statement is Java's multiway branch statement. It provides an easy way to dispatch execution to different parts of your code based on the value of an expression. As such, it often provides a better alternative than a large series of **if-else-if** statements. Here is the general form of a **switch** statement:

```
switch (expression) {
  case value1:
    // statement sequence
    break;
  case value2:
    // statement sequence
    break;
.
.
.
  case valueN:
    // statement sequence
    break;
  default:
    // default statement sequence
}
```

For versions of Java prior to JDK 7, expression must be of type byte, short, int, char, or an enumeration. Beginning with JDK 7, expression can also be of type String. Each value specified in the case statements must be a unique constant expression (such as a literal value). Duplicate case values are not allowed. The type of each value must be compatible with the type of expression.

The switch statement works like this: The value of the expression is compared with each of the values in the case statements. If a match is found, the code sequence following that case statement is executed. If none of the constants matches the value of the expression, then the default statement is executed. However, the default statement is optional. If no case matches and no default is present, then no further action is taken.

The break statement is used inside the switch to terminate a statement sequence. When a break statement is encountered, execution branches to the first line of code that follows the entire switch statement. This has the effect of "jumping out" of the switch.

Example-9: WAP java program to illustrate the concept of Switch statement using Loop.

```
class SampleSwitch {
    public static void main(String args[]) {
        for(int i=0; i<6; i++)
            switch(i) {
                case 0:
                    System.out.println("i is zero.");
                    break;
                case 1:
                    System.out.println("i is one.");
                    break;
                case 2:
                    System.out.println("i is two.");
                    break;
                case 3:
                    System.out.println("i is three.");
                    break;
                default:
                System.out.println("i is greater than 3.");
            }
        }
    }
}
```

Example-10: WAP java program to illustrate the concept of Switch statement using String control variables.

```
// Use a string to control a switch statement.
class StringSwitch {
    public static void main(String args[]) {
        String str = "two";
        switch(str) {
            case "one":
                System.out.println("one");
                break;
            case "two":
                System.out.println("two");
                break;
            case "three":
                System.out.println("three");
                break;
                default:
                System.out.println("no match");
                break;
        }
    }
}
```

Example-11: WAP java program to display the season name using Switch statement.

```
class Switch {
    public static void main(String args[]) {
    int month = 4;   String season;
    switch (month) {
        case 12:
        case 1:
        case 2:
            season = "Winter";  break;
        case 3:
        case 4:
        case 5:
            season = "Spring";  break;
        case 6:
        case 7:
        case 8:
            season = "Summer";  break;
        case 9:
        case 10:
        case 11:
            season = "Autumn";  break;
        default:
            season = "Bogus Month";
    }
    System.out.println("April is in the " + season + ".");
    }
}
```

Nested switch Statements

You can use a **switch** as part of the statement sequence of an outer **switch**. This is called a *nested* **switch**. Since a **switch** statement defines its own block, no conflicts arise between the **case** constants in the inner **switch** and those in the outer **switch**. For example, the following fragment is perfectly valid:

```
switch(count) {
  case 1:
    switch(target) { // nested switch
      case 0:
        System.out.println("target is zero");
        break;
      case 1: // no conflicts with outer switch
        System.out.println("target is one");
        break;
    }
    break;
  case 2: // ...
```

Here, the **case 1:** statement in the inner switch does not conflict with the **case 1:** statement in the outer switch. The **count** variable is compared only with the list of cases at the outer level. If **count** is 1, then **target** is compared with the inner list cases.

In summary, there are three important features of the **switch** statement to note:

- The **switch** differs from the **if** in that **switch** can only test for equality, whereas **if** can evaluate any type of Boolean expression. That is, the **switch** looks only for a match between the value of the expression and one of its **case** constants.
- No two **case** constants in the same **switch** can have identical values. Of course, a **switch** statement and an enclosing outer **switch** can have **case** constants in common.
- A **switch** statement is usually more efficient than a set of nested **if**s.

while

The **while** loop is Java's most fundamental loop statement. It repeats a statement or block while its controlling expression is true. Here is its general form:

```
while(condition) {
  // body of loop
}
```

The *condition* can be any Boolean expression. The body of the loop will be executed as long as the conditional expression is true. When *condition* becomes false, control passes to the next line of code immediately following the loop. The curly braces are unnecessary if only a single statement is being repeated.

Example-12: WAP java program to illustrate the concept of simple While Statement.

```
1  class While {
2      public static void main(String args[]) {
3          int n = 10;
4          while(n > 0) {
5              System.out.println("tick " + n);
6              n--;
7          }
8      }
9  }
```

```
do {
  // body of loop
} while (condition);
```

Each iteration of the **do-while** loop first executes the body of the loop and then evaluates the conditional expression. If this expression is true, the loop will repeat. Otherwise, the loop terminates. As with all of Java's loops, *condition* must be a Boolean expression.

Example-13: WAP java program to illustrate the concept of simple Do-while Statement.

```
class DoWhile {
    public static void main(String args[]) {
        int n = 10;
        do {
            System.out.println("tick " + n);
            n--;
            } while(n > 0);
        }
    }
}
```

Here is the general form of the traditional **for** statement:

```
for(initialization; condition; iteration) {
  // body
}
```

If only one statement is being repeated, there is no need for the curly braces.

Example-13: WAP java program to check whether a given number is prime or not.

```
class FindPrime {
    public static void main(String args[]) {
        int num;  boolean isPrime;
        num = 23;  isPrime = true; // assume num is prime initally.

        for(int i=2; i <= num/2; i++) {
            if((num % i) == 0) {
                isPrime = false;
                break;
            }
        }
        if(isPrime)
            System.out.println(num +" is Prime Number.");
        else
            System.out.println(num +" is Not Prime Number.");
    }
}
```

Here is one more **for** loop variation. You can intentionally create an infinite loop (a loop that never terminates) if you leave all three parts of the **for** empty. For example:

```
for( ; ; ) {
  // ...
}
```

Example-14: WAP java program to illustrate the concept of part of loop can be empty.

```
1   // Parts of the for loop can be empty.
2   class ForVar {
3       public static void main(String args[]) {
4           int i;
5           boolean done = false;
6           i = 0;
7           for( ; !done; ) {
8               System.out.println("i is " + i);
9               if(i == 10)
10                  done = true;
11              i++;
12          }
13      }
14  }
```

Java for each loop

The general form of the for-each version of the **for** is shown here:

for(*type itr-var*: *collection*) *statement-block*

Here, *type* specifies the type and *itr-var* specifies the name of an *iteration variable* that will receive the elements from a collection, one at a time, from beginning to end. The collection being cycled through is specified by *collection*. There are various types of collections that can

Example-15: WAP java program to illustrate the concept of for each loop.

```
1   class ForEach {
2       public static void main(String args[]) {
3           int nums[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
4           int sum = 0;
5           // use for-each style for to display and sum the values
6           for(int x : nums) {
7               System.out.println("Value is: " + x);
8               sum += x;
9           }
10          System.out.println("Summation: " + sum);
11          sum=0;
12          // use traditional for loop
13          for(int i=0; i<10;i++) {
14              System.out.println("Value is: " + nums[i]);
15              sum += nums[i];
16          }
17          System.out.println("Summation: " + sum);
18      }
```

Example-16: WAP java program to search a value of array using for each loop.

```java
// Search an array using for-each style for.
class Search {
    public static void main(String args[]) {
        int nums[] = { 6, 8, 3, 7, 5, 6, 1, 4 };
        int val = 15;
        boolean found = false;
        // use for-each style for to search nums for val
        for(int x : nums) {
            if(x == val) {
                found = true;
                break;
            }
        }
        if(found)
            System.out.println("Value found!");
        else
            System.out.println("Value not found!");
    }
}
```

Reading Input

Method	Description
int nextInt()	It is used to scan the next token of the input as an integer.
float nextFloat()	It is used to scan the next token of the input as a float.
double nextDouble()	It is used to scan the next token of the input as a double.
byte nextByte()	It is used to scan the next token of the input as a byte.
String nextLine()	Advances this scanner past the current line.
boolean nextBoolean()	It is used to scan the next token of the input into a boolean value.
long nextLong()	It is used to scan the next token of the input as a long.
short nextShort()	It is used to scan the next token of the input as a Short.
BigInteger nextBigInteger()	It is used to scan the next token of the input as a BigInteger.
BigDecimal nextBigDecimal()	It is used to scan the next token of the input as a BigDecimal.

Example-17: WAP java program to take input from users and display the result.

```
1  import java.util.Scanner;
2  class ReadDemo {
3      public static void main(String args[]){
4          // Using Scanner for Getting Input from User
5          Scanner in = new Scanner(System.in);
6
7          System.out.println("Enter string: ");
8          String s = in.nextLine();
9          System.out.println("You entered string " + s);
10
11         System.out.println("Enter Integer ");
12         int a = in.nextInt();
13         System.out.println("You entered integer " + a);
14
15         System.out.println("Enter Float");
16         float b = in.nextFloat();
17         System.out.println("You entered float " + b);
18     }
19 }
```

```
Scanner in = new Scanner(System.in);
```

(We discuss constructors and the `new` operator in detail in Chapter 4.)

Now you use the various methods of the `Scanner` class to read input. For example, the `nextLine` method reads a line of input.

```
System.out.print("What is your name? ");
String name = in.nextLine();
```

Here, we use the `nextLine` method because the input might contain spaces. To read a single word (delimited by whitespace), call

```
String firstName = in.next();
```

To read an integer, use the `nextInt` method.

```
System.out.print("How old are you? ");
int age = in.nextInt();
```

Similarly, the `nextDouble` method reads the next floating-point number.

Chapter – 3: Class and Object

OOPs (Object-Oriented Programming System)

Object-Oriented Programming is a methodology or paradigm to design a program using classes and objects. Object means a real-world entity such as a Mobile, Laptop, College, computer, watch, etc. Java is an object-oriented programming language. The core concept of the object-oriented approach is to break complex problems into smaller objects.

The main features of OOPS are:

Class and Object: An object is an instance of a class. An object is any entity that has a state and behavior. Collection of objects is called class. It is a logical entity. A class can also be defined as a blueprint from which you can create an individual object.

An object is an instance of a class. A class is a template or blueprint from which objects are created. So, an object is the instance(result) of a class.

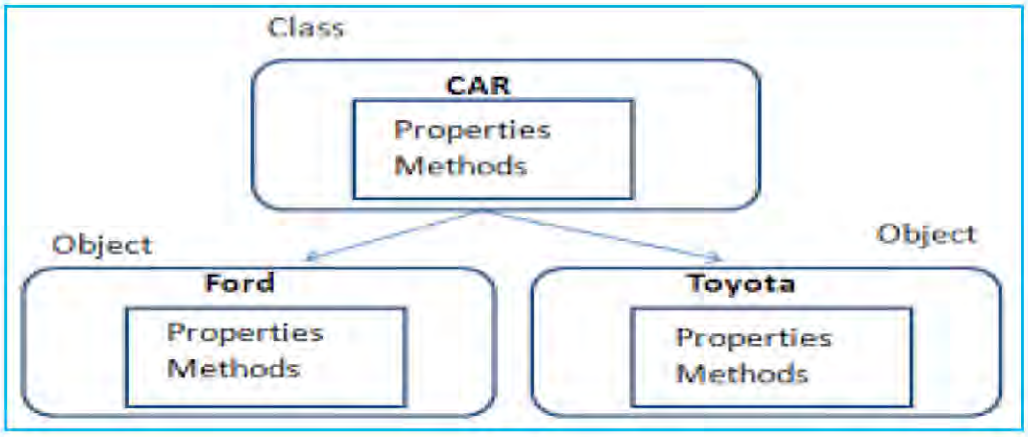
Object Definitions:

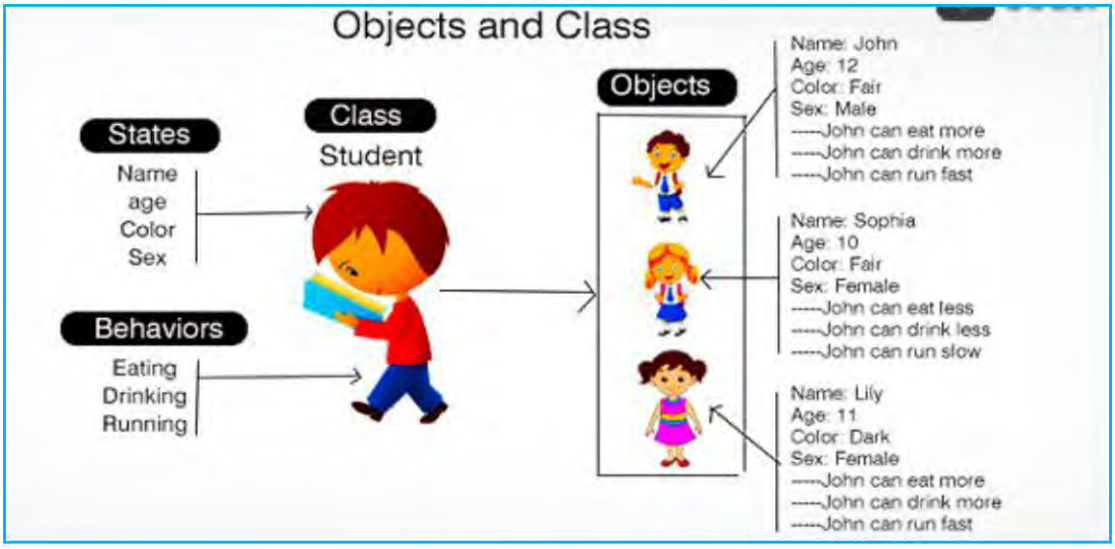
- An object is a real-world entity.
- An object is a runtime entity.
- The object is an entity which has state and behavior.
- The object is an instance of a class.

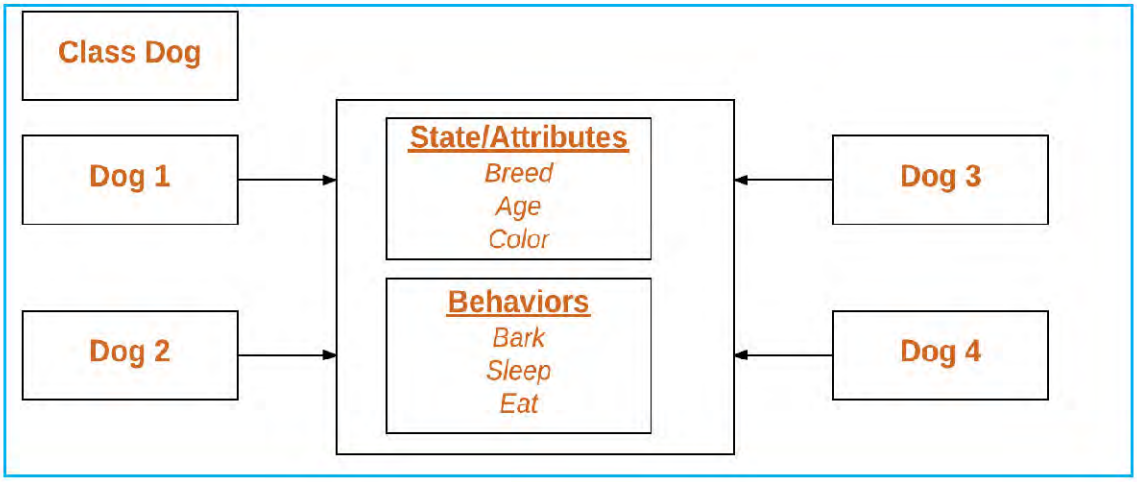
Three key characteristics of objects:

- ***The object's behavior***—what can you do with this object, or what methods can you apply to it?
- ***The object's state***—how does the object react when you invoke those methods?
- ***The object's identity***—how is the object distinguished from others that may have the same behavior and state?

All objects that are instances of the same class share a family resemblance by supporting the same behavior. The behavior of an object is defined by the methods that you can call.







OBJECT	CLASS
Object is an instance ofa class.	Class is a blue print from which objects are created
Object is a real world entity such as chair, pen. table, laptop etc.	Class is a group of similar objects.
Object is a physical entity.	Class is a logical entity.
Object is created many times as per requirement.	Class is declared once.
Object allocates memory when it is created.	Class doesn't allocated memory when it is created.
Object is created through new keyword. Employee ob = new Employee();	Class is declared using class keyword. class Employee{}
There are different ways to create object in java:- New keyword, newinstance() method, clone() method, And deserialization.	There is only one way to define a class, i.e., by using class keyword.

Encapsulation: Encapsulation means putting together all the variables (instance variables) and the methods into a single unit called Class. It also means hiding data and methods within an Object. Encapsulation provides the security that keeps data and methods safe from inadvertent changes. Programmers sometimes refer to encapsulation as using a "black box," or a device that you can use without regard to the internal mechanisms. A programmer can access and use the methods and data contained in the black box but cannot change them.

Inheritance: An important feature of object-oriented programs is inheritance—the ability to create classes that share the attributes and methods of existing classes, but with more specific features. Object acquires the properties and behaviors of the parent object. It's creating a parent-child relationship between two classes. It offers robust and natural mechanism for organizing and structure of any software. Inheritance is mainly used for code reusability.

Polymorphism: In Core, Java Polymorphism is one of easy concept to understand. Polymorphism definition is that Poly means many and morphos means forms. It describes the feature of languages that allows the same word or symbol to be interpreted correctly in different situations based on the context. There are two types of Polymorphism available in Java. For example, in English, the verb "run" means different things if you use it with "a footrace," a "business," or "a computer." You understand the meaning of "run" based on the other words used with it. Object-oriented programs are written so that the methods having the same name works differently in different context. Java provides two ways to implement polymorphism.

Abstraction: It is hiding internal details and showing functionality is known as abstraction. Abstraction is one of the OOP Concepts in Java which is an act of representing essential features without including background details. It is a technique of creating a new data type that is suited for a specific application. For example, while driving a car, you do not have to be concerned with its internal working. Here you just need to concern about parts like steering wheel, Gears, accelerator, etc. In Java, we use abstract class and interface to achieve abstraction.

Declaration of Java Classes

In order to bring class into existence, we should declare it. We can declare a class with the use of a class keyword. The components of the **Java Class declaration are** –

1. ***Access Modifiers*** – We can access Java classes using any access modifiers such as ***public, private, protected and default.***
2. ***Class Name*** – In Java, the class name generally represents nouns which should begin with a capital letter without any spaces.
3. ***Superclass (if any)*** – The name of the parent class is a superclass and its child class is a subclass, and child class inherits the properties of a parent using the extends keyword. A subclass can only inherit a single parent.
4. ***Interfaces (if any)*** – To declare an interface, we just write the keyword interface followed by the interface name.
5. ***Class Body*** – The class body follows the class declaration and embeds within curly braces {}.

The syntax for declaring classes:

```
1.   <access-modifier> class <ClassName>
2.     {
3.       //Class body containing variables and methods
4.     }
```

Creating Objects from a Java Class

We know that an object is an instance of a class. To create an object of a class, first, we need to declare it and then instantiate it with the help of a **"new"** keyword.

Syntax of creating an object of a class: To create an object of a class, specify the class name, followed by the object name, by using the **new** keyword:

ClassName objectName = new ClassName();

Accessing the members of a Java Class

We can access the data members of a class using the object of the class. We just write the name of the object which is followed by a dot operator then we write the name of the data member (either variables or methods) which we want to access.

Syntax of accessing data members and methods of a Java Class:

objectName.variableName; //accessing the variables
objectName.MethodName(); //accessing the methods

Methods in Java

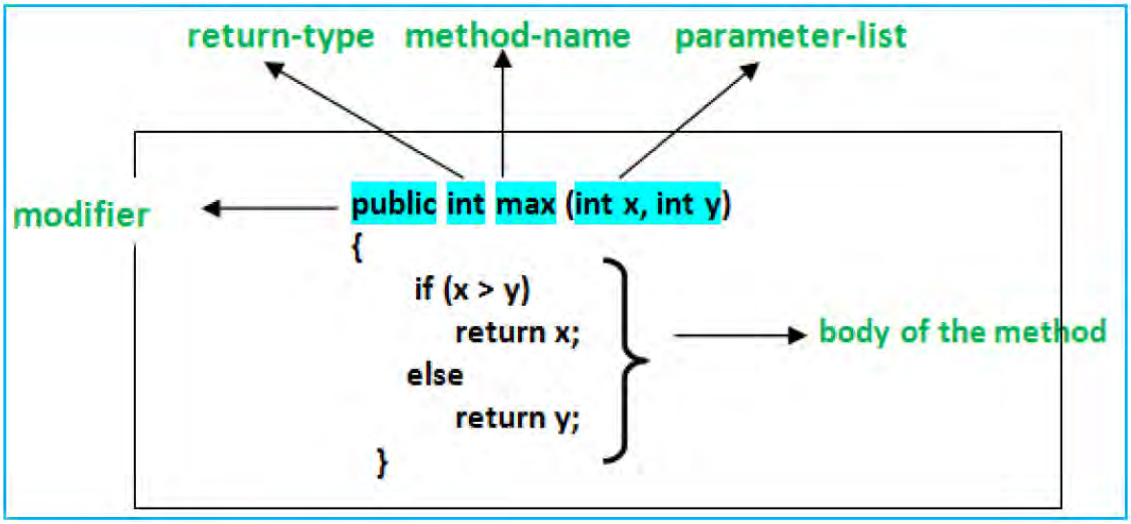
A method in Java or Java Method is a collection of statements that perform some specific tasks and return the result to the caller. In Java, every method must be part of some class that is different from languages like C, C++, and Python.

Method Declaration: In general, method declaration has six components:

1. ***Modifier:*** It defines the access type of the method i.e., from where it can be accessed in your application. In Java, there 4 types of access specifiers.
   - **Public:** It is accessible in all classes in your application.
   - **Protected:** It is accessible within the class in which it is defined and, in its subclass, /es
   - **Private:** It is accessible only within the class in which it is defined.
   - **Default:** It is declared/defined without using any modifier. It is accessible within the same class and package within which its class is defined.

	default	private	protected	public
Same Class	Yes	Yes	Yes	Yes
Same package subclass	Yes	No	Yes	Yes
Same package non-subclass	Yes	No	Yes	Yes
Different package subclass	No	No	Yes	Yes
Different package non-subclass	No	No	No	Yes

2. ***The return type:*** The data type of the value returned by the method or void if does not return a value.
3. **Method Name**: the rules for field names apply to method names as well, but the convention is a little different.
4. **Parameter list:** Comma-separated list of the input parameters is defined, preceded with their data type, within the enclosed parenthesis. If there are no parameters, you must use empty parentheses ().
5. **Exception list:** The exceptions you expect by the method can throw; you can specify these exception(s).
6. **Method body:** it is enclosed between braces. The code you need to be executed to perform your intended operations.



Example – 1: Write a Java program to illustrate the concept of Class and object.

```
1   // " Student.java"
2   class Student{
3       int crn;//field or data member or instance variable
4       String name;
5       public static void main(String args[]){
6           //Creating an object or instance
7           Student S1=new Student();//creating an object of Student
8           S1.crn=101;
9           S1.name="Ram Sharma";
10
11          System.out.println("Name: "+S1.name);
12          System.out.println("Roll Num: "+S1.crn);
13      }
14  }
```

Example – 2: Write a Java program to illustrate the concept of Class, object and Method.

```
1   // " Student1.java"
2   class Student1{
3       int crn;    String name;
4
5       void setData(int x, String y){
6         crn=x;   name=y;}
7
8       public static void main(String args[]){
9           Student1 S1=new Student1();
10
11          S1.setData(101,"Ram Sharma");
12          System.out.println("Name: "+S1.name);
13          System.out.println("Roll Num: "+S1.crn);
14      }
15  }
```

Example – 3: Write a Java program to calculate volume of Box. Use SetDim() to set the dimension and volume() to calculate the volume of Box.

```
// "BoxDemo.java"
class Box {
    double width;       double height;    double depth;
    void setDim(double w, double h, double d) { // sets dimensions of box
        width = w;   height = h;   depth = d; }
    double volume() { // compute and return volume
        return width * height * depth; }
}
class BoxDemo{
    public static void main(String[] args) {
        Box mybox1 = new Box();
        Box mybox2 = new Box();
        double vol;
        // initialize each box
        mybox1.setDim(10, 20, 15);
        mybox2.setDim(3, 6, 9);
        // get volume of first box
        vol = mybox1.volume();
        System.out.println("Volume is " + vol);
        // get volume of second box
        vol = mybox2.volume();
        System.out.println("Volume is " + vol);
    }
}
```

Constructors in Java

A constructor in Java is a special method that is used to initialize objects. The constructor is called when an object of a class is created. In Java, ***a constructor is a block of codes similar to the method.*** At the time of calling the constructor, memory for the object is allocated in the memory. ***It is a special type of method which is used to initialize the object.*** Every time an object is created using the ***new ()*** keyword, at least one constructor is called.

Note: It is not necessary to write a constructor for a class. It is because java compiler creates a default constructor if your class doesn't have any.

Rules for Writing Constructors in Java

Following are some rules for writing constructors in Java:

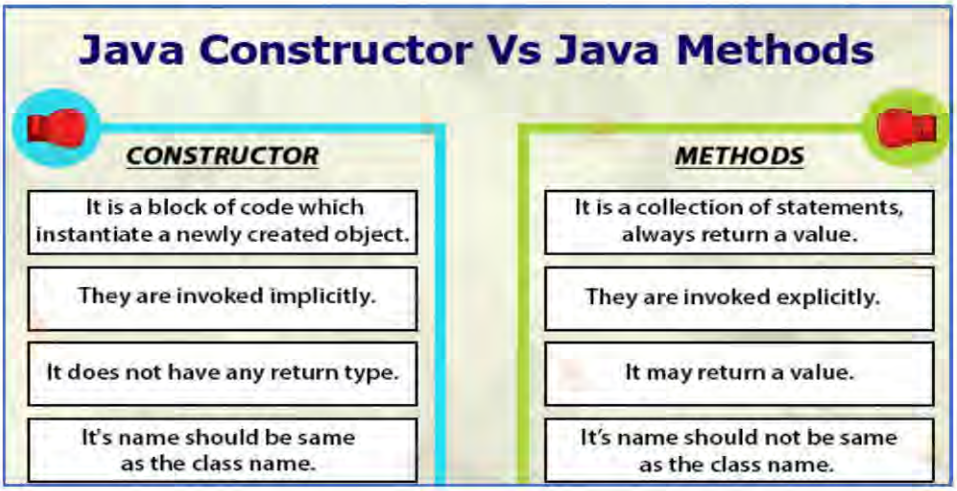
- The name of the constructor must be the same as the name of its class.
- A constructor must have no return type. It cannot have not even void as its return type.
- We can use the access modifiers with a constructor to control its access so that other classes can call the constructor.
- We cannot declare a constructor as final, abstract, abstract and synchronized.

How Constructors are Different from Methods in Java?

- A constructor is a block of code that instantiates a newly created object, while a method is a set of statements that always return value depending upon its execution.
- The constructor's name should be the same as the class name. On the other hand, the name of the method should not be the same as the class name.
- Constructors are called implicitly, while we call the methods explicitly.
- A constructor should not have any return type, not even void, but methods must have a return type.
- The compiler automatically creates the constructor if there is no constructor in the class. But, in the case of the method, there is no default method provided by the compiler.
- We can override a method but we can't override a constructor.

The syntax for writing a Constructor: A constructor has the same name as the class and we can write a constructor as follows:

```
public class MyClass {
    //This is the Constructor
    MyClass() {
        //Constructor body
    }
..
}
```

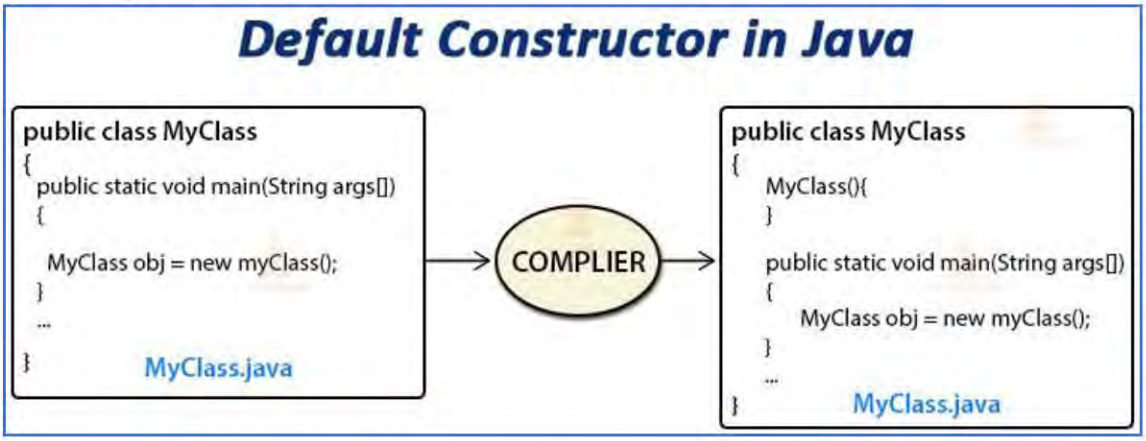


Types of Constructor in Java

There are two types of constructors in Java, which are:

1. ***Default Constructor:***
   - A Default Constructor is a ***constructor with no parameter***.
   - The Java ***compiler automatically creates a default constructor*** if we do not write any constructor in our program.
   - This default constructor is not present in source code or the java file as the compiler automatically puts it into the Java code during the compilation process and therefore, we can't find it in our java file, rather ***it exists in the bytecode or .class file.***

- If we do not provide a user-defined constructor in a class, the ***compiler initializes member variables to its default values such as:***
  - *Numeric data types set to 0*
  - *Char data types set to a null character ('\0')*
  - *Reference variables set to null*



2. ***Parameterized Constructor:*** A Parameterized constructor is a constructor with a specific number of parameters. We can use parameterized constructor mainly to initialize the members of the class with different values or objects.

Example – 4: Write a Java program to illustrate the concept of Default Constructor.

```
1  //"DefaultConstructor.java"
2  class DefaultConstructor {
3      String name;
4      DefaultConstructor() {
5          System.out.println("DefaultConstructor Called:");
6          name = "Default Constructor";
7        }
8
9      public static void main(String[] args) {
10         // constructor is invoked while creating an object of the DefaultConstructor class
11         DefaultConstructor obj = new DefaultConstructor();
12         System.out.print("Name is " + obj.name);
13     }
14 }
```

Example – 5: Write a Java program to illustrate the concept of parameterized constructor.

```
// "ParameterizedConstructor.java"
class ParameterizedConstructor {
    String languages;
    int year;
    ParameterizedConstructor(String lang,int yr) {
        languages = lang;
        year=yr;
        System.out.println(languages + " Programming Language "+ "started in "+ year);
    }
    public static void main(String[] args) {
        ParameterizedConstructor obj1 = new ParameterizedConstructor("Java",1995);
        ParameterizedConstructor obj2 = new ParameterizedConstructor("Python",1991);
        ParameterizedConstructor obj3 = new ParameterizedConstructor("C",1972);
    }
}
```

Example – 6: Write a Java program to set the name and roll number of students using parameterized constructor and display the information.

```
// "ParaConstructor.java"
class Student{
    String name;
    int crn;
    Student(String name, int crn){
        this.name = name;
        this.crn = crn;
    }
}
public class ParaConstructor{
    public static void main (String[] args){
        Student object = new Student("Ram Sharma", 105);
        System.out.println("Name: " + object.name );
        System.out.println("Roll Num: " + object.crn);

        Student object1 = new Student("Sita Sharma", 124);
        System.out.println("Name: " + object1.name );
        System.out.println("Roll Num: " + object1.crn);
    }
}
```

3. ***Overloaded Constructor in Java:*** Overloading generally means "to have multiple instances of the same thing". Constructor Overloading in Java is a process ***of having more than one constructor with different parameters list***. It allows the constructor to behave differently and perform a different task with respect to its parameters. ***Constructor overloading is the same as method overloading in Java.***

Example – 6: Write a Java program to illustrate the concept of overloaded constructor.

```
// "OverloadedConstructor.java"
class Student{
    Student(String name){
        System.out.println("Constructor with one parameter: String: ");
        System.out.println("Name: " +name); }

    Student(int crn){
        System.out.println("Constructor with one parameter: Integer: ");
        System.out.println("Roll Number: " + crn); }

    Student(String name, int crn){
        System.out.println("Constructor with two parameters: String and Integer: ");
        System.out.println("Name: " + name);
        System.out.println("Roll Number: " + crn);}
}
public class OverloadedConstructor{
    public static void main (String[] args){
        Student obj1 = new Student("Ram Sharma");
        Student obj2 = new Student(112);
        Student obj3 = new Student("Ram Sharma", 112);
    }
}
```

How to Initialize Object in Java?

There are three ways by which we can initialize object. They are as follows:

1. ***By using constructor:*** Using default or parameterized constructor.
2. ***By using a reference variable:*** Using dot operator.
3. ***By using a method:*** Using instance method.

Example – 7: Write a Java program to illustrate the concept of object initialization.

```
// ObjectInitializeEx.java
class ObjectInitializeEx {
    String name;   int rollNo;   // instance variables

    ObjectInitializeEx(){}

    ObjectInitializeEx(String name,int rollNo){
       this.name = name; this.rollNo = rollNo; }

    void setValue(String x, int y){ //instance method
         name=x; rollNo=y; }

    void display(){ //instance method
        System.out.println("Student's name:" +name);
        System.out.println("Student's roll no: " +rollNo); }

    public static void main(String[] args) {
    System.out.println("object initialization By using constructor");
    ObjectInitializeEx obj1 = new ObjectInitializeEx("Ram Sharma",205);
    obj1.display();
```

```
21
22      System.out.println("Object Initialization by using Reference variable");
23      ObjectInitializeEx obj2 = new ObjectInitializeEx();
24      obj2.name="Hari Sharma"; obj2.rollNo=305;
25      obj2.display();
26
27      System.out.println("Object Initialization by using Method");
28      ObjectInitializeEx obj3 = new ObjectInitializeEx();
29      obj3.setValue("Shyam Sharma",310);
30      obj3.display();
31      } // main function end
32  }// class end
```

Static Variables and Methods in Java

The static keyword is used to construct methods that will exist regardless of whether or not any instances of the class are generated. Any method that uses the static keyword is referred to as a static method.

Features of static method:

- A static method in Java is a method that is part of a class rather than an instance of that class.
- Every instance of a class has access to the method.
- Static methods have access to class variables (static variables) without using the class's object (instance).
- Only static data may be accessed by a static method. It is unable to access data that is not static (instance variables).
- In both static and non-static methods, static methods can be accessed directly.

Syntax to declare the static method:

```
Access_modifier static void methodName() {
        // Method body.
}
```

The name of the class can be used to invoke or access static methods.

Syntax to call a static method:

```
className.methodName();
```

Restrictions in Static Methods:

- Non-static data members or non-static methods cannot be used by static methods, and static methods cannot call non-static methods directly.
- In a static environment, this and super aren't allowed to be used.

Sr. No	Instance Methods	Static (Class) Methods
1	Instance methods are used to perform repetitive tasks like reading records from the file, reading records from the DBMS etc…	Static methods are used to perform single operation like opening the files, obtaining a DBMS connection etc…

2	Methods definition does not require a static keyword to start.
	Syntax:
	void net_salary (parameters if any)
	{
	Block of statements;
	}
		Methods definition must start with the static keyword.
		Syntax:
		static void basic_salary (parameters if any)
		{
		Block of statements;
		}

3	Each and every instance method must be accessed with the respective Object name	Each and every static method must be accessed with the respective Class name
4	Result of instance method is not shared. Every object has its own copy of instance method.	Results of static method always shared by objects of the same class.

Example – 8: Write a Java program to illustrate the concept of static variables and methods.

```
1  //StaticExample.java
2  import java.io.*;
3  public class StaticExample {
4      static int num = 100;
5      static String str = "Static Variable";
6      int b = 50;   // instance variable
7
8      static void display() {  // This is Static method
9          System.out.println("Accessing Static data: " + num);
10         System.out.println("Accessing Static data: " + str); }
11         // System.out.println("Accessing Non-Static data: " + b);}
12
13     void nonstatic() {  // non-static method
14         System.out.println("Accessing Static data: " + num);
15         System.out.println("Accessing Static data: " + str);
16         System.out.println("Accessing Non-Static data: " + b);  }
17
18     public static void main(String args[]) {
19         StaticExample obj = new StaticExample();
20         // This is object to call non static function
21         obj.nonstatic();
22         // static method can called directly without an object
23         display();
24     }
25 }
```

Example – 9: Write a Java program to change the common property of all objects (Static field).

```java
//Program to change the common property of all objects (static field).
// StaticMember.java
class Student{
    int rollno;  String name;
    static String college = "WRC";

    static void change(){
        college = "PCM"; }

    Student(int r, String n){
     rollno = r;  name = n; }

    void display (){
        System.out.println(rollno +" "+ name +" "+ college); }
}
class StaticMember{
    public static void main(String args[]){
        Student s1 = new Student (931,"Kiran");
        Student s2 = new Student (732,"Bimal");
        s1.display();   s2.display();
        Student.change();
        s1.display();   s2.display();

    }
}
```

```
C:\Users\balkr\Documents\Java>javac StaticMember.java

C:\Users\balkr\Documents\Java>java StaticMember
931 Kiran PCM
732 Bimal PCM
```

Java static variable

- The variables declared with the ***static*** keyword.
- The static variable can be used to refer the ***common property of all objects*** (that is not unique for each object) e.g., company name of employees, college name of students etc.
- The static variable gets ***memory only once in class area*** at the time of class loading.
- Advantage of static variable: It makes your program memory efficient (i.e., it saves memory).
- Static variables are, essentially, global variables. All instances of the class share the same static variable.
- Default values are same as instance variables. For numbers, the default value is 0; for Booleans, it is false; and for object references, it is null. Values can be assigned

during the declaration or within the constructor. Additionally, values can be assigned in special static initializer blocks.

Example – 10: Write a Java program to illustrate the concept of static variables.

```
// StaticVariables.java
class Student{
    int rollno;
    String name;
    static String college ="PCM";

    Student(int r,String n){
        rollno = r;  name = n;  }

    void display (){
        System.out.println(rollno + " "+name+" "+college);}
}
class StaticVariables{
    public static void main(String args[]){
        Student s1 = new Student(111,"Karan");
        Student s2 = new Student(222,"Aryan");

        s1.display();
        s2.display();
    }
}
```

Java Static Block

- We know that a ***block is nothing but the lines of code*** enclosed within curly braces. If we declare any block as static, then such blocks are called static blocks in Java.
- We use the static ***block when we need to initialize the static variables***. The static blocks load or execute at the time of loading a class. There can be multiple static blocks inside a Java program.
- And these ***multiple static blocks always execute in a sequential manner according to which they are written*** in the program.

Example – 11: Write a Java program to illustrate the concept of Static Blocks.

```
// StaticBlocks.java
class StaticBlocks {
    static int ID;   static String name;

    static { // Static Block one
        System.out.println("First Static Block:");
        ID = 1505;   name = "WRC";  }
    static { // Static Block two
        System.out.println("Second Static Block:");
        ID = 1210;  name = "PCM"; }
    static { // Static Block three
        System.out.println("Third Static Block:");
        ID = 1200;  name = "PN"; }

    public static void main(String[] args) {
        System.out.println("Static Number: " + ID);
        System.out.println("Static String: " + name);
    }
}
```

```
First Static Block:
Second Static Block:
Third Static Block:
Static Number: 1200
Static String: PN
```

Final Keyword in Java

In Java, the final keyword is used to denote constants. It can be used with variables, methods, and classes. Once any entity (variable, method or class) is declared final, it can be assigned only once. That is,

- The final variable cannot be reinitialized with another value;
- The final method cannot be overridden;
- The final class cannot be extended.

```
class Main {
  public static void main(String[] args) {

    // create a final variable
    final int AGE = 32;

    // try to change the final variable
    AGE = 45;
    System.out.println("Age: " + AGE);
  }
}
```

Example - 12: WAP java program to illustrate the concept of Final method.

```
class FinalDemo {
      // create a final method
      public final void display() {
        System.out.println("This is a final method.");
      }
}

class Main extends FinalDemo {
    // try to override final method
    public final void display() {
      System.out.println("The final method is overridden.");
    }

    public static void main(String[] args) {
      Main obj = new Main();
      obj.display();
    }
}
```

```
display() in Main cannot override display() in FinalDemo
 public final void display() {
                   ^
 overridden method is final
```

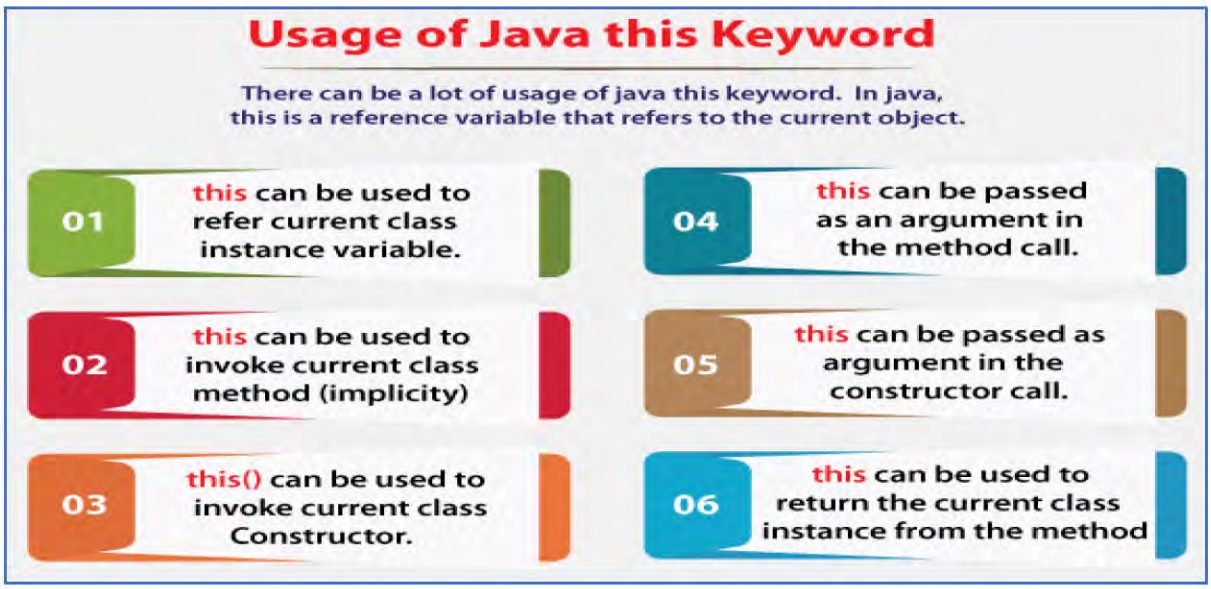
Example - 13: WAP java program to illustrate the concept of Final Class.

```
// create a final class
final class FinalClass {
  public void display() {
    System.out.println("This is a final method.");
  }
}

// try to extend the final class
class Main extends FinalClass {
  public  void display() {
    System.out.println("The final method is overridden.");
  }

  public static void main(String[] args) {
    Main obj = new Main();
    obj.display();
  }
}
```

```
cannot inherit from final FinalClass
class Main extends FinalClass {
                   ^
```

Java this Keyword

The *"this"* keyword refers to the current object in a method or constructor. The most common use of the *"this"* keyword is to eliminate the confusion between class attributes and parameters with the same name (because a class attribute is shadowed by a method or constructor parameter).



Example - 14: WAP java program to illustrate the concept of "this" keyword.

```
1   // thisExample.java
2   class thisExample {
3       String name;
4       void setName( String name ) {
5          this.name = name;}
6
7       String getName()  { return this.name;   }
8
9       public static void main( String[] args ) {
10          thisExample obj = new thisExample();
11          obj.setName("Core Java");
12          System.out.println("obj.name: "+obj.getName());
13      }
14  }
```

```
C:\Users\balkr\Documents\Java>javac thisExample.java

C:\Users\balkr\Documents\Java>java thisExample
obj.name: Core Java
```

Example - 15: WAP java program to illustrate the concept of "this" keyword.

```java
// Complex.java
class Complex {
    int a, b;
    Complex( int i, int j ){
        this.a = i;   this.b = j;  }
    Complex(int i){
        this(i, i); } // invokes the constructor with 2 parameters

    Complex(){// invokes the constructor with single parameter
        this(0); }
    @Override
    public String toString(){
        return this.a + " + " + this.b + "i";    }

    public static void main( String[] args ) {
        Complex c1 = new Complex(2, 3);
        Complex c2 = new Complex(3);
        Complex c3 = new Complex();

        System.out.println(c1);
        System.out.println(c2);
        System.out.println(c3);
    }
}
```

```
C:\Users\balkr\Documents\Java>javac Complex.java

C:\Users\balkr\Documents\Java>java Complex
2 + 3i
3 + 3i
0 + 0i
```

Java Method Overloading

- In Java, two or more methods may have the same name if they differ in parameters (different number of parameters, different types of parameters, or both). These methods are called overloaded methods and this feature is called method overloading.
- Overloaded methods may have the same or different return types, but they must differ in parameters.
- Method overloading increases the readability of the program.
- There are two ways to overload the method in java
    1. By changing number of arguments

2. By changing the data type

Example - 16: Write a Java program to calculate the sum of numbers using concept of method overloading.

```
1   //overloadMethod.java
2   public class overloadMethod {
3       int sum(int x, int y) {
4           return (x + y); }
5
6       int sum(int x, int y, int z){
7           return (x + y + z); }
8
9       double sum(double x, double y){
10          return (x + y);}
11
12      public static void main(String args[]){
13          overloadMethod  s = new overloadMethod ();
14          System.out.println(s.sum(100, 200));
15          System.out.println(s.sum(100, 200, 380));
16          System.out.println(s.sum(105.5, 260.5));
17      }
18  }
```

Java Inner Class

- An inner class is a class that is defined inside another class.
- ***Why would you want to do that? There are two reasons:***
  1. Inner classes can be hidden from other classes in the same package.
  2. Inner class methods can access the data from the scope in which they are defined—including the data that would otherwise be private.
- ***Advantage of Java inner classes***
  - ***It is a way of logically grouping classes that are only used in one place***: If a class is useful to only one other class, then it is logical to embed it in that class and keep the two together. Nesting such "helper classes" makes their package more streamlined.
  - ***It increases encapsulation:*** Consider two top-level classes, A and B, where B needs access to members of A that would otherwise be declared private. By hiding class B within class A, A's members can be declared private and B can access them. In addition, B itself can be hidden from the outside world.
  - ***It can lead to more readable and maintainable code:*** Nesting small classes within top-level classes places the code closer to where it is used.
  - Nested classes represent a particular type of relationship that is it ***can access all the members*** (data members and methods) of the outer class, ***including private.***
  - ***Code Optimization:*** It requires less code to write.

- ***Syntax of Inner class***

```
class OuterClass {
  ...
  class NestedClass {
    ...
  }
}
```

Example - 17: Write a Java program to illustrate the concept of Java inner class.

```
1  //innerClassExample.java
2  class outerClass {
3      int out;
4      void setDataOut(int x){
5          out=x; }
6      // nested inner class
7      class innerClass {
8          int in;
9          void setDataIn(int x){
10             in=x;  }
11         public void show(){
12             System.out.println("From Inner Class Method: ");
13             System.out.println("\tout= "+ out+ "\n\tin= "+ in);
14         }
15     } // end of inner class.
16 }// end of outer class
17 class innerClassExample {
18     public static void main(String[] args){
19         outerClass myOuter = new outerClass();
20         outerClass.innerClass myInner = myOuter.new innerClass();
21         myOuter.setDataOut(100);     myInner.setDataIn(200);  myInner.show();
22         System.out.print("out + in = ");
23         System.out.println( myOuter.out + myInner.in);
24     }
```

```
From Inner Class Method:
        out= 100
        in= 200
out + in = 300
```

Example - 18: Write a Java program to illustrate the concept of Java inner class from outer class method.

```java
//MethodLocalVariableDemo.java
class Outer {
void outerMethod() {
    final int x=1000;
    System.out.println("From outer Method......");
    class Inner {
        void innerMethod() {
            System.out.println("x = "+x);
        }
    }// end of inner class
    Inner in = new Inner();
    in.innerMethod();
}
}
class MethodLocalVariableDemo {
    public static void main(String[] args){
    Outer x = new Outer();
    x.outerMethod();
    }
}
```

```
From outer Method......
x = 1000
```

Chapter – 4
Inheritance, Interfaces and Packages

Introduction

Inheritance is one of the most important concepts of Object-Oriented Programming. Inheritance is the capability of one class to inherit capabilities or properties from another class in Java. For instance, we are humans. We inherit certain properties from the class 'Human' such as the ability to speak, breathe, eat, drink, etc.

Why Java Inheritance?

- The capability to express inheritance relationships ensures the closeness with the real-world models.
- Another reason is the idea of reusability. One can derive a new class (sub-class) from an existing class and add new features to it without modifying its parent class. There is no need to rewrite the parent class in order to inherit it.
- One reason is the transitive nature. If class A inherits properties from another class B, then all subclasses of A will automatically inherit properties from B. This property is called the transitive nature of inheritance.

Important Terms in Java Inheritance

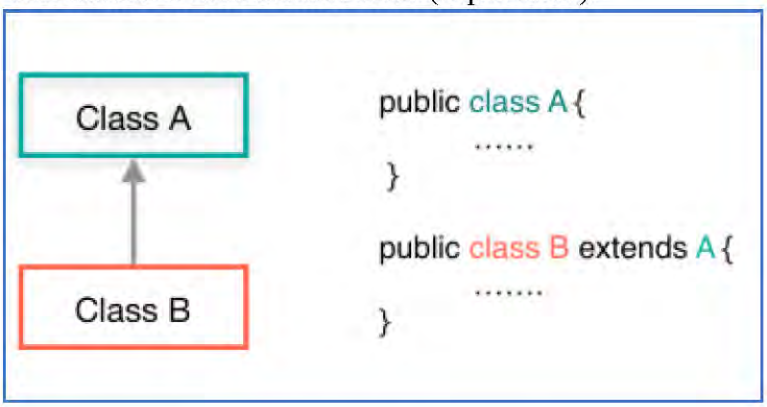
1. ***Class:*** Class is a user-defined datatype in Java that is basically a group of objects. It is a blueprint or template from which we create objects.
2. ***Super Class:*** The class whose features and functionalities are being inherited or used is known as the superclass or a base class or a parent class.
3. ***Sub Class:*** The class that inherits the properties and features from another class is known as a subclass or a derived class or extended class or child class. The subclass can add its own features and functions in addition to the fields and methods of its superclass or the parent class.
4. ***The extends keyword:*** The keyword extends is used by child class while inheriting the parent class.
5. ***The super keyword:*** The super keyword is similar to this keyword. The following are some cases where we use super keyword:
   - There are some situations where the members of the superclass and the subclass have the same names, then the super keyword is used to differentiate the members of the superclass from the members of the subclass.
   - To invoke the superclass constructor from the subclass.

Syntax of using Inheritance in Java

```
class BaseClass
{
   //methods and fields
}
class DerivedClass extends BaseClass
{
   //methods and fields
}
```

Types of Inheritance

Single Inheritance: A Single Inheritance is a basic inheritance, where one class (sub class) inherits the features of one another class (super class).

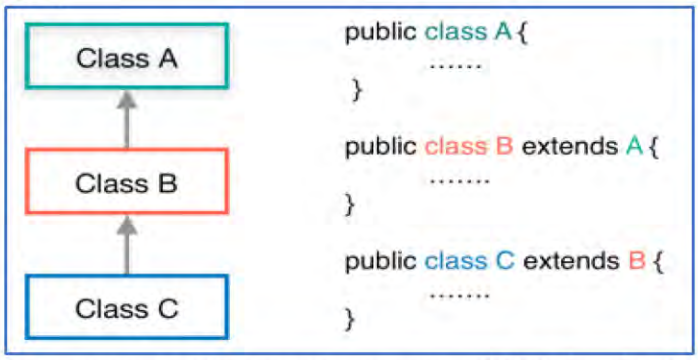


Example -1: Write a Java program to illustrate the concept of Simple inheritance. Use ***Vehicle*** as a base class and T***wo Wheeler*** as a derived class.

```
1  // SimpleInheritance.java
2  class Vehicle {
3      void run(){
4          System.out.println("It is Super Class: Vehicle");    }
5  }
6  class TwoWheeler extends Vehicle {
7      void wheel(){
8          System.out.println("It is Sub Class: Two Wheeler");
9      }
10 }
11
12 class SimpleInheritance{
13     public static void main(String args[]) {
14         TwoWheeler twoWheeler = new TwoWheeler();
15         twoWheeler.run();
16         twoWheeler.wheel();
17     }
18 }
```

Multilevel Inheritance: In Multilevel Inheritance, a derived class will be inheriting a base class and as well as the derived class also act as the base class to other class. In the below

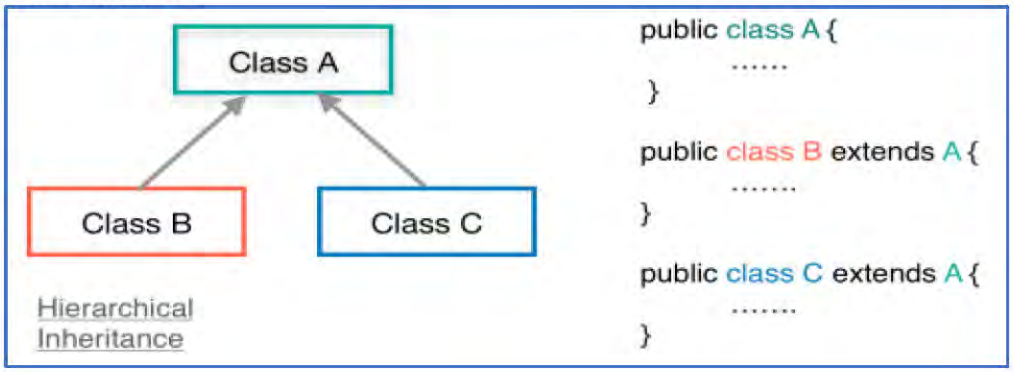
image, class A serves as a base class for the derived class B, which in turn serves as a base class for the derived class C.



Example -2: Write a Java program to illustrate the concept of Multilevel inheritance. Use ***Vehicle*** as a base class for ***TwoWheeler*** and ***TwoWheeler*** as base class for ***MotorBike.***

```
1   // MultilevelInheritance.java
2   class Vehicle {
3       void run(){
4           System.out.println("It is Super Class: Vehicle");    }
5   }
6   class TwoWheeler extends Vehicle {
7       void wheel(){
8           System.out.println("It is Sub Class: Two Wheeler");
9       }
10  }
11  class MotorBike extends TwoWheeler{
12      void fuel(){
13          System.out.println("Fuel Required : Petrol");
14      }
15  }
16  class MultilevelInheritance{
17      public static void main(String args[]) {
18          MotorBike motorBike = new MotorBike();
19          motorBike.run();
20          motorBike.wheel();
21          motorBike.fuel();
22      }
23  }
```

Hierarchical Inheritance: In Hierarchical Inheritance, one class serves as a superclass (base class) for more than one subclass. In the below image, class A serves as a base class for the derived class B, and C.

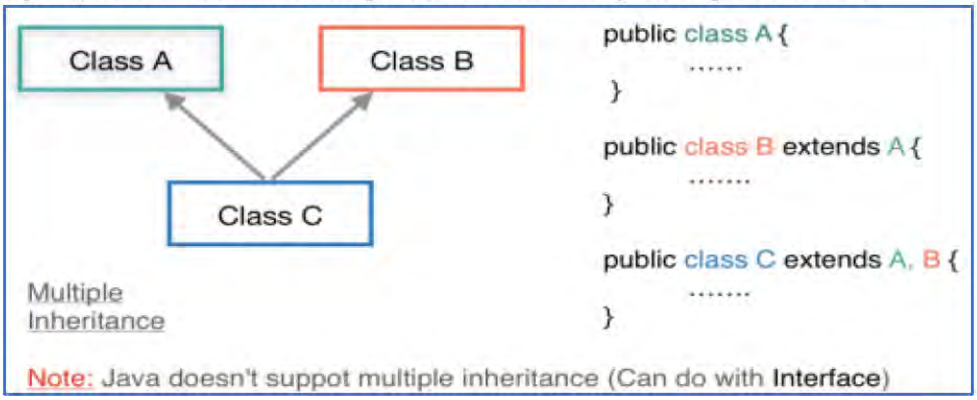


Example -3: Write a Java program to illustrate the concept of Hierarchical inheritance. Use ***Vehicle***, ***TwoWheeler*** and ***FourWheeler*** for hierarchical inheritance.

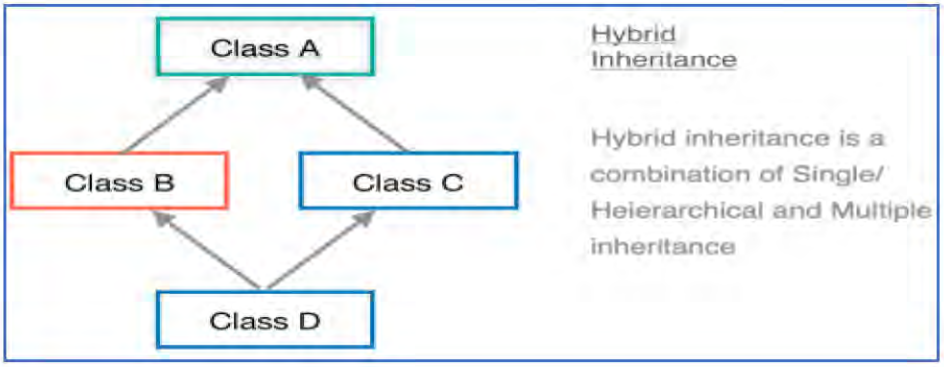
```
// HierarchicalInheritance.java
class Vehicle {
    void run(){
        System.out.println("It is Super Class: Vehicle");    }
}
class TwoWheeler extends Vehicle {
    void wheel(){
        System.out.println("It is Derived Class: Two Wheeler");
    }
}
class FourWheeler extends Vehicle{
    void wheel(){
        System.out.println("It is Derived Class:Four Wheeler");
    }
}
```

```
class HierarchicalInheritance{
    public static void main(String args[]) {
        FourWheeler fourWheeler = new FourWheeler();
        fourWheeler.run();
        fourWheeler.wheel();

        TwoWheeler twoWheeler = new TwoWheeler();
        twoWheeler.run();
        twoWheeler.wheel();
    }
}
```

Multiple Inheritance: In Multiple inheritances, one class can have more than one superclass and inherit features from all parent classes. ***Java does not support multiple inheritances*** with classes. In java, we can achieve multiple inheritances only through Interfaces.



Hybrid Inheritance: It is a mix of two or more of the above types of inheritance. Since java doesn't support multiple inheritances with classes, hybrid inheritance is also not possible with classes. In java, we can achieve hybrid inheritance only through Interfaces.



Java Method Overriding

If the same method is defined in both the superclass and the subclass, then the method of the subclass class overrides the method of the superclass. This is ***known as method overriding.***

In any object-oriented programming language, Overriding is a feature that allows a subclass or child class to provide a specific implementation of a method that is already provided by one of its super-classes or parent classes. When a method in a subclass has the same name, same parameters or signature, and same return type (or sub-type) as a method in its super-class, then the method in the subclass is said to override the method in the super-class.

Java Overriding Rules

- Both the superclass and the subclass must have the same method name, the same return type and the same parameter list.
- We cannot override the method declared as final and static.
- We should always override abstract methods of the superclass.

Example -4: Write a Java program to illustrate the concept of method overriding. Animal is base class and Dog is derived class. Use Display as a member function for both.

```
1  // MethodOverriding.java
2  class Animal {
3      public void Display() {
4          System.out.println("It is Base Class: Animal.");  }
5  }
6  class Dog extends Animal {
7      public void Display() {
8          super.Display();//called base class function
9          System.out.println("It is Derived Class: Dog.");
10     }
11 }
12
13 class MethodOverriding {
14     public static void main(String[] args) {
15         Dog d = new Dog();
16         d.Display();
17     }
18 }
```

Constructor in Inheritance

- In single level inheritance, the constructor of the base class is executed first.
- In multilevel inheritance, all the supper class constructors are executed when an instance of bottom most child class is created.

Example -4: Write a Java program to illustrate the concept of constructor in simple inheritance.

```
1  // ConstructorInheritamce.java
2  class Box {
3      double width;  double height;  double depth;
4      Box() { // use -1 to indicate an uninitialized box
5          width = -1;  height = -1; depth = -1;
6          System.out.println("I am Base default constructor."); }
7
8      Box(double w, double h, double d) {
9          width = w;  height = h;  depth = d; }
10
11     double volume() {
12         return width * height * depth; }
13 } // end of Box class
14 class BoxWeight extends Box {
15     double weight; // weight of box
16     BoxWeight() {
17         super();  weight = -1;
18         System.out.println("I am Derived default constructor.");}
19
20     BoxWeight(double w, double h, double d, double m) {
21         super(w, h, d); // call superclass constructor
22         weight = m;
23     }
24 } //end of BoxWeight class
```

```
25
26 class ConstructorInheritamce {
27     public static void main(String args[]) {
28         BoxWeight mybox1 = new BoxWeight(10, 20, 15, 34.3);
29         BoxWeight mybox3 = new BoxWeight(); // default
30
31         double vol;
32         vol = mybox1.volume();
33         System.out.println("Volume of mybox1 is " + vol);
34         System.out.println("Weight of mybox1 is " + mybox1.weight);
35
36         vol = mybox3.volume();
37         System.out.println("Volume of mybox3 is " + vol);
38         System.out.println("Weight of mybox3 is " + mybox3.weight);
39     }
40 }
```

```
I am Base default constructor.
I am Derived default constructor.
Volume of mybox1 is 3000.0
Weight of mybox1 is 34.3
Volume of mybox3 is -1.0
Weight of mybox3 is -1.0
```

Usage of Java super Keyword

1. super can be used to refer immediate parent class instance variable.
2. super can be used to invoke immediate parent class method.
3. super can be used to invoke immediate parent class constructor.

Dynamic Method Dispatch

- Dynamic method dispatch is the mechanism by which a call to an overridden method is resolved at run time, rather than compile time. Dynamic method dispatch is important because this is how Java implements run-time polymorphism.
- ***Principle:*** *a superclass reference variable can refer to a subclass object.*
- When an overridden method is called through a superclass reference, Java determines which version of that method to execute based upon the type of the object being referred to at the time the call occurs. Thus, this determination is made at run time.
- When different types of objects are referred to, different versions of an overridden method will be called. In other words, it is the type of the object being referred to (not the type of the reference variable) that determines which version of an overridden method will be executed.
- Therefore, if a superclass contains a method that is overridden by a subclass, then when different types of objects are referred to through a superclass reference variable, different versions of the method are executed.

Example -5: Write a Java program to illustrate the concept of Dynamic Dispatch.

```
1   // DynamicDispatch
2  class A {
3      void callme() {
4          System.out.println("Inside A's callme method");}
5  }
6  class B extends A {
7      // override callme()
8      void callme() {
9          System.out.println("Inside B's callme method");}
10 }
11 class C extends A {
12     // override callme()
13     void callme() {
14         System.out.println("Inside C's callme method"); }
15     }
```

```
16  class DynamicDispatch {
17      public static void main(String args[]) {
18          A a = new A(); // object of type A
19          B b = new B(); // object of type B
20          C c = new C(); // object of type C
21          A r; // obtain a reference of type A
22          r = a; // r refers to an A object
23          r.callme(); // calls A's version of callme
24          r = b; // r refers to a B object
25          r.callme(); // calls B's version of callme
26          r = c; // r refers to a C object
27          r.callme(); // calls C's version of callme
28      }
29  }
```

Java Abstract Class

An Abstract class is a class that represents a concept and whose objects can't be created. A class that contains at least one abstract method (method without any implementation or method body) is called an abstract class. It can have ***abstract and non-abstract methods*** (method with body). Using an abstract class, we can achieve ***0 to 100% abstraction***.
There is always a default constructor in an abstract class, it can also have a parameterized constructor.
Abstract class in Java: A class that is declared as abstract is known as abstract class. It needs to be extended and its method implemented. It cannot be instantiated.
Abstract method: A method that is declared as abstract and does not have implementation is known as abstract method.

Abstract Methods in Java

- Abstract methods are methods with no implementation. They do not contain any method statement.
- The child classes of this abstract class must provide the implementation of these inherited abstract methods.
- An abstract method is declared with an abstract keyword.
- The declaration of an abstract method must end with a semicolon.

Declaring a Java Abstract Class: To declare an abstract class in Java we use the ***keyword abstract***. ***The syntax is given below:***

```
abstract class ClassName {
  //class body
}
```

Example -6: Write a Java program to illustrate the concept of Abstract Class. Use "Bank" as an abstract class and SBI and PNB as a Derived class.

```
1  //AbstractBank.java
2  abstract class Bank{
3      abstract int getRateOfInterest();
4  }
5  class SBI extends Bank{
6          int getRateOfInterest(){return 7; }
7  }
8  class PNB extends Bank{
9          int getRateOfInterest(){return 8;}
10 }
11
12 class AbstractBank{
13     public static void main(String args[]){
14         Bank b=new SBI();
15         int interest=b.getRateOfInterest();
16         System.out.println("Rate of Interest is: "+interest+" %");
17
18         Bank d=new PNB();
19         interest=d.getRateOfInterest();
20         System.out.println("Rate of Interest is: "+interest+" %");
21     }
22 }
```

Example -7: Write a Java program to illustrate the concept of Abstract Class. Use "Figure" as an abstract class and Rectangle and Triangle as a Derived classes. Use area () as an abstract method.

```
1  //AbstractAreas.java
2  abstract class Figure {
3      double dim1; double dim2;
4      Figure(double a, double b) {
5          dim1 = a; dim2 = b; }
6      // area is now an abstract method
7      abstract double area();
8  }// end of class Figure
9
10 class Rectangle extends Figure {
11     Rectangle(double a, double b) {
12         super(a, b); }
13     // override area for rectangle
14     double area() {
15         System.out.println("From the Rectangle class. ");
16         return dim1 * dim2; }
17 }// end of class Rectangle
```

```
18
19  class Triangle extends Figure {
20      Triangle(double a, double b) {
21          super(a, b);}
22      // override area for right triangle
23      double area() {
24          System.out.println("From the Triangle class. ");
25          return dim1 * dim2 / 2;}
26  }// end of class Triangle
27
28  class AbstractAreas {
29      public static void main(String args[]) {
30          // Figure f = new Figure(10, 10); // illegal now
31          Rectangle r = new Rectangle(9, 5);
32          Triangle t = new Triangle(10, 8);
33
34          Figure fig; // this is OK, no object is created
35          fig = r;// run time polumorphism
36          System.out.println("Area is " + fig.area());
37          fig = t;// run time polumorphism
38          System.out.println("Area is " + fig.area());
39      }
40  }
```

Example -8: Write a Java program to illustrate the concept of Abstract Class. Use "Bike" as an abstract class and "Honda" as a Derived class. Use both abstract and non-abstract method.

```
1   //AbstractBike.java
2   abstract class Bike{
3       Bike(){
4           System.out.println("Bike is created");
5       }
6       abstract void run(); // Abstract class
7       void changeGear(){
8           System.out.println("Gear changed");
9        }
10  }
11
12  class Honda extends Bike{
13      void run(){
14          System.out.println("Running safely..");
15      }
16  }
17  class AbstractBike{
18      public static void main(String args[]){
19          Bike obj = new Honda();
20          obj.run();
21          obj.changeGear();
22      }
23   }
```

Example -9: Write a Java program to illustrate the concept of Abstract Class. Use "BankAccount" as an abstract class and "SavingAccount", "CurrentAccount", and "FixedDepositAccount" as a derived classis. Use deposit () and withdraw () as an abstract method.

```
//AbstractBankDemo.java
abstract class BankAccount {
    //abstract methods
    public abstract void deposit();
    public abstract void withdraw();
}

class SavingsAccount extends BankAccount{
    public void deposit()  {
        System.out.println("This is the concrete deposit method of SavingsAccount"); }
    public void withdraw()  {
        System.out.println("This is the concrete withdraw method of SavingsAccount\n"); }
}

class CurrentAccount extends BankAccount{
    public void deposit()  {
        System.out.println("This is the concrete deposit method of CurrentAccount");   }
    public void withdraw()   {
        System.out.println("This is the concrete withdraw method of CurrentAccount\n");   }
}
```

```
//FixedDepositAccount class extends BankAccount class
class FixedDepositAccount extends BankAccount {
    public void deposit()   {
        System.out.println("This is the concrete deposit method of FixedDepositAccount");  }
    public void withdraw()  {
        System.out.println("This is the concrete withdraw method of FixedDepositAccount");  }
}
public class AbstractBankDemo {
    public static void main(String args[])  {
        BankAccount obj = new SavingsAccount();
        obj.deposit();
        obj.withdraw();

        BankAccount obj1 = new CurrentAccount();
        obj1.deposit();
        obj1.withdraw();

        BankAccount obj2 = new FixedDepositAccount();
        obj2.deposit();
        obj2.withdraw();
    }
}
```

Interface in Java

An interface is another building block of Java which is a blueprint or template of a class. It has static constants and abstract methods only. The interface in java is a mechanism to achieve fully abstraction. It cannot be instantiated just like abstract class. There can be only abstract methods in the java interface not method body. It is used to achieve fully abstraction and multiple inheritance in Java.

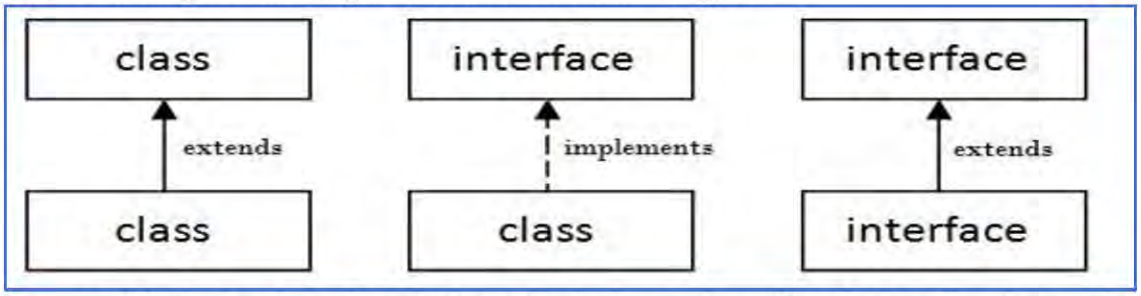
The class that implements the interface should be declared as abstract, otherwise, all the methods of the interface need to be defined in the class. Java Interface also represents IS-A relationship.

The java compiler adds public and abstract keywords before the interface method and public, static and final keywords before data members. In other words, Interface fields are public, static and final by default, and methods are public and abstract.

Why use Java interface?

- It allows us to achieve a complete abstraction.
- Interfaces are mainly designed to support dynamic method resolution at run time.
- Interfaces allow us to achieve loose coupling.
- It also helps us to separate the definition of a method from the inheritance hierarchy.

Understanding relationship between classes and interfaces



Syntax of declaring Interfaces in Java: To declare an interface, the interface keyword is used. ***Here is a syntax to declare an interface:***

```
interface interface-name {
  //abstract methods
}
```

Example -10: Write a Java program to illustrate the concept of Interface. Use "Printable" and "Showable" as an Interfaces and make class to implement the abstract methods of interfaces.

```
1  //InterfaceExample.java
2  interface Printable{
3          void print();
4  }
5  interface Showable extends Printable{
6          void show();
7  }
8  class InterfaceExample implements Showable{
9      public void print(){
10         System.out.println("Interface Printable example.");}
11     public void show(){
12         System.out.println("Interface Showable example.");  }
13
14     public static void main(String args[]){
15         InterfaceExample obj = new InterfaceExample();
16         obj.print();
17         obj.show();
18     }
19 }
```

Example -11: Write a Java program to illustrate the concept of Interface. Use "Polygon" as a Interface and make a class to implement the abstract methods of interface.

```java
// InterfaceDemo.java
interface Polygon {
    public static final int length = 4,breadth = 8;
    //declaring interface methods(without a method body)
    public void getName();
    public void getNumberOfSides();
    public void getArea();
    public void getPerimeter();
}
// Rectangle class "implements" the Polygon interface
class Rectangle implements Polygon {
    public void getName()   {
        System.out.println("The name of the Polygon is: Rectangle"); }
    public void getNumberOfSides(){
        System.out.println("There are 4 sides in a Rectangle"); }
    public void getArea(){
        System.out.println("The Area of Rectangle is: " +length*breadth); }
    public void getPerimeter(){
        System.out.println("The Perimeter of Rectangle is: " +2*(length + breadth)); }
}
```

```java
class InterfaceDemo{
    public static void main(String[] args){
        Rectangle rectangle = new Rectangle(); // Create a Rectangle object
        //calling methods of class Rectangle
        rectangle.getName();
        rectangle.getNumberOfSides();
        rectangle.getArea();
        rectangle.getPerimeter();
    }
}
```

Difference between abstract class and interface

Abstract class	Interface
1) Abstract class can **have abstract and non-abstract** methods.	Interface can have **only abstract** methods.
2) Abstract class **doesn't support multiple inheritance**.	Interface **supports multiple inheritance**.
3) Abstract class **can have final, non-final, static and non-static variables**.	Interface has **only static and final variables**.
4) Abstract class **can have static methods, main method and constructor**.	Interface **can't have static methods, main method or constructor**.
5) Abstract class **can provide the implementation of interface**.	Interface **can't provide the implementation of abstract class**.
6) The **abstract keyword** is used to declare abstract class.	The **interface keyword** is used to declare interface.
7) **Example:** public abstract class Shape { public abstract void draw ();}	**Example:** public interface Drawable { void draw ();}

Java Packages

In Java, Packages are similar to folders, which are mainly used to organize classes and interfaces. A package is a collection of similar types of Java entities such as classes, interfaces, subclasses, exceptions, errors, and enums. A package can also contain sub-packages.

There are several advantages of using Java Packages, some of them, are as follows –

- Make easy searching or locating of classes and interfaces.
- Avoid naming conflicts. For example, there can be two classes with the name Student in two packages, *university.csdept.Student* and college.itdept.Student
- Implement data encapsulation (or data-hiding).
- Provide controlled access: The access specifiers protected and default have access control on package level. A member declared as protected is accessible by classes within the same package and its subclasses. A member without any access specifier that is default specifier is accessible only by classes in the same package.
- Reuse the classes contained in the packages of other programs.
- Uniquely compare the classes in other packages.

Note:

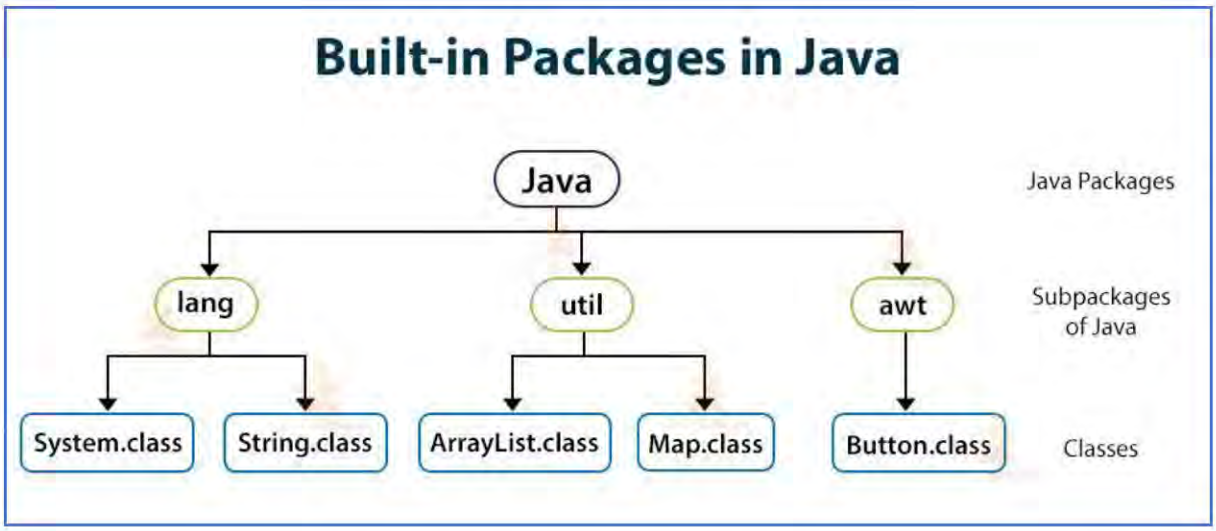
- *Package names are dot-separated, e.g.,* ***java.lang.String***
- *Packages avoid namespace collision: A package cannot contain two classes with the same names, but two different packages can have a class with the same name.*
- *The exact name of the class is identified by the structure of its package.*

Types of Packages in Java: They can be divided into two categories:

1. Java API packages or built-in packages and
2. User-defined packages.

Java API packages or built-in packages: Java provides a large number of classes grouped into different packages based on a particular functionality. Examples:

- ***java.lang:*** It contains classes for primitive types, strings, math functions, threads, and exceptions.
- ***java.util:*** It contains classes such as vectors, hash tables, dates, Calendars, etc.
- ***java.io:*** It has stream classes for Input/Output.
- ***java.awt:*** Classes for implementing Graphical User Interface – windows, buttons, menus, etc.
- ***java.net:*** Classes for networking
- ***java. Applet:*** Classes for creating and implementing applets



User-defined packages: As the name suggests, these packages are defined by the user. We create a directory whose name should be the same as the name of the package. Then we create a class inside the directory.

Package Naming Conventions: Packages names follow the reverse order of domain names. For example, in a university, the recommended convention is ***university.engineering.mech*** or ***university.tech.it*** or ***university.arts.history*** etc.

In the following package: "***university.tech.it***"

- ***university*** is a top-level package
- ***tech*** is a sub-package
- and ***it*** is a class which is present in the sub-package ***tech***.

Compiling and Executing a Java Package

```
1  // PackageEx.java
2  package university.tech; //package
3  class PackageEx{
4      public static void main(String args[])      {
5          System.out.println("Welcome to package Tutorial of Java");
6      }
7  }
```

```
C:\Users\balkr\Documents\Java>javac -d C:\Users\balkr\Documents\Java PackageEx.java

C:\Users\balkr\Documents\Java>java university.tech.PackageEx
Welcome to package Tutorial of Java
```

Accessing Packages or Classes from Another Package

If we want to access all the classes and interfaces of an existing package then we use the import statement. We can do it in three different ways:

1. ***For importing all the classes***

import packageName.;*

By using * after the import statement, we can access all the classes of the package but not the sub-packages.

2. ***For importing a particular class***

import packageName.className;

3. ***Fully qualified name:*** Using a Fully qualified name means we can access the declared class of different packages without using the import statement. But you need to use a fully qualified name every time when you are accessing the class or interface which is present in a different package. This type of technique is generally used when two packages have the same class name example ***class Date*** is present in both the packages ***java.util*** and ***java.sql.***

Wrapper Class, Autoboxing and Unboxing

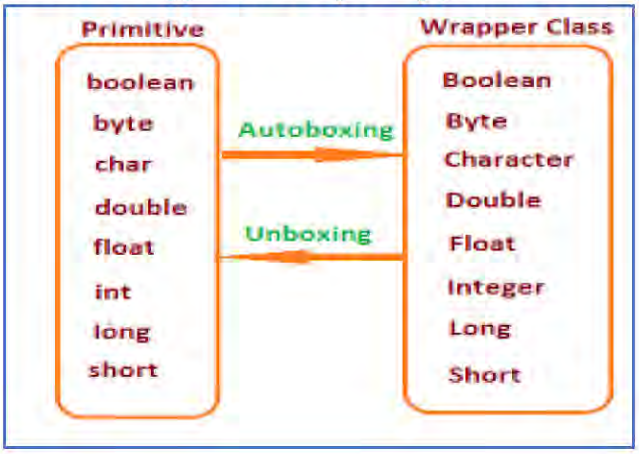
Java Wrapper Class

- Java uses primitive types such as int or double, to hold the basic data types supported by the language. Primitive types, rather than objects, are used for these quantities for the sake of performance. Using objects for these values would add an unacceptable overhead to even the simplest of calculations. Thus, ***the primitive types are not part of the object hierarchy***, and they do not inherit Object.
- Despite the performance benefit offered by the primitive types, there are times when you will need an object representation. For example, ***you can't pass a primitive type by reference to a method.*** Also, many of the ***standard data structures implemented by Java operate on objects***, which means that you can't use these data structures to store primitive types. ***To handle these (and other) situations***, Java *provides type wrappers*, which are classes that *encapsulate a primitive type within an object.*
- Wrapper class provides a mechanism to convert primitive data types into wrapper class objects.

- **Need of Java Wrapper Class**
  - They convert the primitive data types into objects.
  - Objects are needed to modify the arguments in a method.
  - The classes in ***java.util*** package only works with objects.
  - Data structures in the collection framework only store objects.
  - Objects helps in synchronization in multithreading.

- Following are the methods to get the associated value from the wrapper objects.
  - intValue()
  - byteValue()
  - shortValue()
  - longValue()
  - doubleValue()
  - charValue()
  - floatValue()
  - booleanValue()
-

- Following are the ***equivalent wrapper class objects of primitive data types.***



Autoboxing and Unboxing

- In autoboxing, the Java compiler automatically converts primitive types into their corresponding wrapper class objects. For example, converting int to Integer class.
- The Java compiler ***applies autoboxing when*** a primitive value is:
  - Passed as a parameter to a method that ***expects an object*** of the corresponding wrapper class.
  - Assigned to a variable of the corresponding ***wrapper class.***
- In unboxing, the Java compiler automatically converts wrapper class objects into their corresponding primitive types. For example, conversion of Integer to int.
- The Java compiler applies to unbox when an object of a wrapper class is:
  - Passed as a parameter to a method that ***expects a value*** of the corresponding primitive type.
  - Assigned to a variable of the corresponding ***primitive type.***
- **Some benefits of autoboxing:**
  - Less code to write
  - The code looks cleaner and easily readable
  - Programmers do not have to perform Explicit typecasting
  - The best method for conversion is automatically chosen
  - It helps prevent errors but may lead to unexpected results sometimes.

```java
//boxingExample.java
class boxingExample{
    public static void main(String args[]){
        byte b = 100; //Primitive byte data
        Byte B = b; //Auto-Boxing of byte data
        System.out.println("Auto-Boxing of byte data: "+B);
        int i = 200;     //Primitive int Data
        Integer I = i;  //Auto-Boxing of int data
        System.out.println("Auto-Boxing of int data: "+I);
        long l = 250;   Long L = l;
        System.out.println("Auto-Boxing of long data: "+L);
        float f = 120L; Float F = f;
        System.out.println("Auto-Boxing of float data: "+F);
        double d = 18.58;   Double D = d;
        System.out.println("Auto-Boxing of double data: "+D);
        boolean bln = false;    Boolean BLN = bln;
        System.out.println("Auto-Boxing of boolean data: "+BLN);
        char c = 'C';   Character C = c;
        System.out.println("Auto-Boxing of Char data: "+C);

        Integer I1 = 15;  //Integer Object
        int i1 = I1;  //Auto-Unboxing of Integer Object
        System.out.println("Auto-Unboxing of Interger data: "+i1);
        Double D1 = 20.5; //Double Object
        double d1 = D1;  //Auto-Unboxing of Double Object
        System.out.println("Auto-Unboxing of Double data: "+d1);
        Character C1 = 'C';     //Character Object
        char c1 = C1; //Auto-Unboxing of Character Object
        System.out.println("Auto-Unboxing of Character data: "+c1);
    }// main function end
}// end of class
```

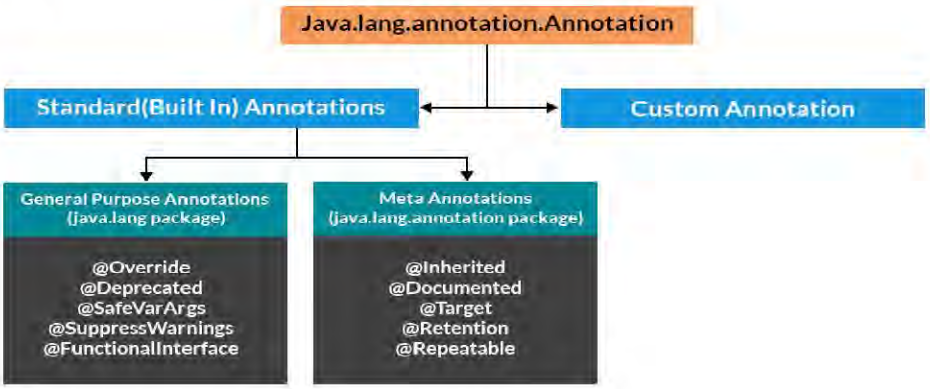
```
C:\Users\balkr\Documents\Java>java boxingExample
Auto-Boxing of byte data: 100
Auto-Boxing of int data: 200
Auto-Boxing of long data: 250
Auto-Boxing of float data: 120.0
Auto-Boxing of double data: 18.58
Auto-Boxing of boolean data: false
Auto-Boxing of Char data: C
Auto-Unboxing of Interger data: 15
Auto-Unboxing of Double data: 20.5
Auto-Unboxing of Character data: C
```

Annotations in Java

- Java Annotation is a tag that represents the metadata i.e., attached with class, interface, methods or fields to indicate some additional information which can be used by java compiler and JVM.
- Annotations are used to represent the syntactic metadata related to the class, interface, methods or fields used in the source code and some additional information which is used by java interpreter and the JVM.
- Annotations, a form of metadata, provide data about a program that is not part of the program itself. Annotations have no direct effect on the operation of the code they annotate.
- The built-in annotations such as @Override, @Deprecated, and @SuppressWarnings provide the interpreter, about the information related to the execution of code. For instance, @Override is used to instruct the interpreter that the annotated method is being overridden.
- Annotations have a number of uses, among them:
  - ***Information for the compiler***: Annotations can be used by the compiler to detect errors or suppress warnings.
  - ***Compile-time and deployment-time processing:*** Software tools can process annotation information to generate code, XML files, and so forth.
  - ***Runtime processing:*** Some annotations are available to be examined at runtime.

- Annotations are used to provide supplemental information about a program.
  - Annotations start with '@'.
  - Annotations do not change the action of a compiled program.
  - Annotations help to associate metadata (information) to the program elements i.e., instance variables, constructors, methods, classes, etc.
  - Annotations are not pure comments as they can change the way a program is treated by the compiler.
  - Annotations basically are used to provide additional information, so could be an alternative to XML and Java marker interfaces.

- **Hierarchy of Annotations in Java**



@Inherited

@Inherited is a marker annotation that can be used only on another annotation declaration. Furthermore, it affects only annotations that will be used on class declarations. **@Inherited** causes the annotation for a superclass to be inherited by a subclass. Therefore, when a request for a specific annotation is made to the subclass, if that annotation is not present in the subclass, then its superclass is checked. If that annotation is present in the superclass, and if it is annotated with **@Inherited**, then that annotation will be returned.

@Override

@Override is a marker annotation that can be used only on methods. A method annotated with **@Override** must override a method from a superclass. If it doesn't, a compile-time error will result. It is used to ensure that a superclass method is actually overridden, and not simply overloaded.

@Deprecated

@Deprecated indicates that a declaration is obsolete and not recommended for use. Beginning with JDK 9, **@Deprecated** also allows you to specify the Java version in which the deprecation occurred and whether the deprecated element is slated for removal.

@SuppressWarnings

@SuppressWarnings specifies that one or more warnings that might be issued by the compiler are to be suppressed. The warnings to suppress are specified by name, in string form.

```
class PersonInfo {
    int age = 25;
    String name = "Joe";
    public void show() {
        System.out.println("Employee Name: " + name);
        System.out.println("Employee Age: " + age);
    }
}
public class AnnotationsExample extends PersonInfo {
    int id = 12;
    @Override
    public void show() {
        System.out.println("Employee id: " + id);
    }
    public static void main(String[] args) {
        AnnotationsExample obj = new AnnotationsExample();
        obj.show();
    }
```

Parent Class

Telling the compiler that subclass is overriding the show method of superclass

Child Class Extending Parent Class

Calling the show method using object of child class

annotationsexample.AnnotationsExample

Output - AnnotationsExample (run)

```
run:
Employee id: 12
BUILD SUCCESSFUL (total time: 0 seconds)
```

Output

```java
//annotationExample.java
class Base{
    void print(){
        System.out.println("I am Base Print method.");
    }
}// end of base class
class Derived extends Base{
    @Override
    void print(){
        System.out.println("I am Derived Print method.");
    }
}// end of derived class
class annotationExample{
    public static void main(String args[]){
        Derived obj=new Derived();
        obj.print();
    }// end of main function
}// end of main class
```

```
C:\Users\balkr\Documents\Java>java annotationExample
I am Derived Print method.
```

Chapter – 5
Exception Handling and Multithreading

Introduction

A Java exception is an object that describes an exceptional (that is, error) condition that has occurred in a piece of code. When an exceptional condition arises, an object representing that exception is created and thrown in the method that caused the error. That method may choose to handle the exception itself, or pass it on. Either way, at some point, the exception is caught and processed.

Exceptions can be generated by the Java run-time system, or they can be manually generated by your code. Exceptions thrown by Java relate to fundamental errors that violate the rules of the Java language or the constraints of the Java execution environment. Manually generated exceptions are typically used to report some error condition to the caller of a method.

Examples of Java Exception: Some common examples of Exceptions in Java are:

- Divide by zero errors
- Trying to access the array elements with an invalid index
- Invalid input data by the user
- Hard disk crash
- Opening a file that does not exist
- Heap memory exhausted
- Network connection loss in the middle of a communication
- JVM has run out of memory.

Java exception handling is managed via five keywords: ***try, catch, throw, throws, and finally.***

This is the general form of an exception-handling block:

```
try {
    // block of code to monitor for errors
}

catch (ExceptionType1 exOb) {
    // exception handler for ExceptionType1
}

catch (ExceptionType2 exOb) {
    // exception handler for ExceptionType2
}
// ...
finally {
    // block of code to be executed after try block ends
}
```

- Program statements that you want to monitor for exceptions are contained within a try block.
- If an exception occurs within the try block, it is thrown. Your code can catch this exception (using catch) and handle it in some rational manner.
- System-generated exceptions are automatically thrown by the Java runtime system.
- To manually throw an exception, ***use the keyword throw***.
- Any exception that is thrown out of a method must be specified as such by a ***throws*** clause.
- Any code that ***absolutely must be executed*** after a try block completes ***is put in a finally block.***

Types of Exception in Java

1. ***Checked Exceptions in java***
   - A checked exception is a ***compile-time exception***, that is, a Java compiler checks or notifies during the compilation-time. ***They occur during the compile-time.***
   - The compiler checks the checked exceptions during compilation to check whether the programmer has written the code to handle them or not. The ***programmer cannot simply ignore these exceptions*** and should take care to handle these exceptions.
   - If the programmer does not write the code to handle them then there will be a compilation error. A method that throws a checked exception needs to either specify or handle it.
   - A ***checked exception extends the Exception class***. Some checked Exceptions are SQLException, IOException, ClassNotFoundException, InvocationTargetException, etc.
   - For example, if we write a program to read data from a file using a FileReader class and ***if the file does not exist, then there is a FileNotFoundException***.

2. *Unchecked Exceptions in Java*

- An exception that occurs ***during the execution of a program is called an unchecked or a runtime exception***. Unlike the checked exceptions, the compiler generally ignores these exceptions during compilation rather, they are checked during the runtime.
- Therefore, the compiler does not check whether the programmer has written the code to handle them or not but it is the responsibility of the programmer to handle the unchecked exceptions and provide a safe exit.
- For example, ***if a program attempts to divide a number by zero***. Or, when there is an illegal arithmetic operation, this impossible event generates a runtime exception.
- Suppose, we declare an array of size 10 in a program, and try to access the 12th element of the array, or with a negative index like -5, then we get an ArrayIndexOutOfBounds exception.
- Some unchecked exceptions are ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, NumberFormatException, InputMismatchException, IllegalStateException, etc.

```java
// UnCheckedExceptions.java
public class UnCheckedExceptions{
  public static void main(String args[])  {
        try{
            int array[] = {1, 2, 3, 4, 5};
            System.out.println(array[7]);} //ArrayIndexOutOfBoundsException
        catch (ArrayIndexOutOfBoundsException e) {
            System.out.println(e); }
        try{
            int number1 = 10;  int number2 = 0;
            int result = number1/number2; //Arithmetic Exception
            System.out.println("Result of 10/0 is: " + result);}
        catch (ArithmeticException e){
            System.out.println(e); }

        try{
            String myString = null; // null value
            System.out.println(myString.charAt(0));} //NullPointerException
        catch (NullPointerException e){
            System.out.println(e); }

        try{
            int number = Integer.parseInt ("Number") ;
            System.out.println(number);} //NumberFormatException
        catch (NumberFormatException e) {
            System.out.println(e); }
    }
}
```

```
C:\Users\balkr\Documents\Java>javac UnCheckedExceptions.java

C:\Users\balkr\Documents\Java>java UnCheckedExceptions
java.lang.ArrayIndexOutOfBoundsException: Index 7 out of bounds for length 5
java.lang.ArithmeticException: / by zero
java.lang.NullPointerException: Cannot invoke "String.charAt(int)" because "<local1>" is null
java.lang.NumberFormatException: For input string: "Number"
```

3. *Errors in java*
   - An Error is not an exception but we consider it as a type of exception.
   - An error is a severe problem or situation that is impossible for a user or a developer to recover. The cause of Errors in Java is when a user overlooks a code or rarely takes care of a mistake. For instance, if a queue is full and overflow occurs, a blunder will emerge.
   - Errors represent irrecoverable conditions such as Java virtual machine (JVM) running out of memory, memory leaks, stack overflow errors, library incompatibility, infinite recursion, etc.
   - Errors are usually beyond the control of the programmer and we should not try to handle errors.

Checked vs Unchecked exceptions

Exceptions that are checked and handled at compile time are checked exceptions.	Exceptions that are not checked and handled at compile time are unchecked exceptions.
They are direct subclasses of Exception but do not inherit from RuntimeException.	They are a direct subclass of RuntimeException class.
The program gives a compilation error if a method throws a checked exception and the compiler is not able to handle the exception on its own.	The program compiles fine because the exceptions escape the notice of compiler. Exceptions occur due to errors in programming logic.
A Checked Exception occurs when the chances of failure are too high.	Unchecked Exception occurs mostly due to programming mistakes.
Common checked exceptions include IOException, DataAccessException, IllegalAccessException, InterruptedException, etc.	Common unchecked exceptions include ArithmeticException, InvalidClassException, NullPointerException, etc.

Java's Built-in Exceptions

Inside the standard package java.lang, Java defines several exception classes.The most general of these exceptions are subclasses of the standard type RuntimeException. In the language of Java, these are called unchecked exceptions because the compiler does not check to see if a method handles or throws these exceptions.

Exception	Meaning
ArithmeticException	Arithmetic error, such as divide-by-zero.
ArrayIndexOutOfBoundsException	Array index is out-of-bounds.
ArrayStoreException	Assignment to an array element of an incompatible type.
ClassCastException	Invalid cast.
EnumConstantNotPresentException	An attempt is made to use an undefined enumeration value.
IllegalArgumentException	Illegal argument used to invoke a method.
IllegalMonitorStateException	Illegal monitor operation, such as waiting on an unlocked thread.
IllegalStateException	Environment or application is in incorrect state.
IllegalThreadStateException	Requested operation not compatible with current thread state.
IndexOutOfBoundsException	Some type of index is out-of-bounds.
NegativeArraySizeException	Array created with a negative size.
NullPointerException	Invalid use of a null reference.
NumberFormatException	Invalid conversion of a string to a numeric format.
SecurityException	Attempt to violate security.
StringIndexOutOfBounds	Attempt to index outside the bounds of a string.
TypeNotPresentException	Type not found.
UnsupportedOperationException	An unsupported operation was encountered.

Table 10-1 Java's Unchecked **RuntimeException** Subclasses Defined in **java.lang**

Exception	Meaning
ClassNotFoundException	Class not found.
CloneNotSupportedException	Attempt to clone an object that does not implement the **Cloneable** interface.
IllegalAccessException	Access to a class is denied.
InstantiationException	Attempt to create an object of an abstract class or interface.
InterruptedException	One thread has been interrupted by another thread.
NoSuchFieldException	A requested field does not exist.
NoSuchMethodException	A requested method does not exist.
ReflectiveOperationException	Superclass of reflection-related exceptions.

Table 10-2 Java's Checked Exceptions Defined in **java.lang**

Exception Examples

```java
//Exception.java
class Exception {
  public static void main(String[] args) {

    try {
      int divideByZero = 5 / 0;
      System.out.println("Rest of code in try block");
    }

    catch (ArithmeticException e) {
      System.out.println("ArithmeticException => " + e.getMessage());
    }
  }
}
```

```
C:\Users\balkr\Documents\Java>javac Exception.java

C:\Users\balkr\Documents\Java>java Exception
ArithmeticException => / by zero
```

Multiple catch Clauses

```java
// MultipleCatches.java
class MultipleCatches {
    public static void main(String args[]) {
        try {
            int a = args.length;
            System.out.println("a = " + a);
            int b = 42 / a;
            int c[] = { 1 };
            c[42] = 99;
        } catch(ArithmeticException e) {
                System.out.println("Divide by 0: " + e);
            }
          catch(ArrayIndexOutOfBoundsException e) {
                System.out.println("Array index oob: " + e);
            }
        System.out.println("After try/catch blocks.");
    }
}
```

```
C:\Users\balkr\Documents\Java>javac MultipleCatches.java

C:\Users\balkr\Documents\Java>java MultipleCatches
a = 0
Divide by 0: java.lang.ArithmeticException: / by zero
After try/catch blocks.

C:\Users\balkr\Documents\Java>javac MultipleCatches.java

C:\Users\balkr\Documents\Java>java MultipleCatches trycatch
a = 1
Array index oob: java.lang.ArrayIndexOutOfBoundsException: Index 42 out of bounds for length 1
After try/catch blocks.
```

```java
//MultipleCatch2.java
import java.util.*;
public class MultipleCatch2 {
  public static void main(String[] args) {
        try {
                Scanner in=new Scanner(System.in);
                System.out.print("Enter b: ");
                int b=in.nextInt();
                int[] array = new int[5];
                array[5] = 30 /b; // if b=0, divide by zero exception.
                System.out.println(array[8]);
            }
        catch(ArithmeticException e) {
                System.out.println("Arithmetic Exception");
            }
        catch(ArrayIndexOutOfBoundsException e) {
                System.out.println("ArrayIndexOutOfBounds");
            }
        catch(java.lang.Exception e) {
                System.out.println("Parent Exception");
            }
        System.out.println("Rest code");
    }
}
```

```
C:\Users\balkr\Documents\Java>javac MultipleCatch2.java

C:\Users\balkr\Documents\Java>java MultipleCatch2
Enter b: 23
ArrayIndexOutOfBounds
Rest code

C:\Users\balkr\Documents\Java>java MultipleCatch2
Enter b: 0
Arithmetic Exception
Rest code
```

Catching Multiple Exceptions

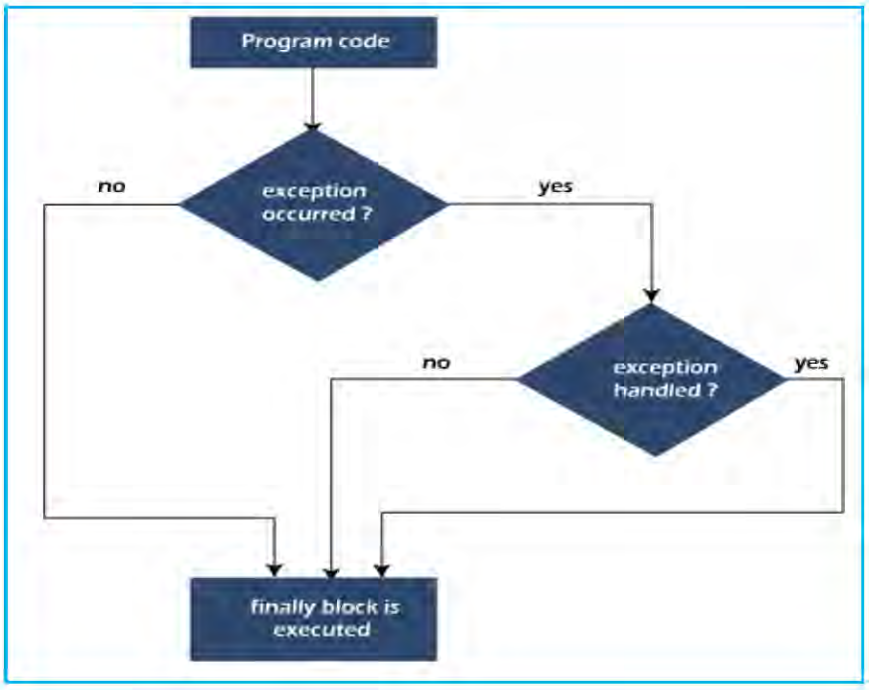
- From Java SE 7 and later, the catch statement ***can handle more than one type of exception with one catch block.*** This reduces code duplication and increases code simplicity and efficiency.
- Each exception type that can be handled by the catch block is separated using a vertical bar "|".
- Its syntax is:

```
try {
  // code
} catch (ExceptionType1 | Exceptiontype2 ex) {
  // catch block
}
```

```
1  //catchWithMultiException.java
2  import java.util.*;
3  public class catchWithMultiException {
4    public static void main(String[] args) {
5          try {
6                  Scanner in=new Scanner(System.in);
7                  System.out.print("Enter b: ");
8                  int b=in.nextInt();
9                  int[] array = new int[5];
10                 array[5] = 30 /b; // if b=0, divide by zero exception
11                 System.out.println(array[0]);
12             }
13         catch(ArithmeticException | ArrayIndexOutOfBoundsException e) {
14                 System.out.println("Arithmetic Exception ==> "+ e.getMessage());
15             }
16        System.out.println("Rest code");
17     }
18 }
```

Java finally blocks

- The ***finally creates a block of code that will be executed after a try /catch block*** has completed and before the code following the try/catch block.
- The finally block follows the try-catch block.
- The finally block ***will execute whether or not an exception is thrown***. If an exception is thrown, the finally block will execute even if no catch statement matches the exception.
- Any time a method is about to return to the caller from inside a try/catch block, via an uncaught exception or an explicit return statement, the finally clause is also executed just before the method returns.
- This can be ***useful for closing file handles*** and ***freeing up any other resources*** that might have been allocated at the beginning of a method with the intent of ***disposing of them before returning.***
- ***The finally clause is optional.*** However, each try statement requires at least one catch or a finally clause.
- ***Note:*** If you don't handle the exception, before terminating the program, JVM executes finally block (if any).



Why use Java finally block?

- Finally block in Java can be used to put "cleanup" code such as closing a file, closing connection, etc.
- The important statements to be printed can be placed in the finally block.

The syntax of the finally block is:

```
try
{
//Protected code
}
catch (ExceptionType1 et1)
{
//Catch block
}
catch (ExceptionType2 et2)
{
//Catch block
}
finally
{
//The finally block always executes.
}
```

```
1  //ExceptionFinallyBlock.java
2  import java.util.*;
3  class ExceptionFinallyBlock {
4    public static void main(String[] args) {
5      try {
6            Scanner in=new Scanner(System.in);
7            System.out.print("Enter Number: ");
8            int num=in.nextInt();
9            int x= 100 / num;
10           } catch(NumberFormatException | ArithmeticException e) {
11              System.out.println("Exception =====>" + e.getMessage());
12
13           }
14           finally {
15              System.out.println("Finally block is always executed."); }
16     }
17 }
```

```
C:\Users\balkr\Documents\Java>javac ExceptionFinallyBlock.java

C:\Users\balkr\Documents\Java>java ExceptionFinallyBlock
Enter Number: 0
Exception =====>/ by zero
Finally block is always executed.
```

Difference between final, finally and finalize

Sr. no.	Key	final	finally	finalize
1.	Definition	final is the keyword and access modifier which is used to apply restrictions	finally is the block in Java Exception Handling to execute the important code	finalize is the ***method in Java which is used to perform clean up processing just before object is garbage collected.***

		on a class, method or variable.	whether the exception occurs or not.	
2.	Applicable to	Final keyword is used with the classes, methods and variables.	Finally, block is always related to the try and catch block in exception handling.	finalize() method is used with the objects.
3.	Functionality	1. Once declared, final variable becomes constant and cannot be modified. 2. final method cannot be overridden by sub class. 3. final class cannot be inherited.	1. finally block runs the important code even if exception occurs or not. 2. finally block cleans up all the resources used in try block	finalize method performs the cleaning activities with respect to the object before its destruction.
4.	Execution	Final method is executed only when we call it.	Finally block is executed as soon as the try-catch block is executed. Its execution is not dependent on the exception.	finalize method is executed just before the object is destroyed.

Java throw and throws keyword

The general form of throw is shown here:

throw ThrowableInstance;

The flow of execution stops immediately after the throw statement; any subsequent statements are not executed. ***The nearest enclosing try block is inspected to see if it has a catch statement that matches the type of exception***. If it does find a match, control is transferred to that statement. ***If not, then the next enclosing try statement is inspected, and so on.*** If no matching catch is found, then the default exception handler halts the program and prints the stack trace.

throws

If a method is capable of causing an exception that it does not handle, it must specify this behavior so that callers of the method can guard themselves against that exception. You do this by including a **throws** clause in the method's declaration. A **throws** clause lists the types of exceptions that a method might throw. This is necessary for all exceptions, except those of type **Error** or **RuntimeException**, or any of their subclasses. All other exceptions that a method can throw must be declared in the **throws** clause. If they are not, a compile-time error will result.

This is the general form of a method declaration that includes a **throws** clause:

type method-name(*parameter-list*) throws *exception-list*
{
  // body of method
}

Here, *exception-list* is a comma-separated list of the exceptions that a method can throw.

Creating Your Own Exception

```
1  //MyExceptionExample.java
2  import java.lang.Exception;
3  class CustomException extends Exception {
4      String message;
5      CustomException(String str) {
6          message = str;
7      }
8      public String toString() {
9        return ("Custom Exception Occurred : " + message);
10     }
11 }// end of class CustomException
12 public class MyExceptionExample {
13     public static void main(String args[]) {
14         try {
15             throw new CustomException("This is a custom message"); }
16         catch(CustomException e) {
17             System.out.println(e); }
18     }// end of main function
19 }// end of class MyExceptionExample
```

```
C:\Users\balkr\Documents\Java>java MyExceptionExample
Custom Exception Occurred : This is a custom message
```

```java
//ownExceptionExample.java
import java.lang.Exception;
class myException extends Exception {
    private int detail;
    myException(int a){
        detail=a;}
    public String toString(){
        return "MyException ["+ detail+"]"; }
}// end of class

class ownExceptionExample{
    static void compute(int a) throws myException{
        System.out.print("Called compute (" +a+ ")");
        if(a>100)
            throw new myException(a);
        System.out.println("  Normal Execution.");
    }
    public static void main(String [] args){
        try{
            compute(50);
            compute(123);}
        catch(myException me){
            System.out.println("  Exception Caught "+ me);}
    }// end of main function
}// end of class
```

```java
//StudentExceptionExample.java
import java.lang.Exception;
class StudentNotFoundException extends Exception {
    public StudentNotFoundException(String message) {
        super(message); }
}// end of class StudentNotFoundException
class StudentManager {
    public String find(String studentID) throws StudentNotFoundException {
        if (studentID.equals("123456")) {
            return "Student ID Found." ; }
        else {
            throw new StudentNotFoundException("ID Not Found: " + studentID); }
    }
}// end of class StudentManager
public class StudentExceptionExample {
    public static void main(String[] args) {
        StudentManager manager = new StudentManager();
        try {
            String msg = manager.find("123456");
            System.out.println(msg);
            msg = manager.find("23456");}
        catch (StudentNotFoundException ex) {
            System.out.println(ex);}
    }
}// end of class StudentExceptionExample
```

```
C:\Users\balkr\Documents\Java>java StudentExceptionExample
Student ID Found.
StudentNotFoundException: ID Not Found: 23456
```

Multithreaded Programming

A multithreaded program contains two or more parts that can run concurrently. Each part of such a program is called a thread, and each thread defines a separate path of execution. Thus, multithreading is a specialized form of multitasking.

There are two distinct types of multitasking:

1. *Process-based and;*
2. *Thread-based.*

A process is, in essence, a program that is executing. Thus, process-based multitasking is the feature that allows your computer to run two or more programs concurrently. For example, process-based multitasking enables you to run the Java compiler at the same time that you are using a text editor or visiting a web site. In process-based multitasking, a program is the smallest unit of code that can be dispatched by the scheduler.

In a thread-based multitasking environment, the thread is the smallest unit of dispatchable code. This means that a single program can perform two or more tasks simultaneously. For instance, a text editor can format text at the same time that it is printing, as long as these two actions are being performed by two separate threads. Thus, process-based multitasking deals with the "big picture," and thread-based multitasking handles the details. Multitasking threads require less overhead than multitasking processes. Processes are heavyweight tasks that require their own separate address spaces. Interprocess communication is expensive and limited. Context switching from one process to another is also costly. Threads, on the other hand, are lighter weight. They share the same address space and cooperatively share the same heavyweight process. Interthread communication is inexpensive, and context switching from one thread to the next is lower in cost. While Java programs make use of process-based multitasking environments, process-based multitasking is not under Java's control.

Life Cycle of a Thread / Thread States

Threads can be in one of six states:

1. New
2. Runnable
3. Blocked
4. Waiting
5. Timed waiting
6. Terminated

New Thread: When you create a thread with the new operator—for example, ***new Thread(r)*** — the thread is not yet running. This means that it is in the new state. When a thread is in the new state, the program has not started executing code inside of it.

Runnable Threads: Once you invoke the start method, the thread is in the runnable state. A runnable thread may or may not actually be running. It is up to the operating system to give the thread time to run. Once a thread is running, it doesn't necessarily

keep running. In fact, it is desirable if running threads occasionally pause so that other threads have a chance to run. The details of thread scheduling depend on the services that the operating system provides.

Blocked and Waiting Threads: When a thread is blocked or waiting, it is temporarily inactive. It doesn't execute any code and it consumes minimal resources. It is up to the thread scheduler to reactivate it. The details depend on how the inactive state was reached.

- When the thread tries to acquire an intrinsic object lock that is currently held by another thread, ***it becomes blocked***. The thread becomes unblocked when all other threads have relinquished the lock and the thread scheduler has allowed this thread to hold it.
- When the thread waits for another thread to notify the scheduler of a condition, ***it enters the waiting state.***
- In practice, ***the difference between the blocked and waiting state is not significant.***
- Several methods have a timeout parameter. Calling them causes the thread to ***enter the timed waiting state.*** This state persists either until the timeout expired or the appropriate notification has been received.

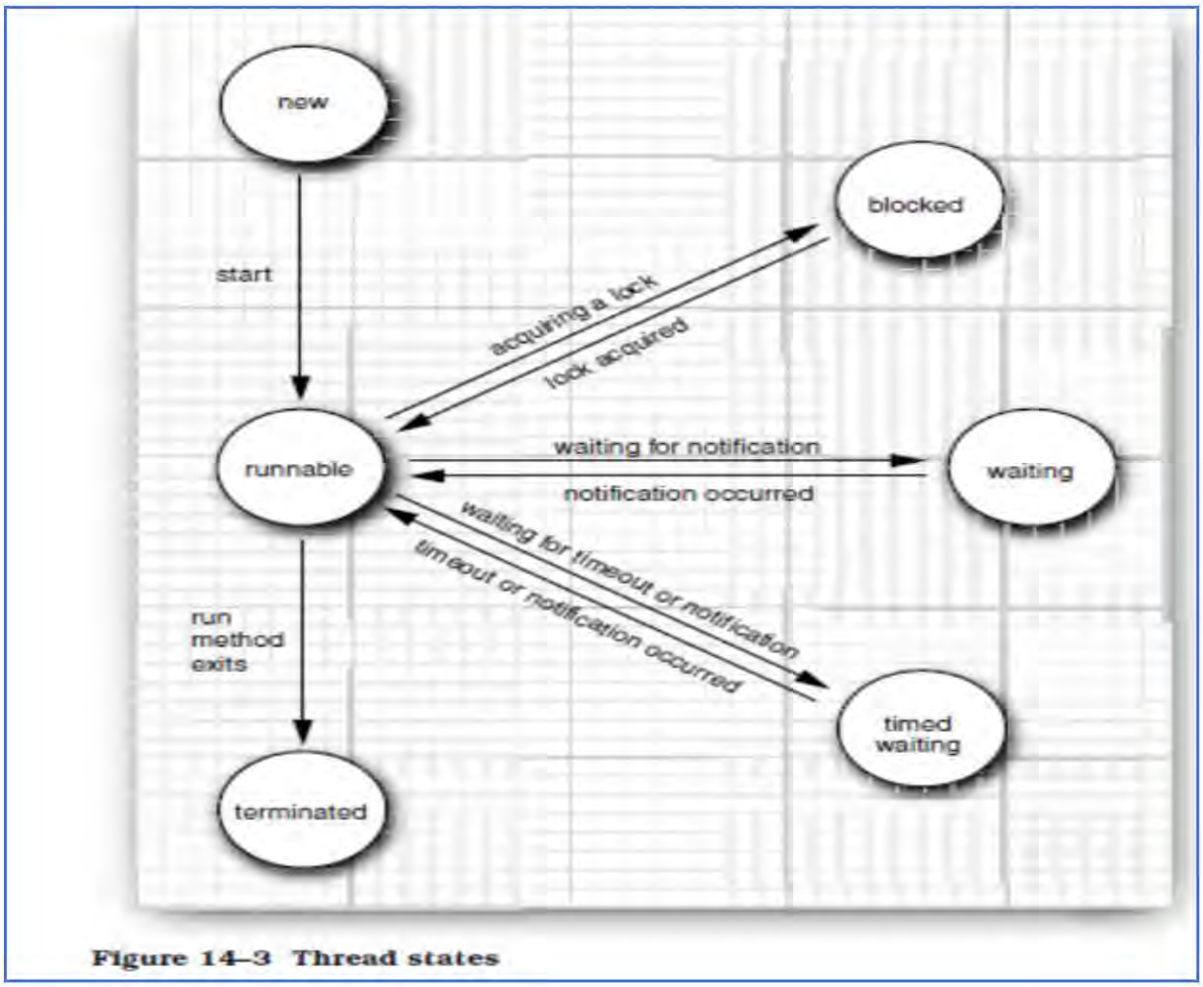


Figure 14–3 Thread states

Terminated Threads: A thread is terminated for one of two reasons:

- It dies a natural death because the run method exits normally.
- It dies abruptly because an uncaught exception terminates the run method.

How to create a thread in Java

Threads can be created by using two mechanisms:

1. ***Extending the Thread class***
2. ***Implementing the Runnable Interface***

Thread creation by extending the Thread class: We create a class that extends the ***java.lang.Thread*** class. This class overrides the ***run()*** method available in the Thread class. A thread begins its life inside ***run()*** method. We create an object of our new class and call ***start()*** method to start the execution of a thread. ***Start()*** invokes the ***run()*** method on the Thread object.

Thread creation by implementing the Runnable Interface: We create a new class which implements ***java.lang. Runnable*** interface and override ***run()*** method. Then we instantiate a Thread object and call ***start()*** method on this object.

```
1  //ThreadDemo.java
2  class NewThread implements Runnable {
3      Thread t;
4      NewThread() {
5          // Create a new, second thread
6          t = new Thread(this, "Demo Thread");
7          System.out.println("Child thread: " + t);
8          t.start(); // Start the thread
9      }
10     // This is the entry point for the second thread.
11     public void run() {
12         try {
13             for(int i = 5; i > 0; i--) {
14                 System.out.println("Child Thread: " + i);
15                 Thread.sleep(500);
16             }
17         } catch (InterruptedException e) {
18                 System.out.println("Child interrupted.");}
19         System.out.println("Exiting child thread.");
20     }
21 }
```

```
22
23  class ThreadDemo {
24      public static void main(String args[ ] ) {
25          new NewThread(); // create a new thread
26          try {
27              for(int i = 5; i > 0; i--) {
28                  System.out.println("Main Thread: " + i);
29                  Thread.sleep(1000);
30              }
31          } catch (InterruptedException e) {
32              System.out.println("Main thread interrupted."); }
33          System.out.println("Main thread exiting.");
34      }
35  }
```

```
C:\Users\balkr\Documents\Java>javac ThreadDemo.java

C:\Users\balkr\Documents\Java>java ThreadDemo
Child thread: Thread[Demo Thread,5,main]
Child Thread: 5
Main Thread: 5
Child Thread: 4
Child Thread: 3
Main Thread: 4
Child Thread: 2
Child Thread: 1
Main Thread: 3
Exiting child thread.
Main Thread: 2
Main Thread: 1
Main thread exiting.
```

```java
//ExtendThread.java
class NewThread extends Thread {
    NewThread() {
        // Create a new, second thread
        super("Demo Thread");
        System.out.println("Child thread: " + this);
        start(); // Start the thread
    }
    // This is the entry point for the second thread.
    public void run() {
        try {
            for(int i = 5; i > 0; i--) {
                System.out.println("Child Thread: " + i);
                Thread.sleep(500);
            }
        } catch (InterruptedException e) {
            System.out.println("Child interrupted.");}
        System.out.println("Exiting child thread.");
    }
}

class ExtendThread {
    public static void main(String args[]) {
        new NewThread(); // create a new thread
        try {
            for(int i = 5; i > 0; i--) {
                System.out.println("Main Thread: " + i);
                Thread.sleep(1000);
            }
        } catch (InterruptedException e) {
            System.out.println("Main thread interrupted."); }
        System.out.println("Main thread exiting.");
    }
}
```

```
C:\Users\balkr\Documents\Java>javac ExtendThread.java

C:\Users\balkr\Documents\Java>java ExtendThread
Child thread: Thread[Demo Thread,5,main]
Child Thread: 5
Main Thread: 5
Child Thread: 4
Main Thread: 4
Child Thread: 3
Child Thread: 2
Main Thread: 3
Child Thread: 1
Exiting child thread.
Main Thread: 2
Main Thread: 1
Main thread exiting.
```

Using isAlive () and join ()

Two ways exist to determine whether a thread has finished. First, you can call **isAlive( )** on the thread. This method is defined by **Thread**, and its general form is shown here:

final boolean isAlive( )

The **isAlive( )** method returns **true** if the thread upon which it is called is still running. It returns **false** otherwise.

While **isAlive( )** is occasionally useful, the method that you will more commonly use to wait for a thread to finish is called **join( )**, shown here:

final void join( ) throws InterruptedException

This method waits until the thread on which it is called terminates. Its name comes from the concept of the calling thread waiting until the specified thread *joins* it. Additional forms of **join( )** allow you to specify a maximum amount of time that you want to wait for the specified thread to terminate.

```java
//exampleIsAliveJoin.java
class NewThread implements Runnable{
    String name; // name of thread
    Thread thread;
    NewThread(String threadName){
        name=threadName;
        thread=new Thread(this, name);
        System.out.println("New Thread: "+ thread);
    }
    // This is the entry point of thread.
    public void run(){
        try{
            for(int i=5; i>0; i--){
                System.out.println(name + " : "+ i);
                Thread.sleep(1000);
            }
        }catch (InterruptedException e){
            System.out.println(name + " interrupted. ");
        }
        System.out.println(name + " exiting...");
    }
}// end of class NewThread
class exampleIsAliveJoin {
    public static void main(String[] args){
        NewThread nt1=new NewThread("One");
        NewThread nt2=new NewThread("Two");
        NewThread nt3=new NewThread("Three");

        // Start the Thread
        nt1.thread.start();
        nt2.thread.start();
        nt3.thread.start();

        System.out.println("Thread One is alive :" + nt1.thread.isAlive());
        System.out.println("Thread One is alive :" + nt2.thread.isAlive());
        System.out.println("Thread One is alive :" + nt3.thread.isAlive());

        // wait for threads to finish
        try{
            System.out.println("Waiting for threads to finish...");
            nt1.thread.join();
            nt2.thread.join();
            nt3.thread.join();
        } catch (InterruptedException e){
            System.out.println("Main thread interrupted.");
        }
        System.out.println("Thread One is alive :" + nt1.thread.isAlive());
        System.out.println("Thread One is alive :" + nt2.thread.isAlive());
        System.out.println("Thread One is alive :" + nt3.thread.isAlive());

        System.out.println("Main thread exiting....");
    }// end of main function
}// end of class exampleIsAliveJoin
```

```
C:\Users\balkr\Documents\Java>java exampleIsAliveJoin
New Thread: Thread[One,5,main]
New Thread: Thread[Two,5,main]
New Thread: Thread[Three,5,main]
Thread One is alive :true
Thread One is alive :true
Thread One is alive :true
Waiting for threads to finish...
Three : 5
Two : 5
One : 5
Three : 4
Two : 4
One : 4
Three : 3
Two : 3
One : 3
Three : 2
Two : 2
One : 2
Three : 1
Two : 1
One : 1
Three exiting...
Two exiting...
One exiting...
Thread One is alive :false
Thread One is alive :false
Thread One is alive :false
Main thread exiting....
```

Thread Priorities

- Thread priorities are used by the thread scheduler to decide when each thread should be allowed to run.
- In theory, over a given period of time, higher-priority threads get more CPU time than lower-priority threads.
- In practice, the amount of CPU time that a thread gets often depends on several factors besides its priority.
- For example, how an operating system implements multitasking can affect the relative availability of CPU time. A higher-priority thread can also preempt a lower-priority one.

- For instance, when a lower-priority thread is running and a higher-priority thread resumes (from sleeping or waiting on I/O, for example), it will preempt the lower-priority thread.
- In theory, threads of equal priority should get equal access to the CPU.
- In practice, even in non-preemptive environments, most threads still get a chance to run, because most threads inevitably encounter some blocking situation, such as waiting for I/O. When this happens, the blocked thread is suspended and other threads can run. But, if you want smooth multithreaded execution, you are better off not relying on this. Also, some types of tasks are CPU-intensive. Such threads dominate the CPU. For these types of threads, you want to yield control occasionally so that other threads can run.
- To set a thread's priority, use the ***setPriority () method***, which is a member of Thread. ***This is its general form:***

  final void setPriority (int level)

  - Here, level specifies the new priority setting for the calling thread. The value of level must be within the range MIN_PRIORITY and MAX_PRIORITY.
  - Currently, these values are 1 and 10, respectively.
  - To return a thread to default priority, specify NORM_PRIORITY, which is currently 5.
  - These priorities are defined as static final variables within Thread.

```
1  //ThreadPriorityExample.java
2  import java.lang.*;
3  public class ThreadPriorityExample extends Thread {
4      public void run(){
5          System.out.println("Inside the run() method"); }
6
7      public static void main(String argvs[]) {
8          // Creating threads with the help of ThreadPriorityExample class
9          ThreadPriorityExample th1 = new ThreadPriorityExample();
10         ThreadPriorityExample th2 = new ThreadPriorityExample();
11         ThreadPriorityExample th3 = new ThreadPriorityExample();
12
13         // We did not mention the priority of the thread.  Therefore, the priorities of the thread is 5, the default value
14         System.out.println("Priority of the thread th1 is : " + th1.getPriority());
15         System.out.println("Priority of the thread th2 is : " + th2.getPriority());
16         System.out.println("Priority of the thread th2 is : " + th3.getPriority());
17
18         // Setting priorities of above threads by  passing integer arguments
19         th1.setPriority(6);
20         th2.setPriority(3);
21         th3.setPriority(9);
```

```
22
23          System.out.println("Priority of the thread th1 is : " + th1.getPriority());
24          System.out.println("Priority of the thread th2 is : " + th2.getPriority());
25          System.out.println("Priority of the thread th3 is : " + th3.getPriority());
26          System.out.println("Currently Executing The Thread : " + Thread.currentThread().getName());
27          System.out.println("Priority of the main thread is : " + Thread.currentThread().getPriority());
28          Thread.currentThread().setPriority(10);
29          System.out.println("Priority of the main thread is : " + Thread.currentThread().getPriority());
30      }
31  }
```

```
C:\Users\balkr\Documents\Java>javac ThreadPriorityExample.java

C:\Users\balkr\Documents\Java>java ThreadPriorityExample
Priority of the thread th1 is : 5
Priority of the thread th2 is : 5
Priority of the thread th2 is : 5
Priority of the thread th1 is : 6
Priority of the thread th2 is : 3
Priority of the thread th3 is : 9
Currently Executing The Thread : main
Priority of the main thread is : 5
Priority of the main thread is : 10
```

Synchronization

- When two or more threads need access to a shared resource, they need some way to ensure that the resource will be used by only one thread at a time. The process by which this is achieved is ***called synchronization***.
- Key to synchronization is the ***concept of the monitor***. A monitor is an object that is used as a ***mutually exclusive lock***. Only one thread can own a monitor at a given time. When a thread acquires a lock, ***it is said to have entered the monitor***. All other threads attempting to enter the locked monitor ***will be suspended until the first thread exits the monitor***. These other threads are said to be ***waiting for the monitor***. A thread that owns a monitor can reenter the same monitor if it so desires.
- **Mutual Exclusive:** Mutual Exclusive helps keep threads from interfering with one another while sharing data. It can be achieved by using the following two ways:
  1. By Using Synchronized Method
  2. By Using Synchronized Statement

Using Synchronized Methods

- Synchronization is easy in Java, because all objects have their own implicit monitor associated with them. To enter an object's monitor, just call a method that has been modified with the synchronized keyword. While a thread is inside a synchronized method, all other threads that try to call it (or any other synchronized method) on the same instance have to wait. To exit the monitor and relinquish control of the object to the

next waiting thread, the owner of the monitor simply returns from the synchronized method.

The Synchronized Statement

While creating **synchronized** methods within classes that you create is an easy and effective means of achieving synchronization, it will not work in all cases. To understand why, consider the following. Imagine that you want to synchronize access to objects of a class that was not designed for multithreaded access. That is, the class does not use **synchronized** methods. Further, this class was not created by you, but by a third party, and you do not have access to the source code. Thus, you can't add **synchronized** to the appropriate methods within the class. How can access to an object of this class be synchronized? Fortunately, the solution to this problem is quite easy: You simply put calls to the methods defined by this class inside a **synchronized** block.

This is the general form of the **synchronized** statement:

```
synchronized(objRef) {
  // statements to be synchronized
}
```

Here, *objRef* is a reference to the object being synchronized. A synchronized block ensures that a call to a synchronized method that is a member of *objRef*'s class occurs only after the current thread has successfully entered *objRef*'s monitor.

Understanding the problem without Synchronization

```
1  //SynchronizationProblem.java
2  import java.lang.*;
3  class Table{
4      void printTable(int n){//method not synchronized
5          for(int i=1;i<=5;i++){
6              System.out.println(n + "*"+ i + "=" + n*i);
7              try{
8                  Thread.sleep(100);
9              }catch(InterruptedException e){
10                 System.out.println(e);
11             }
12         }
13     }
14 }
15 class MyThreadFirst extends Thread{
16     Table t;
17     MyThreadFirst(Table t){
18         this.t=t;}
19     public void run(){
20         t.printTable(10); }
21 } // end of class MyThreadFirst
```

```
22  class MyThreadSecond extends Thread{
23      Table t;
24      MyThreadSecond(Table t){
25          this.t=t; }
26      public void run(){
27          t.printTable(100); }
28  }// end of class MyThreadSecond
29
30  class SynchronizationProblem{
31      public static void main(String args[]){
32          Table obj = new Table();//only one object
33          MyThreadFirst t1=new MyThreadFirst(obj);
34          MyThreadSecond t2=new MyThreadSecond(obj);
35          t1.start();
36          t2.start();
37      }
38  } // end of class SynchronizationProblem
```

```
C:\Users\balkr\Documents\Java>java SynchronizationProblem
10*1=10
100*1=100
100*2=200
10*2=20
100*3=300
10*3=30
100*4=400
10*4=40
100*5=500
10*5=50
```

```
1   //synchronizationUsingMethod.java
2   import java.lang.*;
3   class Table{
4       synchronized void printTable(int n){//method synchronized
5           for(int i=1;i<=5;i++){
6               System.out.println(n + "*"+ i + "=" + n*i);
7               try{
8                   Thread.sleep(100);
9               }catch(InterruptedException e){
10                  System.out.println(e);
11              }
12          }
13      }
14  }
15  class MyThreadFirst extends Thread{
16      Table t;
17      MyThreadFirst(Table t){
18          this.t=t;}
19      public void run(){
20          t.printTable(10); }
21  } // end of class MyThreadFirst
```

```
class MyThreadSecond extends Thread{
    Table t;
    MyThreadSecond(Table t){
        this.t=t; }
    public void run(){
        t.printTable(100); }
}// end of class MyThreadSecond

class synchronizationUsingMethod{
    public static void main(String args[]){
        Table obj = new Table();//only one object
        MyThreadFirst t1=new MyThreadFirst(obj);
        MyThreadSecond t2=new MyThreadSecond(obj);
        t1.start();
        t2.start();
    }
} // end of class synchronizationUsingMethod
```

```
C:\Users\balkr\Documents\Java>java synchronizationUsingMethod
10*1=10
10*2=20
10*3=30
10*4=40
10*5=50
100*1=100
100*2=200
100*3=300
100*4=400
100*5=500
```

```
//synchronizationBlock.java
import java.lang.*;
class Table{
    void printTable(int n){
        synchronized(this){//synchronized block
            for(int i=1;i<=5;i++){
                System.out.println(n + "*"+ i + "=" + n*i);
                try{
                    Thread.sleep(100);
                }catch(InterruptedException e){
                    System.out.println(e);
                }
            }
        }
    }
}
class MyThreadFirst extends Thread{
    Table t;
    MyThreadFirst(Table t){
        this.t=t;}
    public void run(){
        t.printTable(10); }
} // end of class MyThreadFirst
```

```java
class MyThreadSecond extends Thread{
    Table t;
    MyThreadSecond(Table t){
        this.t=t; }
    public void run(){
        t.printTable(100); }
}// end of class MyThreadSecond

class synchronizationBlock{
    public static void main(String args[]){
        Table obj = new Table();//only one object
        MyThreadFirst t1=new MyThreadFirst(obj);
        MyThreadSecond t2=new MyThreadSecond(obj);
        t1.start();
        t2.start();
    }
} // end of class synchronizationBlock
```

```
C:\Users\balkr\Documents\Java>java synchronizationBlock
100*1=100
100*2=200
100*3=300
100*4=400
100*5=500
10*1=10
10*2=20
10*3=30
10*4=40
10*5=50
```

Chapter – 6

Java Input/Output

Streams

- Java programs perform I/O through streams. A ***stream is an abstraction that either produces or consumes information.***
- A stream is linked to a physical device by the Java I/O system. All streams behave in the same manner, even if the actual physical devices to which they are linked differ. Thus, the ***same I/O classes and methods can be applied to different types of devices.***
- This means that an input stream can abstract many different kinds of input: from a disk file, a keyboard, or a network socket. Likewise, ***an output stream may refer to the console, a disk file, or a network connection.***
- Streams are a clean way to deal with input/output without having every part of your code understand the difference between a keyboard and a network, for example.
- Java implements streams within class hierarchies defined in the ***java.io*** package.

Byte Streams and Character Streams

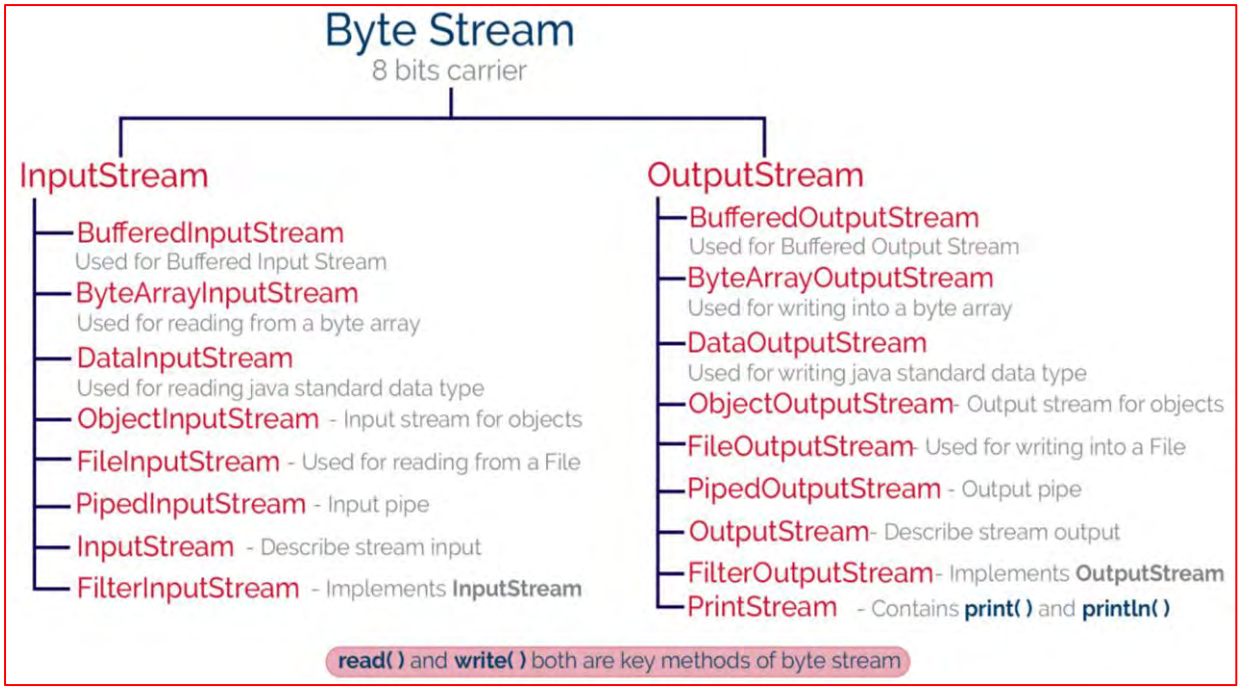
- Java defines two types of streams: ***byte and character***.
- ***Byte streams*** provide a convenient means for handling input and output of bytes. Byte streams are used, for example, when reading or writing binary data.
- ***Character streams*** provide a convenient means for handling input and output of characters. They use Unicode and, therefore, can be internationalized. Also, in some cases, character streams are more efficient than byte streams.

The Byte Stream Classes

- Byte streams are defined by using two class hierarchies. At the top are two abstract classes***: InputStream*** and ***OutputStream***.
- Each of these abstract classes has several concrete subclasses that handle the differences among various devices, such as disk files, network connections, and even memory buffers.

The Character Stream Classes

- Character streams are defined by using two class hierarchies. At the top are two abstract classes: ***Reader*** and ***Writer***.
- These abstract classes handle Unicode character streams. Java has several concrete subclasses of each of these.



- ***BufferedInputStream:*** contains methods to read bytes from the buffer.
- ***ByteArrayInputStream:*** contains methods to read bytes from a byte array.
- ***DataInputStream:*** contains methods to read Java primitive data types.
- ***FileInputStream:*** contains methods to read bytes from a file.
- ***FilterInputStream:*** This class contains methods to read bytes from the other input streams, which are used as the primary source of data.
- ***ObjectInputStream:*** contains methods to read objects.
- ***PipedInputStream:*** contains methods to read from a piped output stream. A piped input stream must be connected to a piped output stream.
- ***SequenceInputStream:*** contains methods to concatenate multiple input streams and then read from the combined stream.
- ***BufferedOutputStream:*** Contains methods to write bytes into the buffer.
- ***ByteArrayOutputStream:*** Contains methods to write bytes into a byte array.
- ***DataOutputStream:*** Contains methods to write Java primitive data types.
- ***FileOutputStream:*** Contains methods to write bytes to a file.
- ***FilterOutputStream:*** Contains methods to write to other output streams.
- ***ObjectOutputStream:*** Contains methods to write objects.
- ***PipedOutputStream:*** Contains methods to write to a piped output stream.
- ***PrintStream:*** Contains methods to print Java primitive data types.

InputStream class

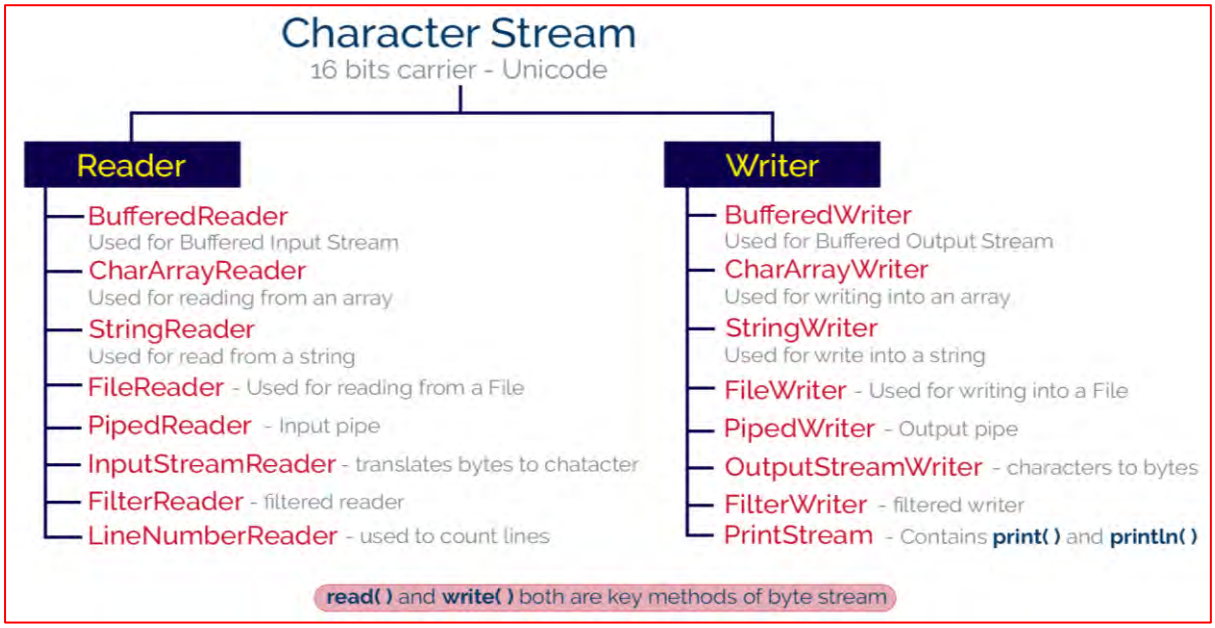
The InputStream class has defined as an abstract class, and it has the following methods which have implemented by its concrete classes.

S.No.	Method with Description
1	**int available()** It returns the number of bytes that can be read from the input stream.
2	**int read()** It reads the next byte from the input stream.
3	**int read(byte[] b)** It reads a chunk of bytes from the input stream and store them in its byte array, b.
4	**void close()** It closes the input stream and also frees any resources connected with this input stream.

OutputStream class

The OutputStream class has defined as an abstract class, and it has the following methods which have implemented by its concrete classes.

S.No.	Method with Description
1	**void write(int n)** It writes byte(contained in an int) to the output stream.
2	**void write(byte[] b)** It writes a whole byte array(b) to the output stream.
3	**void flush()** It flushes the output steam by forcing out buffered bytes to be written out.
4	**void close()** It closes the output stream and also frees any resources connected with this output stream.



Reader class

The Reader class has defined as an abstract class, and it has the following methods which have implemented by its concrete classes.

S.No.	Method with Description
1	**int read()** It reads the next character from the input stream.
2	**int read(char[] cbuffer)** It reads a chunk of charaters from the input stream and store them in its byte array, cbuffer.
3	**int read(char[] cbuf, int off, int len)** It reads charaters into a portion of an array.
4	**int read(CharBuffer target)** It reads charaters into into the specified character buffer.
5	**String readLine()** It reads a line of text. A line is considered to be terminated by any oneof a line feed ('\n'), a carriage return ('\r'), or a carriage returnfollowed immediately by a linefeed.
6	**boolean ready()** It tells whether the stream is ready to be read.
7	**void close()** It closes the input stream and also frees any resources connected with this input stream.

Writer class

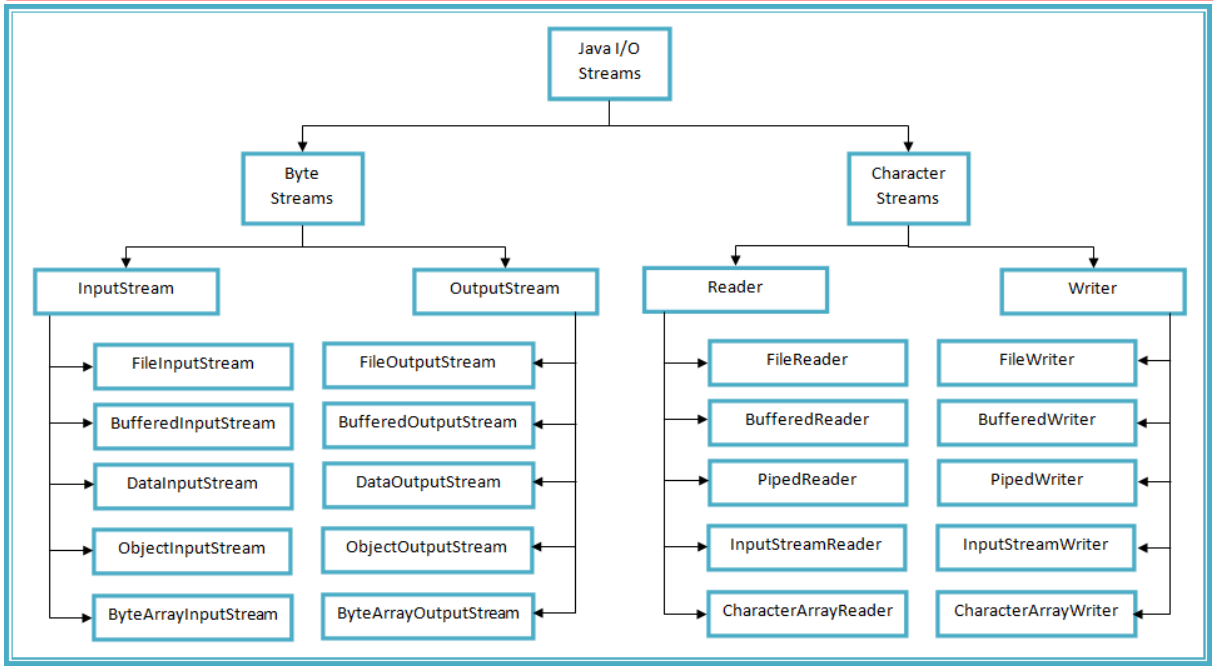
The Writer class has defined as an abstract class, and it has the following methods which have implemented by its concrete classes.

S.No.	Method with Description
1	**void flush()** It flushes the output steam by forcing out buffered bytes to be written out.
2	**void write(char[] cbuf)** It writes a whole array(cbuf) to the output stream.
3	**void write(char[] cbuf, int off, int len)** It writes a portion of an array of characters.
4	**void write(int c)** It writes single character.

5	**void write(String str)** It writes a string.
6	**void write(String str, int off, int len)** It writes a portion of a string.
7	**Writer append(char c)** It appends the specified character to the writer.
8	**Writer append(CharSequence csq)** It appends the specified character sequence to the writer
9	**Writer append(CharSequence csq, int start, int end)** It appends a subsequence of the specified character sequence to the writer.
10	**void close()** It closes the output stream and also frees any resources connected with this output stream.

Byte Stream Vs Character Stream In Java :

Byte Stream	Character Stream
They process the data byte by byte.	They process the data character by character.
They read/write data 8 bits maximum at a time.	They read/write data 16 bits maximum at a time.
They are most suitable to process binary files.	They are most suitable to process text files.
All byte stream classes in Java are descendants of InputStream and OutputStream.	All character stream classes in Java are descendants of Reader and Writer.



```java
//bufferReadesExample.java
import java.io.*;
public class bufferReadesExample {
    public static void main(String[] args) throws IOException {
        InputStreamReader isr = new InputStreamReader(System.in);
        BufferedReader in = new BufferedReader(isr);

        System.out.print("Please enter your name: ");
        String name = in.readLine(); //Reading String
        System.out.println("My name is: " + name );

        System.out.print("Please enter your Age: ");
        int age = Integer.parseInt(in.readLine());
        System.out.println("My Age is: " + age );

        System.out.print("Please enter your Salary: ");
        double salary = Double.parseDouble(in.readLine());
        System.out.println("My salary is: " + salary);

        System.out.print("Enter any character : ");
        char c = (char)in.read();
        System.out.print("Entered character is: " + c);
    }
}
```

```
C:\Users\balkr\Documents\Java>java bufferReadesExample
Please enter your name: asdas asdjhasf
My name is: asdas asdjhasf
Please enter your Age: 13
My Age is: 13
Please enter your Salary: 12412.4325235
My salary is: 12412.4325235
Enter any character : e
e
```

```java
// Reading data from console until user writes "stop".
//bufferReadesLine.java
import java.io.*;
public class bufferReadesLine {
    public static void main(String[] args) throws IOException {
        InputStreamReader isr = new InputStreamReader(System.in);
        BufferedReader in = new BufferedReader(isr);
        String str;
        System.out.println("Enter \"stop\" to quit. ");
        System.out.println("Enter line of text: ");
        do{
            str = in.readLine(); //Reading String
            System.out.println(str);
        }while(!str.equals("stop"));
    }
}
```

```
C:\Users\balkr\Documents\Java>java bufferReadesLine
Enter "stop" to quit.
Enter line of text:
hello students.
hello students.
Java is fun.
Java is fun.
programming nevsr stop.
programming nevsr stop.
stop
stop
```

```java
//FileOutputStreamExample.java
import java.io.*;
public class FileOutputStreamExample {
    public static void main(String args[]){
          try{
              String path="C:/Users/balkr/Documents/Java/Input-Output/fileout.txt";
              FileOutputStream fout=new FileOutputStream(path);
              String s="Welcome to File Output Stream Example.";
              byte b[]=s.getBytes();//converting string into byte array
              fout.write(b);
              fout.close();
              System.out.println("success...");
           }catch(Exception e){
              System.out.println(e);}
     }
}
```

Example: Write a Java Program to read data from keyboard and write those data into file “Records.txt” until user say “no”.

```java
//fileReadWriteExample.java
import java.io.*;
class fileReadWriteExample {
    public static void main(String[] args) throws IOException {
        try{
            InputStreamReader isr = new InputStreamReader(System.in);
            BufferedReader in = new BufferedReader(isr);

            File file = new File("C:/Users/balkr/Documents/Java/Records.txt");
            FileWriter write = new FileWriter(file) ;

            String option;
            System.out.println("Enter \"no\" to quit. ");
            do{
                System.out.print("Please enter your name: ");
                String name = in.readLine(); //Reading String
                System.out.print("Please enter your Age: ");
                int age = Integer.parseInt(in.readLine());
                System.out.print("Please enter your Salary: ");
                double salary = Double.parseDouble(in.readLine());
```

```
                String str=" Name: "+name+"  Age: "+ age +"  Salary: "+ salary;
                write.write(str);
                write.write(System.lineSeparator());

                System.out.print("Do you want to continue....yes/no?   ");
                option = in.readLine(); //Reading String
            }while(option.equals("yes"));

            write.close();
        }catch (IOException  e){
            System.out.print(e);
        }
    }
}
```

```
C:\Users\balkr\Documents\Java>java fileReadWriteExample
Enter "no" to quit.
Please enter your name: Ram sharma
Please enter your Age: 56
Please enter your Salary: 2335533.12
Do you want to continue....yes/no?   yes
Please enter your name: Hari Karki
Please enter your Age: 43
Please enter your Salary: 45632413.12
Do you want to continue....yes/no?   yes
Please enter your name: kiran Thapa
Please enter your Age: 65
Please enter your Salary: 214543534.12
Do you want to continue....yes/no?   no
```

Records - Notepad

File Edit Format View Help

```
Name: Ram sharma  Age: 56  Salary: 2335533.12
Name: Hari Karki  Age: 43  Salary: 4.563241312E7
Name: kiran Thapa  Age: 65  Salary: 2.1454353412E8
```

Example: Write a Java Program to read data from file "student.txt" and display the content of file into the console.

```
//fileInputStreamExample.java
import java.io.*;
public class fileInputStreamExample {
    public static void main(String args[]) {
        byte[] array = new byte[200];
        try {
            String path="C:/Users/balkr/Documents/Java/student.txt";
            InputStream input = new FileInputStream(path);
            System.out.println("Total bytes in the file: " + input.available());
            // Read byte from the input stream
            input.read(array);
            System.out.println("Data read from the file: ");
            // Convert byte array into string
            String data = new String(array);
            System.out.println(data);
            // Close the input stream
            input.close();
        } catch (IOException e) {
            e.getStackTrace();
        }
    }
}
```

```
C:\Users\balkr\Documents\Java>java fileInputStreamExample
Total bytes in the file: 152
Data read from the file:
 Name: Ram sharma  Age: 56  Salary: 2335533.12
 Name: Hari Karki  Age: 43  Salary: 4.563241312E7
 Name: kiran Thapa  Age: 65  Salary: 2.1454353412E8
```

```java
//ReadFileLineByLineUsingBufferedReader.java
import java.io.*;
class ReadFileLineByLineUsingBufferedReader {
    public static void main(String[] args) {
        BufferedReader reader;
        try {
            String path="C:/Users/balkr/Documents/Java/student.txt";
            reader = new BufferedReader(new FileReader(path));
            String line = reader.readLine();

            while (line != null) {
                System.out.println(line);
                System.out.println("============================");
                line = reader.readLine();
            }

            reader.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

```
C:\Users\balkr\Documents\Java>java ReadFileLineByLineUsingBufferedReader
 Name: Ram sharma  Age: 56  Salary: 2335533.12
============================
 Name: Hari Karki  Age: 43  Salary: 4.563241312E7
============================
 Name: kiran Thapa  Age: 65  Salary: 2.1454353412E8
============================
```

```java
//copyFileOne2Another.java
import java.io.*;
public class copyFileOne2Another {
    public static void main(String[] args) throws IOException {
        FileInputStream sourceStream = null;
        FileOutputStream targetStream = null;
        try {
            String path1="C:/Users/balkr/Documents/Java/Input-Output/source.txt";
            String path2="C:/Users/balkr/Documents/Java/Input-Output/destination.txt";
            sourceStream = new FileInputStream(path1);
            targetStream = new FileOutputStream(path2);
            int temp;
            while ((temp = sourceStream.read())!= -1)
                targetStream.write((byte)temp);
        }
        finally {
            if (sourceStream != null)
                sourceStream.close();
            if (targetStream != null)
                targetStream.close();
        }
    }
}
```

```java
//CopyFileCharacterWise.java
import java.io.*;
public class CopyFileCharacterWise {
    public static void main(String args[]) throws IOException {
        FileReader in = null;
        FileWriter out = null;
      try {
            String path1="C:/Users/balkr/Documents/Java/Input-Output/source1.txt";
            String path2="C:/Users/balkr/Documents/Java/Input-Output/destination1.txt";
            in = new FileReader(path1);
            out = new FileWriter(path2);
            int c;
            while ((c = in.read()) != -1) {
                out.write(c);}
        }finally {
            if (in != null) {
                in.close(); }
            if (out != null) {
                out.close();}
        }
    }
}
```

Object handling with File

```java
//ObjectHandlingwithFile.java
import java.util.*;
import java.io.*;
class Student implements Serializable{
    int rollno;  String name;  int age;
    Student(int rollno,String name,int age){
        this.rollno=rollno;
        this.name=name;  this.age=age;
    }
}// end of class Student
class ObjectHandlingwithFile{
    public static void main(String args[]){
        try{
            InputStreamReader isr = new InputStreamReader(System.in);
            BufferedReader in = new BufferedReader(isr);
            String path="C:/Users/balkr/Documents/Java/Input-Output/student.txt";

            FileOutputStream fos=new FileOutputStream(path);
            ObjectOutputStream output=new ObjectOutputStream(fos);
            FileInputStream fis=new FileInputStream(path);
            ObjectInputStream input=new ObjectInputStream(fis);

              String option;
              System.out.println("Enter \"no\" to quit. ");
              do{
                  System.out.print("Enter Roll Number: ");
                  int crn = Integer.parseInt(in.readLine());
                  System.out.print("Enter your name: ");
                  String name = in.readLine(); //Reading String
                  System.out.print("Enter your Age: ");
                  int age = Integer.parseInt(in.readLine());

                  Student s1=new Student(crn,name,age);
                  output.writeObject(s1);
```

```
36                  System.out.print("Do you want to continue....yes/no?    ");
37                  option = in.readLine(); //Reading String
38              }while(option.equals("yes"));
39
40              boolean isExist = true;  Student st =null;
41              while(isExist){
42                  if(fis.available() != 0){
43                      st = (Student) input.readObject();
44                      System.out.println(st.rollno+" "+st.name+" "+st.age);
45                  }
46                  else{
47                      isExist =false; }
48              }
49              output.close();  input.close();
50
51          }catch(Exception e){
52              System.out.println(e);}
53      }
54  }
```

```
C:\Users\balkr\Documents\Java\Input-Output>java ObjectHandlingwithFile
Enter "no" to quit.
Enter Roll Number: 123
Enter your name: Ram Sharma
Enter your Age: 23
Do you want to continue....yes/no?   yes
Enter Roll Number: 124
Enter your name: Shyam Thapa
Enter your Age: 43
Do you want to continue....yes/no?   no
123 Ram Sharma 23
124 Shyam Thapa 43
```

Chapter – 7

Java Collections and Java API Library

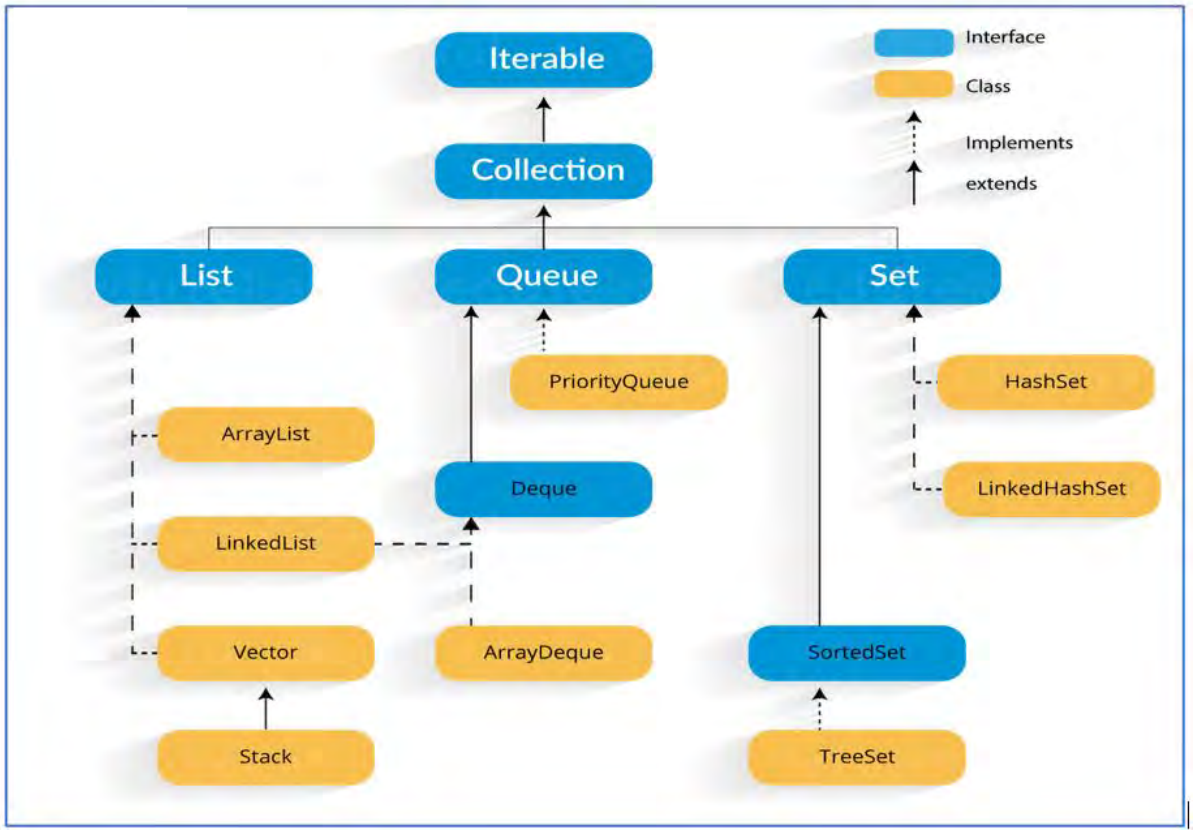
The Collection Interfaces

- A Java collection framework provides an architecture to store and manipulate a group of objects. A Java collection framework includes the following:
  1. Interfaces
  2. Classes
  3. Algorithm
- ***Interfaces:*** Interface in Java refers to the abstract data types. They allow Java collections to be manipulated independently from the details of their representation. Also, they form a hierarchy in object-oriented programming languages.
- ***Classes:*** Classes in Java are the implementation of the collection interface. It basically refers to the data structures that are used again and again.
- ***Algorithm:*** Algorithm refers to the methods which are used to perform operations such as searching and sorting, on objects that implement collection interfaces. Algorithms are polymorphic in nature as the same method can be used to take many forms or you can say perform different implementations of the Java collection interface.

Why use Java collection?

- There are several benefits of using Java collections such as:
  - Reducing the effort required to write the code by providing useful data structures and algorithms.
  - Java collections provide high-performance and high-quality data structures and algorithms thereby increasing the speed and quality.
  - Unrelated APIs can pass collection interfaces back and forth.
  - Decreases extra effort required to learn, use, and design new API's.
  - Supports reusability of standard data structures and algorithms.

Interface	Description
Collection	Enables you to work with groups of objects; it is at the top of the collections hierarchy.
Deque	Extends **Queue** to handle a double-ended queue.
List	Extends **Collection** to handle sequences (lists of objects).
NavigableSet	Extends **SortedSet** to handle retrieval of elements based on closest-match searches.
Queue	Extends **Collection** to handle special types of lists in which elements are removed only from the head.
Set	Extends **Collection** to handle sets, which must contain unique elements.
SortedSet	Extends **Set** to handle sorted sets.



Java Collections: Interface

- **Iterator interface:** Iterator is an interface that iterates the elements. It is used to traverse the list and modify the elements. Iterator interface has three methods which are mentioned below:
  - ***public boolean hasNext()*** – This method returns true if the iterator has more elements.
  - ***public object next ()*** – It returns the element and moves the cursor pointer to the next element.
  - ***public void remove ()*** – This method removes the last elements returned by the iterator.

Java collections: List

- A List is an ordered Collection of elements which may contain duplicates. It is an interface that extends the Collection interface. Lists are further classified into the following:
  1. ArrayList
  2. LinkedList
  3. Vectors

List

- List in Java provides the facility to maintain the ordered collection. It contains the index-based methods to insert, update, delete and search the elements. It can have the duplicate elements also. We can also store the null elements in the list.
- The List interface is found in the ***java.util*** package and inherits the Collection interface.
- It is a factory of ***ListIterator*** interface. Through the ***ListIterator***, we can iterate the list in forward and backward directions. The implementation classes of List interface are ***ArrayList, LinkedList, Stack and Vector***.
- The ArrayList and LinkedList are widely used in Java programming.
- ***The Vector class is deprecated since Java 5.***

How to create List

The List interface is declared as:

public interface List<E> extends Collection<E>;

- Creating a List of type String using ArrayList
  List<String> list=new ArrayList<String> ();
- Creating a List of type Integer using ArrayList
  List<Integer> list=new ArrayList<Integer> ();
- Creating a List of type Book using ArrayList
  List<Book> list=new ArrayList<Book> ();
- Creating a List of type String using LinkedList
  List<String> list=new LinkedList<String> ();

Methods of List

- Some of the commonly used methods of the Collection interface that's also available in the List interface are:
  - ***add()*** - adds an element to a list
  - ***addAll()*** - adds all elements of one list to another
  - ***get()*** - helps to randomly access elements from lists
  - ***iterator()*** - returns iterator object that can be used to sequentially access elements of lists
  - ***set()*** - changes elements of lists
  - ***remove()*** - removes an element from the list
  - ***removeAll()*** - removes all the elements from the list
  - ***clear()*** - removes all the elements from the list (more efficient than removeAll())
  - ***size()*** - returns the length of lists
  - ***toArray()*** - converts a list into an array
  - ***contains()*** - returns true if a list contains specified element

```java
//ListExample.java
import java.util.*;
class ListExample {
    public static void main(String[] args) {
        // Creating list using the ArrayList class
        List<Integer> num = new ArrayList<>();
        // Add elements to the list
        num.add(100); num.add(200); num.add(300);
        num.add(400); num.add(200); num.add(500);
        System.out.println("List: " + num);
        // Access element from the list
        System.out.println("Accessed Element: " + num.get(2));
        // Remove element from the list  present at 1st index
        num.remove(1);
        System.out.println("List After Removed from 1st index: " + num);
        // Creating List class object
        List<String> str = new ArrayList<>();
        str.add("Hello"); str.add("Java"); str.add("Programming");
        // Adding For at 1st indexes
        str.add(1, "Basic");
        System.out.println("String List: " + str);
        // access particular element
        System.out.print("3rd index string: "+str.get(3));
    }
}
```

```
C:\Users\balkr\Documents\Java\Collections>java ListExample
List: [100, 200, 300, 400, 200, 500]
Accessed Element: 300
List After Removed from 1st index: [100, 300, 400, 200, 500]
String List: [Hello, Basic, Java, Programming]
3rd index string: Programming
```

Java ArrayList

- ArrayList is the implementation of List Interface where the elements can be dynamically added or removed from the list. Also, the size of the list is increased dynamically if the elements are added more than the initial size.
- The important points about the ***Java ArrayList*** class are:
  - Java ***ArrayList*** class can contain duplicate elements.
  - Java ***ArrayList*** class maintains insertion order.
  - Java ***ArrayList*** class is non synchronized.
  - Java ***ArrayList*** allows random access because the array works on an index basis.
  - In ArrayList, manipulation is a little bit slower than the LinkedList in Java because a lot of shifting needs to occur if any element is removed from the array list.
  - We cannot create an array list of the primitive types, such as int, float, char, etc. It is required to use the required wrapper class in such cases. For example:
    - ✓ ***ArrayList<int> al = ArrayList<int>(); // does not work***
    - ✓ ***ArrayList<Integer> al = new ArrayList<Integer>(); // works fine***
  - Java ***ArrayList*** gets initialized by the size. The size is dynamic in the array list, which varies according to the elements getting added or removed from the list.

ArrayList has the constructors shown here:

ArrayList( )
ArrayList(Collection<? extends E> *c*)
ArrayList(int *capacity*)

The first constructor builds an empty array list. The second constructor builds an array list that is initialized with the elements of the collection c. The third constructor builds an array list that has the specified initial *capacity*. The capacity is the size of the underlying array that is used to store the elements. The capacity grows automatically as elements are added to an array list.

```
1  //ArrayListExample.java
2  import java.util.*;
3  class ArrayListExample{
4      public static void main(String args[]){
5          ArrayList<String> alist=new ArrayList<String>();
6          //adding elements
7          alist.add("Ram Sharma"); alist.add("Hari Karki");
8          alist.add("Sita Nepal"); alist.add("Gita Thapa");
9          //length of ArrayList
10         System.out.println("Length of list: "+ alist.size());
11         //Printing the arraylist object
12         System.out.println(alist);
13         System.out.println("### After Removing element #####");
14         //remove the item from arraylist
15         alist.remove("Sita Nepal");
16         //Traversing list through Iterator
17         Iterator itr=alist.iterator();
18         while(itr.hasNext())
19             System.out.println(itr.next());
20         //accessing the element.....return the 2nd element.
21         System.out.println("2nd element: " + alist.get(1));
22         System.out.println("### After Sorting Element #####");
23         //changing the element
24         alist.set(1,"Hari Nepal");
25         //Sorting the list
26         Collections.sort(alist);
27         //Traversing list through for-each loop
28         for(String name:alist)
29             System.out.println(name);
30         //create an ArrayList
31         ArrayList<String> myList = new ArrayList<String>();
32         //use addAll() method to add list elements to ArrayList
33         myList.addAll(alist);
34         System.out.println("### From another ArrayList #####");
35         for(String name:myList)
36             System.out.println(name);
37     }
38 }
```

```
C:\Users\balkr\Documents\Java\Collections>java ArrayListExample
Length of list: 4
[Ram Sharma, Hari Karki, Sita Nepal, Gita Thapa]
### After Removing element #####
Ram Sharma
Hari Karki
Gita Thapa
2nd element: Hari Karki
### After Sorting Element #####
Gita Thapa
Hari Nepal
Ram Sharma
### From another ArrayList #####
Gita Thapa
Hari Nepal
Ram Sharma
```

The LinkedList Class

The **LinkedList** class extends **AbstractSequentialList** and implements the **List**, **Deque**, and **Queue** interfaces. It provides a linked-list data structure. **LinkedList** is a generic class that has this declaration:

```
class LinkedList<E>
```

Here, **E** specifies the type of objects that the list will hold. **LinkedList** has the two constructors shown here:

```
LinkedList( )
LinkedList(Collection<? extends E> c)
```

The first constructor builds an empty linked list. The second constructor builds a linked list that is initialized with the elements of the collection *c*.

- Java LinkedList class uses a doubly linked list to store the elements. It provides a linked-list data structure. It inherits the ***AbstractList*** class and implements List and Deque interfaces.
- The important points about Java LinkedList are:
  - Java LinkedList class can contain duplicate elements.
  - Java LinkedList class maintains insertion order.
  - Java LinkedList class is non synchronized.
  - In Java LinkedList class, manipulation is fast because no shifting needs to occur.
  - Java LinkedList class can be used as a list, stack or queue.
- Creating a Java LinkedList

  LinkedList<Type> linkedList = new LinkedList<>();
- Create Integer type linked list

  LinkedList<Integer> linkedList = new LinkedList<>();

- Create String type linked list

  LinkedList<String> linkedList = new LinkedList<>();

```java
//LinkedListExample.java
import java.util.LinkedList;
class LinkedListExample {
    public static void main(String[] args){
        LinkedList<String> animals = new LinkedList<>();
        // Add elements to LinkedList
        animals.add("Dog");  animals.add("Tiger");
        animals.add("Lion");   animals.add("Cat");
        System.out.println("Linked List: " + animals);
        // add() method with the index parameter
        animals.add(1, "Horse");
        System.out.println("Updated LinkedList: " + animals);
        // get the element from the linked list
        String str = animals.get(1);
        System.out.println("Element at index 1: " + str);
        // change elements at index 3
        animals.set(3, "Big Lion");
        System.out.println("Changed at index 3: " + animals);
        // remove elements from index 1
        str = animals.remove(1);
        System.out.println("Removed Element: " + str);
        System.out.println("Updated LinkedList: " + animals);
        // Using the for each loop
        System.out.print("Linked List using For each loop: " );
        for (String animal : animals)
            System.out.print(animal + "   ");
    }
}
```

```
C:\Users\balkr\Documents\Java\Collections>java LinkedListExample
Linked List: [Dog, Tiger, Lion, Cat]
Updated LinkedList: [Dog, Horse, Tiger, Lion, Cat]
Element at index 1: Horse
Changed at index 3: [Dog, Horse, Tiger, Big Lion, Cat]
Removed Element: Horse
Updated LinkedList: [Dog, Tiger, Big Lion, Cat]
Linked List using For each loop: Dog   Tiger   Big Lion   Cat
```

Difference between ArrayList and LinkedList in Java

ArrayList	LinkedList
ArrayList internally uses a dynamic array to store the elements	LinkedList internally uses a doubly linked list to store the elements
Manipulation with ArrayList is slow because it internally uses an array. If any element is removed from the array, all the bits are shifted in memory.	Manipulation with LinkedList is faster than ArrayList because it uses a doubly linked list, so no bit shifting is required in memory.
ArrayList consumes less memory than LinkedList	A LinkedList consumes more memory than an ArrayList because it also stores the next and previous references along with the data.
An ArrayList class can **act as a list** only because it implements List only.	LinkedList class can **act as a list and queue** both because it implements List and Deque interfaces.
ArrayList is better for storing and accessing data.	LinkedList is **better for manipulating** data.

Set in Java

- The set interface is present in ***java.util*** package and extends the Collection interface.
- It is an unordered collection of objects in which ***duplicate values cannot be stored***.
- It is an interface that implements the mathematical set.
- This interface contains the methods inherited from the Collection interface and adds a feature that restricts the insertion of the duplicate elements.
- There are two interfaces that extend the set implementation namely ***SortedSet*** and ***NavigableSet***.
- Creating Set Objects

  Set<Obj> set = new HashSet<Obj> ();
- ***Methods present in the Set interface***
  - ***add(element):*** This method is used to add a specific element to the set.
  - ***addAll(collection):*** This method is used to append all of the elements from the mentioned collection to the existing set. The elements are added randomly without following any specific order.
  - ***clear():***This method is used to remove all the elements from the set but not delete the set. The reference for the set still exists.
  - ***contains(element):*** This method is used to check whether a specific element is present in the Set or not.
  - ***containsAll(collection):*** This method is used to check whether the set contains all the elements present in the given collection or not.
  - ***isEmpty():***This method is used to check whether the set is empty or not.
  - ***iterator():*** This method is used to return the iterator of the set. The elements from the set are returned in a random order.
  - ***remove(element):*** This method is used to remove the given element from the set. This method returns True if the specified element is present in the Set otherwise it returns False.
  - ***removeAll(collection):*** This method is used to remove all the elements from the collection which are present in the set.
  - ***size():*** This method is used to get the size of the set.
  - ***toArray():***This method is used to form an array of the same elements as that of the Set.

HashSet

- Java HashSet class is used to create a collection that uses a hash table for storage. It inherits the ***AbstractSet*** class and implements Set interface.
- The important points about Java HashSet class are:
  - The underlying data structure for HashSet is Hashtable.
  - HashSet stores the elements by using a mechanism called hashing.
  - As it implements the Set Interface, duplicate values are not allowed.
  - HashSet allows null value.
  - Objects that you insert in HashSet are not guaranteed to be inserted in the same order. Objects are inserted based on their hash code.
  - HashSet also implements Serializable and Cloneable interfaces.
  - HashSet is the best approach for search operations.

- HashSet default initial capacity is 16 and the default load factor is 0.75.

```java
//SetExample.java
import java.io.*;
import java.util.*;
class SetExample {
    public static void main(String[] args) {
        Set<Integer> set1 = new HashSet<>();
        set1.add(20); set1.add(30); set1.add(40);
        set1.add(50); set1.add(60);
        System.out.println("Set1: " + set1);

        Set<Integer> set2 = new HashSet<>();
        set2.add(100); set2.add(200); set2.add(300);
        System.out.println("Set2: " + set2);
        System.out.println("Set contains 9o?:  " + set2.contains(90));
        // Union of two sets
        set2.addAll(set1);
        System.out.println("Union is: " + set2);
        //clear the set 1
        set1.clear();
        System.out.println("Set1 after clear(): " + set1);
        System.out.println("Is set1 empty?: "+ set1.isEmpty());
        System.out.println("Is set2 empty?: "+ set2.isEmpty());

        System.out.println("size of the Set2 : " + set2.size());
        //remove element
        set2.remove(100);
        System.out.print("Set1 after removing elements:  ");
        Iterator itr = set2.iterator();
        while (itr.hasNext())
            System.out.print("  "+itr.next());
    }
}
```

```
C:\Users\balkr\Documents\Java\Collections>java SetHashSetExample
Set1: [50, 20, 40, 60, 30]
Set2: [100, 200, 300]
Set contains 9o?:  false
Union is: [50, 100, 20, 200, 40, 300, 60, 30]
Set1 after clear(): []
Is set1 empty?: true
Is set2 empty?: false
size of the Set2 : 8
Set1 after removing elements:    50  20  200  40  300  60  30
```

```java
//HashSetExample.java
import java.util.*;
class HashSetExample {
    public static void main(String[] args) {
        HashSet<Integer> num = new HashSet<>();
        num.add(5); num.add(25); num.add(30);
        System.out.println("HashSet: " + num);

        HashSet<Integer> num2 = new HashSet<>();
        // addAll() method.....Union of two set
        num2.addAll(num);  num2.add(3); num2.add(6);
        System.out.println("New HashSet: " + num2);
        // Calling iterator() method
        Iterator<Integer> iterate = num2.iterator();
        System.out.print("HashSet using Iterator: ");
        while(iterate.hasNext())
            System.out.print(iterate.next()+ ", ");
        // Intersection of two sets
        num.retainAll(num2);
        System.out.println("\nIntersection is: " + num);
```

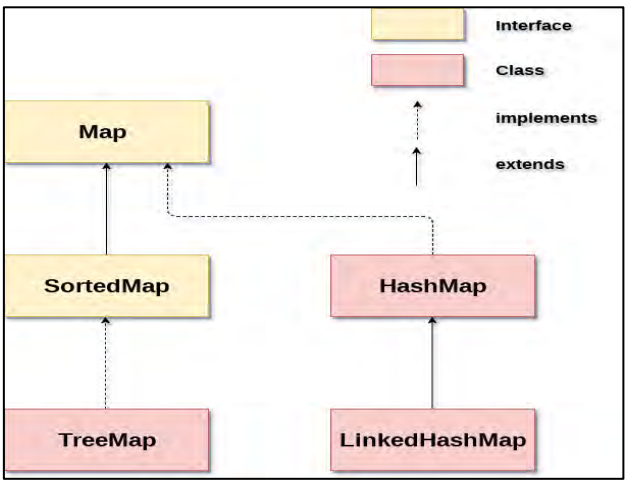
```
22          HashSet<Integer> prime = new HashSet<>();
23          prime.add(2);  prime.add(3); prime.add(5);
24          System.out.println("HashSet1: " + prime);
25
26          HashSet<Integer> odd = new HashSet<>();
27          odd.add(1);odd.add(3);odd.add(5);odd.add(7);
28          System.out.println("HashSet2: " + odd);
29          // Difference between HashSet1 and HashSet2
30          prime.removeAll(odd);
31          System.out.println("Difference : " + prime);
32      }
33  }
```

```
C:\Users\balkr\Documents\Java\Collections>java HashSetExample
HashSet: [5, 25, 30]
New HashSet: [3, 5, 6, 25, 30]
HashSet using Iterator: 3, 5, 6, 25, 30,
Intersection is: [5, 25, 30]
HashSet1: [2, 3, 5]
HashSet2: [1, 3, 5, 7]
Difference : [2]
```

Java Map Interface

- The map interface is present in ***java.util*** package represents a mapping between a key and a value.
- A map contains values on the basis of key, i.e. key and value pair. Each key and value pair is known as an entry. ***A Map contains unique keys***.
- Maps are perfect to use for key-value association mapping such as dictionaries. The maps are ***used to perform lookups by keys*** or when someone ***wants to retrieve and update elements by keys***. Some common scenarios are as follows:
  - A map of error codes and their descriptions.
  - A map of zip codes and cities.
  - A map of managers and employees. Each manager (key) is associated with a list of employees (value) he manages.
  - A map of classes and students. Each class (key) is associated with a list of students (value).
- ***Characteristics of a Map Interface***
  - A Map cannot contain duplicate keys and each key can map to at most one value. Some implementations allow null key and null values like the HashMap and LinkedHashMap, but some do not like the TreeMap.
  - The order of a map depends on the specific implementations. For example, TreeMap and LinkedHashMap have predictable orders, while HashMap does not.

- There are two interfaces for implementing Map in java. They are Map and SortedMap, and three classes: HashMap, TreeMap, and LinkedHashMap.
- **Java Map Hierarchy:** There are two interfaces for implementing Map in java: Map and SortedMap, and three classes: HashMap, LinkedHashMap, and TreeMap. The hierarchy of Java Map is given below:



- **Map implementation using HashMap**

Map<Key, Value> numbers = new HashMap<>();

```java
//HashMapExample.java
import java.util.*;
class HashMapExample {
    public static void main(String[] args) {
        HashMap<String, Integer> numbers = new HashMap<>();
        numbers.put("One", 1); numbers.put("Two", 2);
        numbers.put("Three", 3); numbers.put("Four", 4);
        System.out.println("Map: " + numbers);
        // Access keys of the map
        System.out.println("Keys: " + numbers.keySet());
        // Access values of the map
        System.out.println("Values: " + numbers.values());
        // Access entries of the map
        System.out.println("Entries: " + numbers.entrySet());
        // Remove Elements from the map
        int value = numbers.remove("Two");
        System.out.println("Removed Value: " + value);
        // Traversing through Map using for-each loop
        for (Map.Entry<String, Integer> hmap :numbers.entrySet()) {
            System.out.print(hmap.getKey() + ":");
            System.out.println(hmap.getValue()); }
    }
}
```

```
C:\Users\balkr\Documents\Java\Collections>java HashMapExample
Map: {One=1, Four=4, Two=2, Three=3}
Keys: [One, Four, Two, Three]
Values: [1, 4, 2, 3]
Entries: [One=1, Four=4, Two=2, Three=3]
Removed Value: 2
One:1
Four:4
Three:3
```

String Handling

- String is an object that represents sequence of characters. In Java, String is represented by String class which is located into ***java.lang package***
- It is probably the most commonly used class in java library. In java, every string that we create is actually an object of type String.
- One important thing to notice about string object is that ***string objects are immutable*** that means once a string object is created it cannot be changed.
- The Java String class implements ***Serializable, Comparable and CharSequence*** interface that we have represented using the below image.
- In Java***, CharSequence Interface*** is used for representing a sequence of characters. CharSequence interface is implemented by ***String, StringBuffer and StringBuilder*** classes. This three classes can be used for creating strings in java.

How to create a string object?

- There are two ways to create String object:
    1. By string literal
    2. By new keyword

1. **String Literal:** Java ***String literal*** is created by ***using double quotes***. For Example:

String s="Java String";

- Each time you create a string literal, the JVM checks ***the "string constant pool" first***. If the string already exists in the pool, a reference to the pooled instance is returned. If the string doesn't exist in the pool, a new string instance is created and placed in the pool. For example:
    - *String s1="Welcome";*
    - *String s2="Welcome"; //* It doesn't create a new instance

2. **By new keyword:** We can create a new string object by using new operator that allocates memory for the object.
    - *String s=new String("Welcome");*

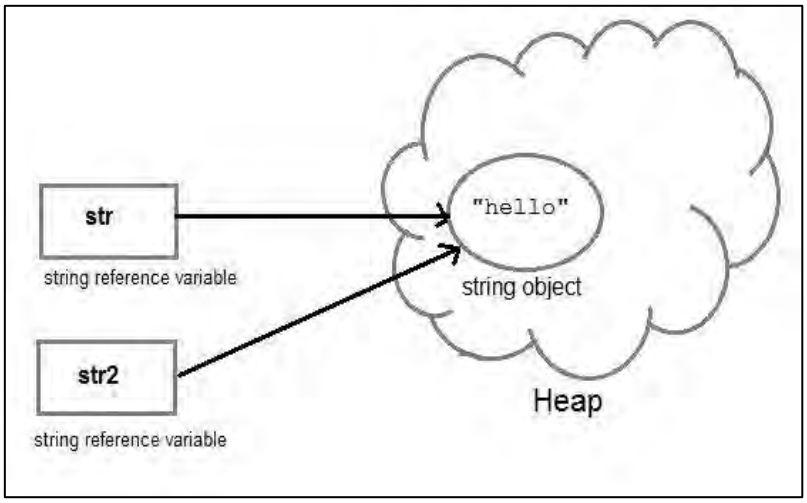
- In such case, JVM will create a new string object in normal (non-pool) heap memory, and the literal "Welcome" will be placed in the string constant pool. The variable s will refer to the object in a heap (non-pool).

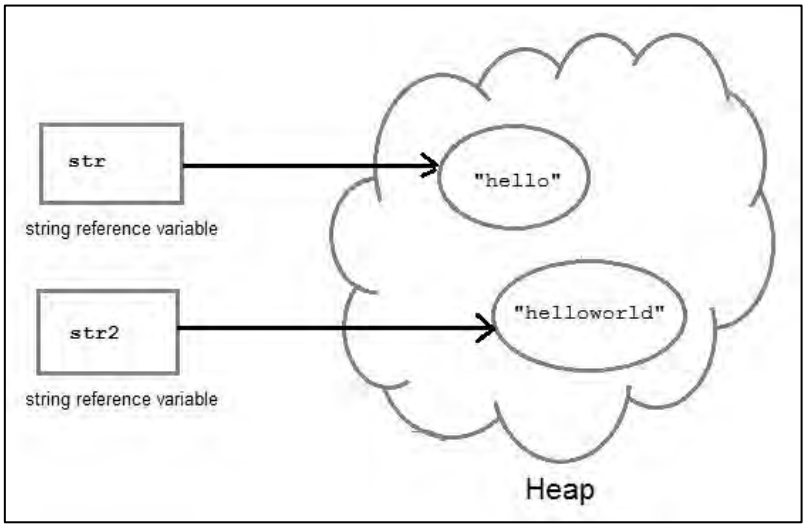
What is an Immutable object?

- An object whose state cannot be changed after it is created is known as an Immutable object.
- The Java String is immutable which means it cannot be changed. Whenever we change any string, a new instance is created. For mutable strings, you can use StringBuffer and StringBuilder classes.

Why string objects are immutable in java?

- Because java uses the concept of string literal. Suppose there are 5 reference variables, all refers to one object “String immutable”. If one reference variable changes the value of the object, it will be affected to all the reference variables. That is why string objects are immutable in java.





The String Constructors

The **String** class supports several constructors. To create an empty **String**, call the default constructor. For example,

```
String s = new String();
```

will create an instance of **String** with no characters in it.

Frequently, you will want to create strings that have initial values. The **String** class provides a variety of constructors to handle this. To create a **String** initialized by an array of characters, use the constructor shown here:

String(char *chars*[ ])

Here is an example:

```
char chars[] = { 'a', 'b', 'c' };
String s = new String(chars);
```

This constructor initializes **s** with the string "abc".

You can specify a subrange of a character array as an initializer using the following constructor:

String(char *chars*[ ], int *startIndex*, int *numChars*)

Here, *startIndex* specifies the index at which the subrange begins, and *numChars* specifies the number of characters to use. Here is an example:

```
char chars[] = { 'a', 'b', 'c', 'd', 'e', 'f' };
String s = new String(chars, 2, 3);
```

This initializes **s** with the characters **cde**.

You can construct a **String** object that contains the same character sequence as another **String** object using this constructor:

String(String *strObj*)

Here, *strObj* is a **String** object. Consider this example:

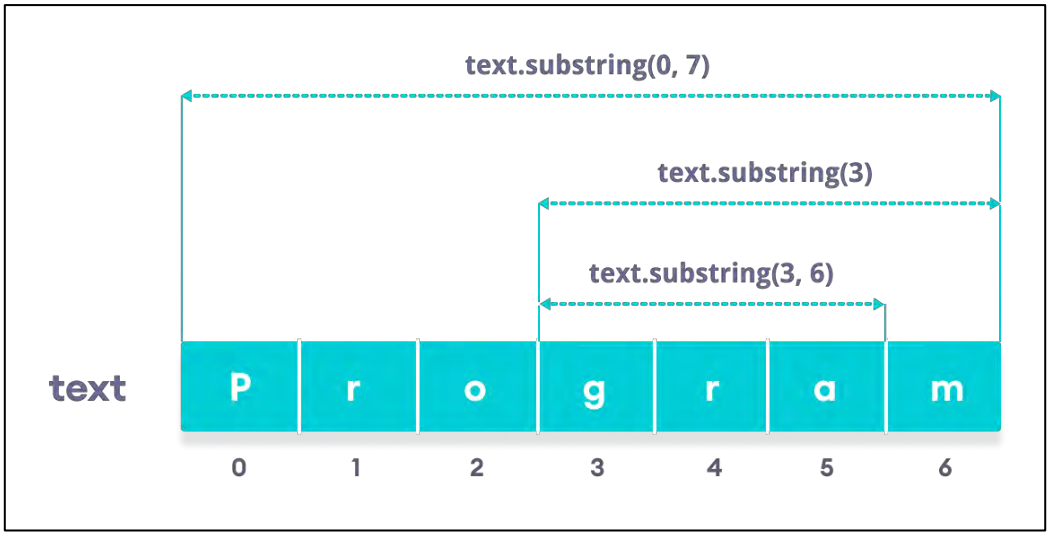
```
// Construct one String from another.
class MakeString {
  public static void main(String args[]) {
    char c[] = {'J', 'a', 'v', 'a'};
    String s1 = new String(c);
    String s2 = new String(s1);

    System.out.println(s1);
    System.out.println(s2);
  }
}
```

The output from this program is as follows:

```
Java
Java
```

As you can see, **s1** and **s2** contain the same string.

- **The syntax of the substring () method is:**
  string.substring(int startIndex, int endIndex)
- Here, string is an object of the String class.
- The ***substring ()*** method returns a substring from the given string.
- The substring begins with the character at the ***startIndex*** and extends to the character at index ***endIndex - 1***.
- If the ***endIndex*** is not passed, the substring begins with the character at the specified index and extends to the end of the string.



- **The syntax of the replace () method is either**

string.replace(char oldChar, char newChar)

or

string.replace(CharSequence oldText, CharSequence newText)

- To replace a single character, the ***replace ()*** method takes these two parameters:
  - ***oldChar*** - the character to be replaced in the string
  - ***newChar*** - matching characters are replaced with this character
- To replace a substring, the ***replace ()*** method takes these two parameters:
  - ***oldText*** - the substring to be replaced in the string
  - ***newText*** - matching substrings are replaced with this string

Java String indexOf()

- The indexOf() method returns the index of the first occurrence of the specified character/substring within the string.
- The syntax of the String indexOf() method either

string.indexOf(int ch, int fromIndex)

or

string.indexOf(String str, int fromIndex)

To find the index of a character, `indexOf()` takes these two parameters:

- **ch** - the character whose starting index is to be found
- **fromIndex** (optional) - if `fromIndex` is passed, the `ch` character is searched starting from this index

To find the index of the specified substring within the string, `indexOf()` takes these two parameters:

- **str** - the string whose starting index is to be found
- **fromIndex** (optional) - if `fromIndex` is passed, the `str` string is searched starting from this index

The `toLowerCase()` method converts all characters in the string to lowercase characters.

Syntax of toLowerCase()

The syntax of the string `toLowerCase()` method is:

```
string.toLowerCase()
```

Here, `string` is an object of the `String` class.

The Java String toUpperCase() method converts all characters in the string to upper case characters.

The syntax of the string `toUpperCase()` method is:

```
string.toUpperCase()
```

The `trim()` method removes any leading (starting) and trailing (ending) whitespaces from the specified string.

Syntax of trim()

The syntax of the string `trim()` method is:

```
string.trim()
```

Here, `string` is an object of the `String` class.

trim() Return Value

- returns a string with leading and trailing whitespace removed
- returns the original string if there is no whitespace in the start or the end of the string

The `replaceAll()` method replaces each substring that matches the regex of the string with the specified text.

Syntax of replaceAll()

The syntax of the `replaceAll()` method is:

```
string.replaceAll(String regex, String replacement)
```

Here, `string` is an object of the `String` class.

The `replaceAll()` method

- returns a new string where each occurrence of the matching substring is replaced with the **replacement** string.

```java
//StringExample.java
class StringExample{
    public static void main(String[] args) {
        //Using a String literal
        String str1 = "Hello Java";
        //Using new Keyword
        String str2 = new String("Hello Java");
        String str3 = new String("Programming.");
        System.out.println(str1);
        System.out.println(str2);

        // get the length of greet
        int len = str2.length();
        System.out.println("Length of String-2: " + len);

        // join two strings
        String catStr = str2.concat(str3);
        System.out.println("Concatinated String: " + catStr);

        //Using + operator
        String str4 = "One to one "+ 11111;
        System.out.println(str4);

```

```
24              // compare first and second strings
25              boolean result = str1.equals(str2);
26              System.out.println("Strings s1 and s2 are equal: " + result);
27
28              //compareTo() method
29              int cmp = str1.compareTo(str4);
30              System.out.println("Compare to method: " + cmp);
31
32              // substring() example
33              // from the 5th character to the end
34              System.out.println(str1.substring(4));
35              // from the 3th character to 8th
36              System.out.println(str1.substring(2, 7));
37
38              //replace() Syntax
39              // all occurrences of 'l' is replaced with 'L'
40              System.out.println(str1.replace('l', 'L'));
41              // all occurences of 'e' is replaced with 'E'
42              System.out.println("Elephant".replace('e', 'E'));
43              System.out.println("Java and C++".replace("C++", "Python"));
44      }
45  }
```

```
C:\Users\balkr\Documents\Java\Collections>java StringExample
Hello Java
Hello Java
Length of String-2: 10
Concatinated String: Hello JavaProgramming.
One to one 11111
Strings s1 and s2 are equal: true
Compare to method: -7
o Java
llo J
HeLLo Java
ElEphant
Java and Python
```

```
1   //StringExample2.java
2   class StringExample2 {
3       public static void main(String[] args) {
4           String str1 = "Advanced Java Programming.";
5           int result;
6           result = str1.indexOf('P');
7           System.out.println(" Index of P: "+result);
8           // character not in the string
9           result = str1.indexOf('z');
10          System.out.println("Character not in the string: "+result);
11          // getting the index of substring
12          result = str1.indexOf("van");
13          System.out.println("Index of substring: "+result);
14          // getting the index of substring start from 10 index
15          result = str1.indexOf("van",10);
16          System.out.println("Index of substring from 10 index: "+result);
17          // substring not in the string
18          result = str1.indexOf("kava");
19          System.out.println("Substring not in the string:  "+result);
20
```

```
21          // convert to upper case letters
22          String str2=str1.toUpperCase();
23          System.out.println("toUpperCase()of str1: "+ str2);
24          String str3=str2.toLowerCase();
25          System.out.println("toLowerCase()of str2: "+ str3);
26
27          //Java String trim()
28          String str4 = "     Learn   Java Programming ";
29          System.out.println("After trim(): "+str4.trim());
30
31          //Java String replaceAll()
32          String str5 = "aaqqaaaabbaasdacsdfs";
33          String str6 = "Learn22334Java57675@@#$%";
34          // regex for sequence of digits
35          String regex = "\\d+";
36          // all occurrences of "aa" is replaceAll with "zz"
37          System.out.println(str5.replaceAll("aa", "@@"));
38          System.out.println(str6.replaceAll(regex, " "));
39      }
40  }
```

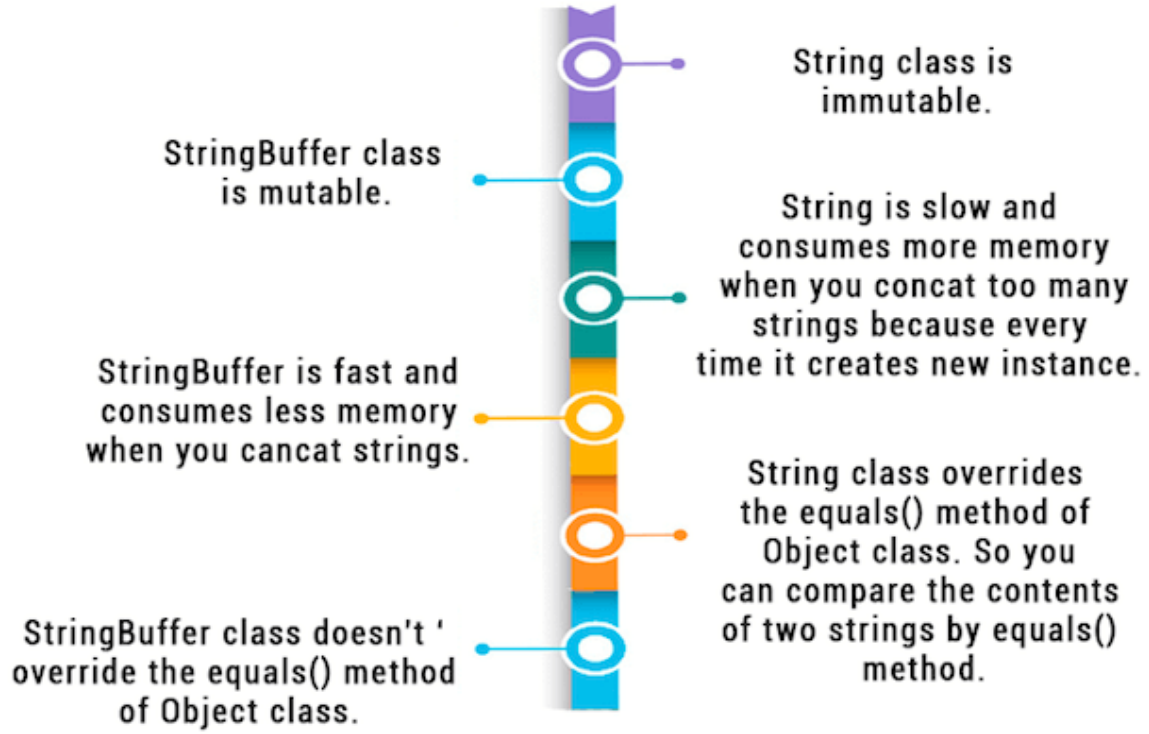
```
C:\Users\balkr\Documents\Java\Collections>java StringExample2
 Index of P: 14
Character not in the string: -1
Index of substring: 2
Index of substring from 10 index: -1
Substring not in the string:  -1
toUpperCase()of str1: ADVANCED JAVA PROGRAMMING.
toLowerCase()of str2: advanced java programming.
After trim(): Learn   Java Programming
@@qq@@@@bb@@sdacsdfs
Learn Java @@#$%
```

String buffer and String builder

- String buffer and StringBuilder both are mutable classes which can be used to do operation on string objects such as reverse of string, concating string and etc.
- We can modify string without creating a new object of the string.

StringBuffer	StringBuilder
1. Every method present in stringBuffer is synchronized.	1. No method present in stringBuilder is synchronized.
2. At a time only one thread is allowed to operate on a StringBuffer object and hence it is thread safe.	2. At a time multiple threads are allowed to operate on a StringBuilder object and hence it is not thread safe.
3. Threads are required to wait to operate on StringBuffer object and hence relatively performance is slow	3. Threads are not required to wait to operate on StringBuffer object and hence relatively performance is high
4. Introduced in 1.0 version	4. Introduced in 1.5 version

StringBuffer vs String




```java
//StringBufferExample.java
class StringBufferExample {
    public static void main(String[] args) {
        // create a string buffer
        StringBuffer str = new StringBuffer();
        // add string to string buffer
        str.append("Basic");
        str.append(" Java");
        str.append(" Programming.");
        System.out.println("String Buffer: " + str);
        // clear the string using new and  here new object is created and assigned to str.
        str = new StringBuffer();
        System.out.println("Updated StringBuffer: " + str);

        StringBuilder str2 = new StringBuilder("Basic Java");
        str2.append(" Programming.");
        System.out.println("Updated StringBuilder: " + str2);
    }
}
```

```
C:\Users\balkr\Documents\Java\Collections>java StringBufferExample
String Buffer: Basic Java Programming.
Updated StringBuffer:
Updated StringBuilder: Basic Java Programming.
```

Calendar Class in Java

- Calendar class in Java is an abstract class that ***provides methods for converting date between a specific instant*** in time and a set of calendar fields such as ***MONTH, YEAR, HOUR***, etc.
- It inherits Object class and implements the ***Comparable, Serializable, Cloneable interfaces***.
- The ***java.util.calendar*** class is an abstract class that provides methods for converting between a specific instant in time and a set of calendar fields such as YEAR, MONTH, DAY_OF_MONTH, HOUR, and so on, and for manipulating the calendar fields, such as getting the date of the next week.
- Following are the important points about Calendar –
  - This class also provides additional fields and methods for implementing a concrete calendar system outside the package.
  - Calendar defines the range of values returned by certain calendar fields.

static Calendar getInstance( )	Returns a **Calendar** object for the default locale and time zone.
static Calendar getInstance(TimeZone *tz*)	Returns a **Calendar** object for the time zone specified by *tz*. The default locale is used.
static Calendar getInstance(Locale *locale*)	Returns a **Calendar** object for the locale specified by *locale*. The default time zone is used.
static Calendar getInstance(TimeZone *tz*, Locale *locale*)	Returns a **Calendar** object for the time zone specified by *tz* and the locale specified by *locale*.
final Date getTime( )	Returns a **Date** object equivalent to the time of the invoking object.
TimeZone getTimeZone( )	Returns the time zone for the invoking object.
final void set(int *year*, int *month*, int *dayOfMonth*)	Sets various date and time components of the invoking object.
final void set(int *year*, int *month*, int *dayOfMonth*, int *hours*, int *minutes*)	Sets various date and time components of the invoking object.
final void set(int *year*, int *month*, int *dayOfMonth*, int *hours*, int *minutes*, int *seconds*)	Sets various date and time components of the invoking object.
final void setTime(Date *d*)	Sets various date and time components of the invoking object. This information is obtained from the **Date** object *d*.
void setTimeZone(TimeZone *tz*)	Sets the time zone for the invoking object to that specified by *tz*.
final Instant toInstant( )	Returns an **Instant** object corresponding to the invoking **Calendar** instance.

Table 20-5 A Sampling of the Methods Defined by **Calendar**

```java
//CalendarExample.java
import java.util.*;
import java.text.SimpleDateFormat;
public class CalendarExample {
    public static void main(String args[]){
        SimpleDateFormat dateFormat = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
        Calendar calendar = Calendar.getInstance();

        System.out.println("The Current Date is: " + dateFormat.format(calendar.getTime()));
        System.out.println("Type of the calendar is: "+calendar.getCalendarType());
        System.out.println("Current Year is: " + calendar.get(Calendar.YEAR));
        System.out.println("Current month is :"+ calendar.get(Calendar.MONTH));
        System.out.println("Current Date is: " + calendar.get(Calendar.DATE));
        System.out.println("Current day is :"+ calendar.get(Calendar.DAY_OF_WEEK));
        System.out.println("Current hour is :"+ calendar.get(Calendar.HOUR_OF_DAY));
        System.out.println("Current MINUTE: " + calendar.get(Calendar.MINUTE));
        System.out.println("Current SECOND: " + calendar.get(Calendar.SECOND));
        System.out.println("MILLISECOND: " + calendar.get(Calendar.MILLISECOND));
        System.out.println("AM_PM: " + calendar.get(Calendar.AM_PM));
        System.out.println("Current Week of year: " + calendar.get(Calendar.WEEK_OF_YEAR));
        System.out.println("Current Week of month: " + calendar.get(Calendar.WEEK_OF_MONTH));
    }
}
```

```
C:\Users\balkr\Documents\Java\Collections>java CalendarExample
The Current Date is: 2022-12-15 21:01:04
Type of the calendar is: gregory
Current Year is: 2022
Current month is :11
Current Date is: 15
Current day is :5
Current hour is :21
Current MINUTE: 1
Current SECOND: 4
MILLISECOND: 478
AM_PM: 1
Current Week of year: 51
Current Week of month: 3
```

```java
//CalendarExample2.java
import java.util.*;
import java.text.SimpleDateFormat;
public class CalendarExample2 {
    public static void main(String[] args) {
        Calendar calendar = new GregorianCalendar();
        System.out.println("Current date is : " + calendar.getTime());
        int min = calendar.getMinimum(Calendar.DAY_OF_WEEK);
        System.out.println("Minimum number of days in week: " + min);
        min = calendar.getMinimum(Calendar.WEEK_OF_YEAR);
        System.out.println("Minimum number of weeks in year: " + min);
        int max = calendar.getMaximum(Calendar.DAY_OF_WEEK);
        System.out.println("Maximum number of days in week: " + max);
        max = calendar.getMaximum(Calendar.WEEK_OF_YEAR);
        System.out.println("Maximum number of weeks in year: " + max);

```

```
17          calendar.add(Calendar.DATE, -15);
18          System.out.println("15 days ago: " + calendar.getTime());
19          calendar.add(Calendar.MONTH,2);
20          System.out.println("After 2 months  : "+ calendar.getTime());
21          calendar.add(Calendar.YEAR, 4);
22          System.out.println("After 4 years  : "+ calendar.getTime());
23          calendar.add(Calendar.YEAR, -12);
24          System.out.println("Before 12 years  : "+ calendar.getTime());
25      }
26  }
```

```
C:\Users\balkr\Documents\Java\Collections>java CalendarExample2
Current date is : Thu Dec 15 21:14:04 NPT 2022
Minimum number of days in week: 1
Minimum number of weeks in year: 1
Maximum number of days in week: 7
Maximum number of weeks in year: 53
15 days ago: Wed Nov 30 21:14:04 NPT 2022
After 2 months  : Mon Jan 30 21:14:04 NPT 2023
After 4 years  : Sat Jan 30 21:14:04 NPT 2027
Before 12 years  : Fri Jan 30 21:14:04 NPT 2015
```

Java SimpleDateFormat

- Java ***SimpleDateFormat*** class is used for formatting date and time.
- A ***SimpleDateFormat*** is a concrete class for formatting and parsing dates in a locale-sensitive manner. It allows for ***formatting*** (date -> text), ***parsing*** (text -> date), and ***normalization***.
- A ***SimpleDateFormat*** allows you to start by choosing any user-defined patterns for date-time formatting.

Constructor and Description
`SimpleDateFormat()` Constructs a `SimpleDateFormat` using the default pattern and date format symbols for the default locale.
`SimpleDateFormat(String pattern)` Constructs a `SimpleDateFormat` using the given pattern and the default date format symbols for the default locale.
`SimpleDateFormat(String pattern, DateFormatSymbols formatSymbols)` Constructs a `SimpleDateFormat` using the given pattern and date format symbols.
`SimpleDateFormat(String pattern, Locale locale)` Constructs a `SimpleDateFormat` using the given pattern and the default date format symbols for the given locale.

Date Format in Java

- Date, original version, defined by Java 1.0. When Java 1.1 was released, many of the functions carried out by the original Date class were moved into the ***Calendar*** and ***DateFormat classes***, and as a result, many of the ***original 1.0 Date methods were deprecated***. Since the deprecated 1.0 methods should not be used for new code.

- The ***DateFormat*** class in Java is used for formatting dates. A specified date can be formatted into the Data/Time string. For example, a date can be formatted into: ***mm/dd/yyyy***.
- ***DateFormat*** is an abstract class for ***date/time formatting*** subclasses which formats and parses dates or time in a ***language-independent manner***.
- The date/time formatting subclass, such as ***SimpleDateFormat***, allows for formatting (i.e., date -> text), parsing (text -> date), and normalization.
- The date is represented as a Date object or as the milliseconds since January 1, 1970, 00:00:00 GMT.
- The formatting styles include FULL, LONG, MEDIUM, and SHORT.

  - `SHORT` is completely numeric, such as `12.13.52` or `3:30pm`
  - `MEDIUM` is longer, such as `Jan 12, 1952`
  - `LONG` is longer, such as `January 12, 1952` or `3:30:32pm`
  - `FULL` is pretty completely specified, such as `Tuesday, April 12, 1952 AD or 3:30:42pm PST.`
- Date Format classes are not synchronized.

```java
//SimpleDateFormatExample.java
import java.text.SimpleDateFormat;
import java.util.Date;
class SimpleDateFormatExample {
    public static void main(String args[]){
        Date date = new Date();
        SimpleDateFormat dateFormat = new SimpleDateFormat("dd-MM-yyyy");
        String stringDate = dateFormat.format(date);
        System.out.println("Date Format with dd-MM-yyyy: "+ stringDate);

        dateFormat = new SimpleDateFormat("MM/dd/yyyy");
        stringDate = dateFormat.format(date);
        System.out.println("Date Format with MM/dd/yyyy : "+ stringDate);

        dateFormat = new SimpleDateFormat("dd-M-yyyy hh:mm:ss");
        stringDate = dateFormat.format(date);
        System.out.println("Date Format with dd-M-yyyy hh:mm:ss : "+ stringDate);
        //MMMM means Month Name and MMM means Month name in Short.
        dateFormat = new SimpleDateFormat("dd MMMM yyyy");
        stringDate = dateFormat.format(date);
        System.out.println("Date Format with dd MMMM yyyy : "+ stringDate);
        //z for Time zon --Eastern Standard Time
        dateFormat = new SimpleDateFormat("dd MMMM yyyy zzzz");
        stringDate = dateFormat.format(date);
        System.out.println("Date Format with dd MMMM yyyy zzzz : "+ stringDate);
        //E for Day in week
        dateFormat = new SimpleDateFormat("E, dd MMM yyyy HH:mm:ss z");
        stringDate = dateFormat.format(date);
        System.out.println("Date Format with E, dd MMM yyyy HH:mm:ss z :"+ stringDate);
    }
}
```

```
C:\Users\balkr\Documents\Java\Collections>java SimpleDateFormatExample
Date Format with dd-MM-yyyy: 15-12-2022
Date Format with MM/dd/yyyy : 12/15/2022
Date Format with dd-M-yyyy hh:mm:ss : 15-12-2022 10:09:52
Date Format with dd MMMM yyyy : 15 December 2022
Date Format with dd MMMM yyyy zzzz : 15 December 2022 Nepal Time
Date Format with E, dd MMM yyyy HH:mm:ss z :Thu, 15 Dec 2022 22:09:52 NPT
```

```java
//SimpleDateFormatExample2.java
import java.text.DateFormat;
import java.util.Date;
class SimpleDateFormatExample2{
    public static void main(String args[]){
        Date currentDate = new Date();
        System.out.println("Current date is: "+currentDate);

        String dateShort = DateFormat.getDateInstance(DateFormat.SHORT).format(currentDate);
        System.out.println("Formatting the Date using DateFormat.SHORT: "+dateShort);
        String dateMedium = DateFormat.getDateInstance(DateFormat.MEDIUM).format(currentDate);
        System.out.println("Formatting the Date using DateFormat.MEDIUM: "+dateMedium);
        String dateLong = DateFormat.getDateInstance(DateFormat.LONG).format(currentDate);
        System.out.println("Formatting the Date using DateFormat.LONG: "+dateLong);

        String timeShort = DateFormat.getTimeInstance(DateFormat.SHORT).format(currentDate);
        System.out.println("Formatting the Time using DateFormat.SHORT: "+timeShort);
        String timeMedium = DateFormat.getTimeInstance(DateFormat.MEDIUM).format(currentDate);
        System.out.println("Formatting the Time using DateFormat.MEDIUM: "+timeMedium);
        String timeLong = DateFormat.getTimeInstance(DateFormat.LONG).format(currentDate);
        System.out.println("Formatting the Time using DateFormat.LONG: "+timeLong);
    }
}
```

```
C:\Users\balkr\Documents\Java\Collections>java SimpleDateFormatExample2
Current date is: Thu Dec 15 22:11:37 NPT 2022
Formatting the Date using DateFormat.SHORT: 12/15/22
Formatting the Date using DateFormat.MEDIUM: Dec 15, 2022
Formatting the Date using DateFormat.LONG: December 15, 2022
Formatting the Time using DateFormat.SHORT: 10:11 PM
Formatting the Time using DateFormat.MEDIUM: 10:11:37 PM
Formatting the Time using DateFormat.LONG: 10:11:37 PM NPT
```

```java
//DaysbetweentwoDates.java
import java.util.Date;
import java.util.concurrent.TimeUnit;
import java.text.SimpleDateFormat;
class DaysbetweentwoDates{
    public static void main(String args[]){
        SimpleDateFormat myFormat = new SimpleDateFormat("dd/MM/yyyy");
        String date1 = "01/01/2024"; // 2024 is Leap year
        String date2 = "01/01/2025";
        try {
            Date dateBefore = myFormat.parse(date1);
            Date dateAfter = myFormat.parse(date2);
            long difference = dateAfter.getTime() - dateBefore.getTime();
            float daysBetween = TimeUnit.DAYS.convert(difference, TimeUnit.MILLISECONDS);
            System.out.println("Number of Days between dates: "+daysBetween);
        } catch (Exception e) {
            e.printStackTrace();}
    }
}
```

Random

The **Random** class is a generator of pseudorandom numbers. These are called *pseudorandom* numbers because they are simply uniformly distributed sequences. **Random** defines the following constructors:

Random( )
Random(long *seed*)

The first version creates a number generator that uses a reasonably unique seed. The second form allows you to specify a seed value manually.

If you initialize a **Random** object with a seed, you define the starting point for the random sequence. If you use the same seed to initialize another **Random** object, you will extract the same random sequence. If you want to generate different sequences, specify different seed values. One way to do this is to use the current time to seed a **Random** object. This approach reduces the possibility of getting repeated sequences.

Method	Description
boolean nextBoolean( )	Returns the next **boolean** random number.
void nextBytes(byte *vals*[ ])	Fills *vals* with randomly generated values.
double nextDouble( )	Returns the next **double** random number.
float nextFloat( )	Returns the next **float** random number.
double nextGaussian( )	Returns the next Gaussian random number.
int nextInt( )	Returns the next **int** random number.
int nextInt(int *n*)	Returns the next **int** random number within the range zero to *n*.
long nextLong( )	Returns the next **long** random number.
void setSeed(long *newSeed*)	Sets the seed value (that is, the starting point for the random number generator) to that specified by *newSeed*.

Table 20-7 The Core Methods Defined by **Random**

```
1  //RandomNumberExample.java
2  import java.util.Random;
3  class RandomNumberExample {
4      public static void main(String[] args)  {
5          Random random = new Random();
6          System.out.println("Random int:  "+ random.nextInt(1760));
7          System.out.println("Random Boolean:  "+random.nextBoolean());
8          System.out.println("Random Double:  "+random.nextDouble());
9          System.out.println("Random Float:  "+random.nextFloat());
10         System.out.println("Random Gaussian:  "+random.nextGaussian());
11
12         byte[] bytes = new byte[10];
13         random.nextBytes(bytes);
14         System.out.printf("[");
15         for(int i = 0; i< bytes.length; i++)
16             System.out.printf("%d ", bytes[i]);
17         System.out.printf("]\n");
```

```
18
19          System.out.println(random.nextLong());
20          System.out.println(random.nextInt());
21
22          long seed = 950;
23          random.setSeed(seed);
24          //value after setting seed
25          System.out.println("With seed : "+random.nextInt());
26          //return the next pseudorandom long value
27          Long val = random.nextLong();
28          System.out.println("With seed Random Long value : "+ val);
29      }
30  }
```

```
C:\Users\balkr\Documents\Java\Collections>java RandomNumberExample
Random int:  429
Random Boolean:  false
Random Double:  0.3467931735712063
Random Float:  0.8678646
Random Gaussian:  0.47157871658689715
[82 3 101 -84 -34 55 109 -66 69 52 ]
-8613149130654390766
922766467
With seed : -1211657916
With seed Random Long value : 2031812215823661910
```

Chapter 1 and 2
AWT, AWT Controls, Layout Managers, Menus

1. An Overview of the AWT

- **AWT:** Abstract Window Toolkit.
- It is a part of JFC (Java Foundation Classes) developed by Sun Microsystems in 1995.
- It is a platform-dependent API to develop GUI (Graphical User Interface) or window-based applications in Java.
- It is heavy-weight in use because it is generated by the system's host operating system.
- One of the ***important features of AWT is that it is platform dependent***. This means that the AWT tools use the native toolkits of the platforms they are being implemented. This approach helps in preserving the look and feel of each platform.
- But as said everything comes with a price, there is a major drawback of this approach. When executed on various platforms because of platform dependency it will look different on each platform. ***This hampers the consistency and aesthetics of an application.***
- The AWT API in Java primarily consists of a comprehensive set of classes and methods that are required for creating and managing the Graphical User Interface (GUI) in a simplified manner.
- The AWT defines a basic set of controls, windows, and dialog boxes that support a usable, but limited graphical interface. One reason for the limited nature of the AWT is that it translates its various visual components into their corresponding, platform-specific equivalents, or peers. ***This means that the look and feel of a component is defined by the platform***, ***not by Java***. Because the AWT components use native code resources, ***they are referred to as heavyweight.***

Features of AWT

- AWT has a set of native user interface components.
- It provides various classes for graphical representation of components like font, shape, color.
- It provides a robust event-handling model.
- It has Layout managers which is helpful for changing window size or screen resolution.
- It provides a large range of libraries that can be used for designing graphics for gaming applications or educational applications.
- It has data transfer classes through which cut and paste operation can be performed using the local clipboard.

Advantages of GUI over CUI (Character User Interface)

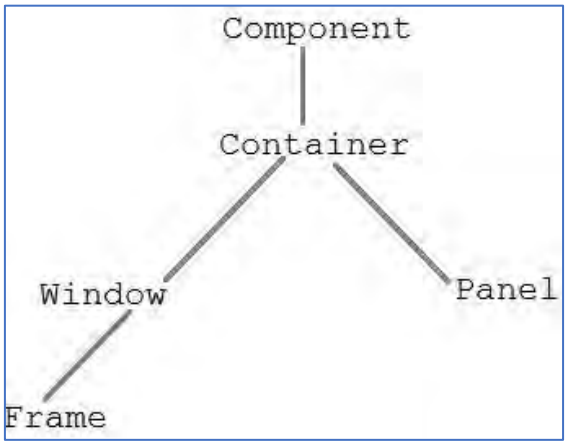
- GUI provides graphical icons to interact while the CUI offers the simple text-based interfaces. GUI makes the application more entertaining and interesting.
- GUI offers click and execute environment while in CUI every time we have to enter the command for a task.
- New user can easily interact with GUI by the visual indicators but it is difficult in CUI.
- GUI offers a lot of controls of file system and the operating system while in CUI you have to use commands which is difficult to remember.
- Windows concept in GUI allow the user to view, manipulate and control the multiple applications at once while in CUI user can control one task at a time.

- GUI provides multitasking environment so as the CUI also does but CUI does not provide same ease as the GUI do.
- Using GUI, it is easier to control and navigate the operating system which becomes very slow in command user interface. GUI can be easily customized.

Java AWT vs. Java Swing

Java AWT	Java Swing
AWT components are platform-dependent.	Java swing components are platform-independent.
AWT components are heavyweight.	Swing components are lightweight.
AWT doesn't support pluggable look and feel.	Swing supports pluggable look and feel.
AWT provides less components than Swing.	Swing provides more powerful components such as tables, lists, scrollpanes, colorchooser, tabbedpane, etc.
AWT doesn't follows MVC(Model View Controller) where model represents data, view represents presentation and controller acts as an interface between model and view.	Swing follows MVC.

Java AWT Hierarchy

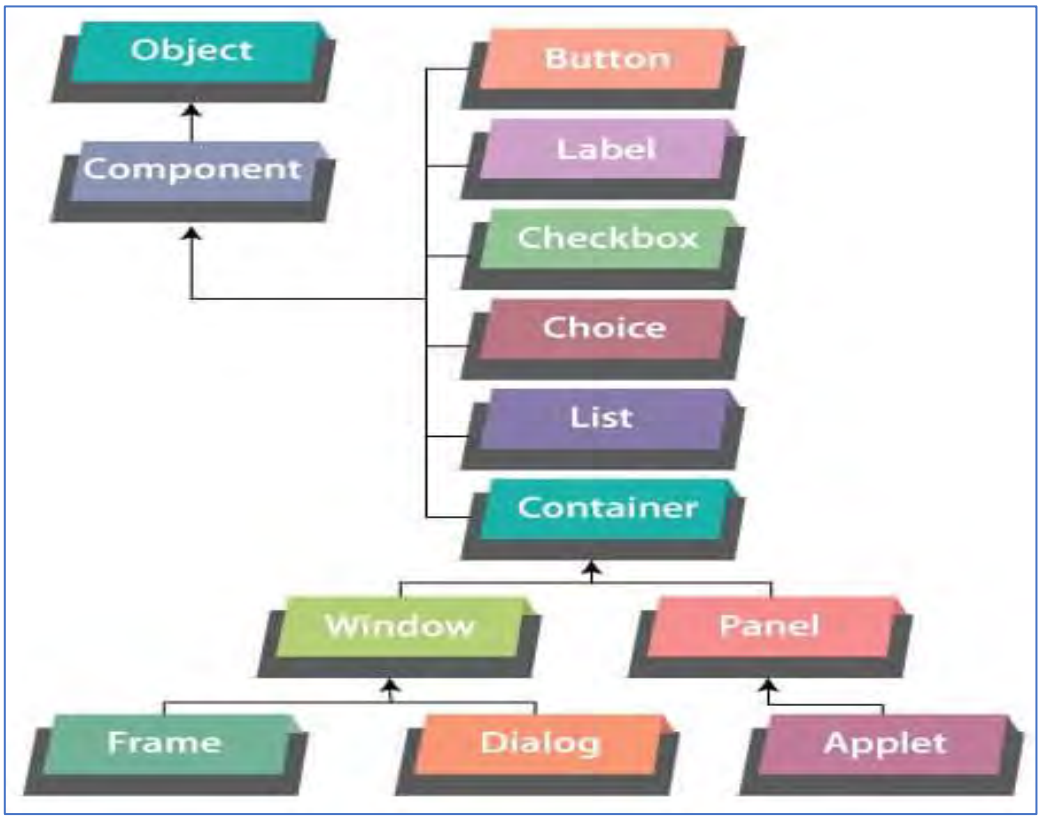


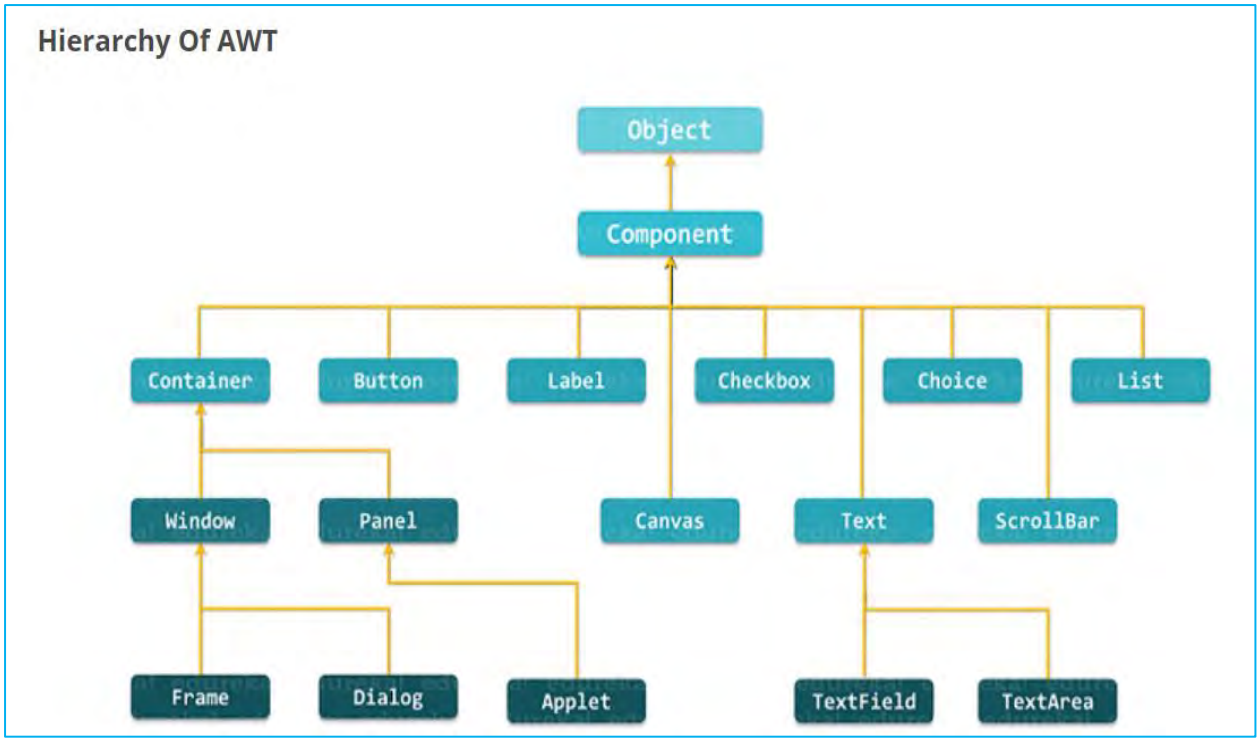
Component class: In the AWT class hierarchy, the Component class is derived from the Object class. Component class is at the top of AWT hierarchy. Component class is an abstract class. It is the superclass of all the GUI element ***classes like Container, Button, Label, Checkbox, Choice, and List.*** Therefore, Component class is responsible for an overall graphical interface. A component object is responsible for remembering the current foreground and background colors and the currently selected text font.

Container: Container in Java AWT is a component that is used to hold other components such as text fields, buttons, etc. It is a subclass ***of java.awt.Component*** and is responsible for keeping

a track of components being added. ***There are four types of containers provided by AWT in Java.***

1. ***Window:*** It is an instance of the Window class having neither border nor title. It is used for creating a top-level window.
2. ***Frame:*** Frame is a subclass ***of Window and contains title, border and menu bars***. It comes with a resizing canvas and is the most widely used container for developing AWT applications. It is capable of holding various components such as buttons, text fields, scrollbars, etc. You can create a Java AWT Frame in two ways:
    a. By Instantiating Frame class
    b. By extending Frame class
3. ***Dialog:*** Dialog class is also a subclass of Window and comes with the border as well as the title. Dialog class's instance always needs an associated Frame class instance to exist.
4. ***Panel:*** Panel is the concrete subclass of Container and doesn't contain any title bar, menu bar or border. Panel class is a generic container for holding the GUI components. You need the instance of the Panel class in order to add the components.





Working with Frame Windows

- Typically, the type of AWT-based application window you will most often create is derived from Frame. As mentioned, it creates a standard-style, top level window that has all of the features normally associated with an application window, such as a close box and title.
- **Here are two of Frame's constructors:**
  - *Frame () throws HeadlessException*
  - *Frame (String title) throws HeadlessException*
- The first form creates a standard window that does not contain a title.
- The second form creates a window with the title specified by title.
- Notice that you cannot specify the dimensions of the window. Instead, you must set the size of the window after it has been created.
- A *HeadlessException* is thrown if an attempt is made to create a Frame instance in an environment that does not support user interaction.

Setting the Window's Dimensions

- The ***setSize()*** method is used to set the dimensions of the window. It is shown here:
  - *void setSize (int newWidth, int newHeight)*
  - *void setSize (Dimension newSize)*
- The new size of the window is specified by newWidth and newHeight, or by the width and height fields of the Dimension object passed in newSize. The dimensions are specified in terms of pixels.
- The ***getSize()*** method is used to obtain the current size of a window. One of its forms is shown here:
  - *Dimension getSize()*
- This method returns the current size of the window contained within the width and height fields of a Dimension object.

Hiding and Showing a Window

- After a frame window has been created, it will not be visible until you call ***setVisible().*** Its signature is shown here:
  - *void setVisible(boolean visibleFlag)*
- The component is visible if the argument to this method is true. Otherwise, it is hidden.

Setting a Window's Title

- You can change the title in a frame window using ***setTitle(),*** which has this general form:
  - *void setTitle(String newTitle)*
- Here, *newTitle* is the new title for the window.

Closing a Frame Window

- When using a frame window, your program must remove that window from the screen when it is closed. If it is not the top-level window of your application, this is done by calling ***setVisible(false).***
- For the main application window, you can simply terminate the program by calling ***System.exit().***

2. AWT Control Fundamentals

- The AWT supports the following types of controls:
  - Labels
  - Push buttons
  - Check boxes
  - Choice lists
  - Lists
  - Scroll bars
  - Text Editing

- All AWT controls are subclasses of Component. Although the set of controls provided by the ***AWT is not particularly rich***, it is sufficient for simple applications, such as short utility programs intended for your own use.
- It is important to point out, however, that ***Swing provides a substantially larger***, more ***sophisticated set of controls*** better suited for creating commercial applications.

Adding and Removing Controls

- To include a control in a window, you must add it to the window. To do this, you must first create an instance of the desired control and then add it to a window by calling ***add (),*** which is defined by Container.
- The ***add ()*** method has several forms.

Component add (Component compRef)

- Here, ***compRef*** is a reference to an instance of the control that you want to add. A reference to the object is returned. Once a control has been added, it will ***automatically be visible*** whenever its parent window is displayed.
- To ***remove a control from a window*** when the control is no longer needed, ***call remove ().*** This method is also defined by Container. Here is one of its forms

void remove (Component compRef)

- Here, ***compRef*** is a reference to the control you want to remove. You can remove all controls by calling ***removeAll().***

The HeadlessException

- Most of the AWT controls constructors can throw a ***HeadlessException*** when an attempt is made ***to instantiate a GUI component in a non-interactive environment*** (such as one in which no display, mouse, or keyboard is present).
- You can use this exception to write code that can adapt to non-interactive environments.

Labels

- The easiest control to use is a label. A label is an object of type Label, and it contains a string, which it displays.
- Labels are passive controls that do not support any interaction with the user.
- Label defines the following constructors:

1. ***Label ():*** It constructs an empty label.
2. ***Label (String text):*** It constructs a label with the given string (left justified by default).
3. ***Label (String text, int alignement):*** It constructs a label with the specified string and the specified alignment. The value of alignement must be one of these three constants: ***Label.LEFT***, ***Label.RIGHT***, or ***Label.CENTER***.

- You can ***set the alignment*** of the string within the label by calling ***setAlignment().***
- To obtain the current alignment***, call getAlignment().***
- The methods are as follows:
  void setAlignment (int alignment)
  int getAlignment()

```
1  //awtLabelExample.java
2  import java.awt.*;
3  import java.awt.event.*;
4  class awtLabelExample extends Frame{
5      awtLabelExample(){
6          setTitle("AWT Label Exampe");
7          setLayout(new FlowLayout());
8          setSize(300,100);
9          // creating the labels
10         Label l1 = new Label ("First Label.");
11         Label l2 = new Label ("Second Label.");
12         Label l3 = new Label ("Third Label.");
13         // adding labels to the frame
14         add(l1);    add(l2);    add(l3);
15         //to close the frame
16         addWindowListener (new WindowAdapter() {
17             public void windowClosing (WindowEvent e) {
18                 dispose();     //System.exit(0);
19             }
20         });
21         setVisible(true);
22     }
23     public static void main(String args[]){
24         new awtLabelExample(); }
25 } // end of class
```

Buttons

- The most widely used control is the push button. A push button is a component that contains a label and generates an event when it is pressed.
- Push buttons are objects of type Button. Button defines these two constructors:
  1. ***Button ():*** Constructs a button with an empty string for its label.
  2. ***Button (String text):*** Constructs a new button with specified label.

```java
//awtButtonExample.java
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class awtButtonExample extends Frame implements ActionListener{
    awtButtonExample(){
        /* Set properties of the Frame. */
        setTitle("AWT Button Example"); setLayout(new FlowLayout());
        setSize(300,300);
        Label label=new Label();
        Button bt1=new Button("Open");   Button bt2=new Button("Submit");
        Button bt3=new Button();    bt3.setLabel("Re-submit");
        /* Add Buttons on the Frame. */
        add(bt1);  add(bt2);  add(bt3); add(label);
        //Add action listener for Buttons
        bt1.addActionListener(this);
        bt2.addActionListener(this);  bt3.addActionListener(this);
        // the windowClosing() method of WindowAdapter class
        addWindowListener (new WindowAdapter() {
            public void windowClosing (WindowEvent e) {
                dispose();    }
        });
        setVisible(true);
     }
    // Handling button action events
    public void actionPerformed(ActionEvent e) {
        String str=e.getActionCommand();
        String msg;
        if(str.equals("Open"))
            msg="You pressed SUbmit Open.";
        else if (str.equals("Submit"))
            msg="You pressed SUbmit Button.";
        else
            msg="You pressed Re-Submit Button.";
        JOptionPane.showMessageDialog(this,msg);
        }

    public static void main(String[] ar){
        new awtButtonExample(); }
} // end of class
```

Checkbox Group

- It is possible to create a set of mutually exclusive check boxes in which one and only one check box in the group can be checked at any one time. These check boxes are **often called radio buttons.**
- To create a ***set of mutually exclusive*** check boxes, you must first ***define the group*** to which they will belong and then specify that group when you construct the check boxes. Check box groups are objects of type ***CheckboxGroup***.
- Only the ***default constructor is defined***, which creates an empty group
  CheckboxGroup (): Creates a new instance of CheckboxGroup.
- You can determine which check box in a group is currently selected by calling ***getSelectedCheckbox ().***
- You can set a check box by calling ***setSelectedCheckbox().***

- These methods are as follows:
  Checkbox getSelectedCheckbox ()
  void setSelectedCheckbox (Checkbox which)
- Here, which is the check box that you want to be selected. The previously selected check box will be turned off.

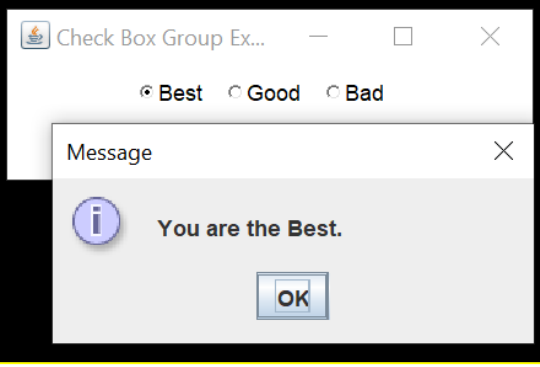
Checkbox

- Checkbox control is used to turn an option on(true) or off(false). There is label for each checkbox representing what the checkbox does. The state of a checkbox can be changed by clicking on it.
- ***Constructors for CheckBox***
  1. ***Checkbox ():*** Creates a check box with an empty string for its label.
  2. ***Checkbox (String label):*** Creates a check box with the specified label.
  3. ***Checkbox (String label, boolean state):*** Creates a check box with the specified label and sets the specified state.
  4. ***Checkbox (String label, boolean state, CheckboxGroup group):*** Constructs a Checkbox with the specified label, set to the specified state, and in the specified check box group.
  5. ***Checkbox (String label, CheckboxGroup group, boolean state):*** Creates a check box with the specified label, in the specified check box group, and set to the specified state.

```
1  //awtCheckBox.java
2  import java.awt.*;
3  import java.awt.event.*;
4  public class awtCheckBox {
5      awtCheckBox(){
6          Frame fm= new Frame("AWT Checkbox Example");
7          Checkbox ckbox1 = new Checkbox("Best", true);
8          ckbox1.setBounds(100,100, 60,60);
9          Checkbox ckbox2 = new Checkbox("Good");
10         ckbox2.setBounds(100,180, 60,60);
11         Checkbox ckbox3 = new Checkbox("Bad");
12         ckbox3.setBounds(100,260, 60,60);
13         fm.add(ckbox1);   fm.add(ckbox2);   fm.add(ckbox3);
14         fm.setSize(400,400);   fm.setLayout(null);
15         // the windowClosing() method of WindowAdapter class
16         fm.addWindowListener (new WindowAdapter() {
17             public void windowClosing (WindowEvent e) {
18                 fm.dispose(); }
19         });
20         fm.setVisible(true);
21     }
22   public static void main(String args[])  {
23     new awtCheckBox();  }
24 } // end of class
```

```java
//awtCheckBoxGroup.java
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class awtCheckBoxGroup extends Frame implements ItemListener {
    CheckboxGroup cbg; Checkbox ckbox1,ckbox2,ckbox3;
    awtCheckBoxGroup(){
        /* Set properties of the Frame. */
        setTitle("Check Box Group Example"); setLayout(new FlowLayout());
        setSize(300,100);
        cbg = new CheckboxGroup();   // to create Check Box Group
        ckbox1 = new Checkbox("Best",cbg,true);
        ckbox2 = new Checkbox("Good", cbg,false);
        ckbox3 = new Checkbox("Bad", cbg, false);
        // check box adding to the frame
        add(ckbox1);   add(ckbox2);     add(ckbox3);
        //add item listener to check box
        ckbox1.addItemListener(this);  ckbox2.addItemListener(this);
        ckbox3.addItemListener(this);
        // the windowClosing() method of WindowAdapter class
        addWindowListener (new WindowAdapter() {
            public void windowClosing (WindowEvent e) {
                dispose();
            }
        });
        setVisible(true);
    }
    public void itemStateChanged(ItemEvent e) {
        String msg;
        if(cbg.getSelectedCheckbox()== ckbox1)  {
            msg= "You are the Best."; }
        else if(cbg.getSelectedCheckbox()== ckbox2)  {
            msg= "You are the Good."; }
        else
            msg= "You are the Bad.";
        JOptionPane.showMessageDialog(this,msg);
    }
    public static void main(String args[])  {
    new awtCheckBoxGroup(); }
}// end of the class
```

```
C:\Users\balkr\Documents\Java\awt>java awtCheckBoxGroup
```



AWT Choice

- In Java, AWT contains a Choice Class. It is used for creating a drop-down menu of choices. When a user selects a particular item from the drop-down then it is shown on the top of the menu.
- **Choice Class constructor**

  Choice (): It constructs a new choice menu.

AWT List

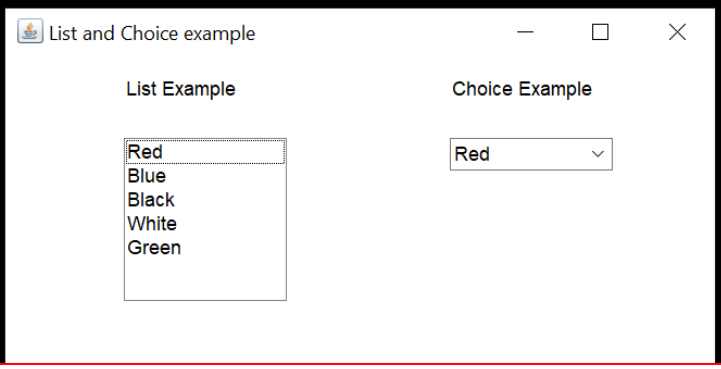
- The List class provides a compact, multiple-choice, scrolling selection list.
- Unlike the Choice object, which shows only the single selected item in the menu, a List object can be constructed to show any number of choices in the visible window.
- **AWT List Class Constructors**
  1. ***List ():*** It constructs a new scrolling list.
  2. ***List (int row_num):*** It constructs a new scrolling list initialized with the given number of rows visible.
  3. ***List (int row_num, Boolean multipleMode):*** It constructs a new scrolling list initialized which displays the given number of rows.
- For lists that allow only single selection, you can determine which item is currently selected by calling either ***getSelectedItem( )*** or ***getSelectedIndex( ).***
- These methods are shown here:

  String getSelectedItem( )
  int getSelectedIndex( )
- The ***getSelectedItem()*** method returns a string containing the name of the item. If more than one item is selected, or if no selection has yet been made, null is returned.
- The ***getSelectedIndex()*** returns the index of the item. The first item is at index 0. If more than one item is selected, or if no selection has yet been made, –1 is returned.

```
1  //awtListChoiceExample.java
2  import java.awt.*;
3  import java.awt.event.*;
4  public class awtListChoiceExample {
5    awtListChoiceExample() {
6          Frame fm= new Frame("List and Choice example");
7          Label lab1 = new Label("List Example");
8
9          lab1.setBounds(80,40, 100,20);   fm.add(lab1);
10         List ls=new List(5);   ls.setBounds(80,80, 100,100);
11
12         ls.add("Red");    ls.add("Blue");   ls.add("Black");
13         ls.add("White");  ls.add("Green");
14         fm.add(ls);   // adding list to the frame
15
16         Label lab2 = new Label("Choice Example");
17         lab2.setBounds(280,40, 100,20);
18         fm.add(lab2);
19         // creating a choice
20         Choice choice=new Choice();
21         choice.setBounds(280,80, 100,100);
22         choice.add("Red");       choice.add("Blue");
23         choice.add("Black");     choice.add("Pink");
24         choice.add("White");     choice.add("Green");
```

```
25          //adding choice to the frame
26          fm.add(choice);
27          // the windowClosing() method of WindowAdapter class
28          fm.addWindowListener (new WindowAdapter() {
29              public void windowClosing (WindowEvent e) {
30                  fm.dispose();
31              }
32          });
33          fm.setSize(450,250);     fm.setLayout(null);
34          fm.setVisible(true);
35      }
36      public static void main(String args[]){
37          new awtListChoiceExample();  }
38  } // end of class
```

```
C:\Users\balkr\Documents\Java\awt>java awtListChoiceExample
```



TextField

- The TextField class implements a single-line text-entry area, usually called an edit control. Text fields allow the user to enter strings and to edit the text using the arrow keys, cut and paste keys, and mouse selections.
- TextField is a subclass of TextComponent. TextField Class constructors
  1. ***TextField():*** It constructs a new text field component.
  2. ***TextField(String text):*** It constructs a new text field initialized with the given string text to be displayed.
  3. ***TextField(int numChars):*** It constructs a new textfield (empty) with given numChars characters.
  4. ***TextField(String text, int numChars):*** It constructs a new text field with the given text and given numChars characters wide.
- To obtain the string currently contained in the text field, call ***getText( ).*** To set the text, ***call setText( ).*** These methods are as follows:

  String getText()
  void setText(String str)
- We can obtain the currently selected text by calling ***getSelectedText( ).*** These methods are shown here:

  String getSelectedText()
  void select (int startIndex, int endIndex)

- ***getSelectedText()*** returns the selected text. The ***select()*** method selects the characters beginning at startIndex and ending at endIndex –1.
- You can control whether the contents of a text field may be modified by the user by calling setEditable( ).

  void setEditable(boolean canEdit)
- In ***setEditable(),*** if canEdit is true, the text may be changed. If it is false, the text cannot be altered.

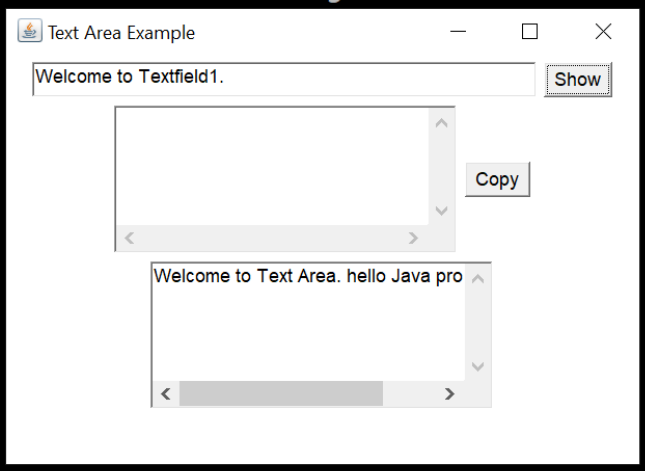
TextArea

- The AWT includes a simple multiline editor called TextArea.
- ***Constructors of TextArea are:***
  1. ***TextArea():*** It constructs a new and empty text area with no text in it.
  2. ***TextArea (int row, int column):*** It constructs a new text area with specified number of rows and columns and empty string as text.
  3. ***TextArea (String text):*** It constructs a new text area and displays the specified text in it.
  4. ***TextArea (String text, int row, int column):*** It constructs a new text area with the specified text in the text area and specified number of rows and columns.
  5. ***TextArea (String text, int row, int column, int scrollbars):*** It construcst a new text area with specified text in text area and specified number of rows and columns and visibility.
- Fields of TextArea Class: The fields of java.awt.TextArea class are as follows:
  - static int ***SCROLLBARS_BOTH***: It creates and displays both horizontal and vertical scrollbars.
  - static int ***SCROLLBARS_HORIZONTAL_ONLY***: It creates and displays only the horizontal scrollbar.
  - static int ***SCROLLBARS_VERTICAL_ONLY:*** It creates and displays only the vertical scrollbar.
  - static int ***SCROLLBARS_NONE:*** It doesn't create or display any scrollbar in the text area.
- It supports the ***getText(), setText(), getSelectedText(), select(), isEditable(), and setEditable()*** methods.

```
1  // awtTextFieldAreaExample.java
2  import javax.swing.*;
3  import java.awt.*;
4  import java.awt.event.*;
5  class awtTextFieldAreaExample extends Frame implements ActionListener{
6      TextField tf;     TextArea ta1,ta2;   Button btn1,btn2;
7      awtTextFieldAreaExample(){
8          setTitle("Text Area Example");      setLayout(new FlowLayout());
9          setSize(450,300);
10
11         tf=new TextField("Welcome to Textfield1.",30);  add(tf);
12         btn1=new Button("Show");      add(btn1);
13         int scrollBoth = TextArea.SCROLLBARS_BOTH;
14         ta1=new TextArea("Welcome to Text Area.",5,20,scrollBoth);     add(ta1);
15         btn2=new Button("Copy");      add(btn2);
16         ta2=new TextArea("Welcome to Text Area.",5,20,scrollBoth);     add(ta2);
17
```

```
18          btn1.addActionListener(this);
19          btn2.addActionListener(this);
20          // the windowClosing() method of WindowAdapter class
21          addWindowListener (new WindowAdapter() {
22              public void windowClosing (WindowEvent e) {
23                  dispose();  }
24          });
25          setVisible(true);
26      }
27      // Handling button action events
28      public void actionPerformed(ActionEvent e) {
29          String str=e.getActionCommand();
30          if(str.equals("Show"))
31              JOptionPane.showMessageDialog(this,"You pressed Show Button.");
32          if(str.equals("Copy")){
33             String copyStr=ta1.getText();
34             ta2.setText(copyStr);
35             ta1.setText(""); }
36      }
37      public static void main(String args[]){
38          new awtTextFieldAreaExample(); }
39  } // end of Class
```

```
C:\Users\balkr\Documents\Java\awt>java awtTextFieldAreaExample
```



Understanding Layout Managers

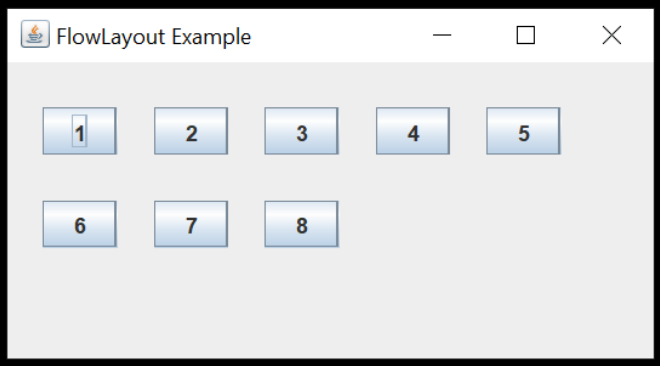
FlowLayout

- FlowLayout implements a simple layout style, which is similar to how words flow in a text editor.
- The direction of the layout is governed by the container's component orientation property, which, by default, is left to right, top to bottom. Therefore, by default, components are laid out line-by-line beginning at the upper-left corner.
- In all cases, when a line is filled, layout advances to the next line. A small space is left between each component, above and below, as well as left and right.
- The **constructors for FlowLayout**:
  - *FlowLayout()*
  - *FlowLayout(int **how**)*
  - *FlowLayout(int **how**, int **horz**, int **vert**)*
- The first form creates the default layout, which centers components and leaves five pixels of space between each component.
- The second form lets you specify how each line is aligned. Valid values for how are as follows:
  - FlowLayout.LEFT
  - FlowLayout.CENTER
  - FlowLayout.RIGHT
  - FlowLayout.LEADING
  - FlowLayout.TRAILING
- These values specify left, center, right, leading edge, and trailing edge alignment, respectively.
- The third constructor allows you to specify the horizontal and vertical space left between components in ***horz*** and ***vert***, respectively.

```
1  // flowLayoutExample.java
2  import java.awt.*;
3  import javax.swing.*;
4  public class flowLayoutExample extends JFrame {
5      flowLayoutExample() {
6          setLayout(new FlowLayout(FlowLayout.LEFT, 20, 25));
7          setTitle("FlowLayout Example");
8          setSize(370,200);
9          setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
10         // creating the buttons
11         JButton b1 = new JButton("1");
12         JButton b2 = new JButton("2");
13         JButton b3 = new JButton("3");
14         JButton b4 = new JButton("4");
15         JButton b5 = new JButton("5");
16         JButton b6 = new JButton("6");
17         JButton b7 = new JButton("7");
18         JButton b8 = new JButton("8");
19         // adding the buttons to frame
20         add(b1); add(b2); add(b3); add(b4);
21         add(b5); add(b6);  add(b7);  add(b8);
22          setVisible(true);
```

```
23          }       // end of constructor
24      public static void main(String argvs[]) {
25              new flowLayoutExample(); }
26  }// end of class
```

```
C:\Users\balkr\Documents\Java\awt>java flowLayoutExample
```

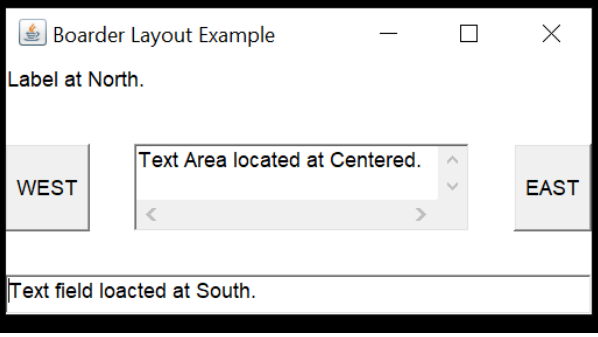


BorderLayout

- The BorderLayout class implements a layout style that has four narrows, fixed width components at the edges and one large area in the center.
- The four sides are referred to as ***north, south, east, and west***. The middle area is called the center.
- BorderLayout is the ***default layout manager for Frame***.
- Here are the constructors defined by BorderLayout:
  - ***BorderLayout()***
  - ***BorderLayout (int horz, int vert)***
- The ***first form creates*** a default border layout. The ***second allows*** you to specify the horizontal and vertical space left between components in ***horz and vert***, respectively.
- BorderLayout defines the following commonly used constants that specify the regions:
  - *BorderLayout.CENTER*
  - *BorderLayout.NORTH*
  - *BorderLayout.SOUTH*
  - *BorderLayout.EAST*
  - *BorderLayout.WEST*
- When adding components, you will use these constants with the following form of ***add(),*** which is defined by Container:
  void ***add****(Component* ***compRef****, Object region)*
- Here, ***compRef*** is a reference to the component to be added, and region specifies where the component will be added.

```java
// boarderLayoutExample.java
import java.awt.*;
import java.awt.event.*;
public class boarderLayoutExample extends Frame {
    boarderLayoutExample() {
        setTitle("Boarder Layout Example");
        Label label = new Label("Label at North.");
        TextField tf= new TextField("Text field loacted at South.");
        Button btn1 = new Button("EAST");
        Button btn2 = new Button("WEST");
        TextArea ta = new TextArea("Text Area located at Centered.");
        // adding componebts to the layout
        setLayout(new BorderLayout(25,25));
        add(label, BorderLayout.NORTH);
        add(tf, BorderLayout.SOUTH);
        add(btn1, BorderLayout.EAST);
        add(btn2, BorderLayout.WEST);
        add(ta, BorderLayout.CENTER);
        // the windowClosing() method of WindowAdapter class
        addWindowListener (new WindowAdapter() {
            public void windowClosing (WindowEvent e) {
                dispose(); }
        });
        setSize(350,350);    setVisible(true);
    }
    public static void main(String argvs[]){
            new boarderLayoutExample(); }
} // end of class
```

```
C:\Users\balkr\Documents\Java\awt>java boarderLayoutExample
```



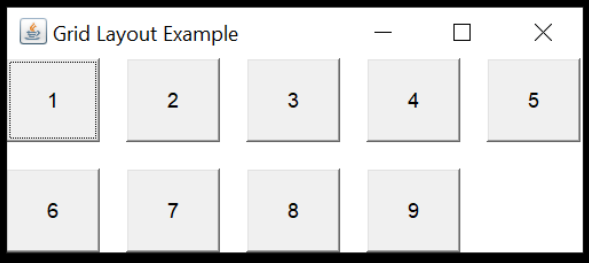
GridLayout

- GridLayout lays out components in a ***two-dimensional grid.***
- When you instantiate a GridLayout, you define the number of rows and columns.
- The constructors supported by GridLayout are shown here:
  - *GridLayout ()*
  - *GridLayout (int **numRows**, int **numColumns**)*
  - *GridLayout (int **numRows**, int **numColumns**, int **horz**, int **vert**)*
- The ***first form*** creates a single-column grid layout.
- The ***second form creates a grid layout*** with the specified number of ***rows and columns.***

- The third form allows you to ***specify the horizontal and vertical space*** left between components in ***horz and vert***, respectively.
- Either numRows or numColumns can be zero. Specifying ***numRows as zero*** allows for unlimited-length columns. Specifying ***numColumns as zero*** allows for unlimited-length rows.

```java
// gridLayoutExample.java
import java.awt.*;
import java.awt.event.*;
public class gridLayoutExample extends Frame {
    gridLayoutExample(){
        setTitle("Grid Layout Example");
        setLayout(new GridLayout(2,5, 15, 15));
        Button btn1 = new Button("1");
        Button btn2 = new Button("2");
        Button btn3 = new Button("3");
        Button btn4 = new Button("4");
        Button btn5 = new Button("5");
        Button btn6 = new Button("6");
        Button btn7 = new Button("7");
        Button btn8 = new Button("8");
        Button btn9 = new Button("9");

        add(btn1); add(btn2); add(btn3);
        add(btn4); add(btn5); add(btn6);
        add(btn7); add(btn8); add(btn9);
        // the windowClosing() method of WindowAdapter class
        addWindowListener (new WindowAdapter() {
            public void windowClosing (WindowEvent e) {
                dispose(); }
        });
        setSize(350,150);    setVisible(true);
    }
    public static void main(String[] argvs){
            new gridLayoutExample(); }
} // end of class
```

```
C:\Users\balkr\Documents\Java\awt>java gridLayoutExample
```



GridBagLayout

- The key to the grid bag is that each component can be a different size, and each row in the grid can have a different number of columns. This is why the layout is called a grid bag. It's a collection of small grids joined together.
- The ***location and size*** of each component in a grid bag are determined by a set of constraints linked to it. The constraints are contained in an object of type ***GridBagConstraints.*** Constraints include ***the height and width of a cell***, and the ***placement*** of a component, its alignment, and its anchor point within the cell.
- The general procedure for using a grid bag is to first create a new GridBagLayout object and to make it the current layout manager. Then, set the constraints that apply to each component that will be added to the grid bag. Finally, add the components to the layout manager.
- GridBagLayout defines **only one constructor**, which is shown here:
  GridBagLayout()
- The key to successfully using ***GridBagLayout*** is the proper setting of the constraints, which are stored in a ***GridBagConstraints*** object. ***GridBagConstraints*** defines several fields that you can set to govern the ***size, placement, and spacing*** of a component.
- ***weightx and weighty:*** Controls how additional space in the row or column is allotted to the component

Field	Purpose
int anchor	Specifies the location of a component within a cell. The default is **GridBagConstraints.CENTER**.
int fill	Specifies how a component is resized if the component is smaller than its cell. Valid values are **GridBagConstraints.NONE** (the default), **GridBagConstraints.HORIZONTAL**, **GridBagConstraints.VERTICAL**, **GridBagConstraints.BOTH**.
int gridheight	Specifies the height of component in terms of cells. The default is 1.
int gridwidth	Specifies the width of component in terms of cells. The default is 1.
int gridx	Specifies the X coordinate of the cell to which the component will be added. The default value is **GridBagConstraints.RELATIVE**.
int gridy	Specifies the Y coordinate of the cell to which the component will be added. The default value is **GridBagConstraints.RELATIVE**.
Insets insets	Specifies the insets. Default insets are all zero.
int ipadx	Specifies extra horizontal space that surrounds a component within a cell. The default is 0.
int ipady	Specifies extra vertical space that surrounds a component within a cell. The default is 0.
double weightx	Specifies a weight value that determines the horizontal spacing between cells and the edges of the container that holds them. The default value is 0.0. The greater the weight, the more space that is allocated. If all values are 0.0, extra space is distributed evenly between the edges of the window.
double weighty	Specifies a weight value that determines the vertical spacing between cells and the edges of the container that holds them. The default value is 0.0. The greater the weight, the more space that is allocated. If all values are 0.0, extra space is distributed evenly between the edges of the window.

TABLE 24-1 Constraint Fields Defined by **GridBagConstraints**

- **GridBagConstraints** also defines several static fields that contain standard constraint values, such as **GridBagConstraints.CENTER** and **GridBagConstraints.VERTICAL**.
- When a component is smaller than its cell, you can use the **anchor** field to specify where within the cell the component's top-left corner will be located.
- There are three types of values that you can give to **anchor**. The first are absolute:

GridBagConstraints.CENTER	GridBagConstraints.SOUTH
GridBagConstraints.EAST	GridBagConstraints.SOUTHEAST
GridBagConstraints.NORTH	GridBagConstraints.SOUTHWEST
GridBagConstraints.NORTHEAST	GridBagConstraints.WEST
GridBagConstraints.NORTHWEST	

- The second type of values that can be given to **anchor** is relative, which means the values are relative to the container's orientation. Their names describe the placement.

GridBagConstraints.FIRST_LINE_END	GridBagConstraints.LINE_END
GridBagConstraints.FIRST_LINE_START	GridBagConstraints.LINE_START
GridBagConstraints.LAST_LINE_END	GridBagConstraints.PAGE_END
GridBagConstraints.LAST_LINE_START	GridBagConstraints.PAGE_START

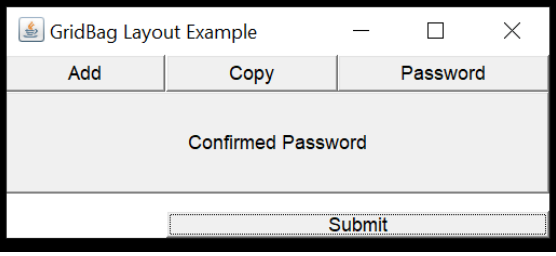
- The third type of values that can be given to **anchor** were added by Java SE 6. They allow you to position components vertically relative to the baseline of the row.

GridBagConstraints.BASELINE	GridBagConstraints.BASELINE_LEADING
GridBagConstraints.BASELINE_TRAILING	GridBagConstraints.ABOVE_BASELINE
GridBagConstraints.ABOVE_BASELINE_LEADING	GridBagConstraints.ABOVE_BASELINE_TRAILING
GridBagConstraints.BELOW_BASELINE	GridBagConstraints.BELOW_BASELINE_LEADING
GridBagConstraints. BELOW_BASELINE_TRAILING	

- The horizontal position can be either centered, against the leading edge (LEADING), or against the trailing edge (TRAILING).
- The **gridwidth** variable lets you specify the width of a cell in terms of cell units. The default is 1.
- To specify that a component use the remaining space in a row, use **GridBagConstraints.REMAINDER**.
- To specify that a component use the next-to-last cell in a row, use **GridBagConstraints.RELATIVE**.
- The **gridheight** constraint works the same way, but in the vertical direction.
- You can specify a padding value that will be used to increase the minimum size of a cell.
- To pad horizontally, assign a value to **ipadx**. To pad vertically, assign a value to **ipady**.

```java
// gridBagLayoutExample.java
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class gridBagLayoutExample extends Frame {
    gridBagLayoutExample(){
        setTitle("GridBag Layout Example");  setSize(350,150);
        setLayout(new GridBagLayout());
        GridBagConstraints gbc = new GridBagConstraints();

        gbc.fill = GridBagConstraints.HORIZONTAL;
        Button btn = new Button("Add");
        gbc.weightx = 0.5;  gbc.gridx = 0;   gbc.gridy = 0;
        add(btn, gbc);

        Button  btn2 = new Button("Copy");
        gbc.gridx = 1;  gbc.gridy = 0;  add(btn2, gbc);

        Button  btn3 = new Button("Password");
        gbc.gridx = 2;  gbc.gridy = 0;  add(btn3, gbc);

        Button btn4 = new Button("Confirmed Password");
        gbc.ipady = 40; gbc.weightx = 0.0;  gbc.gridwidth = 3;
        gbc.gridx = 0;  gbc.gridy = 1; add(btn4, gbc);

        Button btn5 = new Button("Submit");
        gbc.ipady = 0;  gbc.weighty = 1.0;
        gbc.anchor = GridBagConstraints.PAGE_END; // bottom of space
        gbc.gridx = 1;  gbc.gridwidth = 2; gbc.gridy = 2;
        add(btn5, gbc);

        // the windowClosing() method of WindowAdapter class
        addWindowListener (new WindowAdapter() {
            public void windowClosing (WindowEvent e) {
                dispose();
            }
        });
        setVisible(true);       //set frame visibilty true
    }
    public static void main(String[] args)  {
        new gridBagLayoutExample(); }
}// end of class
```

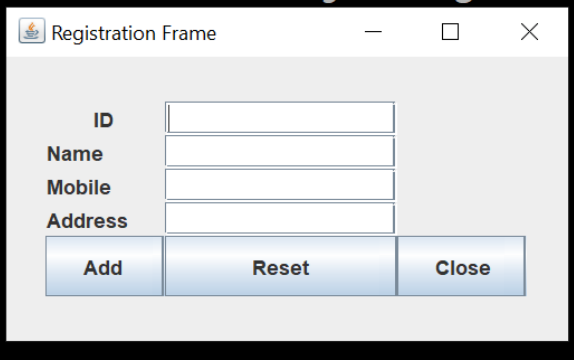
```
C:\Users\balkr\Documents\Java\awt>java gridBagLayoutExample
```



```
1  //RegisterFrameGBagL.java
2  import java.awt.event.*;
3  import javax.swing.*;
4  import java.awt.*;
5  public class RegisterFrameGBagL extends JFrame {
6      RegisterFrameGBagL(){
7          setSize(350,250);   setTitle("Registration Frame");
8          setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
9          setLayout(new GridBagLayout());
10         GridBagConstraints gbc = new GridBagConstraints();
11         //gbc.fill = GridBagConstraints.HORIZONTAL;
12         gbc.gridx = 0;  gbc.gridy = 0;
13         gbc.gridwidth = 1;      gbc.gridheight=1;
14         JLabel idLabel=new JLabel("ID ");   add(idLabel,gbc);
15         //gbc.fill = GridBagConstraints.HORIZONTAL;
16         gbc.gridx = 1;      gbc.gridy = 0;
17         gbc.gridwidth = 1;  gbc.gridheight=1;
18         JTextField idTF=new JTextField(15);  add(idTF,gbc);
19
20         gbc.fill = GridBagConstraints.HORIZONTAL;
21         gbc.gridx = 0;      gbc.gridy = 2;
22         gbc.gridwidth = 1;  gbc.gridheight=1;
23         JLabel fnameLabel=new JLabel("Name ");  add(fnameLabel,gbc);
24
25         gbc.fill = GridBagConstraints.HORIZONTAL;
26         gbc.gridx = 1;      gbc.gridy = 2;
27         gbc.gridwidth = 1;  gbc.gridheight=1;
28         JTextField fnameTF=new JTextField(15);  add(fnameTF,gbc);
29
30         gbc.fill = GridBagConstraints.HORIZONTAL;
31         gbc.gridx = 0;      gbc.gridy = 3;
32         gbc.gridwidth = 1;   gbc.gridheight=1;
33         JLabel lnameLabel=new JLabel("Mobile "); add(lnameLabel,gbc);
34
35         gbc.fill = GridBagConstraints.HORIZONTAL;
36         gbc.gridx = 1;  gbc.gridy = 3;
37         gbc.gridwidth = 1;  gbc.gridheight=1;
38         JTextField lnameTF=new JTextField(15);      add(lnameTF,gbc);
39
40        gbc.fill = GridBagConstraints.HORIZONTAL;
41        gbc.gridx = 0;      gbc.gridy = 4;
42        gbc.gridwidth = 1;  gbc.gridheight=1;
43        JLabel cLabel=new JLabel("Address ");   add(cLabel,gbc);
44
45        gbc.fill = GridBagConstraints.HORIZONTAL;
46        gbc.gridx = 1;      gbc.gridy = 4;
47        gbc.gridwidth = 1;  gbc.gridheight=1;
48        JTextField cTF=new JTextField(15);      add(cTF,gbc);
49
50        gbc.fill = GridBagConstraints.HORIZONTAL;
51        gbc.gridx = 0;      gbc.gridy = 5;  gbc.ipadx = 10;
52        gbc.ipady = 10;     gbc.gridwidth = 1;  gbc.gridheight=1;
53        JButton loginButton=new JButton("Add "); add(loginButton,gbc);
54
```

```
55          gbc.fill = GridBagConstraints.HORIZONTAL;
56          gbc.gridx = 1;      gbc.gridy = 5;      gbc.ipadx = 10;
57          gbc.ipady = 10;     gbc.gridwidth = 1;  gbc.gridheight=1;
58          JButton resetButton=new JButton("Reset"); add(resetButton,gbc);
59
60          gbc.gridx = 2;  gbc.gridy = 5;  gbc.ipadx = 10;
61          gbc.ipady = 10;     gbc.gridwidth = 1;   gbc.gridheight=1;
62          JButton closeButton=new JButton("Close"); add(closeButton,gbc);
63
64          setVisible(true);
65      }
66      public static void main(String[] args){
67          new RegisterFrameGBagL();
68      }
69  }// end of class
```

```
C:\Users\balkr\Documents\Java\awt>java RegisterFrameGBagL
```



CardLayout

- The CardLayout class is unique among the other layout managers in that it stores several different layouts. Each layout can be thought of as being on a separate index card in a deck that can be shuffled so that any card is on top at a given time.
- This can be useful for user interfaces with optional components that can be dynamically enabled and disabled upon user input.
- You can prepare the other layouts and have them hidden, ready to be activated when needed.
- CardLayout provides these two constructors:
  - *CardLayout()*
  - *CardLayout (int horz, int vert)*
- The first form creates a default card layout. The second form allows you to specify the horizontal and vertical space left between components in horz and vert, respectively.

```
1  //CardLayoutExample.java
2  import javax.swing.*;
3  import java.awt.*;
4  import java.awt.event.*;
5  public class CardLayoutExample extends JFrame implements ActionListener {
6      private CardLayout dealer;    private JPanel deckPanel;
7      public CardLayoutExample() {
8          setSize(350,150);
9          setTitle("CardLayout Demonstration");
10         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
11         Container contentPane = getContentPane( );
12         contentPane.setLayout(new BorderLayout( ));
13
14         deckPanel = new JPanel( );
15         dealer = new CardLayout( );
16         deckPanel.setLayout(dealer);
17
18         JPanel startCardPanel = new JPanel( );
19         startCardPanel.setLayout(new FlowLayout( ));
20         startCardPanel.setBackground(Color.LIGHT_GRAY);
21         JLabel startLabel = new JLabel("Hello");
22         startCardPanel.add(startLabel);
23         deckPanel.add("start", startCardPanel);
24
25             JPanel greenCardPanel = new JPanel( );
26             greenCardPanel.setLayout(new FlowLayout( ));
27             greenCardPanel.setBackground(Color.GREEN);
28             JLabel goLabel = new JLabel("Go");
29             greenCardPanel.add(goLabel);
30             deckPanel.add("green", greenCardPanel);
31
32             JPanel redCardPanel = new JPanel( );
33             redCardPanel.setLayout(new FlowLayout( ));
34             redCardPanel.setBackground(Color.RED);
35             JLabel stopLabel = new JLabel("Stop");
36             redCardPanel.add(stopLabel);
37             deckPanel.add("red", redCardPanel);
38
39             contentPane.add(deckPanel, BorderLayout.CENTER);
40
41             JPanel buttonPanel = new JPanel( );
42             buttonPanel.setBackground(Color.WHITE);
43             buttonPanel.setLayout(new FlowLayout( ));
44             JButton stopButton = new JButton("Red");
45             stopButton.addActionListener(this);
46             buttonPanel.add(stopButton);
47
48             JButton goButton = new JButton("Green");
49             goButton.addActionListener(this);
50             buttonPanel.add(goButton);
51
```

```
52              JButton resetButton = new JButton("Reset");
53              resetButton.addActionListener(this);
54              buttonPanel.add(resetButton);
55              contentPane.add(buttonPanel, BorderLayout.SOUTH);
56              dealer.first(deckPanel);//Optional
57              setVisible(true);
58          }// end of constructor
59      public void actionPerformed(ActionEvent e){
60          String actionCommand = e.getActionCommand( );
61
62          if (actionCommand.equals("Red"))
63              dealer.show(deckPanel, "red");
64          else if (actionCommand.equals("Green"))
65              dealer.show(deckPanel, "green");
66          else if (actionCommand.equals("Reset"))
67             dealer.show(deckPanel, "start");
68          else
69              System.out.println("Error in CardLayout Demo.");
70      }
71      public static void main(String[] args) {
72          new CardLayoutExample(); } // end of main function
73  }// end of class
```

```
C:\Users\balkr\Documents\Java\awt>java CardLayoutExample
```

Menu Bars and Menus

- A menu bar displays a list of top-level menu choices. Each choice is associated with a dropdown menu. This concept is implemented in the AWT by the following classes: ***MenuBar, Menu, and MenuItem***.
- In general, a menu bar contains one or more Menu objects. Each Menu object contains a list of ***MenuItem*** objects. Each MenuItem object represents something that can be selected by the user.
- Since Menu is a subclass of ***MenuItem***, a hierarchy of nested submenus can be created. It is also possible to include checkable menu items. These are menu options of type ***CheckboxMenuItem*** and will have a check mark next to them when they are selected.
- To create a ***menu bar***, first create an instance of MenuBar. This class defines only the default constructor.
    - MenuBar()

- The constructors for Menu:
  - *Menu ()*
  - *Menu (String **optionName**)*
  - *Menu (String **optionName**, boolean **removable**)*
- Here, ***optionName*** specifies the name of the menu selection. If ***removable is true***, the menu ***can be removed*** and allowed ***to float free***. Otherwise, it will remain attached to the menu bar.
- Individual menu items are of type ***MenuItem***. It defines these constructors:
  - *MenuItem ()*
  - *MenuItem (String **itemName**)*
  - *MenuItem (String **itemName**, MenuShortcut **keyAccel**)*
- Here, ***itemName*** is the name shown in the menu, and ***keyAccel*** is the menu shortcut for this item.
- You can disable or enable a menu item by using the ***setEnabled()*** method.
  void setEnabled(boolean enabledFlag)
- If the argument enabledFlag is true, the menu item is enabled. If false, the menu item is disabled.
- You can determine an item's status by calling isEnabled().
  boolean isEnabled()
- ***isEnabled()*** returns true if the menu item on which it is called is enabled. Otherwise, it returns false.
- You can change the name of a menu item by calling ***setLabel().*** You can retrieve the current name by using ***getLabel().***
  void setLabel (String newName)
  String getLabel ()
- You can create a checkable menu item by using a subclass of MenuItem called ***CheckboxMenuItem.*** It has these constructors:
  - CheckboxMenuItem()
  - CheckboxMenuItem (String ***itemName***)
  - CheckboxMenuItem (String ***itemName***, boolean ***on***)
- Here, ***itemName*** is the name shown in the menu. ***Checkable items operate as toggles.*** Each time one is selected, its state changes. If ***on*** is true, the checkable entry is initially checked. Otherwise, it is cleared.
- Once you have created a menu item, you must add the item to a Menu object by using ***add (),*** which has the following general form:
  *MenuItem **add** (MenuItem item)*
- Here, item is the item being added. Items are added to a menu ***in the order in which the calls to add () take place***. The item is returned.

- Once you have added all items to a Menu object, you can add that object to the menu bar by using this version of ***add ()*** defined by MenuBar:
  Menu add (Menu menu)
- Here, menu is the menu being added. The menu is returned. Menus ***generate events*** only when an item of type ***MenuItem or CheckboxMenuItem*** is selected.

- They do not generate events when a menu bar is accessed to display a drop-down menu, for example. Each time a menu item is selected, an ***ActionEvent*** object is generated. ***By default, the action command string is the name of the menu item.***
- Each time a check box menu item is checked or unchecked, an ***ItemEvent*** object is generated. Thus, you must implement the ActionListener and/or ***ItemListener*** interfaces in order to handle these menu events.

Dialog Box

- Dialog boxes are primarily used to obtain user input and are often child windows of a top-level window. Dialog boxes don't have menu bars, but in other respects, they function like frame windows.
- The Dialog control represents a top-level window with a border and a title used to take some form of input from the user. It inherits the Window class.
- ***Frame and Dialog both inherits Window class. Frame has maximized and minimize buttons but Dialog doesn't have.***
- In the AWT, dialog boxes are of type Dialog. Two commonly used constructors are shown here:
  - *Dialog (Frame parentWindow, boolean mode)*
  - *Dialog (Frame parentWindow, String title, boolean mode)*
- Here, ***parentWindow*** is the owner of the dialog box. If mode is ***true***, the dialog box uses the default modality. Otherwise, ***it is modeless***. The title of the dialog box can be passed in title. Generally, you will subclass Dialog, adding the functionality required by your application.

FileDialog

- The ***FileDialog*** class displays a dialog window from which the user can select a file. Since it is a modal dialog, when the application calls its show method to display the dialog, it blocks the rest of the application until the user has chosen a file.
- **Constructor Summary**
  - ***FileDialog (Frame parent, String title, int mode):*** Creates a file dialog window with the specified title for loading or saving a file.
  - ***FileDialog (Frame parent, String title):*** Creates a file dialog window with the specified title for loading a file.
  - ***FileDialog (Frame parent):*** Creates a file dialog for loading a file.
  - ***FileDialog (Dialog parent):*** Creates a file dialog for loading a file.
  - ***FileDialog (Dialog parent, String title):*** Creates a file dialog window with the specified title for loading a file.
  - ***FileDialog (Dialog parent, String title, int mode):*** Creates a file dialog window with the specified title for loading or saving a file.
- If **mode** is ***FileDialog.LOAD***, then the box is selecting a file for reading.
- If **mode** is ***FileDialog.SAVE***, the box is selecting a file for writing.
- If mode is omitted, the box is selecting a file for reading.
- **Common methods**
  - ***String getDirectory ():*** Gets the directory of this file dialog.

- ***String getFile ():*** Gets the selected file of this file dialog.
- ***FilenameFilter getFilenameFilter():*** Determines this file dialog's filename filter.
- ***Int getMode ():*** Indicates whether this file dialog box is for loading from a file or for saving to a file.
- ***String paramString ():*** Returns a string representing the state of this FileDialog window.
- ***Void setDirectory (String dir):*** Sets the directory of this file dialog window to be the specified directory.
- ***Void setFile(String file):*** Sets the selected file for this file dialog window to be the specified file.
- ***Void setFilenameFilter (FilenameFilter filter):*** Sets the filename filter for this file dialog window to the specified filter.
- ***Void setMode (int mode):*** Sets the mode of the file dialog.

```
1  //menuBarExample.java
2  import javax.swing.*;
3  import java.awt.*;
4  import java.awt.event.*;
5  import java.io.*;
6  public class menuBarExample extends Frame implements ActionListener{
7      MenuBar menuBar;    Menu menu1, menu2;
8      MenuItem mItem1, mItem2, mItem3, mItem4;
9      MenuItem mItem5, mItem6, mItem7;
10     FileDialog fg;              Label label1;
11     menuBarExample(){
12         setTitle("MenuBar Example");
13         //Creating a menu bar
14         menuBar= new MenuBar();
15         //Creating and adding menu item in file menu
16         menu1 = new Menu("File");
17         menu1.add(mItem1=new MenuItem("New"));
18         menu1.add(mItem2=new MenuItem("Open"));
19         menu1.add(mItem3=new MenuItem("Exit"));
20         mItem4=new MenuItem("-");        menu1.add(mItem4);
21         //Creating a second sub-menu
22         menu2 = new Menu("Save-as");
23         menu2.add(mItem5=new MenuItem(".jpeg"));
24         menu2.add(mItem6=new MenuItem("png"));
25         menu2.add(mItem7=new MenuItem(".pdf"));
26         //Adding the sub-menu to the first menu
27         menu1.add(menu2);
28         //Adding our menu to the menu bar
29         menuBar.add(menu1);
30         //Adding listener to "Open" and "Exit" menu items
31         mItem2.addActionListener(this); mItem1.addActionListener(this);
32         mItem3.addActionListener(this);
33         label1 = new Label("", Label.CENTER);
34         //Adding my menu bar to the frame by calling setMenuBar() method
```

```
35          setMenuBar(menuBar);
36          add(label1,BorderLayout.CENTER);
37          setSize(350,250);
38          setVisible(true);
39      }// end of constructor
40      public void actionPerformed(ActionEvent ae){
41          if(ae.getActionCommand().equals("Open")){
42              //Creating an object of FileDialog
43              fg = new FileDialog(this, "Open a file");
44              fg.setVisible(true);
45              String file = fg.getDirectory()+ fg.getFile();
46              label1.setText("File to Open - " + file);
47          }
48          if(ae.getActionCommand().equals("Exit")){
49          System.exit(0);}
50          if(ae.getActionCommand().equals("New")){
51          JOptionPane.showMessageDialog(this,"You Pressed menu item--New");}
52      }// end of action performed function
53      public static void main(String[] ar){
54          new menuBarExample();}
55  }// end of class
```

Chapter 3
Event Handling

Two Event Handling Mechanisms

- The way in which events are handled changed significantly between the original version of Java (1.0) and all subsequent versions of Java, beginning with version 1.1.
- Although the ***1.0 method of event handling*** is still supported, it ***is not recommended*** for new programs. Also, many of the methods that support the old 1.0 event model have been deprecated.
- The modern approach is the way that events should be handled by all new programs.

The Delegation Event Model

- The modern approach to handling events is based on the delegation event model, which defines standard and consistent mechanisms to generate and process events.
- Its concept is quite simple: a ***source generates an event*** and ***sends*** it to one or ***more listeners***. In this scheme, the ***listener simply waits until it receives an event***. ***Once an event*** is received, the listener ***processes the event and then returns.***
- The advantage of this design is that ***the application logic*** that processes events is ***cleanly separated*** from the user ***interface logic*** that generates those events.
- A user interface element is able to "delegate" the processing of an event to a separate piece of code.
- In the delegation event model, listeners must register with a source in order to receive an event notification. ***This provides an important benefit:*** notifications are ***sent only to listeners*** that want to receive them.

Events

- In the delegation model, ***an event is an object*** that describes a state change in a source. An ***event can be generated*** as a consequence of a ***person interacting with the elements*** in a graphical user interface.
- Some of the ***activities that cause events to be generated*** are pressing a button, entering a character via the keyboard, selecting an item in a list, and clicking the mouse. Events may also occur that are not directly caused by interactions with a user interface.
- ***For example***, an event may be generated when a ***timer expires***, a counter exceeds a value, ***software or hardware failure occurs***, or an operation is completed.

Event Sources

- A ***source is an object that generates an event***. This occurs when the internal state of that object changes in some way. Sources may generate more than one type of event.
- A source ***must register listeners*** in order for the listeners ***to receive notifications about a specific type of event.***
- Each type of ***event has its own registration method***. Here is ***the general form:***

```
public void addTypeListener (TypeListener el )
```

Here, *Type* is the name of the event, and *el* is a reference to the event listener. For example, the method that registers a keyboard event listener is called **addKeyListener( )**. The method that registers a mouse motion listener is called **addMouseMotionListener( )**. When an event occurs, all registered listeners are notified and receive a copy of the event object. This is known as *multicasting* the event. In all cases, notifications are sent only to listeners that register to receive them.

Some sources may allow only one listener to register. The general form of such a method is this:

```
public void addTypeListener(TypeListener el )
    throws java.util.TooManyListenersException
```

Here, *Type* is the name of the event, and *el* is a reference to the event listener. When such an event occurs, the registered listener is notified. This is known as *unicasting* the event.

A source must also provide a method that allows a listener to unregister an interest in a specific type of event. The general form of such a method is this:

```
public void removeTypeListener(TypeListener el )
```

Here, *Type* is the name of the event, and *el* is a reference to the event listener. For example, to remove a keyboard listener, you would call **removeKeyListener( )**.

Event Listeners

- A ***listener is an object*** that is ***notified when an event occurs***. It has two major requirements.
    - ***First,*** it must have been ***registered with one or more sources*** to receive notifications about specific types of events.
    - ***Second,*** it ***must implement methods to receive and process*** these notifications. In other words, the listener must supply the event handlers.
- The methods that receive and process events ***are defined in a set of interfaces***, such as those found in ***java.awt.event***.
- For example, the ***MouseMotionListener*** interface defines two methods to receive notifications when the mouse is dragged or moved. Any object may handle one or both of these events if it provides an implementation of this interface.

Event Classes

- The classes that represent events are at the core of Java's event handling mechanism. Thus, a discussion of event handling must begin with the event classes.
- Arguably, the most widely used events at the time of this writing are those defined by the AWT and those defined by Swing.
- This chapter focuses on the AWT events. ***Most of these events also apply to Swing***.
- The package ***java.awt.event*** defines many types of events that are generated by various user interface elements. The ***below table*** shows several commonly used event classes and provides a brief description of when they are generated.

Event Class	Description
ActionEvent	Generated when a button is pressed, a list item is double-clicked, or a menu item is selected.
AdjustmentEvent	Generated when a scroll bar is manipulated.
ComponentEvent	Generated when a component is hidden, moved, resized, or becomes visible.
ContainerEvent	Generated when a component is added to or removed from a container.
FocusEvent	Generated when a component gains or loses keyboard focus.
InputEvent	Abstract superclass for all component input event classes.
ItemEvent	Generated when a check box or list item is clicked; also occurs when a choice selection is made or a checkable menu item is selected or deselected.
KeyEvent	Generated when input is received from the keyboard.
MouseEvent	Generated when the mouse is dragged, moved, clicked, pressed, or released; also generated when the mouse enters or exits a component.
MouseWheelEvent	Generated when the mouse wheel is moved.
TextEvent	Generated when the value of a text area or text field is changed.
WindowEvent	Generated when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

The ActionEvent Class

An **ActionEvent** is generated when a button is pressed, a list item is double-clicked, or a menu item is selected. The **ActionEvent** class defines four integer constants that can be used to identify any modifiers associated with an action event: **ALT_MASK**, **CTRL_MASK**, **META_MASK**, and **SHIFT_MASK**. In addition, there is an integer constant, **ACTION_PERFORMED**, which can be used to identify action events.

ActionEvent has these three constructors:

ActionEvent(Object *src*, int *type*, String *cmd*)
ActionEvent(Object *src*, int *type*, String *cmd*, int *modifiers*)
ActionEvent(Object *src*, int *type*, String *cmd*, long *when*, int *modifiers*)

Here, *src* is a reference to the object that generated this event. The type of the event is specified by *type*, and its command string is *cmd*. The argument *modifiers* indicates which modifier keys (ALT, CTRL, META, and/or SHIFT) were pressed when the event was generated. The *when* parameter specifies when the event occurred.

You can obtain the command name for the invoking **ActionEvent** object by using the **getActionCommand( )** method, shown here:

String getActionCommand( )

For example, when a button is pressed, an action event is generated that has a command name equal to the label on that button.

The AdjustmentEvent Class

An **AdjustmentEvent** is generated by a scroll bar. There are five types of adjustment events. The **AdjustmentEvent** class defines integer constants that can be used to identify them. The constants and their meanings are shown here:

BLOCK_DECREMENT	The user clicked inside the scroll bar to decrease its value.
BLOCK_INCREMENT	The user clicked inside the scroll bar to increase its value.
TRACK	The slider was dragged.
UNIT_DECREMENT	The button at the end of the scroll bar was clicked to decrease its value.
UNIT_INCREMENT	The button at the end of the scroll bar was clicked to increase its value.

In addition, there is an integer constant, **ADJUSTMENT_VALUE_CHANGED**, that indicates that a change has occurred.

Here is one **AdjustmentEvent** constructor:

AdjustmentEvent(Adjustable *src*, int *id*, int *type*, int *val*)

Here, *src* is a reference to the object that generated this event. The *id* specifies the event. The type of the adjustment is specified by *type*, and its associated value is *val*.

The **getAdjustable( )** method returns the object that generated the event. Its form is shown here:

Adjustable getAdjustable( )

The ComponentEvent Class

A **ComponentEvent** is generated when the size, position, or visibility of a component is changed. There are four types of component events. The **ComponentEvent** class defines integer constants that can be used to identify them. The constants and their meanings are shown here:

COMPONENT_HIDDEN	The component was hidden.
COMPONENT_MOVED	The component was moved.
COMPONENT_RESIZED	The component was resized.
COMPONENT_SHOWN	The component became visible.

ComponentEvent has this constructor:

ComponentEvent(Component *src*, int *type*)

Here, *src* is a reference to the object that generated this event. The type of the event is specified by *type*.

ComponentEvent is the superclass either directly or indirectly of **ContainerEvent**, **FocusEvent**, **KeyEvent**, **MouseEvent**, and **WindowEvent**, among others.

The ContainerEvent Class

A **ContainerEvent** is generated when a component is added to or removed from a container. There are two types of container events. The **ContainerEvent** class defines **int** constants that can be used to identify them: **COMPONENT_ADDED** and **COMPONENT_REMOVED**. They indicate that a component has been added to or removed from the container.

ContainerEvent is a subclass of **ComponentEvent** and has this constructor:

ContainerEvent(Component *src*, int *type*, Component *comp*)

Here, *src* is a reference to the container that generated this event. The type of the event is specified by *type*, and the component that has been added to or removed from the container is *comp*.

You can obtain a reference to the container that generated this event by using the **getContainer ( )** method, shown here:

Container getContainer( )

The **getChild( )** method returns a reference to the component that was added to or removed from the container. Its general form is shown here:

Component getChild( )

The FocusEvent Class

A **FocusEvent** is generated when a component gains or loses input focus. These events are identified by the integer constants **FOCUS_GAINED** and **FOCUS_LOST**.

FocusEvent is a subclass of **ComponentEvent** and has these constructors:

FocusEvent(Component *src*, int *type*)
FocusEvent(Component *src*, int *type*, boolean *temporaryFlag*)
FocusEvent(Component *src*, int *type*, boolean *temporaryFlag*, Component *other*)

Here, *src* is a reference to the component that generated this event. The type of the event is specified by *type*. The argument *temporaryFlag* is set to **true** if the focus event is temporary. Otherwise, it is set to **false**. (A temporary focus event occurs as a result of another user interface operation. For example, assume that the focus is in a text field. If the user moves the mouse to adjust a scroll bar, the focus is temporarily lost.)

The other component involved in the focus change, called the *opposite component*, is passed in *other*. Therefore, if a **FOCUS_GAINED** event occurred, *other* will refer to the component that lost focus. Conversely, if a **FOCUS_LOST** event occurred, *other* will refer to the component that gains focus.

The ItemEvent Class

An **ItemEvent** is generated when a check box or a list item is clicked or when a checkable menu item is selected or deselected. (Check boxes and list boxes are described later in this book.) There are two types of item events, which are identified by the following integer constants:

DESELECTED	The user deselected an item.
SELECTED	The user selected an item.

In addition, **ItemEvent** defines the integer constant, **ITEM_STATE_CHANGED**, that signifies a change of state.

ItemEvent has this constructor:

ItemEvent(ItemSelectable *src*, int *type*, Object *entry*, int *state*)

Here, *src* is a reference to the component that generated this event. For example, this might be a list or choice element. The type of the event is specified by *type*. The specific item that generated the item event is passed in *entry*. The current state of that item is in *state*.

The **getItem( )** method can be used to obtain a reference to the item that changed. Its signature is shown here:

Object getItem( )

The **getItemSelectable( )** method can be used to obtain a reference to the **ItemSelectable** object that generated an event. Its general form is shown here:

ItemSelectable getItemSelectable( )

Lists and choices are examples of user interface elements that implement the **ItemSelectable** interface.

The **getStateChange( )** method returns the state change (that is, **SELECTED** or **DESELECTED**) for the event. It is shown here:

int getStateChange( )

The MouseEvent Class

There are eight types of mouse events. The **MouseEvent** class defines the following integer constants that can be used to identify them:

MOUSE_CLICKED	The user clicked the mouse.
MOUSE_DRAGGED	The user dragged the mouse.
MOUSE_ENTERED	The mouse entered a component.
MOUSE_EXITED	The mouse exited from a component.
MOUSE_MOVED	The mouse moved.
MOUSE_PRESSED	The mouse was pressed.
MOUSE_RELEASED	The mouse was released.
MOUSE_WHEEL	The mouse wheel was moved.

MouseEvent is a subclass of **InputEvent**. Here is one of its constructors:

MouseEvent(Component *src*, int *type*, long *when*, int *modifiers*,
int *x*, int *y*, int *clicks*, boolean *triggersPopup*)

Here, *src* is a reference to the component that generated the event. The type of the event is specified by *type*. The system time at which the mouse event occurred is passed in *when*. The *modifiers* argument indicates which modifiers were pressed when a mouse event occurred. The coordinates of the mouse are passed in *x* and *y*. The click count is passed in *clicks*. The *triggersPopup* flag indicates if this event causes a pop-up menu to appear on this platform.

Two commonly used methods in this class are **getX( )** and **getY( )**. These return the X and Y coordinates of the mouse within the component when the event occurred. Their forms are shown here:

int getX( )
int getY( )

Alternatively, you can use the **getPoint( )** method to obtain the coordinates of the mouse. It is shown here:

Point getPoint( )

It returns a **Point** object that contains the X,Y coordinates in its integer members: **x** and **y**.

The **translatePoint( )** method changes the location of the event. Its form is shown here:

void translatePoint(int *x*, int *y*)

Here, the arguments *x* and *y* are added to the coordinates of the event.

The **getClickCount( )** method obtains the number of mouse clicks for this event. Its signature is shown here:

int getClickCount( )

The **isPopupTrigger( )** method tests if this event causes a pop-up menu to appear on this platform. Its form is shown here:

boolean isPopupTrigger( )

Also available is the **getButton( )** method, shown here:

int getButton( )

It returns a value that represents the button that caused the event. For most cases, the return value will be one of these constants defined by **MouseEvent**:

NOBUTTON	BUTTON1	BUTTON2	BUTTON3

The **NOBUTTON** value indicates that no button was pressed or released.

Also available are three methods that obtain the coordinates of the mouse relative to the screen rather than the component. They are shown here:

Point getLocationOnScreen( )

int getXOnScreen( )

int getYOnScreen( )

The **getLocationOnScreen( )** method returns a **Point** object that contains both the X and Y coordinate. The other two methods return the indicated coordinate.

The TextEvent Class

Instances of this class describe text events. These are generated by text fields and text areas when characters are entered by a user or program. **TextEvent** defines the integer constant **TEXT_VALUE_CHANGED**.

The one constructor for this class is shown here:

TextEvent(Object *src*, int *type*)

Here, *src* is a reference to the object that generated this event. The type of the event is specified by *type*.

The WindowEvent Class

There are ten types of window events. The **WindowEvent** class defines integer constants that can be used to identify them. The constants and their meanings are shown here:

WINDOW_ACTIVATED	The window was activated.
WINDOW_CLOSED	The window has been closed.
WINDOW_CLOSING	The user requested that the window be closed.
WINDOW_DEACTIVATED	The window was deactivated.
WINDOW_DEICONIFIED	The window was deiconified.
WINDOW_GAINED_FOCUS	The window gained input focus.
WINDOW_ICONIFIED	The window was iconified.
WINDOW_LOST_FOCUS	The window lost input focus.
WINDOW_OPENED	The window was opened.
WINDOW_STATE_CHANGED	The state of the window changed.

WindowEvent is a subclass of **ComponentEvent**. It defines several constructors. The first is

WindowEvent(Window *src*, int *type*)

Here, *src* is a reference to the object that generated this event. The type of the event is *type*.

The next three constructors offer more detailed control:

WindowEvent(Window *src*, int *type*, Window *other*)
WindowEvent(Window *src*, int *type*, int *fromState*, int *toState*)
WindowEvent(Window *src*, int *type*, Window *other*, int *fromState*, int *toState*)

Here, *other* specifies the opposite window when a focus or activation event occurs. The *fromState* specifies the prior state of the window, and *toState* specifies the new state that the window will have when a window state change occurs.

A commonly used method in this class is **getWindow( )**. It returns the **Window** object that generated the event. Its general form is shown here:

Window getWindow( )

WindowEvent also defines methods that return the opposite window (when a focus or activation event has occurred), the previous window state, and the current window state. These methods are shown here:

Window getOppositeWindow( )
int getOldState( )
int getNewState( )

Sources of Events

Event Source	Description
Button	Generates action events when the button is pressed.
Check box	Generates item events when the check box is selected or deselected.
Choice	Generates item events when the choice is changed.
List	Generates action events when an item is double-clicked; generates item events when an item is selected or deselected.
Menu item	Generates action events when a menu item is selected; generates item events when a checkable menu item is selected or deselected.
Scroll bar	Generates adjustment events when the scroll bar is manipulated.
Text components	Generates text events when the user enters a character.
Window	Generates window events when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

Table 24-2 Event Source Examples

Event Listener Interfaces

- The delegation event model has two parts: sources and listeners. Listeners are created by implementing one or more of the interfaces defined by the ***java.awt.event*** package.
- When an event occurs, the event source invokes the appropriate method defined by the listener and provides an event object as its argument.

Interface	Description
ActionListener	Defines one method to receive action events.
AdjustmentListener	Defines one method to receive adjustment events.
ComponentListener	Defines four methods to recognize when a component is hidden, moved, resized, or shown.
ContainerListener	Defines two methods to recognize when a component is added to or removed from a container.
FocusListener	Defines two methods to recognize when a component gains or loses keyboard focus.
ItemListener	Defines one method to recognize when the state of an item changes.
KeyListener	Defines three methods to recognize when a key is pressed, released, or typed.
MouseListener	Defines five methods to recognize when the mouse is clicked, enters a component, exits a component, is pressed, or is released.
MouseMotionListener	Defines two methods to recognize when the mouse is dragged or moved.
MouseWheelListener	Defines one method to recognize when the mouse wheel is moved.
TextListener	Defines one method to recognize when a text value changes.
WindowFocusListener	Defines two methods to recognize when a window gains or loses input focus.
WindowListener	Defines seven methods to recognize when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

Table 24-3 Commonly Used Event Listener Interfaces

The ActionListener Interface

This interface defines the **actionPerformed( )** method that is invoked when an action event occurs. Its general form is shown here:

```
void actionPerformed(ActionEvent ae)
```

The AdjustmentListener Interface

This interface defines the **adjustmentValueChanged( )** method that is invoked when an adjustment event occurs. Its general form is shown here:

```
void adjustmentValueChanged(AdjustmentEvent ae)
```

The ContainerListener Interface

This interface contains two methods. When a component is added to a container, **componentAdded( )** is invoked. When a component is removed from a container, **componentRemoved( )** is invoked. Their general forms are shown here:

```
void componentAdded(ContainerEvent ce)
void componentRemoved(ContainerEvent ce)
```

The FocusListener Interface

This interface defines two methods. When a component obtains keyboard focus, **focusGained( )** is invoked. When a component loses keyboard focus, **focusLost( )** is called. Their general forms are shown here:

```
void focusGained(FocusEvent fe)
void focusLost(FocusEvent fe)
```

The ItemListener Interface

This interface defines the **itemStateChanged( )** method that is invoked when the state of an item changes. Its general form is shown here:

```
void itemStateChanged(ItemEvent ie)
```

The MouseMotionListener Interface

This interface defines two methods. The **mouseDragged( )** method is called multiple times as the mouse is dragged. The **mouseMoved( )** method is called multiple times as the mouse is moved. Their general forms are shown here:

```
void mouseDragged(MouseEvent me)
void mouseMoved(MouseEvent me)
```

The MouseListener Interface

This interface defines five methods. If the mouse is pressed and released at the same point, **mouseClicked( )** is invoked. When the mouse enters a component, the **mouseEntered( )** method is called. When it leaves, **mouseExited( )** is called. The **mousePressed( )** and **mouseReleased( )** methods are invoked when the mouse is pressed and released, respectively.

The general forms of these methods are shown here:

void mouseClicked(MouseEvent *me*)
void mouseEntered(MouseEvent *me*)
void mouseExited(MouseEvent *me*)
void mousePressed(MouseEvent *me*)
void mouseReleased(MouseEvent *me*)

The TextListener Interface

This interface defines the **textValueChanged( )** method that is invoked when a change occurs in a text area or text field. Its general form is shown here:

void textValueChanged(TextEvent *te*)

The WindowListener Interface

This interface defines seven methods. The **windowActivated( )** and **windowDeactivated( )** methods are invoked when a window is activated or deactivated, respectively. If a window is iconified, the **windowIconified( )** method is called. When a window is deiconified, the **windowDeiconified( )** method is called. When a window is opened or closed, the **windowOpened( )** or **windowClosed( )** methods are called, respectively. The **windowClosing( )** method is called when a window is being closed. The general forms of these methods are

void windowActivated(WindowEvent *we*)
void windowClosed(WindowEvent *we*)
void windowClosing(WindowEvent *we*)
void windowDeactivated(WindowEvent *we*)
void windowDeiconified(WindowEvent *we*)
void windowIconified(WindowEvent *we*)
void windowOpened(WindowEvent *we*)

Using the Delegation Event Model

- Using the delegation event model is actually quite easy. Just follow these two steps:
  1. Implement the appropriate interface in the listener so that it can receive the type of event desired.
  2. Implement code to register and unregister (if necessary) the listener as a recipient for the event notifications.
- Remember that a source may generate several types of events. Each event must be registered separately. Also, an object may register to receive several types of events, but it must implement all of the interfaces that are required to receive these events.
- To see how the delegation model works in practice, we will look at examples that handle two commonly used event generators: the mouse and keyboard.

```
1  //TextEventExample.java
2  import java.awt.*;
3  import java.awt.event.*;
4  import javax.swing.*;
5  class TextEventExample extends Frame implements TextListener {
6     TextField tf;
7     public TextEventExample() {
8          setTitle("Text Listener Example");
9          setLayout(new FlowLayout());    setSize(250,100);
10         tf=new TextField(15);
11         tf.addTextListener(this);        add(tf);
12         // the windowClosing() method of WindowAdapter class
13         addWindowListener (new WindowAdapter() {
14             public void windowClosing (WindowEvent e) {
15                 dispose();    }
16         });
17         setVisible(true);
18    }
19    public void textValueChanged(TextEvent tet)  {
20         setTitle(tf.getText());
21    }
22    public static void main(String[] args){
23         new TextEventExample();
24    }
25 } // end of class
```

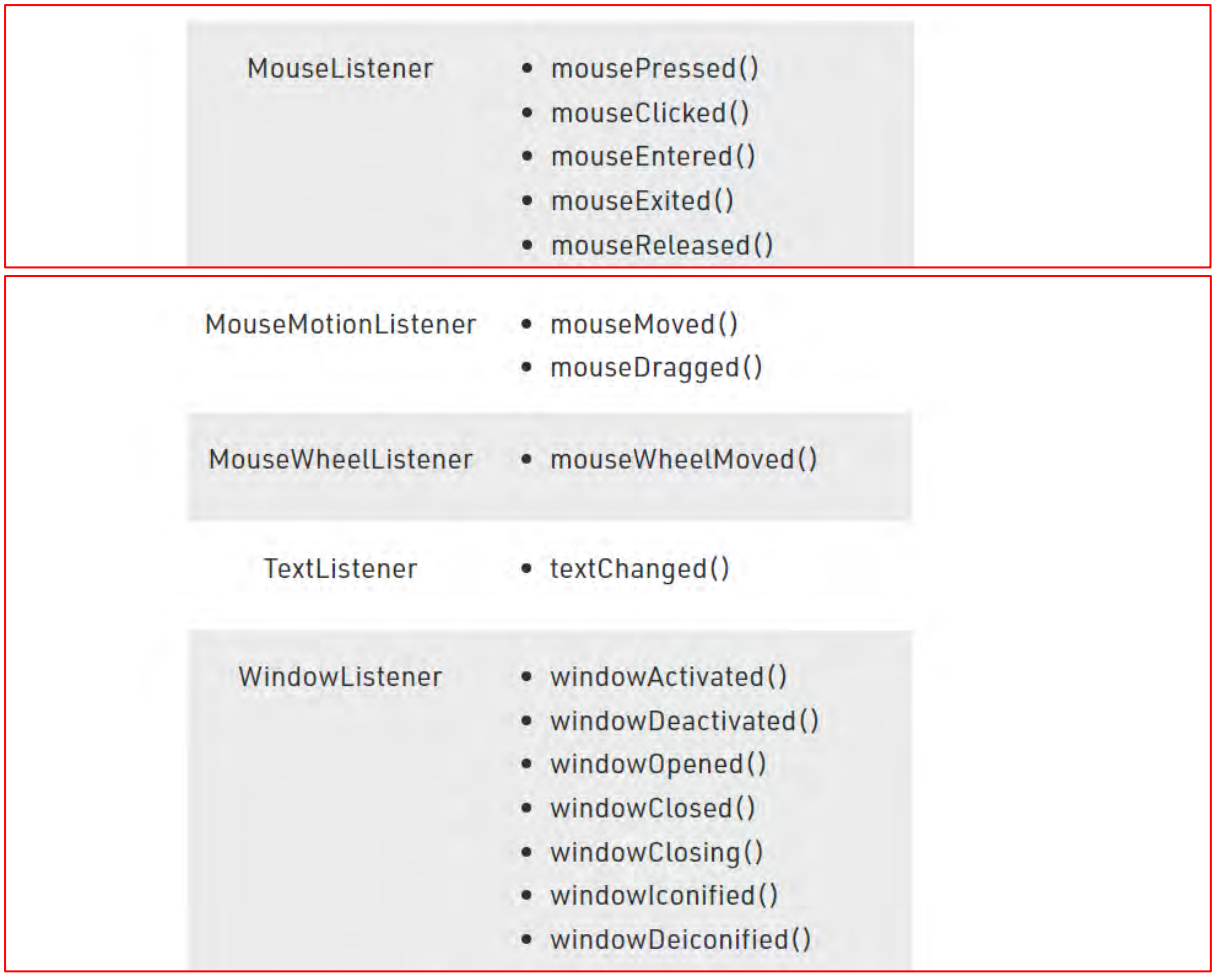
Event Classes in Java

Event Class	Listener Interface	Description
ActionEvent	ActionListener	An event that indicates that a component-defined action occurred like a button click or selecting an item from the menu-item list.
AdjustmentEvent	AdjustmentListener	The adjustment event is emitted by an Adjustable object like Scrollbar.
ComponentEvent	ComponentListener	An event that indicates that a component moved, the size changed or changed its visibility.
ContainerEvent	ContainerListener	When a component is added to a container (or) removed from it, then this event is generated by a container object.
FocusEvent	FocusListener	These are focus-related events, which include focus, focusin, focusout, and blur.
ItemEvent	ItemListener	An event that indicates whether an item was selected or not.
KeyEvent	KeyListener	An event that occurs due to a sequence of keypresses on the keyboard.
MouseEvent	MouseListener & MouseMotionListener	The events that occur due to the user interaction with the mouse (Pointing Device).

MouseWheelEvent	MouseWheelListener	An event that specifies that the mouse wheel was rotated in a component.
TextEvent	TextListener	An event that occurs when an object's text changes.
WindowEvent	WindowListener	An event which indicates whether a window has changed its status or not.

Listener Interface	Methods
ActionListener	• actionPerformed()
AdjustmentListener	• adjustmentValueChanged()
ComponentListener	• componentResized() • componentShown() • componentMoved() • componentHidden()
ContainerListener	• componentAdded() • componentRemoved()
FocusListener	• focusGained() • focusLost()

FocusListener	• focusGained() • focusLost()
ItemListener	• itemStateChanged()
KeyListener	• keyTyped() • keyPressed() • keyReleased()



MouseListener	• mousePressed() • mouseClicked() • mouseEntered() • mouseExited() • mouseReleased()
MouseMotionListener	• mouseMoved() • mouseDragged()
MouseWheelListener	• mouseWheelMoved()
TextListener	• textChanged()
WindowListener	• windowActivated() • windowDeactivated() • windowOpened() • windowClosed() • windowClosing() • windowIconified() • windowDeiconified()

Flow of Event Handling

1. User Interaction with a component is required to generate an event.
2. The object of the respective event class is created automatically after event generation, and it holds all information of the event source.
3. The newly created object is passed to the methods of the registered listener.
4. The method executes and returns the result.

Code-Approaches

The three approaches for performing event handling are by placing the event handling code in one of the below-specified places.

1. Within Class
2. Other Class
3. Anonymous Class

```java
//approachesWithinClass.java
import java.awt.*;
import java.awt.event.*;
class approachesWithinClass extends Frame implements ActionListener {
    TextField textField;
    approachesWithinClass(){
        setTitle(" Example for approaches Within Class");
        setSize(450,100);   setLayout(new FlowLayout());

        textField = new TextField(30);
        Button button = new Button("Click");
        button.addActionListener(this);
        add(textField); add(button);
        // the windowClosing() method of WindowAdapter class
        addWindowListener (new WindowAdapter() {
            public void windowClosing (WindowEvent e) {
                dispose(); }
        });
        setVisible(true);
    }
    public void actionPerformed(ActionEvent e)  {
        textField.setText("Implemented Within Class!!!");
    }
```

```java
//approachesOtherClass.java
import java.awt.*;
import java.awt.event.*;
class approachesOtherClass extends Frame {
    public TextField textField;
    approachesOtherClass()  {
        textField = new TextField(30);
        setTitle(" Example for approaches Other Class");
        setSize(450,100);   setLayout(new FlowLayout());

        textField = new TextField(30);
        Button button = new Button("Click");
        add(textField); add(button);
        EventHandler eventhandler = new EventHandler(this);
        button.addActionListener(eventhandler);

        // the windowClosing() method of WindowAdapter class
        addWindowListener (new WindowAdapter() {
            public void windowClosing (WindowEvent e) {
                dispose(); }
        });
        setVisible(true);
    }
```

```
    public static void main(String[] args)  {
        new approachesOtherClass();
    }
}// end of approachesOtherClass
// implementing the listener interface
class EventHandler implements ActionListener {
    approachesOtherClass obj;
    EventHandler(approachesOtherClass obj){
        this.obj=obj;
    }
    public void actionPerformed(ActionEvent e){
        obj.textField.setText("Aapproaches from Other Class!!!");
    }
}//end of class
```

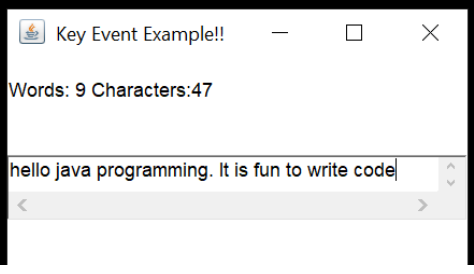
```
//usingAnonymousClass.java
import java.awt.*;
import java.awt.event.*;
class usingAnonymousClass extends Frame {
    TextField textField;
    usingAnonymousClass(){
        textField = new TextField(30);
        setTitle(" Example for approaches Other Class");
        setSize(450,100);   setLayout(new FlowLayout());

        textField = new TextField(30);
        Button button = new Button("Click");
        add(textField); add(button);
        // Registering component with listener anonymously
        button.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e){
                textField.setText("Implemented form Anonymous Class!!!");
            }
        });
        // the windowClosing() method of WindowAdapter class
        addWindowListener (new WindowAdapter() {
            public void windowClosing (WindowEvent e) {
                dispose(); }
        });
        setVisible(true);
    }
    public static void main(String[] args){
        new usingAnonymousClass();
    }
}// end of class
```

```java
//TextEventHandling.java
import java.awt.*;
import java.awt.event.*;

public class TextEventHandling extends Frame{
    Label label1,label2;   TextField txtField;

    public TextEventHandling() {
        setTitle("TextEvent Examples");
        setSize(300, 200);
        setLayout(new GridLayout(3, 1));

        label1 = new Label("Text Listener in Action", Label.CENTER);
        label2 = new Label(" ", Label.CENTER);
        txtField = new TextField(15);
        add(label1);   add(txtField);   add(label2);

        txtField.addTextListener(new TextListener() {
            @Override
            public void textValueChanged(TextEvent e) {
                label2.setText("Entered text: " + txtField.getText());
            }
        });
        // the windowClosing() method of WindowAdapter class
        addWindowListener (new WindowAdapter() {
            public void windowClosing (WindowEvent e) {
                dispose();    }
        });
        setVisible(true);
    }

    public static void main(String[] args) {
        new TextEventHandling();
    }
}
```

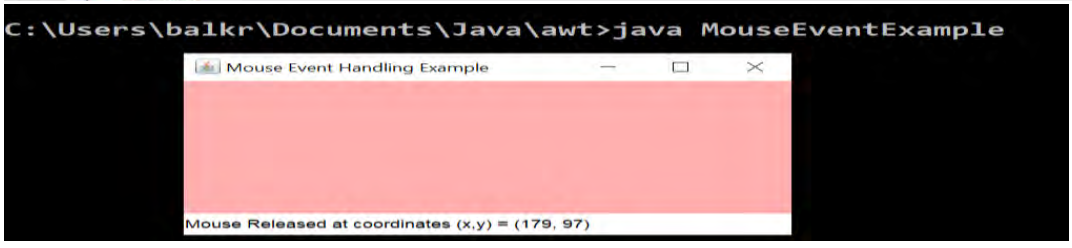
```java
//awtListEventHandling.java
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class awtListEventHandling extends Frame implements ItemListener {
    List ls;
    awtListEventHandling()  {
        setTitle("List Event Handling");
        setLayout(null);   setSize(300,200);
        Label lab1=new Label("Label Example");
        lab1.setBounds(80,40,100,20);    add(lab1);

        ls=new List(5);
        ls.setBounds(80,80,100,100);
        ls.add("Red");  ls.add("Yellow");    ls.add("Blue");
        ls.add("Green");   ls.add("Black");    add(ls);

        ls.addItemListener(this);
        addWindowListener (new WindowAdapter(){
            public void windowClosing(WindowEvent e){
                dispose();
        }});
        setVisible(true);
    }
    public void itemStateChanged(ItemEvent e){
        String msg;
        if(ls.getSelectedItem()=="Red")
            msg="The selected color is Red.";
        else if(ls.getSelectedItem()=="Yellow")
            msg="The selected color is Yellow.";
        else if(ls.getSelectedItem()=="Blue")
            msg="The selected color is Blue.";
        else if(ls.getSelectedItem()=="Green")
            msg="The selected color is Green.";
        else
            msg="The selected color is Black.";

        JOptionPane.showMessageDialog(this, msg);
    }
    public static void main(String args[]){
        new awtListEventHandling();
    }
} // end of class
```

```java
// keyEventExample.java
import java.awt.*;
import java.awt.event.*;
public class keyEventExample extends Frame implements KeyListener {
    Label label;     TextArea ta;
    keyEventExample() {
        setTitle(" Key Event Example!!");
        setLayout(new GridLayout(3,1, 20, 20));
        label= new Label();
        //label.setBounds (20, 50, 200, 20);
        ta = new TextArea();
        //ta.setBounds (20, 80, 300, 300);
        ta.addKeyListener(this);
        add(label);    add(ta);
        // the windowClosing() method of WindowAdapter class
        addWindowListener (new WindowAdapter() {
            public void windowClosing (WindowEvent e) {
                dispose(); }
        });
        setSize(300,200);
        setVisible (true);
    }// end of constructor
    public void keyPressed(KeyEvent e) {}

    public void keyReleased (KeyEvent e) {
        String text = ta.getText();
        String words[] = text.split ("\\s");
        label.setText ("Words: " + words.length + " Characters:" + text.length());
    }

    public void keyTyped(KeyEvent e) {}

    public static void main(String[] args) {
        new keyEventExample();
    } // end of main class
}// end of class
```

```
C:\Users\balkr\Documents\Java\awt>java keyEventExample
```

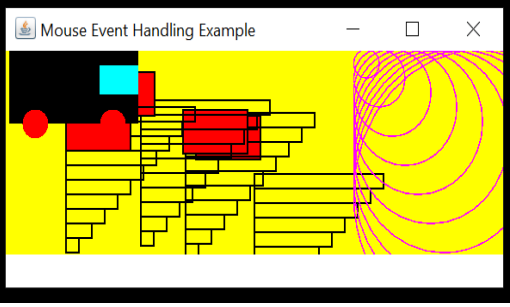


```java
//MouseEventExample.java
import java.awt.Color;
import java.awt.*;
import java.awt.event.*;
class MouseEventExample extends Frame implements MouseListener {
    Label label;
    public MouseEventExample(){
        setTitle("Mouse Event Handling Example");  setSize(400,200);
        // adding componebts to the layout
        // adding componebts to the layout
        setLayout(new BorderLayout(25,25));
        label=new Label();  add(label, BorderLayout.SOUTH);
        addMouseListener(this);
        // the windowClosing() method of WindowAdapter class
        addWindowListener (new WindowAdapter() {
            public void windowClosing (WindowEvent e) {
                dispose();    }
        });
        setVisible(true);
    }// end of constructor
    public void mousePressed(MouseEvent me) {
        setBackground(Color.green);
        String msg= new String("Mouse Pressed at coordinates (x,y) = ");
        label.setText(msg +"("+me.getX()+", "+me.getY()+")");
    }
    public void mouseReleased(MouseEvent me){
        setBackground(Color.pink);
        String msg= new String("Mouse Released at coordinates (x,y) = ");
        label.setText(msg +"("+me.getX()+", "+me.getY()+")");
    }
    public void mouseClicked(MouseEvent me) {
        setBackground(Color.yellow);
        String msg= new String("Mouse Clicked at coordinates (x,y) = ");
        label.setText(msg +"("+me.getX()+", "+me.getY()+")");
    }
    public void mouseEntered(MouseEvent me) {
        setBackground(Color.red);
        String msg= new String("Mouse Entered at coordinates (x,y) = ");
        label.setText(msg +"("+me.getX()+", "+me.getY()+")");
    }
    public void mouseExited(MouseEvent me)  {
        setBackground(Color.white);
        String msg= new String("Mouse Exited at coordinates (x,y) = ");
        label.setText(msg +"("+me.getX()+", "+me.getY()+")");
    }
    public static void main(String[] args){
         new MouseEventExample();
    }
}//end of class
```



```java
//mouseEventPaintExample.java
import java.awt.Color;
import java.awt.*;
import java.awt.event.*;
class mouseEventPaintExample extends Frame implements MouseListener {
    Label label;
    public mouseEventPaintExample(){
        setTitle("Mouse Event Handling Example");  setSize(400,200);
        // adding componebts to the layout
        // adding componebts to the layout
        setLayout(new BorderLayout(25,25));
        label=new Label();  add(label, BorderLayout.SOUTH);
        addMouseListener(this);
        // the windowClosing() method of WindowAdapter class
        addWindowListener (new WindowAdapter() {
            public void windowClosing (WindowEvent e) {
                dispose();    }
        });
        setVisible(true);
    }// end of constructor
    public void mousePressed(MouseEvent e) {
        Graphics g=getGraphics();
        for (int i = 0; i < 10; i++)
                g.drawRect(e.getX(), e.getY() + 10 * i,100 - 10 * i, 10);
    }
    public void mouseReleased(MouseEvent e){
        Graphics g=getGraphics();
        // inner red fill
        g.setColor(Color.RED);
        g.fillRect(e.getX(), e.getY(),50,30);
        // black outline
        g.setColor(Color.BLACK);
        g.drawRect(e.getX(), e.getY(),50,30);

    }
    public void mouseClicked(MouseEvent e)  {
        Graphics g=getGraphics();
        g.setColor(Color.BLACK);
        g.fillRect(10, 30, 100, 50);
        g.setColor(Color.RED);
        g.fillOval(20, 70, 20, 20);
        g.fillOval(80, 70, 20, 20);
        g.setColor(Color.CYAN);
        g.fillRect(80, 40, 30, 20);
    }
    public void mouseEntered(MouseEvent e)  {
        repaint();
    }
    public void mouseExited(MouseEvent e) {
        Graphics g=getGraphics();
        setBackground(Color.YELLOW);
        g.setColor(Color.MAGENTA);
        for (int i = 1; i <= 10; i++)
            g.drawOval(e.getX(), e.getY(), 20 * i, 20 * i);
    }
    public static void main(String[] args){
        new mouseEventPaintExample();
    }
}//end of class
```

```
C:\Users\balkr\Documents\Java\awt>java mouseEventPaintExample
```

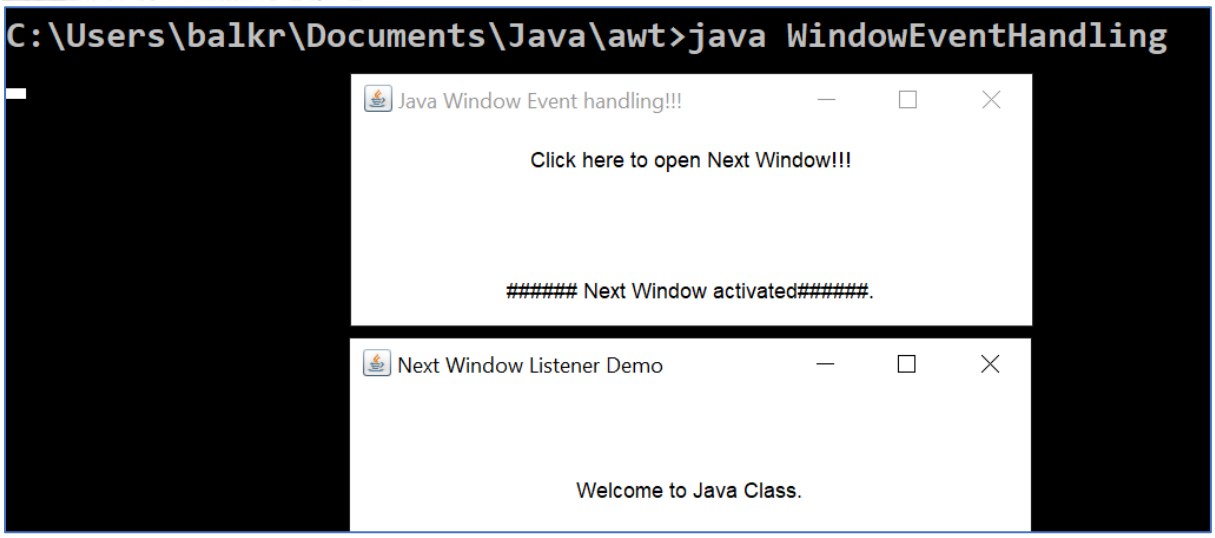



```java
//mouseMotionEventExample.java
import java.awt.*;
import java.awt.event.*;
class mouseMotionEventExample extends Frame implements MouseMotionListener{
    mouseMotionEventExample(){
        setTitle("Mouse Motion Drawing!!!");  setSize(300,200);
        setLayout(null);
        addMouseMotionListener(this);
        // the windowClosing() method of WindowAdapter class
        addWindowListener (new WindowAdapter() {
            public void windowClosing (WindowEvent e) {
                dispose();    }
        });
        setVisible(true);
    }
    public void mouseDragged(MouseEvent e) {
        Graphics g=getGraphics();
        g.setColor(Color.BLUE);
        g.fillOval(e.getX(),e.getY(),10,10);
    }
    public void mouseMoved(MouseEvent e) {}

    public static void main(String[] args) {
        new mouseMotionEventExample();
    } // end of main function
} // end of class
```

```
C:\Users\balkr\Documents\Java\awt>java mouseMotionEventExample
```

```java
//WindowEventHandling.java
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class WindowEventHandling {
    private Frame mainFrame,nextFrame;
    private Label mainLabel, nextFrameLabel;
    private Panel panel;

    public WindowEventHandling() {
        mainFrame = new Frame("Java Window Event handling!!!");
        mainFrame.setSize(400, 150);
        mainFrame.setLayout(new GridLayout(3, 1));

        mainLabel = new Label();     nextFrameLabel = new Label();
        mainLabel.setAlignment(Label.CENTER);
        nextFrameLabel.setAlignment(Label.CENTER);
        nextFrameLabel.setSize(100, 100);

        panel = new Panel();
        panel.setLayout(new FlowLayout());

        mainFrame.add(mainLabel);
        mainFrame.add(panel);
        mainFrame.add(nextFrameLabel);
        mainFrame.setVisible(true);

        mainLabel.setText("Click here to open Next Window!!!");
        nextFrame = new Frame("Next Window Listener Demo");
        nextFrame.setSize(400, 150);;

        nextFrame.addWindowListener(new CustomWindowListener());

        Label msgLabel = new Label("Welcome to Java Class.");
        msgLabel.setAlignment(Label.CENTER);
        msgLabel.setSize(100, 100);
        nextFrame.add(msgLabel);
        // the windowClosing() method of WindowAdapter class
        mainFrame.addWindowListener (new WindowAdapter() {
            public void windowClosing (WindowEvent e){
                mainFrame.dispose();  }
        });
        mainLabel.addMouseListener(new MouseAdapter() {
            @Override
            public void mouseClicked(MouseEvent e) {
                nextFrame.setVisible(true);}
        });

    }
```

```
50      class CustomWindowListener implements WindowListener {
51          public void windowOpened(WindowEvent e) {
52              nextFrameLabel.setText("Next Window Opened.");
53              }
54          public void windowClosing(WindowEvent e) {
55                      nextFrame.dispose();
56              }
57          public void windowClosed(WindowEvent e) {
58              nextFrameLabel.setText("###### Next Window closed######.");
59              }
60          public void windowIconified(WindowEvent e) {
61               JOptionPane.showMessageDialog(null,"Next Window Iconified.");
62              }
63          public void windowDeiconified(WindowEvent e) {
64              JOptionPane.showMessageDialog(null,"Next Window Deiconified.");
65              }
66          public void windowActivated(WindowEvent e) {
67              nextFrameLabel.setText("###### Next Window activated######.");
68              }
69          public void windowDeactivated(WindowEvent e) {
70              nextFrameLabel.setText("######Next Window deactivated######.");
71              }
72      }// end of CustomWindowListener class
73      public static void main(String[] args) {
74          new WindowEventHandling();
75      }// end of main
76  }// end of WindowEventHandling class
```



Adapter Class

- Java provides a special feature, called an adapter class, that can simplify the creation of event handlers in certain situations.
- An ***adapter class*** provides ***an empty implementation of all methods*** in an event listener interface. Adapter classes are useful when you want to receive and process only some of the events that are handled by a particular event listener interface. You ***can define a new class to act*** as an event listener ***by extending one of the adapter classes*** and implementing only those ***events in which you are interested.***

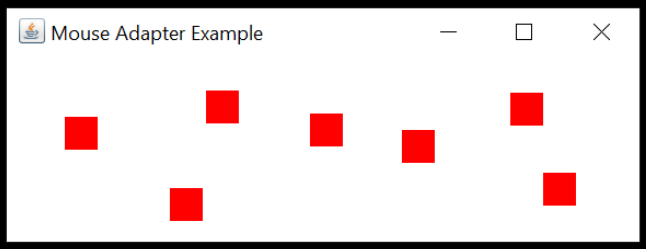
- For example, the ***MouseMotionAdapter*** class has two methods, ***mouseDragged() and mouseMoved(),*** which are the methods defined by the ***MouseMotionListener*** interface. If you were interested in ***only mouse drag events***, then you could simply extend ***MouseMotionAdapter*** and ***override mouseDragged().*** The empty implementation of mouseMoved() would handle the mouse motion events for you.

Adapter Class	Listener Interface
ComponentAdapter	ComponentListener
ContainerAdapter	ContainerListener
FocusAdapter	FocusListener
KeyAdapter	KeyListener
MouseAdapter	MouseListener, MouseMotionListener, and MouseWheelListener
MouseMotionAdapter	MouseMotionListener
WindowAdapter	WindowListener, WindowFocusListener, and WindowStateListener

Table 24-4 Commonly Used Listener Interfaces Implemented by Adapter Classes

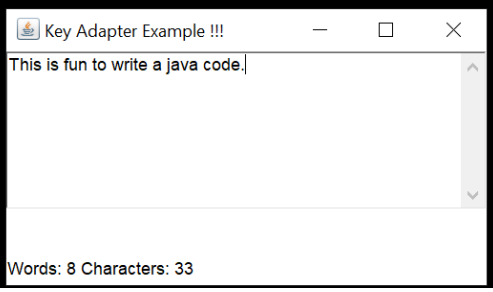
```
1  //MouseAdapterExample.java
2  import java.awt.*;
3  import java.awt.event.*;
4  public class MouseAdapterExample extends MouseAdapter{
5      Frame frame;
6      MouseAdapterExample(){
7          frame=new Frame("Mouse Adapter Example");
8          frame.addMouseListener(this);
9          frame.setSize(400,150);
10         frame.setLayout(null);
11         // the windowClosing() method of WindowAdapter class
12         frame.addWindowListener (new WindowAdapter() {
13             public void windowClosing (WindowEvent e) {
14                 frame.dispose(); }
15         });
16         frame.setVisible(true);
17     }
18     public void mouseClicked(MouseEvent e) {
19         Graphics g=frame.getGraphics();
20         g.setColor(Color.RED);
21         g.fillRect(e.getX(),e.getY(),20,20);
22      }
24         new MouseAdapterExample();
25     } // end of main function
26 } // end of main class
```

```
C:\Users\balkr\Documents\Java\awt>java MouseAdapterExample
```




```java
// KeyAdapterExample.java
import java.awt.*;
import java.awt.event.*;
public class KeyAdapterExample extends KeyAdapter{
    Label label;  TextArea tarea;  Frame frame;
    KeyAdapterExample(){
        frame=new Frame("Key Adapter Example !!!");
        frame.setSize(350,200);
        frame.setLayout(new BorderLayout(25,25));
        label=new Label();
        frame.add(label, BorderLayout.SOUTH);
        tarea=new TextArea(6,30);
        frame.add(tarea, BorderLayout.NORTH);
        tarea.addKeyListener(this);
        // the windowClosing() method of WindowAdapter class
        frame.addWindowListener (new WindowAdapter() {
            public void windowClosing (WindowEvent e) {
                frame.dispose(); }
        });
        frame.setVisible(true);
    }// end of constructor
    public void keyReleased(KeyEvent e) {
        String text=tarea.getText();
        String words[]=text.split("\s");
        label.setText("Words: "+ words.length +" Characters: "+text.length());
    }
    public static void main(String[] args) {
        new KeyAdapterExample();
    }// end of main function
}// end of class
```

```
C:\Users\balkr\Documents\Java\awt>java KeyAdapterExample
```

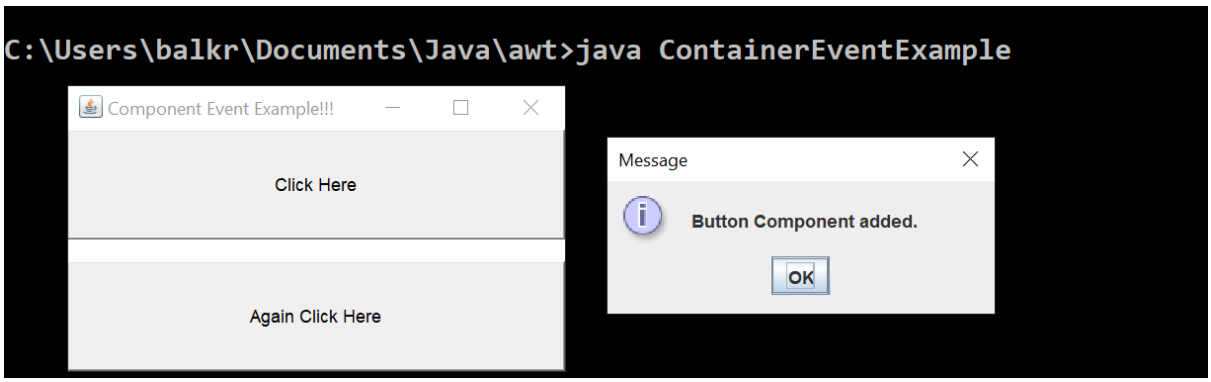


```java
//MouseMotionAdapterExample.java
import java.awt.*;
import java.awt.event.*;
public class MouseMotionAdapterExample extends MouseMotionAdapter{
    Frame frame;
    MouseMotionAdapterExample(){
        frame=new Frame("Mouse Motion Adapter");
        frame.addMouseMotionListener(this);
        frame.setSize(400,200);    frame.setLayout(null);
        // the windowClosing() method of WindowAdapter class
        frame.addWindowListener (new WindowAdapter() {
            public void windowClosing (WindowEvent e) {
                frame.dispose(); }
        });
        frame.setVisible(true);
    }
    public void mouseDragged(MouseEvent e) {
        Graphics g=frame.getGraphics();
        g.setColor(Color.ORANGE);
        g.fillOval(e.getX(),e.getY(),10,10);
    }
    public static void main(String[] args) {
        new MouseMotionAdapterExample();
    }// end of main function
}// end of class
```

```
C:\Users\balkr\Documents\Java\awt>java MouseMotionAdapterExample
```

```java
//ContainerEventExample.java
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class ContainerEventExample {
    public ContainerEventExample() {
        Frame f = new Frame("Component Event Example!!!");
        f.setLayout(new GridLayout(2,1,15,15));
        Button b1 = new Button("Click Here");
        Button b2 = new Button("Again Click Here");
        f.setSize(350, 200);  f.add(b1);

        f.addContainerListener(new ContainerListener() {
            public void componentAdded(ContainerEvent e) {
                JOptionPane.showMessageDialog(null,"Button Component added.");
            }
            @Override
            public void componentRemoved(ContainerEvent e) {
                JOptionPane.showMessageDialog(null,"Buttin Component removed.");
            }
        });

```

```
23          b1.addActionListener(new ActionListener() {
24              @Override
25              public void actionPerformed(ActionEvent e) {
26                  f.add(b2); f.setVisible(true);}
27          });
28
29          b2.addActionListener(new ActionListener() {
30              @Override
31              public void actionPerformed(ActionEvent e) {
32                  f.remove(b2);  f.setVisible(true);}
33          });
34          // the windowClosing() method of WindowAdapter class
35          f.addWindowListener (new WindowAdapter() {
36              public void windowClosing (WindowEvent e) {
37                  f.dispose(); }
38          });
39          f.setVisible(true);
40      }
41      public static void main(String[] args) {
42          new ContainerEventExample();
43      }// end of main function
44  }// end of class
```



Chapter – 4 and 5
Java Swing

Java AWT problems

1. First, because ***of variations between operating systems***, a component might look, or even act, ***differently on different platforms***. This potential variability threatened the overarching philosophy of Java: write once, run anywhere.
2. Second, ***the look and feel of each component was fixed*** (because it is defined by the platform) and could not be (easily) changed.
3. Third, the use of ***heavyweight components caused some frustrating restrictions***. For example, a heavyweight component was always opaque.

- The ***solution was Swing***. Introduced in 1997, Swing was included as part of the Java Foundation Classes (JFC). Swing was initially available for use with Java 1.1 as a separate library.
- Swing is ***built on the foundation of the AWT***. This is why the AWT is still a crucial part of Java. ***Swing also uses the same event handling mechanism as the AWT***. Therefore, a basic understanding of the AWT and of event handling is required to use Swing.

Two Key Swing Features

1. ***Swing Components Are Lightweight:*** This means that they ***are written entirely in Java*** and ***do not map directly to platform-specific peers***. Thus, lightweight components are ***more efficient and more flexible***. Furthermore, because lightweight components do not translate into native peers, ***the look and feel of each component is determined by Swing***, not by the underlying operating system. As a result, each component will work in a consistent manner across all platforms.

2. ***Swing Supports a Pluggable Look and Feel:*** Each Swing component is rendered by Java code rather than by native peers, the look and feel of a component is under the control of Swing. ***Advantages of Pluggable look-and-feels***
   - Possible to define a look and feel that is consistent across all platforms.
   - It is also possible to design a custom look and feel.
   - The look and feel can be changed dynamically at run time.

   Java provides look-and-feels, such as metal and Nimbus. The metal look and feel is also called the Java look and feel. ***Default Java look and feel is metal*** because it is platform independent.

The MVC Connection

In general, a visual component is a composite of three distinct aspects:

- The way that the component looks when rendered on the screen.
- The way that the component reacts to the user.
- The state information associated with the component.

Over the years, one component architecture has proven itself to be exceptionally effective: ***Model-View-Controller***, or ***MVC for short***. In MVC terminology,

- ***Model:*** The model corresponds to the state information associated with the component. For example, in the case of a check box, the model contains a field that indicates if the box is checked or unchecked.
- ***View:*** The view determines how the component is displayed on the screen, including any aspects of the view that are affected by the current state of the model.
- ***Controller:*** The controller determines how the component reacts to the user. For example, when the user clicks a check box, the controller reacts by changing the model to reflect the user's choice (checked or unchecked).

Swing uses a ***modified version of MVC that*** combines the ***view and the controller into a single logical entity*** called the UI delegate.

Swing's approach is called either the ***Model-Delegate architecture*** or the Separable Model architecture. ***Swing's pluggable look and feel is made possible by its Model-Delegate architecture.***

To support the Model-Delegate architecture, most Swing ***components contain two objects***. The ***first represents the model***. The ***second represents the UI delegate***. Models are defined by interfaces.

Components and Containers

A Swing GUI consists of two key items: ***components and containers.***

- The difference between the two is found in their intended purpose: As the term is commonly used, ***a component is an independent visual control***, such as a push button or slider.
- A ***container holds a group of components***. Thus, a container is a ***special type of component*** that is designed to hold other components.
- Furthermore, in order for a component to be displayed, it must be held within a container. Thus, ***all Swing GUIs will have at least one container***.
- Because ***containers are components***, a container can also hold other containers. This ***enables Swing to define what is called a containment hierarchy***, at the top of which must be ***a top-level container.***

Components

- In general, Swing components are derived from the ***JComponent*** class**.**
- ***JComponent*** provides the functionality that ***is common to all components***. For example, JComponent supports the pluggable look and feel.
- ***JComponent*** inherits the ***AWT classes Container and Component***. Thus, a Swing component is built on and compatible with an AWT component.
- All of Swing's components are represented by classes defined within the package ***javax.swing***.
- The ***following table shows the class names for Swing components*** (including those used as containers).

JApplet (Deprecated)	JButton	JCheckBox	JCheckBoxMenuItem
JColorChooser	JComboBox	JComponent	JDesktopPane
JDialog	JEditorPane	JFileChooser	JFormattedTextField
JFrame	JInternalFrame	JLabel	JLayer
JLayeredPane	JList	JMenu	JMenuBar
JMenuItem	JOptionPane	JPanel	JPasswordField
JPopupMenu	JProgressBar	JRadioButton	JRadioButtonMenuItem
JRootPane	JScrollBar	JScrollPane	JSeparator
JSlider	JSpinner	JSplitPane	JTabbedPane
JTable	JTextArea	JTextField	JTextPane
JTogglebutton	JToolBar	JToolTip	JTree
JViewport	JWindow		

Containers: Swing defines two types of containers.

1. ***The first are top-level containers: JFrame, JApplet, JWindow, and JDialog.*** *These containers do not inherit JComponent.* They do, however, inherit the AWT classes Component and Container. Unlike Swing's other components, which are lightweight, the top-level containers are heavyweight. This makes the top-level containers a special case in the Swing component library.
2. ***The second type of containers supported by Swing are lightweight containers.*** Lightweight containers do inherit ***JComponent.*** An example of a lightweight container is ***JPanel***, which is a general-purpose container.

- Lightweight containers are often used to ***organize and manage groups*** of related components because ***a lightweight container can be contained within another container***. Thus, you can use lightweight containers such as ***JPanel*** to create subgroups of related controls ***that are contained within an outer container.***
- Each top-level container defines a set of panes. At the top of the hierarchy is an instance of ***JRootPane***. JRootPane is a lightweight container whose purpose is to

manage the other panes. It also helps manage the optional menu bar. The panes that comprise the root pane are called the glass pane, the content pane, and the layered pane.

JLabel	A JLabel is an object component for placing text in a container
JButton	This class creates a labeled button
JColorChooser	A JColorChooser provides a pane of controls designed to allow the user to manipulate and select a color
JCheckBox	A JCheckBox is a graphical(GUI) component that can be in either an on-(true) or off-(false) state
JRadioButton	The JRadioButton class is a graphical(GUI) component that can be in either an on-(true) or off-(false) state. in the group
JList	A JList component represents the user with the scrolling list of text items
JComboBox	A JComboBox component is Presents the User with a show up Menu of choices
JTextField	A JTextField object is a text component that will allow for the editing of a single line of text
JPasswordField	A JPasswordField object it is a text component specialized for password entry
JTextArea	A JTextArea object s a text component that allows for the editing of multiple lines of text
ImageIcon	A ImageIcon control is an implementation of the Icon interface that paints Icons from Images
JScrollbar	A JScrollbar control represents a scroll bar component in order to enable users to Select from range values
JOptionPane	JOptionPane provides set of standard dialog boxes that prompt users for a value or Something

Java Swing Examples

```java
//FirstSwingExample.java
import javax.swing.*;
class FirstSwingExample {
    public static void main(String args[]) {
        // Create a new JFrame container.
        JFrame jfrm = new JFrame("A Simple Swing Application");
        // Create a text-based label.
        JLabel jlab = new JLabel(" Swing means powerful GUIs.");
        // Add the label to the content pane.
        jfrm.add(jlab);
        // Display the frame.
        jfrm.setVisible(true);
        jfrm.setSize(400,400);
        jfrm.setLayout(null);
        jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

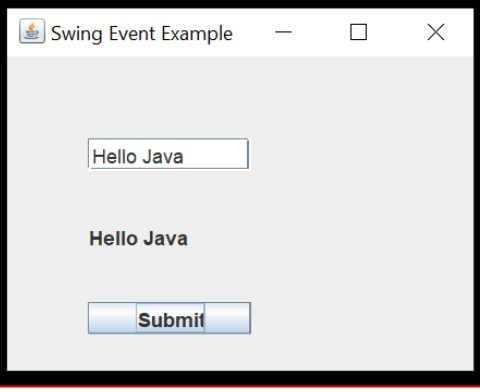
```java
//ButtonExample.java
import java.awt.event.*;
import javax.swing.*;
public class ButtonExample {
    public static void main(String[] args) {
        JFrame f=new JFrame("Button Example");
        JTextField tf=new JTextField();
        tf.setBounds(50,50, 100,20);

        JButton b=new JButton("Click Here");
        b.setBounds(50,100,95,30);

        b.addActionListener(new ActionListener(){
            public void actionPerformed(ActionEvent e){
                    tf.setText("Welcome to Swing.");
                }
            });
        f.add(b);           f.add(tf);      f.setSize(300,300);
        f.setLayout(null);          f.setVisible(true);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

```java
//ButtonExample2.java
import java.awt.event.*;
import javax.swing.*;
import java.awt.*;
public class ButtonExample2 extends JFrame{
    ButtonExample2(){
        setTitle("Button Event Example");

        JButton b=new JButton("Click");
        add(b); //adding button on frame
        JTextField tf=new JTextField();
        add(tf);//adding tf on frame

        b.addActionListener(new ActionListener(){
            public void actionPerformed(ActionEvent e){
                    tf.setText("Swing Text Field.");
                }
            });
        setSize(400,200);       setVisible(true);
        setLayout(new FlowLayout());
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
    public static void main(String[] args) {
        ButtonExample2 bt2=new ButtonExample2(); }      //end of main function
}// end of class
```

```
C:\Users\balkr\Documents\Java\swing>javac ButtonExample2.java

C:\Users\balkr\Documents\Java\swing>java ButtonExample2
```

```java
//swingEventExample.java
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class swingEventExample extends JFrame implements ActionListener{
    JTextField tfield; JLabel label; JButton btn;
    swingEventExample(){
        setTitle("Swing Event Example");
        tfield=new JTextField();     tfield.setBounds(50,50, 100,20);
        label=new JLabel();          label.setBounds(50,100, 100,20);
        btn=new JButton("Submit");   btn.setBounds(50,150,100,20);

        btn.addActionListener(this);
        add(btn);               add(tfield);              add(label);

        setSize(300,300);        setLayout(null);       setVisible(true);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }  // end of constructor
    public void actionPerformed(ActionEvent e) {
        String str=tfield.getText();
        label.setText(str);      }    // end of actioner listerner class

    public static void main(String[] args) {
        new swingEventExample();  } // end of main function
}  // end of class
```

```
C:\Users\balkr\Documents\Java\swing>javac swingEventExample.java

C:\Users\balkr\Documents\Java\swing>java swingEventExample
```



JComboBox

- JComboBox inherits JComponent class. Swing provides a combo box (a combination of a text field and a drop-down list) through the JComboBox class.
- A combo box normally displays one entry, but it will also display a drop-down list that allows a user to select a different entry.
- JComboBox shows a popup menu that shows a list and the user can select an option from that specified list.
- JComboBox can be editable or read- only depending on the choice of the programmer.

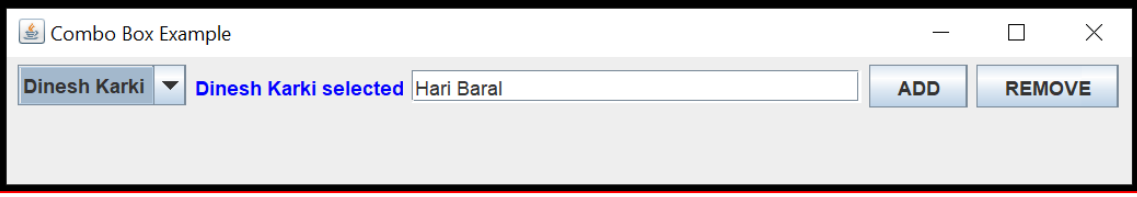
- ***Constructors of the JComboBox are:***
  1. ***JComboBox()*** : creates a new empty JComboBox .
  2. ***JComboBox(ComboBoxModel M)*** : creates a new JComboBox with items from specified ComboBoxModel
  3. ***JComboBox(E [] items)*** : creates a new JComboBox with items. Here items is an array that initializes the combo box.
  4. ***JComboBox(Vector items)*** : creates a new JComboBox with items from the specified vector
- ***Some commonly used Methods are:***
  1. ***addItem(E item)***: adds the item to the JComboBox
  2. ***addItemListener(ItemListener l) :*** adds a ItemListener to JComboBox
  3. ***getItemAt(int i)***: returns the item at index i.
  4. ***getItemCount():*** returns the number of items from the list
  5. ***getSelectedItem():*** returns the item which is selected
  6. ***removeItemAt(int i)***: removes the element at index i
  7. ***setEditable(boolean b):*** the boolean b determines whether the combo box is editable or not .If true is passed then the combo box is editable or vice versa.
  8. ***setSelectedIndex(int i):*** selects the element of JComboBox at index i.
  9. ***showPopup():*** causes the combo box to display its popup window.
  10. ***setEnabled(boolean b):*** enables the combo box so that items can be selected.
  11. ***removeItem(Object anObject)***: removes an item from the item list.
  12. ***getItemCount()***: returns the number of items in the list.

```java
//JComboBoxExample.java
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class JComboBoxExample extends JFrame implements ActionListener{
    String names[] ={"Ram Sharma", "Hari Thapa", "Dinesh Karki"};
    JComboBox<String> jcb;              JLabel label;
    JComboBoxExample(){
        setTitle("Combo Box Example");   setLayout(new FlowLayout());
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);   setSize(400,250);
        // create checkbox
        jcb = new JComboBox<String>(names);
        jcb.addActionListener(this);                add(jcb);

        label = new JLabel("select Name");   label.setForeground(Color.blue);
        add(label);
        setVisible(true);
    }  // end of constructor
    public void actionPerformed(ActionEvent e) {
        String str= jcb.getSelectedItem();
        label.setText(str);      }    // end of actioner listerner class

    public static void main(String[] args) {
        new JComboBoxExample();  } // end of main function
}  // end of class
```

```
C:\Users\balkr\Documents\Java\swing>javac JComboBoxExample.java

C:\Users\balkr\Documents\Java\swing>java JComboBoxExample
```

```java
//JComboBoxExample2.java
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
class JComboBoxExample2 extends JFrame implements ItemListener, ActionListener {
    String names[] ={"Ram Sharma", "Hari Thapa", "Dinesh Karki"};

    JComboBox<String> jcb;          JLabel label;          JTextField tfield;
    JButton btnadd;                      JButton btnremove;

    JComboBoxExample2(){
        setTitle("Combo Box Example");              setLayout(new FlowLayout());
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);      setSize(700,250);
        // create checkbox
        jcb = new JComboBox<String>(names);             add(jcb);
        jcb.addItemListener(this);  // add ItemListener

        label = new JLabel("Display Name");   label.setForeground(Color.blue);
        add(label);
        // create textfield
        tfield = new JTextField(30);   add(tfield);

        btnadd = new JButton("ADD");  btnremove = new JButton("REMOVE");
        add(btnadd);                  add(btnremove);
        // add action listener
        btnadd.addActionListener(this); btnremove.addActionListener(this);
        setVisible(true);
    }  // end of constructor

    public void actionPerformed(ActionEvent e) {
        String str = e.getActionCommand();
        if (str.equals("ADD")) {
                jcb.addItem(tfield.getText());  }
        else {
            jcb.removeItem(tfield.getText()); }
    }    // end of actioner listerner class

    public void itemStateChanged(ItemEvent e){
        if (e.getSource() == jcb) { // if the state combobox is changed
            label.setText(jcb.getSelectedItem() + " selected"); }
    }
    public static void main(String[] args) {
        new JComboBoxExample2();  } // end of main function
}  // end of class
```

```
C:\Users\balkr\Documents\Java\swing>javac JComboBoxExample2.java

C:\Users\balkr\Documents\Java\swing>java JComboBoxExample2
```



JList

- A JComboBox is a component that ***displays a drop-down list and gives users options*** that we can ***select one and only one item*** at a time whereas a ***JList shows multiple items (rows) to the user*** and also gives an option to let the user ***select multiple items.***
- A JList is a component that allows the user to choose either a single selection or multiple selections.
- A JList class itself does not support scrollbar. In order to add scrollbar, we have to use JScrollPane class together with the JList class. The JScrollPane then manages a scrollbar automatically.
- A getSelectedIndex() method returns the index of the first selected item or –1 if no items are selected and getSelectedIndexes() method returns an array with the index of each selected item. The array is empty if no items are selected.
- A getSelectedValue() returns the first selected item or null if no items are selected.
- A ***DefaultListModel*** class provides a simple implementation of a list model, which can be used to manage items displayed by a JList control.
- JList inherits JComponent class. JList was made generic and is now declared like this:

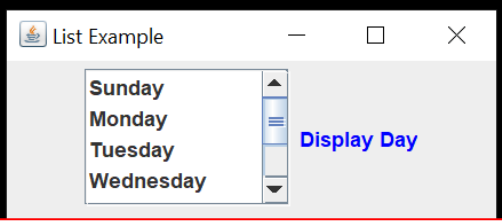
  class JList<E>

  Here, E represents the type of the items in the list.

```java
//JListExample.java
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import javax.swing.event.*;
public class JListExample extends JFrame implements ListSelectionListener{
    String days[] ={"Sunday", "Monday", "Tuesday","Wednesday",
                                    "Thursday", "Friday","Saturday"};
    JList<String> jlist;             JLabel label;

    JListExample(){
        setTitle("List Example");   setLayout(new FlowLayout());
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);   setSize(300,200);
        // create checkbox
        jlist = new JList<String>(days);
        jlist.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);

```

```
        JScrollPane jsp=new JScrollPane(jlist); //add the list to the scroll pane
        jsp.setPreferredSize(new Dimension(120,80));// set size of scroll pane
        jsp.setVerticalScrollBarPolicy(ScrollPaneConstants.VERTICAL_SCROLLBAR_ALWAYS);
        jsp.setHorizontalScrollBarPolicy(ScrollPaneConstants.HORIZONTAL_SCROLLBAR_NEVER);

        label = new JLabel("Display Day");   label.setForeground(Color.blue);
        jlist.addListSelectionListener(this); //add listerner to the list

        add(jsp);              add(label);
        setVisible(true);
    }  // end of constructor

    public void valueChanged(ListSelectionEvent e){
        int idx=jlist.getSelectedIndex();
            if(idx!=-1)
                label.setText("Today is "+ days[idx]);  }    // end of actioner listerner class
    public static void main(String[] args) {
        new JListExample();  } // end of main function
} // end of class
```

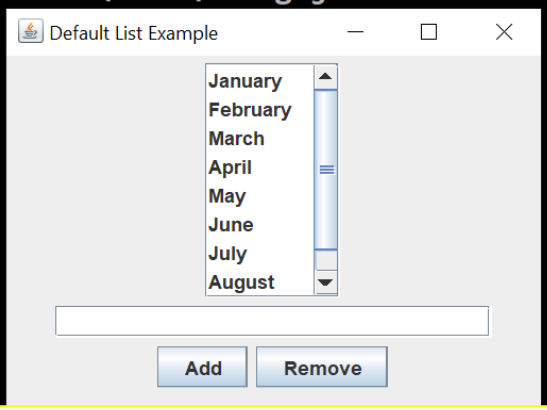
```
C:\Users\balkr\Documents\Java\swing>javac JListExample.java

C:\Users\balkr\Documents\Java\swing>java JListExample
```




```
//DefaultListModelExample.java
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

class DefaultListModelExample extends JFrame {
    String[] months= { "January", "February", "March",
                       "April", "May", "June",
                       "July", "August", "September"};

    JList<String> list;    DefaultListModel<String> dlmodel;
    JButton addButton;   JButton removeButton;  JTextField tfield;

    DefaultListModelExample() {
        setTitle("Default List Example");
        setLayout(new FlowLayout());
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(350,400);

        dlmodel = new DefaultListModel<String>();
        list = new JList<String>(dlmodel);

        JScrollPane pane = new JScrollPane(list);

```

```
25          for (String  month: months)
26              dlmodel.addElement(month);
27          add(pane); //adding pane to frame
28
29          tfield = new JTextField(30);    add(tfield);
30          addButton = new JButton("Add");
31          removeButton = new JButton("Remove");
32          add(addButton);  add(removeButton);
33
34          addButton.addActionListener(new ActionListener() {
35              public void actionPerformed(ActionEvent e) {
36                  dlmodel.addElement(tfield.getText());
37              }
38          });
39
40          removeButton.addActionListener(new ActionListener() {
41              public void actionPerformed(ActionEvent e) {
42                  if (dlmodel.getSize() > 0){
43                      int idx=list.getSelectedIndex();
44                      dlmodel.removeElementAt(idx); }
45              }
46          });
47          setVisible(true);
48      }
49      public static void main(String s[]) {
50          new DefaultListModelExample();
51      }
52  }//end of class
```

```
C:\Users\balkr\Documents\Java\swing>java DefaultListModelExample
```



ImageIcon

- Icon is small fixed size picture, typically used to decorate components. ImageIcon is an implementation of the Icon interface that paints icons from images. Images can be created from a URL, filename, or byte array.
- ImageIcon implements Icon and encapsulates an image. Thus, an object of type ImageIcon can be passed as an argument to the Icon parameter of JLabel's constructor.

- **ImageIcon constructors:** ImageIcon has several constructors, including:
  - ***ImageIcon(byte[] imageData)***: creates an ImageIcon from an array of bytes.
  - ***ImageIcon(Image image):*** creates an ImageIcon from an image object.
  - ***ImageIcon(String filename)***: creates an ImageIcon the specified file.
  - ***ImageIcon(URL location)***: creates an ImageIcon from the specified URL.
- ImageIcon can work with PNG, JPEG, and GIF images. If we want to work with BMP or ICO images, we can use the image4j library.

```java
//imageIconExample.java
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import java.awt.Image;

class imageIconExample extends JFrame{
    imageIconExample(){
        setTitle("Image Icon Example");
        setLayout(new FlowLayout());
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(300,300);
        // Create an Icon
        ImageIcon image=new ImageIcon("C:/Users/balkr/Documents/Java/imgs/java.png");
        int width = image.getIconWidth();
        int height = image.getIconHeight();
        //Create a Labe
        JLabel label=new JLabel("Image",image, JLabel.CENTER);
        add(label);
        JLabel label2=new JLabel(" Height: "+ height+ " Width: " + width );
        add(label2);

        Image scaled = scaleImage(image.getImage(), width/2, height/2);
        ImageIcon scaledIcon = new ImageIcon(scaled);
        JLabel scaleLabel = new JLabel("Scaled Image",scaledIcon,JLabel.CENTER);
        add(scaleLabel);

        setVisible(true);
    }
    public Image scaleImage(Image image, int w, int h) {
        Image scaled = image.getScaledInstance(w, h, Image.SCALE_SMOOTH);
        return scaled;
    }
    public static void main(String args[]){
        new imageIconExample();
    }
}// end of class
```

```
C:\Users\balkr\Documents\Java\swing>java imageIconExample
```

JToggleButton

- A useful variation on the push button is called a toggle button. A toggle button looks just like a push button, but it acts differently because it has **two states**: ***pushed and released.***
- That is, when you press a toggle button, it stays pressed rather than popping back up as a regular push button does. When you press the toggle button a second time, it releases (pops up).
- Therefore, each time a toggle button is pushed, it toggles between its two states.
- Toggle buttons are objects of the JToggleButton class. JToggleButton implements AbstractButton.
- In addition to creating standard toggle buttons, JToggleButton is a superclass for two other Swing components that also represent two-state controls which are JCheckBox and JRadioButton.
- **Constructors in JToggleButton:**
  1. ***JToggleButton():*** Creates an initially unselected toggle button without setting the text or image.
  2. ***JToggleButton(Action a):*** Creates a toggle button where properties are taken from the Action supplied.
  3. ***JToggleButton(Icon icon):*** Creates an initially unselected toggle button with the specified image but no text.
  4. ***JToggleButton(Icon icon, boolean selected):*** Creates a toggle button with the specified image and selection state, but no text.
  5. ***JToggleButton(String text):*** Creates an unselected toggle button with the specified text.
  6. ***JToggleButton(String text, boolean selected):*** Creates a toggle button with the specified text and selection state.
  7. ***JToggleButton(String text, Icon icon):*** Creates a toggle button that has the specified text and image, and that is initially unselected.
  8. ***JToggleButton(String text, Icon icon, boolean selected):*** Creates a toggle button with the specified text, image, and selection state.

JCheckBox

- The ***JCheckBox*** class provides the functionality of a check box. Its immediate superclass is J***ToggleButton***, which provides support for two-state buttons.
- When the user selects or deselects a check box, an ***ItemEvent*** is generated. You can obtain a reference to the ***JCheckBox*** that generated the event by calling ***getItem( )*** on the ***ItemEvent*** passed to the ***itemStateChanged( )*** method defined by ***ItemListener.*** The easiest way to determine the selected state of a check box is to call ***isSelected( )*** on the ***JCheckBox*** instance.
- ***Constructor of the class are:***
  1. ***JCheckBox():*** creates a new checkbox with no text or icon
  2. ***JCheckBox(Icon i):*** creates a new checkbox with the icon specified

3. ***JCheckBox(Icon icon, boolean s):*** creates a new checkbox with the icon specified and the boolean value specifies whether it is selected or not.
4. ***JCheckBox(String t)***: creates a new checkbox with the string specified
5. ***JCheckBox(String text, boolean selected)***: creates a new checkbox with the string specified and the boolean value specifies whether it is selected or not.
6. ***JCheckBox(String text, Icon icon)***: creates a new checkbox with the string and the icon specified.
7. ***JCheckBox(String text, Icon icon, boolean selected):*** creates a new checkbox with the string and the icon specified and the boolean value specifies whether it is selected or not.

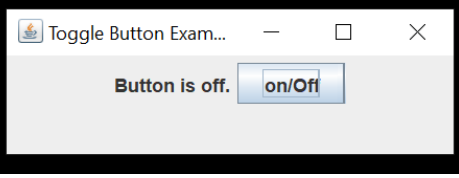
JRadioButton

- Radio buttons are a group of mutually exclusive buttons, in which only one button can be selected at any one time. They are supported by the ***JRadioButton*** class, which extends ***JToggleButton***.
- In order for their ***mutually exclusive nature*** to be activated, radio buttons must be configured into a group. ***Only one of the buttons*** in the group ***can be selected*** at any time.
- For example, if a user presses a radio button that is in a group, any previously selected button in that group is automatically deselected. ***A button group is created by the ButtonGroup class***. Its default constructor is invoked for this purpose.
- A JRadioButton generates ***action events, item events, and change events*** each time the button selection changes. Most often, it is the ***action event that is handled***, which means that you will normally implement the ActionListener interface.

```
1  //JToggleButtonExample.java
2  import java.awt.*;
3  import java.awt.event.*;
4  import javax.swing.*;
5  class JToggleButtonExample extends JFrame implements ItemListener {
6      JToggleButton tbtn;      JLabel label;
7      JToggleButtonExample(){
8              setTitle("Toggle Button Example");
9              setLayout(new FlowLayout());
10             setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
11             setSize(300,100);
12
13             label=new JLabel("Button is Off.");
14             tbtn=new JToggleButton("on/Off");
15
16             tbtn.addItemListener(this);
17             add(label);          add(tbtn);
18             setVisible(true);
19         }
20
```

```
21      public void itemStateChanged(ItemEvent e) {
22              if (tbtn.isSelected())
23                  label.setText("Button is on.");
24              else
25              label.setText("Button is off.");
26          }
27      public static void main(String s[]) {
28          new JToggleButtonExample();
29      }
30  }//end of class
```

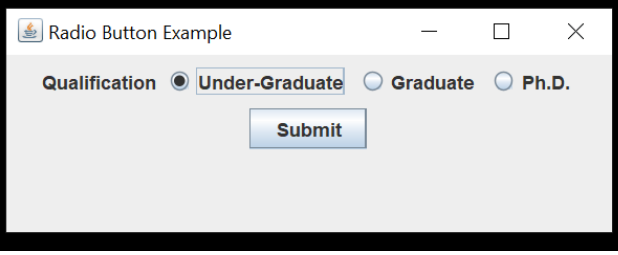
```
C:\Users\balkr\Documents\Java\swing>java JToggleButtonExample
```




```
1   //JRadioButtonExample.java
2   import java.awt.*;
3   import javax.swing.*;
4   import java.awt.event.*;
5
6   public class JRadioButtonExample extends JFrame implements ActionListener{
7           JRadioButton jrb1;  JRadioButton jrb2;  JRadioButton jrb3;
8           JButton submitBtn;   ButtonGroup groupBtn;
9
10      public JRadioButtonExample() {
11          setTitle("Radio Button Example");
12          setLayout(new FlowLayout());
13          setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
14          setSize(400,150);
15
16          jrb1 = new JRadioButton();   jrb1.setText("Under-Graduate");
17          jrb2 = new JRadioButton();   jrb2.setText("Graduate");
18          jrb3 = new JRadioButton("Ph.D.");
19          JLabel label=new JLabel("Qualification");     add(label);
20          submitBtn = new JButton("Submit");
21
22          groupBtn = new ButtonGroup(); // ButtonGroup class. and adding radio button to the group button
23          groupBtn.add(jrb1);  groupBtn.add(jrb2);  groupBtn.add(jrb3);
24          add(jrb1);   add(jrb2);  add(jrb3);   //adding radio button to frame
25          submitBtn.addActionListener(this);    add(submitBtn); //adding submit button
26          setVisible(true);
27      }
```

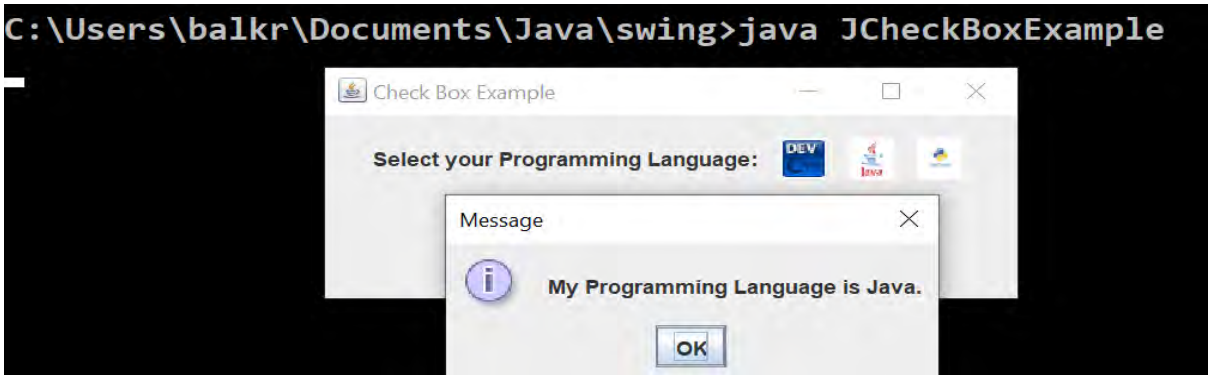
```
28      public void actionPerformed(ActionEvent e){
29          String msg = " ";
30          // If jrb1is selected.
31          if (jrb1.isSelected())
32              msg = "Under-Graduate.";
33          else if (jrb2.isSelected())
34              msg = "Graduate.";
35          else if (jrb2.isSelected())
36              msg = "Ph.D.";
37          else
38              msg = "No Qualification.";
39          // MessageDialog to show information selected radio buttons.
40          JOptionPane.showMessageDialog(this, "Qualification is "+ msg);
41              }
42      public static void main(String s[]) {
43          new JRadioButtonExample();
44      }
45  }//end of class
```

```
C:\Users\balkr\Documents\Java\swing>java JRadioButtonExample
```




```
1   //JCheckBoxExample.java
2   import java.awt.*;
3   import javax.swing.*;
4   import java.awt.event.*;
5   class JCheckBoxExample extends JFrame implements ItemListener {
6       JCheckBox chbox1, chbox2,chbox3; JLabel label;
7
8       JCheckBoxExample (){
9           setTitle("Check Box Example");
10          setLayout(new FlowLayout());
11          setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
12          setSize(400,150);
13          String path1="C:/Users/balkr/Documents/Java/imgs/c++.png";
14          String path2="C:/Users/balkr/Documents/Java/imgs/java.png";
15          String path3="C:/Users/balkr/Documents/Java/imgs/python.png";
16
17          label=new JLabel("Select your Programming Language: ");
18          // create checkbox
19          chbox1 = new JCheckBox(new ImageIcon(path1), false);
20          chbox2 = new JCheckBox(new ImageIcon(path2), false);
21          chbox3 = new JCheckBox(new ImageIcon(path3), false);
22          // add ItemListener
23          chbox1.addItemListener(this);    chbox2.addItemListener(this);
24          chbox3.addItemListener(this);
25          // create a new panel
```

```
        JPanel panel = new JPanel();
        // add checkbox to panel
        panel.add(label);     panel.add(chbox1);   panel.add(chbox2);
        panel.add(chbox3);     add(panel);
        setVisible(true);
    }
    public void itemStateChanged(ItemEvent e) {
        // if the state of checkbox1 is changed
        if (e.getSource() == chbox1) {
            if (e.getStateChange() == 1)
                JOptionPane.showMessageDialog(this, "My Programming Language is C++.");}
        else if (e.getSource() == chbox2) {
            if (e.getStateChange() == 1)
                JOptionPane.showMessageDialog(this, "My Programming Language is Java.");}
        else if (e.getSource() == chbox3) {
            if (e.getStateChange() == 1)
                JOptionPane.showMessageDialog(this, "My Programming Language is Python.");}
        else
            JOptionPane.showMessageDialog(this, "Programming Language Not Selected.");
    }

    public static void main(String s[]) {
        new JCheckBoxExample(); }
}//end of class
```



Java JOptionPane

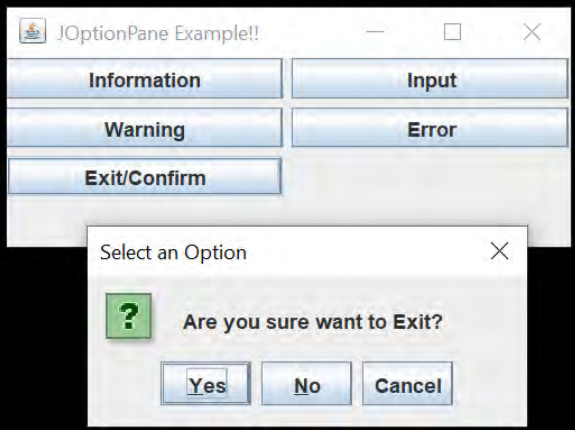
- JOptionPane is a ***subclass of JComponent*** which includes static methods for ***creating and customizing modal dialog boxes*** using a simple code.
- JOptionPane displays the dialog boxes with one of the ***four standard icons*** (***question, information, warning, and error***) or the custom icons specified by the user. The JOptionPane class is used to provide standard dialog boxes such as message dialog box, confirm dialog box and input dialog box. These dialog boxes are used to display information or get input from the user.
- Some of the constructors defined by the JOptionPane class are as follows.
    - JOptionPane ()
    - JOptionPane (Object ***message***)
    - JOptionPane (Object ***message***, int ***messageType***)
    - JOptionPane (Object ***message***, int ***messageType***, int ***optionType***)
    - JOPtionPane (Object ***message***, int ***messageType***, int ***optionType,*** Icon icon)
    - JOPtionPane (Object ***message***, int ***messageType***, int ***optionType***, Icon icon, Object [] options)
    - JOPtionPane (Object ***message***, int ***messageType***, int ***optionType***, Icon icon, Object [] options, Object initialValue)

- Where,
  - ***message*** is the message to be displayed in the dialog box
  - ***messageType*** specifies the type of the message to be displayed. It has various values which are represented as: ***ERROR_MESSAGE***, ***INFORMATION_MESSAGE***, ***WARNING_MESSAGE***, ***QUESTION_MESSAGE*** or ***PLAIN_MESSAGE.***
- ***optionType*** is used to specify the options. It can have various values which are represented as: ***DEFAULT_OPTION***, ***YES_NO_OPTION,*** ***YES_NO_CANCEL_OPTION*** or ***OK_CANCEL_OPTION*** icon specifies the icon to be displayed.
- ***options*** are an array of options to be displayed in the dialog box with none of them selected initial Value specifies the default option selected from the list provided in options.
- ***JOptionPane class is used to display four types of dialog boxes namely,***
  1. ***MessageDialog:*** dialog box that displays a message making it possible to add icons to alert the user.
  2. ***ConfirmDialog:*** dialog box that besides sending a message, enables the user to answer a question.
  3. ***InputDialog:*** dialog box that besides sending a message, allows entry of a text.
  4. ***OptionDialog:*** dialog box that covers the three previous types.

```java
//joptionPaneExample.java
import javax.swing.*;
import java.awt.event.*;
import java.awt.*;
public class joptionPaneExample extends JFrame implements ActionListener{
    JButton btn1,btn2,btn3,btn4,btn5;
    joptionPaneExample(){
        setTitle(" JOptionPane Example!!");
        setSize(350,150);   setLayout(new GridLayout(4,1,5,5));
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        int alert=JOptionPane.WARNING_MESSAGE;
        btn1=new JButton("Information");  add(btn1);
        btn2=new JButton("Input");  add(btn2);
        btn3=new JButton("Warning");  add(btn3);
        btn4=new JButton("Error");  add(btn4);
        btn5=new JButton("Exit/Confirm");  add(btn5);
        btn1.addActionListener(this);  btn2.addActionListener(this);
        btn3.addActionListener(this);  btn4.addActionListener(this);
        btn5.addActionListener(this);
        setVisible(true);
    }
```

```
23      public void actionPerformed(ActionEvent event) {
24          String str=event.getActionCommand();
25          String message=new String();
26          int messageType;
27          if (str.equals("Information")){
28              messageType = JOptionPane.INFORMATION_MESSAGE;
29              message = "Information Message";
30              JOptionPane.showMessageDialog(this,message,
31                                      "Information Dialog",messageType);
32          }else if (str.equals("Input")){
33              messageType = JOptionPane.QUESTION_MESSAGE;
34              String name = JOptionPane.showInputDialog("Enter your name: ");
35                  JOptionPane.showMessageDialog(null, "This is  " + name,
36                                      "Input Dialog", messageType);
37          }else if (str.equals("Error")){
38              messageType = JOptionPane.ERROR_MESSAGE;
39              message = "Error Message";
40              JOptionPane.showMessageDialog(null, message,
41                                      "Error Dialog", messageType);
42          }else if (str.equals("Warning")){
43              messageType = JOptionPane.WARNING_MESSAGE;
44              message = "Warning Message";
45              JOptionPane.showMessageDialog(null, message,
46                                      "Warning Dialog", messageType);
47          }else if (str.equals("Exit/Confirm")){
48              int option=JOptionPane.showConfirmDialog(null,
49                                      "Are you sure want to Exit?");
50              message = "We Exited...";
51              messageType = JOptionPane.INFORMATION_MESSAGE;
52              if(option==JOptionPane.YES_OPTION){
53                  JOptionPane.showMessageDialog(this,message,
54                                      "From Confirmation Dialog",messageType);}
55          }
56      }// end of constructor
57      public static void main(String[] args) {
58          new joptionPaneExample();
59      }// end of main function
60  }// end of class
```

```
C:\Users\balkr\Documents\Java\swing>java joptionPaneExample
```



JPanel

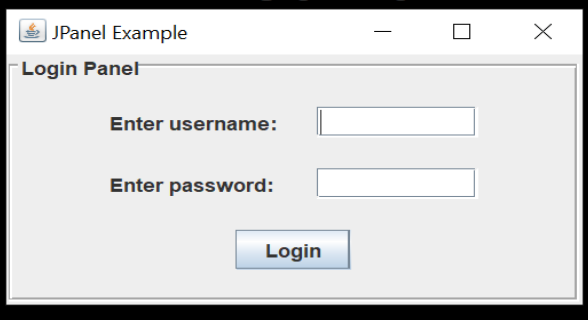
- The ***JPanel*** is a simplest container class. It provides space in which an application can attach any other component. It inherits the ***JComponents class.***
- A panel is a lightweight container that is ***designed to group a set of components***, including other panels. It is visually represented as window that does not contain a title bar, menu bar or border. It is simplest of all the containers. Its ***default layout manager is FlowLayout.***
- After the panel has been created, other components ***can be added to JPanel*** object by calling it ***add (component) method*** inherited from the Container class.
- Commonly used Constructors:
  1. ***JPanel():*** It is used to create a new JPanel with a double buffer and a flow layout.
  2. ***JPanel(boolean isDoubleBuffered):*** It is used to create a new JPanel with FlowLayout and the specified buffering strategy.
  3. ***JPanel(LayoutManager layout):*** It is used to create a new JPanel with the specified layout manager.
- Normally we create new JPanel object as simple as follows:

  JPanel newPanel = new JPanel();

```
1  //jPanelExample.java
2  import java.awt.*;
3  import javax.swing.*;
4  public class jPanelExample extends JFrame {
5      private JLabel labelUsername = new JLabel("Enter username: ");
6      private JLabel labelPassword = new JLabel("Enter password: ");
7      private JTextField textUsername = new JTextField(10);
8      private JPasswordField fieldPassword = new JPasswordField(10);
9      private JButton buttonLogin = new JButton("Login");
10
11     public jPanelExample() {
12         super("JPanel Example");  setSize(350,200);
13         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
14
15         JPanel newPanel = new JPanel(new GridBagLayout());
16         GridBagConstraints constraints = new GridBagConstraints();
17         constraints.anchor = GridBagConstraints.WEST;
18         constraints.insets = new Insets(10, 10, 10, 10);
19
20         constraints.gridx = 0;  constraints.gridy = 0;
21         newPanel.add(labelUsername, constraints);
22         constraints.gridx = 1;
23         newPanel.add(textUsername, constraints);
24         constraints.gridx = 0; constraints.gridy = 1;
```

```
25          newPanel.add(labelPassword, constraints);
26          constraints.gridx = 1;
27          newPanel.add(fieldPassword, constraints);
28          constraints.gridx = 0; constraints.gridy = 2;
29          constraints.gridwidth = 2;
30          constraints.anchor = GridBagConstraints.CENTER;
31          newPanel.add(buttonLogin, constraints);
32          // set border for the panel
33          newPanel.setBorder(BorderFactory.createTitledBorder(
34                  BorderFactory.createEtchedBorder(), "Login Panel"));
35          add(newPanel); // add the panel to this frame
36          setLocationRelativeTo(null);//setLocationRelativeTo(null) is used to center frame in the screen.
37      }// end of constructor
38      public static void main(String[] args) {
39          new jPanelExample().setVisible(true);
40      }// end of main functio
41  }// end of class
```

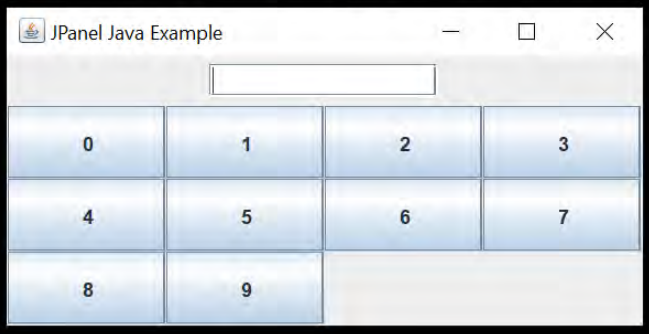
```
C:\Users\balkr\Documents\Java\swing>java jPanelExample
```




```
1   //JPanelExample2.java
2   import javax.swing.*;
3   import java.awt.event.*;
4   import java.awt.*;
5   class JPanelExample2 extends JFrame{
6       JPanel panel1,panel2 ;  JTextField tf;
7       JButton[] btnData = new JButton[12];
8       JPanelExample2() {
9           setTitle("JPanel Java Example");
10          setSize(400,200);
11          setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
12
13          panel1=new JPanel();
14          tf = new JTextField(15);
15          panel1.add(tf);      add(panel1,"North");
16          panel2=new JPanel(new GridLayout(3,4));
17          for(int i=0;i<=9;i++){
18              btnData[i]=new JButton(""+i);
19              panel2.add(btnData[i]);
20          }
21          add(panel2,"Center");
22          setVisible(true);
23      }// end of constructor
24
```

```
25      public static void main(String[]  args) {
26          JPanelExample2 frame = new JPanelExample2();
27      }// end of main function
28  }// end of class
```

```
C:\Users\balkr\Documents\Java\swing>java JPanelExample2
```



JTabbedPane

- JTabbedPane in Swing enables you to group several swing components groups on the same container. To view a particular component group, you need to select the tab corresponding to the component group in the tabbed pane.
- JTabbedPane ***encapsulates a tabbed pane***. It ***manages a set of components*** by linking them with tabs. Selecting a tab causes the component associated with that tab to come to the forefront. Tabbed panes are very common in the modern GUI, and you have no doubt used them many times.
- To create a tabbed pane***, create an instance of JTabbedPane*** class and add various tabs to it using the addTab() method of JTabbedPane class.
- JTabbedPane ***defines three constructors***. We will use its default constructor, which creates an empty control with the tabs positioned across the top of the pane. The other two constructors let you specify the location of the tabs, which can be along any of the four sides.
- JTabbedPane uses the SingleSelectionModel model.
- ***JTabbedPane Constructor***
  1. ***JTabbedPane():*** Creates an empty TabbedPane with a default tab placement of JTabbedPane.Top.
  2. ***JTabbedPane(int tabPlacement):*** Creates an empty TabbedPane with a specified tab placement. Creates an empty TabbedPane with the specified tab placement of either: ***JTabbedPane.TOP***, ***JTabbedPane.BOTTOM***, ***JTabbedPane.LEFT***, or ***JTabbedPane.RIGHT***.
  3. ***JTabbedPane(int tabPlacement, int tabLayoutPolicy):*** Creates an empty TabbedPane with a specified tab placement and tab layout policy.

Tabs are added by calling **addTab( )**. Here is one of its forms:

void addTab(String *name*, Component *comp*)

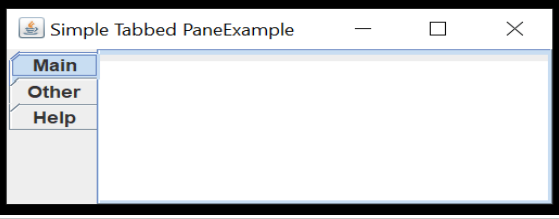
Here, *name* is the name for the tab, and *comp* is the component that should be added to the tab. Often, the component added to a tab is a **JPanel** that contains a group of related components. This technique allows a tab to hold a set of components.

The general procedure to use a tabbed pane is outlined here:

1. Create an instance of **JTabbedPane**.
2. Add eachn tab by calling **addTab( )**.
3. Add the tabbed pane to the content pane.

```
1  //simpleTabbedPane.java
2  import javax.swing.*;
3  import java.awt.*;
4  public class simpleTabbedPane extends JFrame{
5      simpleTabbedPane(){
6          setTitle("Simple Tabbed PaneExample");
7          setSize(350,150);   setLayout(new BorderLayout(15,25));
8          setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
9
10         JTextArea ta=new JTextArea(50,100);
11         JPanel p1=new JPanel();    p1.add(ta);
12         JPanel p2=new JPanel();    JPanel p3=new JPanel();
13         //JTabbedPane tp=new JTabbedPane(JTabbedPane.TOP);
14         //JTabbedPane tp=new JTabbedPane(JTabbedPane.BOTTOM);
15         JTabbedPane tp=new JTabbedPane(JTabbedPane.LEFT);
16
17         tp.add("Main",p1);    tp.add("Other",p2);
18         tp.add("Help",p3);   add(tp);
19         setVisible(true);
20     }
21     public static void main(String[] args) {
22         new simpleTabbedPane();
23     }// end of main function
24 }// end of class
```

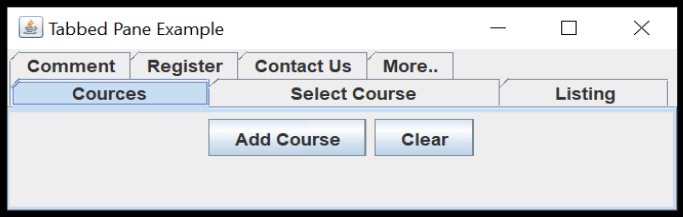
```
C:\Users\balkr\Documents\Java\swing>java simpleTabbedPane
```




```java
//JTabbedPaneExample.java
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class TabbedPane extends JFrame {
    JTabbedPane tabs;    CoursePanel cource;   SelectCoursePanel selectCourse;

    TabbedPane() {
        super("Tabbed Pane Example");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        // Setting the JTabbedPane Position and Layout as Wrap
        tabs = new JTabbedPane(JTabbedPane.TOP, JTabbedPane.WRAP_TAB_LAYOUT);
        //tabs = new JTabbedPane(JTabbedPane.TOP,JTabbedPane.SCROLL_TAB_LAYOUT);
        //tabs = new JTabbedPane(JTabbedPane.LEFT,JTabbedPane.WRAP_TAB_LAYOUT);

        cource = new CoursePanel();
        selectCourse = new SelectCoursePanel();
        // Adding user defined pannels to JTabbedPane
        tabs.addTab("Cources", cource);
        tabs.addTab("Select Course", selectCourse);
            // Adding JPanels to JTabbedPane
            tabs.addTab("Listing", new JPanel());
            tabs.addTab("Comment", new JTextArea(10, 40));
            tabs.addTab("Register", new JPanel());
            tabs.addTab("Contact Us", new JPanel());
            tabs.addTab("More..", new JPanel());
            getContentPane().add(tabs);
        }
}
class CoursePanel extends JPanel {
    JButton addCourse, clear;
    CoursePanel() {
        addCourse = new JButton("Add Course");
        clear = new JButton("Clear");
        setLayout(new FlowLayout());
        add(addCourse);
        add(clear);
    }
}
```

```
41  class SelectCoursePanel extends JPanel {
42      JCheckBox java, swing, hibernate;
43      SelectCoursePanel() {
44          java = new JCheckBox("Java");
45          swing = new JCheckBox("Spring");
46          hibernate = new JCheckBox("Hibernate");
47          setLayout(new FlowLayout());
48          add(java);
49          add(swing);
50          add(hibernate);
51      }
52  }
53  class JTabbedPaneExample {
54      public static void main(String args[]) throws Exception {
55          TabbedPane frame = new TabbedPane();
56          frame.setSize(450, 150);
57          frame.setVisible(true);
58      }// end of main function
59  }// end of class
```

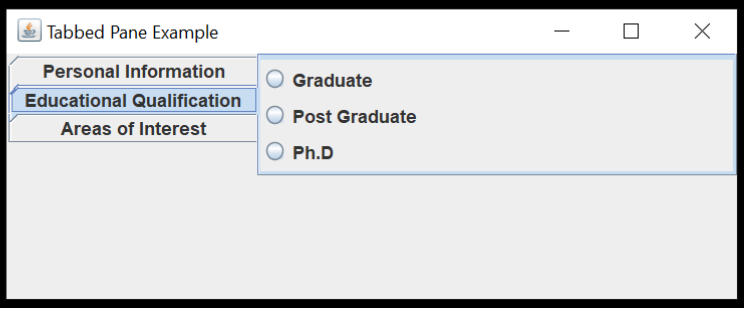
```
C:\Users\balkr\Documents\Java\swing>java JTabbedPaneExample
```




```
1   //JTabbedPaneDemo.java
2   import javax.swing.*;
3   import java.awt.*;
4   import java.awt.event.*;
5   public class JTabbedPaneDemo extends JFrame {
6       JTabbedPane t1;
7       JPanel p1,p2,p3;   Container c;
8       JLabel l1,l2,l3;
9       JTextField text1,text2,text3;
10      JRadioButton r21,r22,r23;
11      JCheckBox ch1,ch2,ch3;
12      public JTabbedPaneDemo(){
13          super("Tabbed Pane Example");  setSize(500,200);
14          setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
15
16          t1=new JTabbedPane(JTabbedPane.LEFT);
17          p1=new JPanel(); p2=new JPanel();  p3=new JPanel();
18          p1.setLayout(new GridLayout(3,2));
19          p2.setLayout(new GridLayout(3,1));
20
```

```
21          l1=new JLabel("Name");
22          l2=new JLabel("Date of Birth (dd.mm.yyyy)");
23          l3=new JLabel("Identification Number");
24          text1=new JTextField(10);  text2=new JTextField(10);
25          text3=new JTextField(10);
26
27          p1.add(l1);  p1.add(text1);   p1.add(l2);
28          p1.add(text2);  p1.add(l3);   p1.add(text3);
29
30          r21=new JRadioButton("Graduate");
31          r22=new JRadioButton("Post Graduate");
32          r23=new JRadioButton("Ph.D");
33          p2.add(r21);   p2.add(r22);   p2.add(r23);
34
35          ch1=new JCheckBox("Computers");
36          ch2=new JCheckBox("Electronics");
37          ch3=new JCheckBox("Marketing");
38          p3.add(ch1);   p3.add(ch2);  p3.add(ch3);
39
40          c=getContentPane();
41          c.setLayout(new GridLayout(2,2));
42
43          t1.addTab("Personal Information",p1);
44          t1.addTab("Educational Qualification",p2);
45          t1.addTab("Areas of Interest",p3);
46          c.add(t1);
47          setVisible(true);
48          }
49      public static void main(String args[]){
50          new JTabbedPaneDemo();
51      }// end of main function
52  }//end of class
```

```
C:\Users\balkr\Documents\Java\swing>java JTabbedPaneDemo
```



JTable

- JTable is a component that displays rows and columns of data. You can drag the cursor on column boundaries to resize columns. You can also drag a column to a new position.
- Depending on its configuration, it is also possible to select a row, column, or cell within the table, and to change the data within a cell.
- It is a component that consists of one or more columns of information. At the top of each column is a heading. In addition to describing the data in a column, the

heading also provides the mechanism by which the user can change the size of a column or change the location of a column within the table.

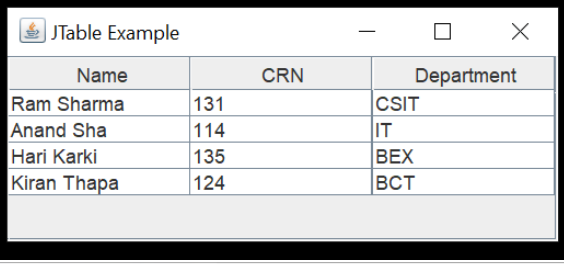
- JTable does not provide any scrolling capabilities of its own. Instead, you will normally ***wrap a JTable inside a JScrollPane***.
- Constructors in JTable:
  1. ***JTable():*** A table is created with empty cells.
  2. ***JTable(int rows, int cols):*** Creates a table of size rows * cols.
  3. ***JTable(Object[][] data, Object []Column):*** A table is created with the specified name where []Column defines the column names.
- ***Functions in JTable:***
  1. ***addColumn(TableColumn []column):*** adds a column at the end of the JTable.
  2. ***clearSelection():*** Selects all the selected rows and columns.
  3. ***editCellAt(int row, int col):*** edits the intersecting cell of the column number col and row number row programmatically, if the given indices are valid and the corresponding cell is editable.
  4. ***setValueAt(Object value, int row, int col):*** Sets the cell value as 'value' for the position row, col in the JTable.

Here are the steps required to set up a simple **JTable** that can be used to display data:

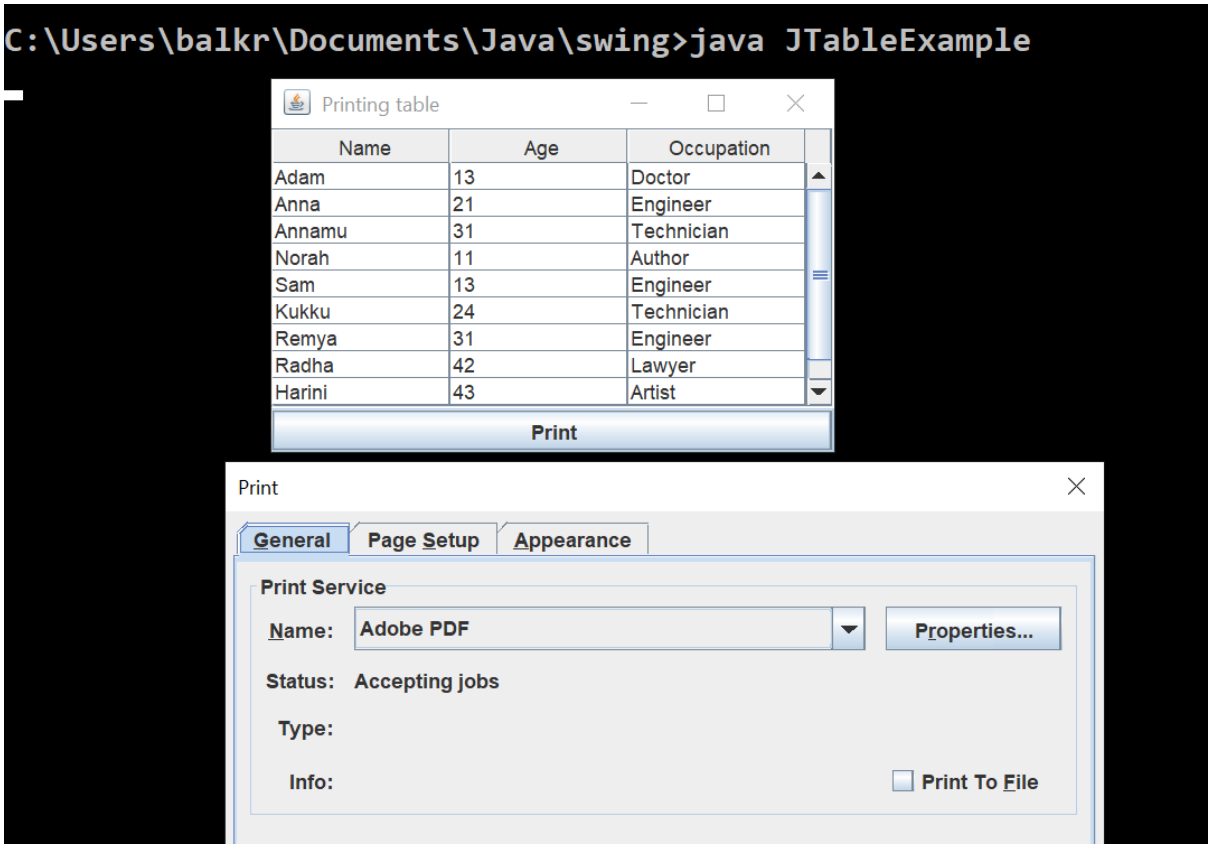
1. Create an instance of **JTable**.
2. Create a **JScrollPane** object, specifying the table as the object to scroll.
3. Add the table to the scroll pane.
4. Add the scroll pane to the content pane.

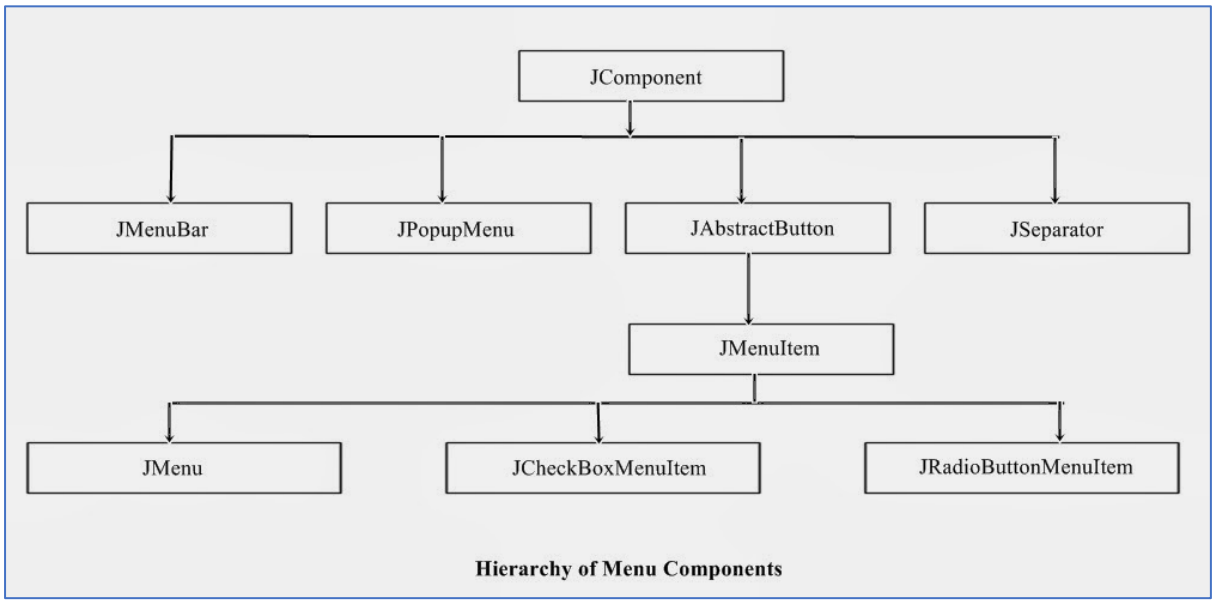
```
1  //simpleJTableExample.java
2  import javax.swing.*;
3  import java.awt.*;
4  public class simpleJTableExample extends JFrame {
5      JTable table;
6      simpleJTableExample(){
7          setTitle("JTable Example");  setSize(350,150);
8          setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
9          String[] columnNames = { "Name", "CRN", "Department" };
10         String[][] data = {
11                 { "Ram Sharma", "131", "CSIT" },
12                 { "Anand Sha", "114", "IT" },
13                 { "Hari Karki", "135", "BEX" },
14                 { "Kiran Thapa", "124", "BCT" }
15         };
16         table = new JTable(data, columnNames);
17         JScrollPane scrol = new JScrollPane(table);
18         add(scrol);
19         setVisible(true);
20     }
21     public static void main(String[] args){
22         new simpleJTableExample();
23     }
24 } // end of class
```

```
C:\Users\balkr\Documents\Java\swing>java simpleJTableExample
```




```java
//JTableExample.java
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.awt.print.*;
class JTableExample {
    public static void main(String args[]) {
        //rows of the table
        Object r[][] = {
            {"Adam", "13" , "Doctor"},
            {"Anna", "21" ,"Engineer"},
            {"Annamu", "31" ,"Technician"},
            {"Norah", "11" , "Author"},
            {"Sam", "13" , "Engineer"},
            {"Kukku", "24" ,"Technician"},
            {"Remya", "31" ,"Engineer"},
            {"Radha", "42" ,"Lawyer"},
            {"Harini", "43", "Artist"},
            {"Vaibhav", "44" , "Engineer"},
        };
        Object h[] = {"Name", "Age" ,"Occupation"};
        JFrame fr = new JFrame(" Printing table");
        fr.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        JTable tb = new JTable(r, h);
        JScrollPane sp = new JScrollPane(tb);
        fr.add(sp , BorderLayout.CENTER);
        JButton button = new JButton("Print");
        ActionListener act = new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                try {
                    tb.print();
                } catch (PrinterException pe) {
                    System.err.println("Error printing: " + pe.getMessage());
                }
            }
        };
        button.addActionListener(act);
        fr.add(button, BorderLayout.SOUTH);
        fr.setSize(350,200);
        fr.setVisible(true);
    }// end of main function
}// end of class
```





Hierarchy of Menu Components

```java
//MenuBarExample.java
import javax.swing.*;
import javax.swing.event.*;
import java.awt.*;
import java.awt.Color;
import java.awt.event.*;
public class MenuBarExample {
    public JPanel createContentPane()    {
        JPanel totalGUI = new JPanel();
        totalGUI.setBackground(Color.white);
        totalGUI.setMinimumSize(new Dimension(300, 200));
        totalGUI.setPreferredSize(new Dimension(300, 200));
        totalGUI.setMaximumSize(new Dimension(300, 200));
        totalGUI.setOpaque(true);
        return totalGUI;
    }
    public JMenuBar createMenuBar() {
        //Create the menu bar.
        JMenuBar menuBar = new JMenuBar();
        //Add a JMenu
        JMenu starter = new JMenu("Starters");
        JMenu maincourse = new JMenu("Main Courses");
        JMenu desserts = new JMenu("Desserts");
        menuBar.add(starter);     menuBar.add(maincourse);
        menuBar.add(desserts);

        JMenuItem soup = new JMenuItem("Soup");
        JMenuItem pate = new JMenuItem("Pate");
        JMenuItem salad = new JMenuItem("Salad");
        starter.add(soup);   starter.add(pate); starter.add(salad);

        JMenuItem fish =  new JMenuItem("Haddock");
        JMenu steak = new JMenu("Steak");
        JMenuItem rare = new JMenuItem("Rare");
        JMenuItem welldone = new JMenuItem("Well Done");
        steak.add(rare);
        steak.add(welldone);

        JMenuItem pie = new JMenuItem("Pie");

        ButtonGroup sides = new ButtonGroup();
        JRadioButtonMenuItem chips = new JRadioButtonMenuItem("Chips");
        JRadioButtonMenuItem bp = new JRadioButtonMenuItem("Baked Potato");
        JRadioButtonMenuItem vege = new JRadioButtonMenuItem("Vegetables");

        sides.add(chips);      sides.add(bp);        sides.add(vege);
        maincourse.add(fish);      maincourse.add(steak);
        maincourse.add(pie);       maincourse.addSeparator();
        maincourse.add(chips);     maincourse.add(bp);
        maincourse.add(vege);

        JCheckBoxMenuItem cake = new JCheckBoxMenuItem("Cake");
        JCheckBoxMenuItem sorbet = new JCheckBoxMenuItem("Sorbet");
        JMenu icecream = new JMenu("Ice Cream");
        JCheckBoxMenuItem choc = new JCheckBoxMenuItem("Chocolate");
        JCheckBoxMenuItem vani = new JCheckBoxMenuItem("Vanilla");
        icecream.add(choc);         icecream.add(vani);
```

```
58
59          desserts.add(cake);   desserts.add(sorbet);  desserts.add(icecream);
60          return menuBar;
61      }
62      private static void createAndShowGUI() {
63          JFrame.setDefaultLookAndFeelDecorated(true);
64          JFrame frame = new JFrame(" JMenuBar Example ");
65          MenuBarExample demo = new MenuBarExample();
66          frame.setContentPane(demo.createContentPane());
67          // We now also set the MenuBar of the Frame to our MenuBar
68          frame.setJMenuBar(demo.createMenuBar());
69          frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
70          frame.pack();     frame.setVisible(true);
71      }
72      public static void main(String[] args) {
73          createAndShowGUI();
74      }
75  }
```

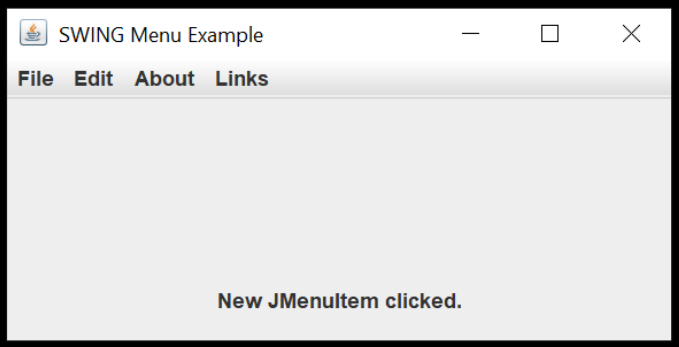
```
C:\Users\balkr\Documents\Java\swing>java MenuBarExample
```

```
1   //swingMenuExample.java
2   import java.awt.*;
3   import java.awt.event.*;
4   import javax.swing.*;
5   public class swingMenuExample extends JFrame {
6       private JLabel headerLabel;
7       private JLabel statusLabel;
8       private JPanel controlPanel;
9
10      public swingMenuExample(){
11          setTitle(" SWING Menu Example");
12          setSize(400,200);  setLayout(new GridLayout(3, 1));
13          setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
14
15          headerLabel = new JLabel("",JLabel.CENTER );
16          statusLabel = new JLabel("",JLabel.CENTER);
17          statusLabel.setSize(250,100);
18          controlPanel = new JPanel();
19          controlPanel.setLayout(new FlowLayout());
20
21          add(headerLabel);    add(controlPanel);
22          add(statusLabel);
23          setVisible(true);
```

```
24      }
25      public static void main(String[] args){
26          swingMenuExample  menuExample = new swingMenuExample();
27          menuExample.showMenu();
28      }
29      private void showMenu(){
30          final JMenuBar menuBar = new JMenuBar();
31          JMenu fileMenu = new JMenu("File");
32          JMenu editMenu = new JMenu("Edit");
33          final JMenu aboutMenu = new JMenu("About");
34          final JMenu linkMenu = new JMenu("Links");
35          JMenuItem newMenuItem = new JMenuItem("New");
36          newMenuItem.setMnemonic(KeyEvent.VK_N);
37          newMenuItem.setActionCommand("New");
38
39          JMenuItem openMenuItem = new JMenuItem("Open");
40          openMenuItem.setActionCommand("Open");
41          JMenuItem saveMenuItem = new JMenuItem("Save");
42          saveMenuItem.setActionCommand("Save");
43          JMenuItem exitMenuItem = new JMenuItem("Exit");
44          exitMenuItem.setActionCommand("Exit");
45          JMenuItem cutMenuItem = new JMenuItem("Cut");
46          cutMenuItem.setActionCommand("Cut");
47          JMenuItem copyMenuItem = new JMenuItem("Copy");
48          copyMenuItem.setActionCommand("Copy");
49          JMenuItem pasteMenuItem = new JMenuItem("Paste");
50          pasteMenuItem.setActionCommand("Paste");
51
52          MenuItemListener menuItemListener = new MenuItemListener();
53
54          newMenuItem.addActionListener(menuItemListener);
55          openMenuItem.addActionListener(menuItemListener);
56          saveMenuItem.addActionListener(menuItemListener);
57          exitMenuItem.addActionListener(menuItemListener);
58          cutMenuItem.addActionListener(menuItemListener);
59          copyMenuItem.addActionListener(menuItemListener);
60          pasteMenuItem.addActionListener(menuItemListener);
61
62          final JCheckBoxMenuItem showWindowMenu = new JCheckBoxMenuItem(
63                                                      "Show About", true);
64          showWindowMenu.addItemListener(new ItemListener() {
65              public void itemStateChanged(ItemEvent e) {
66                  if(showWindowMenu.getState()){
67                      menuBar.add(aboutMenu);
68                  } else {
69                      menuBar.remove(aboutMenu);}
70              }
71          });
```

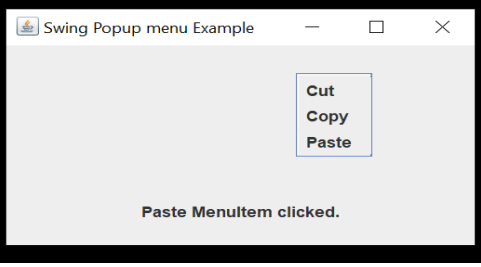
```
72          final JRadioButtonMenuItem showLinksMenu = new JRadioButtonMenuItem(
73                                                          "Show Links", true);
74              showLinksMenu.addItemListener(new ItemListener() {
75              public void itemStateChanged(ItemEvent e) {
76                  if(menuBar.getMenu(3)!= null){
77                      menuBar.remove(linkMenu);
78                      //repaint();
79                  } else {
80                      menuBar.add(linkMenu);
81                      //repaint();
82                      }
83                  }
84              });
85          //add menu items to menus
86          fileMenu.add(newMenuItem);     fileMenu.add(openMenuItem);
87          fileMenu.add(saveMenuItem);    fileMenu.addSeparator();
88          fileMenu.add(showWindowMenu); fileMenu.addSeparator();
89          fileMenu.add(showLinksMenu);  fileMenu.addSeparator();
90          fileMenu.add(exitMenuItem);    editMenu.add(cutMenuItem);
91          editMenu.add(copyMenuItem);    editMenu.add(pasteMenuItem);
92          //add menu to menubar
93          menuBar.add(fileMenu);         menuBar.add(editMenu);
94          menuBar.add(aboutMenu);        menuBar.add(linkMenu);
95          //add menubar to the frame
96          setJMenuBar(menuBar);
97          setVisible(true);
98      }
99      class MenuItemListener implements ActionListener {
100         public void actionPerformed(ActionEvent e) {
101             statusLabel.setText(e.getActionCommand() + " JMenuItem clicked.");
102         }
103     }
104 }
```

```
C:\Users\balkr\Documents\Java\swing>java swingMenuExample
```



```java
//SwingJpopupMenuExample.java
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class SwingJpopupMenuExample{
    private JFrame mainFrame;   private JLabel headerLabel;
    private JLabel statusLabel; private JPanel controlPanel;

    public SwingJpopupMenuExample(){
        mainFrame = new JFrame("Swing Popup menu Example");
        mainFrame.setSize(350,200);
        mainFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        mainFrame.setLayout(new GridLayout(3, 1));

        headerLabel = new JLabel("",JLabel.CENTER );
        statusLabel = new JLabel("",JLabel.CENTER);
        statusLabel.setSize(350,100);
        controlPanel = new JPanel();
        controlPanel.setLayout(new FlowLayout());

        mainFrame.add(headerLabel); mainFrame.add(controlPanel);
        mainFrame.add(statusLabel); mainFrame.setVisible(true);
    }
    public static void main(String[] args){
        SwingJpopupMenuExample  swingMenu = new SwingJpopupMenuExample();
        swingMenu.showPopupMenu();
    }
    private void showPopupMenu(){
        final JPopupMenu editMenu = new JPopupMenu("Edit");
        JMenuItem cutMenuItem = new JMenuItem("Cut");
        cutMenuItem.setActionCommand("Cut");
        JMenuItem copyMenuItem = new JMenuItem("Copy");
        copyMenuItem.setActionCommand("Copy");
        JMenuItem pasteMenuItem = new JMenuItem("Paste");
        pasteMenuItem.setActionCommand("Paste");

        MenuItemListener menuItemListener = new MenuItemListener();

        cutMenuItem.addActionListener(menuItemListener);
        copyMenuItem.addActionListener(menuItemListener);
        pasteMenuItem.addActionListener(menuItemListener);
        editMenu.add(cutMenuItem);  editMenu.add(copyMenuItem);
        editMenu.add(pasteMenuItem);

        mainFrame.addMouseListener(new MouseAdapter() {
            public void mouseClicked(MouseEvent e) {
                editMenu.show(mainFrame, e.getX(), e.getY());
            }
        });
        mainFrame.add(editMenu);
        mainFrame.setVisible(true);
    }
    class MenuItemListener implements ActionListener {
        public void actionPerformed(ActionEvent e) {
            statusLabel.setText(e.getActionCommand() + " MenuItem clicked.");
        }
    }
}
```

```
C:\Users\balkr\Documents\Java\swing>java SwingJpopupMenuExample
```



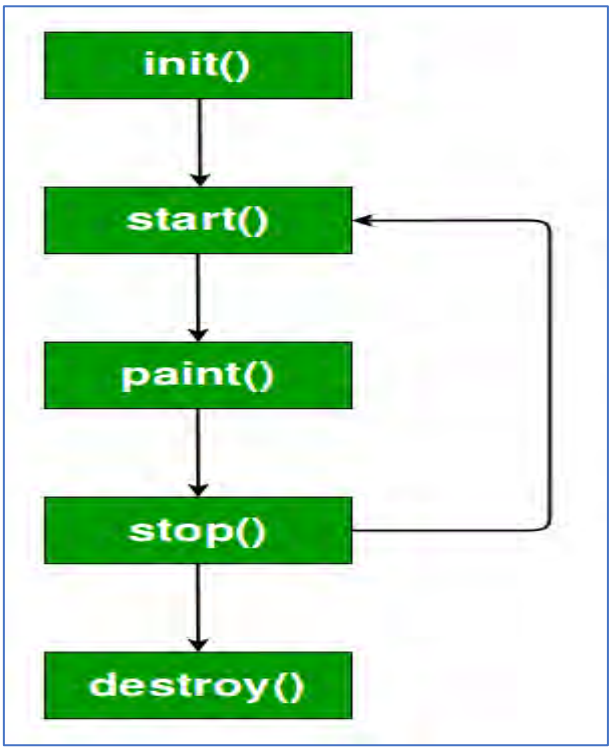
Chapter 6

Java Applet

- An applet is a Java program that can be ***embedded into a web page***. It ***runs inside the web browser*** and ***works at client side***.
- An applet is ***embedded in an HTML page*** using the APPLET or OBJECT tag and hosted on a web server.
- Applets are used to make the ***website more dynamic and entertaining.***
- To create an applet, a class must class extends ***java.applet.Applet*** class.
- An Applet class ***does not have any main ()*** method. It is viewed using JVM. The JVM can use either a plug-in of the Web browser or a separate runtime environment to run an applet application. JVM ***creates an instance of the applet class*** and invokes ***init() method*** to initialize an Applet.
- ***Note:*** Java Applet is deprecated since Java 9. It means Applet API is no longer considered important.
- **Important points:**
  - All applets are sub-classes (either directly or indirectly) of java.applet.Applet class.
  - Applets are not stand-alone programs. Instead, they run within either a web browser or an applet viewer. JDK provides a standard applet viewer tool called applet viewer.
  - In general, execution of an applet does not begin at main () method.
  - Output of an applet window is not performed by ***System.out.println().*** Rather it is handled with various AWT methods, such as ***drawString().***
- There are some important differences ***between an applet and a standalone Java application***, including the following –
    - An applet is a Java class that extends the ***java.applet.Applet*** class.
    - A ***main () method is not invoked*** on an applet, and an applet class will not define main ().
    - Applets are designed to be ***embedded within an HTML*** page.
    - When a user views an HTML page that contains an applet, the code for the applet is downloaded to the user's machine.
    - A JVM is required to view an applet. The JVM can be either a plug-in of the Web browser or a separate runtime environment.
    - The JVM on the user's machine creates an instance of the applet class and invokes various methods during the applet's lifetime.
    - Applets ***have strict security rules*** that are enforced by the Web browser. The security of an applet is often referred to as sandbox security, comparing the applet to a child playing in a sandbox with various rules that must be followed.
    - Other classes that the applet needs can be downloaded in a single Java Archive (JAR) file.

Life cycle of an Applet

- When an applet begins, the following methods are called, in this sequence:
  1. init ()
  2. start ()
  3. paint ()
- When an applet is terminated, the following sequence of method calls takes place:
  1. stop ()
  2. destroy ()



1. ***init():*** This method is intended for ***whatever initialization is needed for your applet***. It is called after the param tags inside the applet tag have been processed. This method is ***called only once*** during the run time of your applet.

2. ***start ():*** This method is ***automatically called after the browser calls the init method***. It is also called to restart an applet after it has been stopped. Note that init () is called once i.e., when the first time an ***applet is loaded whereas start () is called each time*** an applet's HTML document is displayed onscreen. It is ***also called whenever the user returns to the page*** containing the applet after having gone off to other pages.

3. ***paint ():*** Invoked immediately after the start () method, and also any time the applet ***needs to repaint itself*** in the browser. For example, the window in which the applet is running may be overwritten by another window and then uncovered. Or the applet window may be minimized and then restored. ***Whatever the cause, whenever the applet must redraw its output, paint () is called.*** The paint () method has one ***parameter of type Graphics.*** This parameter will contain the graphics context, which describes the graphics environment in which the applet is running. This

context is used whenever output to the applet is required. ***Its prototype is*** ***public void paint (Graphics g);*** where g is an object reference of class Graphic.

4. ***stop ():*** The stop () method is called ***when a web browser leaves the HTML document*** containing the applet—when it goes to another page, for example. When stop () is called, the applet is probably running. ***You should use stop () to suspend threads that don't need to run when the applet is not visible.*** You can ***restart them when start () is*** called if the user returns to the page.

5. ***destroy ():*** This method is only called when the browser shuts down normally. destroy () method is called when your applet needs to be removed completely from memory.

Advantage of Applet

- There are many advantages of applet. They are as follows:
  - It is simple to make it work on Linux, Windows and Mac OS i.e., to make it cross platform.
  - It is supported by most web browsers.
  - It will cache in most web browsers, so will be quick to load when returning to a web page but may get stuck in the cache and have issues when new versions come out.
  - It can have full access to the machine it is running on if the user agrees.
  - It can improve with use: after a first applet is run, the JVM is already running and starts quickly, benefitting regular users of Java but the JVM will need to restart each time the browser starts fresh.
  - It can run at a comparable (but generally slower) speed to other compiled languages such as C++, but many times faster than JavaScript.
  - It can move the work from the server to the client, making a web solution more scalable with the number of users/clients.

Drawback of Applet

- It requires the Java plug-in, which isn't available by default on all web browsers.
- Some browsers, notably mobile browsers running Apple iOS or Android do not run Java applets at all.
- It cannot start until the Java Virtual Machine is running, and this may have significant startup time the first time it is used.
- If untrusted, it has severely limited access to the user's system - in particular having no direct access to the client's disk or clipboard.
- Some organizations only allow software installed by the administrators. As a result, many users cannot view applets by default.
- Applets may require a specific JRE.

Two Types of Applets

- It is important to state at the outset that there are two varieties of applets based on Applet.
    - The first are those based directly on the ***Applet class***. These applets use the Abstract Window Toolkit (AWT) to provide the graphical user interface (or use no GUI at all). This style of applet has been available since Java was first created.
    - The second type of applets are those based on the ***Swing class JApplet***, which inherits Applet. Swing applets use the Swing classes to provide the GUI. Swing offers a richer and often easier-to-use user interface than does the AWT.
- Thus, both AWT- and Swing-based applets are valid.

Applet Skeleton

```
1  import java.awt.*;
2  import java.applet.*;
3  public class AppletTest extends Applet {
4      public void init() {
5          //initialization
6      }
7      public void start () {
8          //start or resume execution
9      }
10     public void stop() {
11         //suspend execution
12     }
13     public void destroy() {
14         //perform shutdown activity
15     }
16     public void paint (Graphics g){
17         //display the content of window
18     }
19 }// end of class
```

HTML Applet Tags

- The <applet> tag in HTML was used to embed Java applets into any HTML document. The <applet> tag ***was deprecated in HTML 4.01***, and its support has been completely discontinued starting from HTML 5. Alternatives available in HTML 5 are the <embed> and the <object> tags.

```
<!DOCTYPE html>
<html>

   <head>
      <title>HTML applet Tag</title>
   </head>

   <body>
      <p>Example of Applet Tag</p>
      <applet code = "javaClassName.class" width = "300" height = "200" align="right">
      <param name1=parameter_name value1=parameter_value>
      .
      .
      .
      <param namen=parameter_name valuen=parameter_value>

      </applet>
   </body>

</html>
```

Java Applet vs Java Application

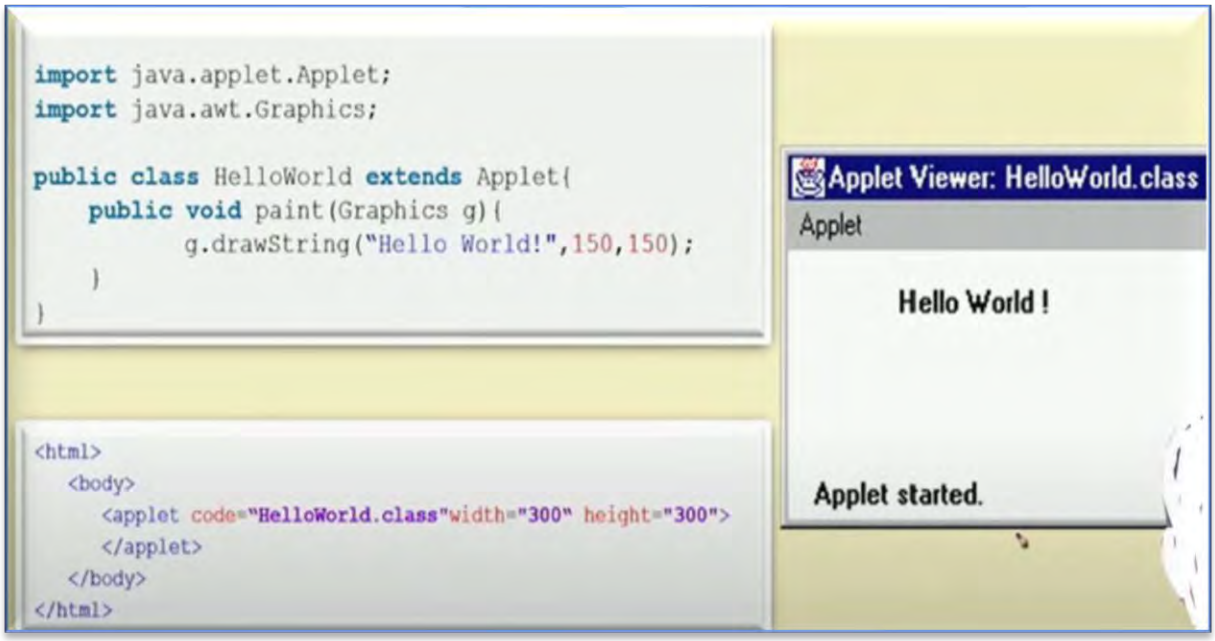
Java Application	Java Applet
Java Applications are the stand-alone programs which can be executed independently	Java Applets are small Java programs which are designed to exist within HTML web document
Java Applications must have main() method for them to execute	Java Applets do not need main() for execution
Java Applications just needs the JRE	Java Applets cannot run independently and require API's
Java Applications do not need to extend any class unless required	Java Applets must extend java.applet.Applet class
Java Applications can execute codes from the local system	Java Applets Applications cannot do so
Java Applications has access to all the resources available in your system	Java Applets has access only to the browser-specific services

Specifying Applet Parameters

- User-define Parameter can be applied in applet ***using <PARAM…> tags***. Each <PARAM…> tag has a ***name and value attribute***.

  Syntax: ***<PARAM name = …… Value = "……" >***

- In an applet code, applet can refer to a parameter by its name and then find its value. The two most important thing to handle and set up the parameter is the <PARAM> tag in the HTML document and an applet code to parse this parameter.
- ***init () method*** is used to get hold of the parameters which is defined in the <PARAM> tags. And ***getParameter ()*** method is used for getting the parameters.
- In Applet, Parameters are passed on applet when it is loaded.




```
// Use of  init( ) to pass value through HTML to applet //

import java.applet.*;
import java.awt.*;

public class RectangleTest extends Applet {
          int x, y, w, h;
          public void init (  ) {
             x  = Integer.parseInt(getParameter (" xValue" ));
             y  = Integer.parseInt(getParameter (" yValue" ));
             w =  Integer.parseInt(getParameter (" wValue" ));
             h  = Integer.parseInt(getParameter (" hValue" ));
          }

          public void paint ( Graphics g ) {
             g.drawRect (x, y, w, h );
       }
    }
```

```
<applet code = " RectangleTest"  width = 150  height = 100 >
    < param name = xValue  value = 20 >
    < param name = yValue  value = 40 >
    < param name = wValue  value = 100>
    < param name = hValue  value = 50 >
</applet >
```

```java
//simpleApplet.java
import javax.swing.*;
import java.awt.*;
@SuppressWarnings("removal")
public class simpleApplet extends JApplet {
    String str;
    public void init(){
        str=getParameter("pname");
        if (str == null)
            str = "Java Applet Example.";
        str = "This is First Applet. " + str;
    }
    public void paint(Graphics g){
        g.drawString(str, 50, 50);
    }
}// end of applet class

//param.html
<html>
    <applet code=simpleApplet.class height=350 width=250>
        <param Name="pname" value="Welcome to Java Applet Example.">
    </applet>
</html>
```

```java
//AppletEventHandling.java
import java.awt.event.*;
import java.applet.Applet;
import java.awt.*;
public class AppletEventHandling extends
                        Applet implements MouseListener {
    StringBuffer strBuf;
    public void init() {
        addMouseListener(this);
        strBuf = new StringBuffer();
        addItem(" Initializing the applet");
        addItem(" Welcome to the Applet Program");
    }
    public void start() {
        addItem(" Starting the applet ");
    }
    public void stop() {
        addItem(" Stopping the applet ");
    }
    public void destroy() {
        addItem(" Destroying the applet");
        addItem(" Good Bye!!");
    }
     void addItem(String word) {
         System.out.println(word);
         strBuf.append(word);
         repaint();
     }
     public void paint(Graphics g) {
         g.drawString(strBuf.toString(), 10, 20);
         setForeground(Color.white);
         setBackground(Color.black);
     }
     public void mouseEntered(MouseEvent event) {}
     public void mouseExited(MouseEvent event) {}
     public void mousePressed(MouseEvent event) {}
     public void mouseReleased(MouseEvent event) {}
     public void mouseClicked(MouseEvent event) {
         addItem(" Mouse is Clicked!!");
     }
}//end of class
//mouseevent.htma
<html>
   <title>Event Handling</title>
   <applet code = "AppletEventHandling.class"
      width = "300" height = "300">
   </applet>
</html>
```

Chapter 7
JDBC

Introduction to SQL

- Structured Query Language (SQL)
- SQL is an ANSI (American National Standards Institute) standard computer language for accessing and manipulating database systems.
- SQL works with database programs like MS Access, DB2, Informix, MySQL, SQL Server, Oracle, Sybase, etc.
- Most of the SQL database programs also have their own proprietary extensions in addition to the SQL standard.

SQL: Syntax for Creating Table

The general form of the **create table** command is:

create table r
$(A_1 \quad D_1,$
$A_2 \quad D_2,$
$\ldots,$
$A_n \quad D_n,$
$\langle$integrity-constraint$_1\rangle,$
$\ldots,$
$\langle$integrity-constraint$_k\rangle);$

where r is the name of the relation, each A_i is the name of an attribute in the schema of relation r, and D_i is the domain of attribute A_i; that is, D_i specifies the type of attribute A_i along with optional constraints that restrict the set of allowed values for A_i.

```
create table section
    (course_id      varchar (8),
     sec_id         varchar (8),
     semester       varchar (6),
     year           numeric (4,0),
     building       varchar (15),
     room_number    varchar (7),
     time_slot_id   varchar (4),
     primary key (course_id, sec_id, semester, year),
     check (semester in ('Fall', 'Winter', 'Spring', 'Summer')));
```

- *Semester must be one of 'Fall', 'Winter', 'Spring', or 'Summer.*

```
create table section
    (course_id      varchar (8),
     sec_id         varchar (8),
     semester       varchar (6) check (semester in
                                ('Fall', 'Winter', 'Spring', 'Summer')),
     year           numeric (4,0) check (year > 1759 and year < 2100),
     building       varchar (15),
     room_number    varchar (7),
     time_slot_id   varchar (4),
     primary key (course_id, sec_id, semester, year),
     foreign key (course_id) references course,
     foreign key (building, room_number) references classroom);
```

```
Create table Employee (
      empno int constraint pk_emp primary key,
      city varchar not null constraint dft_city Default 'KTM',
      ename varchar(50),
      salary float,
      hire_date date,
      gender char(1) constraint chk_gen  check(gender in ('M', 'F', 'm', 'f')),
      email varchar(50) unique,
  );


Create table course
    (
       course_id   char(5) constraint pk_cur primary key,
       empno int,
       title  varchar(20),
       dept_name varchar(20) ,
       constraint fk_cur foreign key (empno) references Employee on delete cascade
            on update cascade

          )
```

- ***INSERT INTO Syntax:*** It is possible to write the INSERT INTO statement in two ways:
  1) Specify both the column names and the values to be inserted:
     INSERT INTO table_name (column1, column2, column3, ...)
     VALUES (value1, value2, value3, ...);
  2) If you are adding values for all the columns of the table, you do not need to specify the column names in the SQL query.
     INSERT INTO table_name
     VALUES (value1, value2, value3, ...);
- ***Insert Data Only in Specified Columns:*** It is also possible to only insert data in specific columns.
  INSERT INTO Customers (CustomerName, City, Country)
  VALUES ('Cardinal', 'Stavanger', 'Norway');
- *CustomerSchema(CustomerName, ContactName, Address, City, PostalCode, Country)*

```
insert into department values("Computer", "RIC",200000);
insert into department values("Electronics", "RIC",300000);
insert into department values("Civil", "D",400000);
insert into department values("Electrical", "B",100000);
insert into department values("Mechanical", "A",200000);
insert into department values("Geomatics", "Msc.",250000);
insert into department values("Humanities", "C",50000);
```

```
SELECT    <attribute list>
FROM      <table list>
WHERE     <condition>;
```

where

- <attribute list> is a list of attribute names whose values are to be retrieved by the query.
- <table list> is a list of the relation names required to process the query.
- <condition> is a conditional (Boolean) expression that identifies the tuples to be retrieved by the query.

- ***Syntax***
  - *SELECT "column_name" FROM "table_name"*
  - *SELECT * FROM "table_name"*
  - *SELECT DISTINCT "column_name" FROM "table_name"*
  - *SELECT "column_name" FROM "table_name" WHERE "condition"*
  - *SELECT "column_name" FROM "table_name" WHERE "simple condition" [AND|OR] "simple condition"*
  - *SELECT "column_name" FROM "table_name" WHERE "column_name" BETWEEN 'value1' AND 'value2'*
  - *SELECT "column_name" FROM "table_name" WHERE "column_name" IN ('value1', 'value2', ...)*

- Pattern matching can be performed on strings using the operator like.
- String patterns describes by using two special characters:
  - *Percent (%): The % character matches any substring.*
  - *Underscore (_): The character matches any character.*
- For Example:
  - *'_ _ _' matches any string of exactly three characters.*
  - *'_ _ _%' matches any string of at least three characters.*
  - *'%Comp%' matches any string containing "Comp" as a substring.*
- SQL also permits strings concatenating using "||". Also supports converting strings to uppercase using the function ***upper(s)*** where s is a string, lowercase using the function ***lower(s)***, removing spaces at the end of the string using ***trim(s)***.

Modification of the Database

- A delete command operates on only one relation.
- If we want to delete tuples from several relations, we must use one delete command for each relation. The predicate in the where clause may be as complex as a select command's where clause.

- Delete all records of the instructor who works in the Finance department.

```
delete from instructor
where dept name = 'Finance';
```

- Delete all instructors with a salary between 13,000 and 15,000.

```
delete from instructor
where salary between 13000 and 15000;
```

- Delete all tuples in the instructor relation for those instructors associated with a department located in the Watson building.

```
delete from instructor
where dept name in (select dept name
from department
where building = 'Watson');
```

- Delete the records of all instructors with salary below the average at the university.

```
delete from instructor
where salary < (select avg (salary)
from instructor);
```

```
insert into course
values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

insert into course (course id, title, dept name, credits)
values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

insert into course (title, course id, credits, dept name)
values ('Database Systems', 'CS-437', 4, 'Comp. Sci.');

insert into instructor
select ID, name, dept name, 18000
from student
where dept name = 'Music' and tot cred > 144;
```

- Increased salaries of all instructors by 5 percent.

```
update instructor set salary= salary * 1.05;
```

- Increase salary of instructors by five percent whose salary is less than 70,000.

```
update instructor
set salary = salary * 1.05
where salary < 70000;
```

- Give a 5 percent salary raise to instructors whose salary is less than average.

```
update instructor
set salary = salary * 1.05
where salary < (select avg (salary)
from instructor);
```

Introduction to JDBC

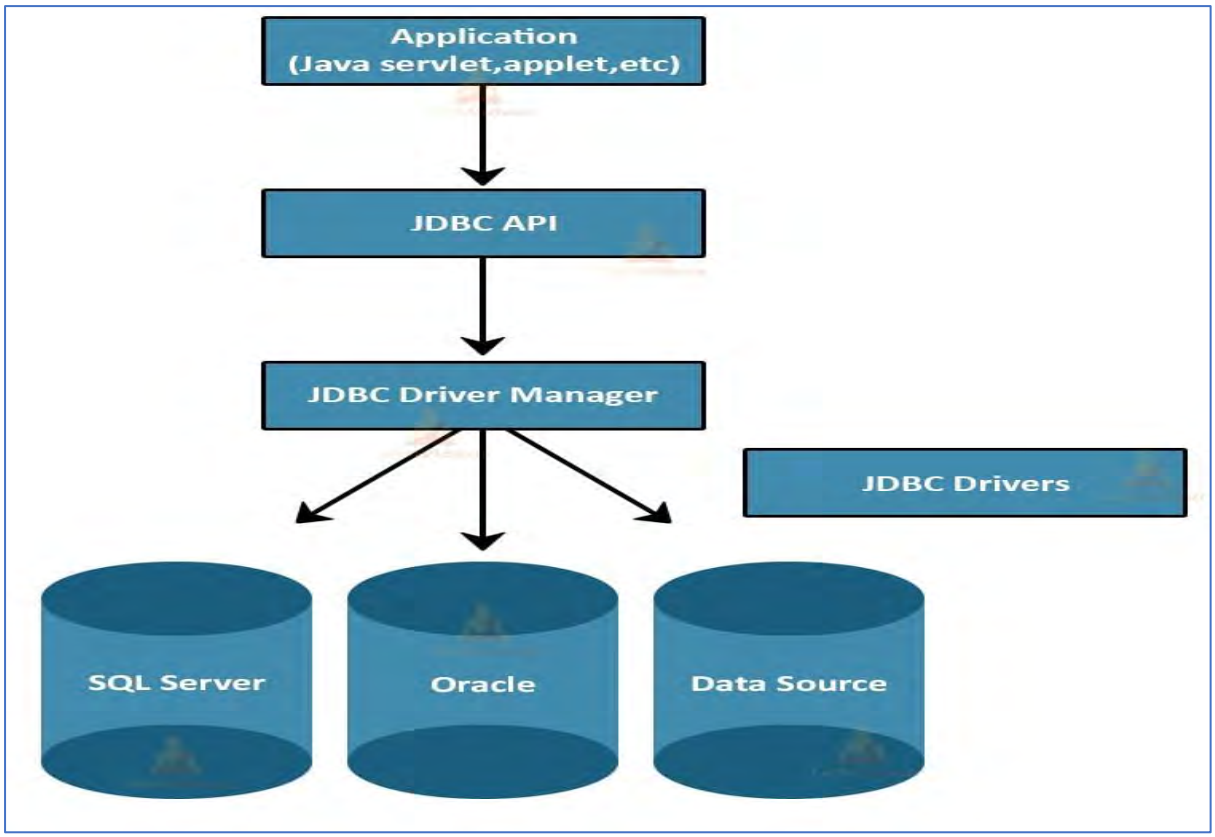
- JDBC stands for Java Database Connectivity.
- ***JDBC is a Java API to connect and execute the query with the database.*** It is a part of JavaSE (Java Standard Edition). JDBC API uses JDBC drivers to connect with the database.
- JDBC is a Java database connectivity technology from Oracle Corporation. This ***technology is an API for the Java programming language that defines how a client may access a database.*** It provides methods for querying and updating data in a database.
- JDBC is oriented towards relational databases. A JDBC-to-ODBC bridge enables connections to any ODBC-accessible data source in the JVM host environment.

- JDBC is a standard Java API for handling database related activities.
- In Java, there is a package java.sql having number of classes for database related programming.
- It includes java.sql.DriverManager class and two interfaces java.sql.Driver and java.sql.Connection.

Why JDBC?

- Write once, run anywhere
  - Multiple client and server platforms.
- Object-relational mapping
  - Databases optimized for searching/indexing.
  - Objects optimized for engineering/flexibility.
- Network independence
  - Works across Internet Protocol.
- Database independence
  - Java can access any database vendor.

Architecture of JDBC



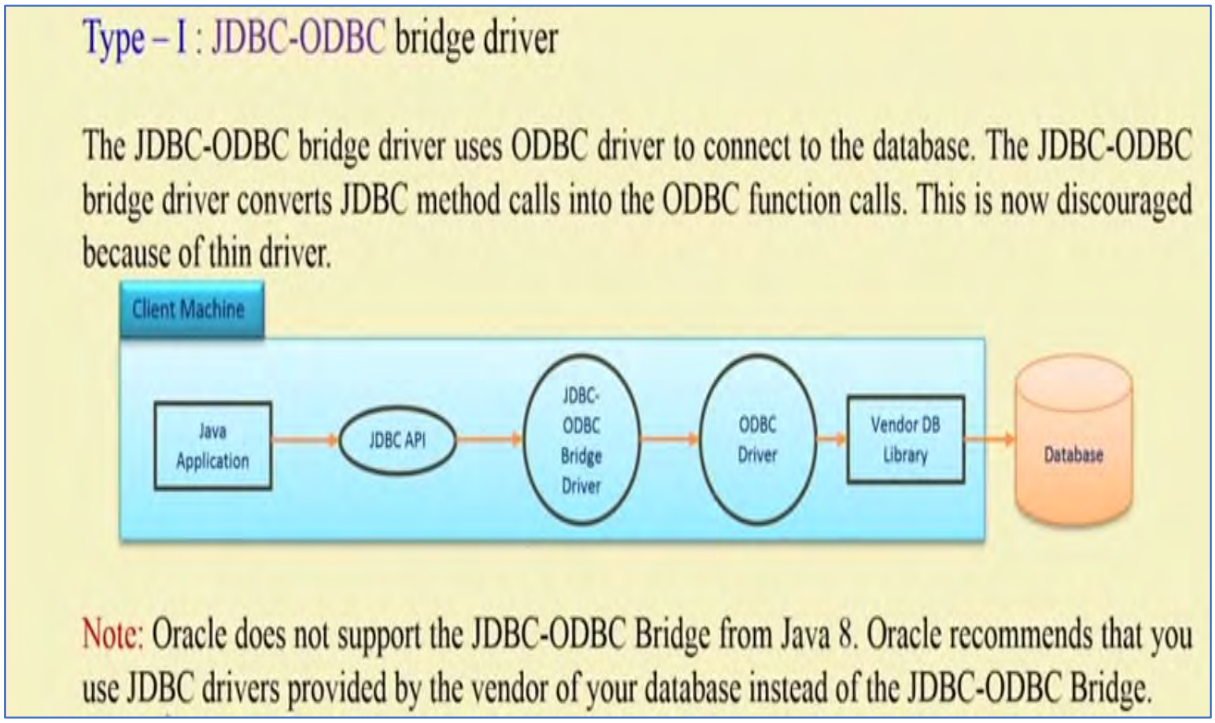
- ***Application:*** It is a java applet or a servlet that communicates with a data source.
- ***The JDBC API:*** The JDBC API allows Java programs to execute SQL statements and retrieve results. Some of the important classes and interfaces defined in JDBC API are as follows:
  - *Driver Manager*
  - *Driver*

- *Connection*
- *Statement*
- *PreparedStatement*
- *CallableStatement*
- *ResultSet*
- *SQL data*

- ***Driver Manager:*** It plays an important role in the JDBC architecture. It uses some database-specific drivers to effectively connect enterprise applications to databases.
- ***JDBC drivers:*** To communicate with a data source through JDBC, you need a JDBC driver that intelligently communicates with the respective data source.

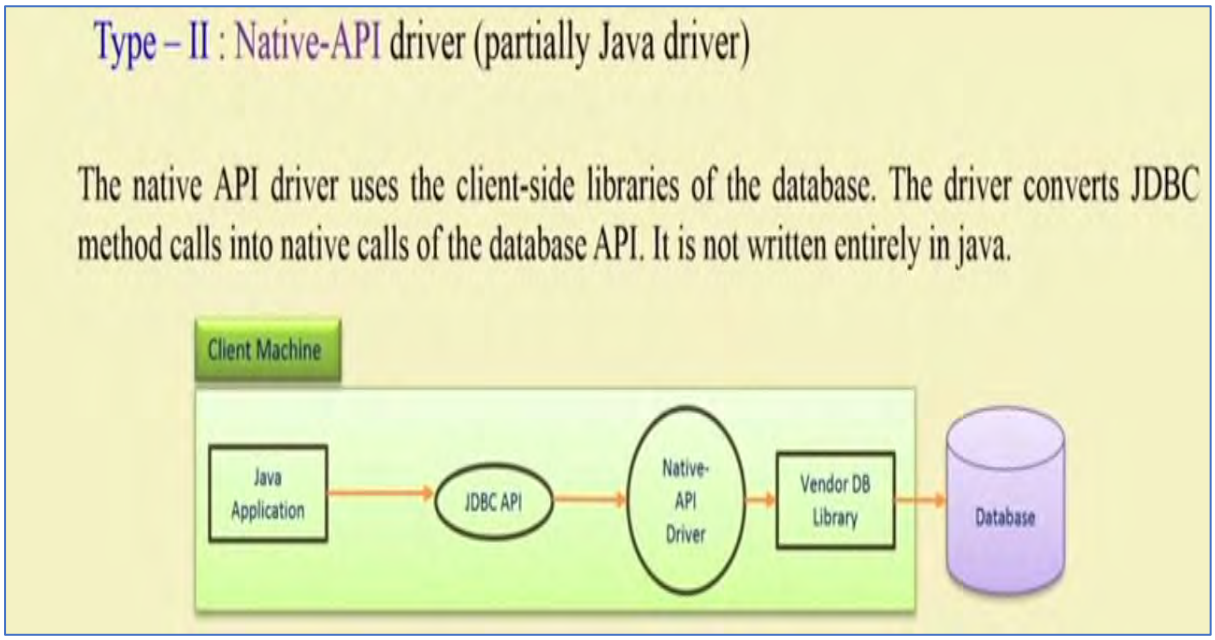
Type of JDBC drivers

- JDBC drivers are client-side adapters (installed on the client machine, not on the server) that convert requests from Java programs to a protocol that the DBMS can understand.
- There are commercial and free drivers available for most relational database servers. JDBC technology drivers fit into one of four categories.
    1. **JDBC-ODBC** bridge driver
    2. **Native-API Driver** (Partially Java Driver)
    3. **Network-Protocol Driver** (Fully Java Driver)
    4. **Thin Driver** (Fully Java Driver)



- **Advantages**
    1. Easy to use.
    2. Can be easily connected to any database.

- **Disadvantages**
  1. Performance degraded because JDBC method call is converted into the ODBC function.
  2. The ODBC driver needs to be installed on the client machine.

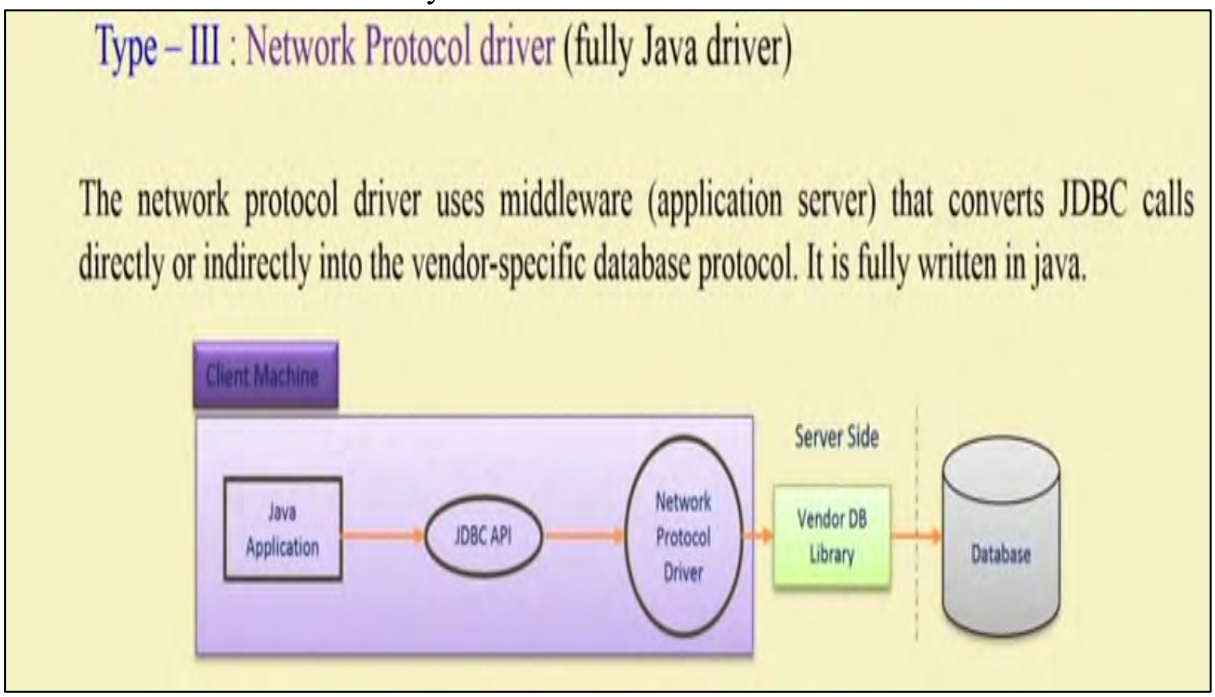


Advantages:

- As there is no implementation of JDBC-ODBC bridge, its considerably faster than a type 1 driver.

Disadvantages:

- The native driver needs to be installed on each client machine.
- The vendor client library needs to be installed on the client machine.

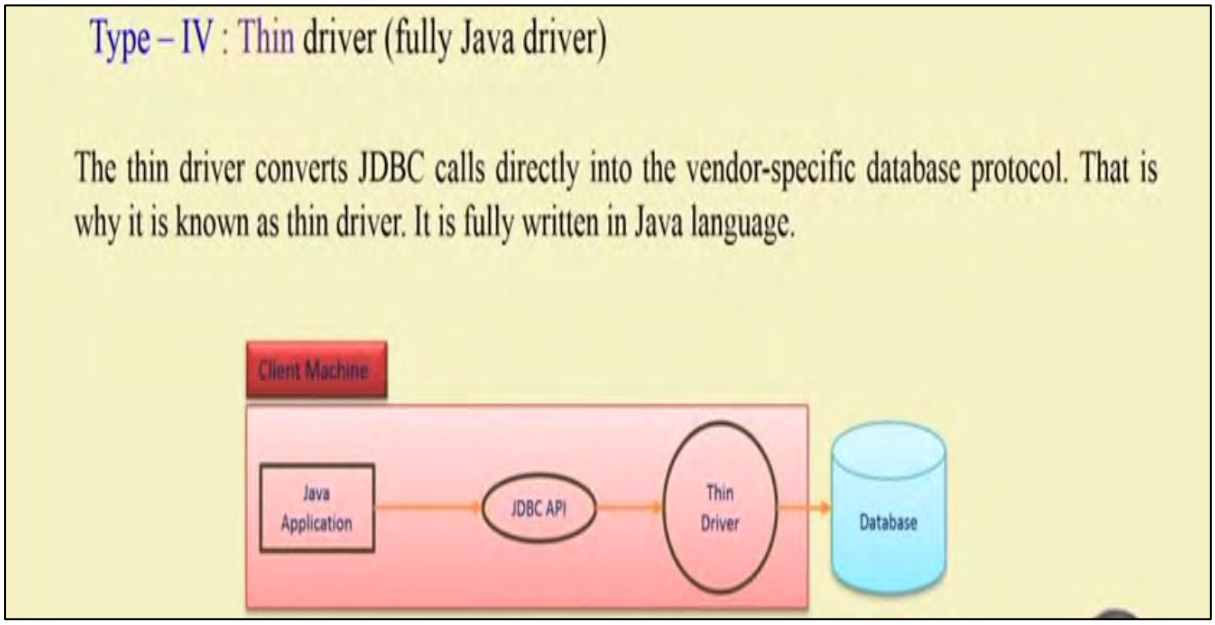


Advantage

1. No client-side library is required because of application server that can perform many tasks like auditing, load balancing, logging etc.

Disadvantage

1. Network support is required on client machine.
2. Requires database-specific coding to be done in the middle tire.
3. Maintenance of Network Protocol driver becomes costly because it requires database-specific coding to be done in the middle tire.



Advantage

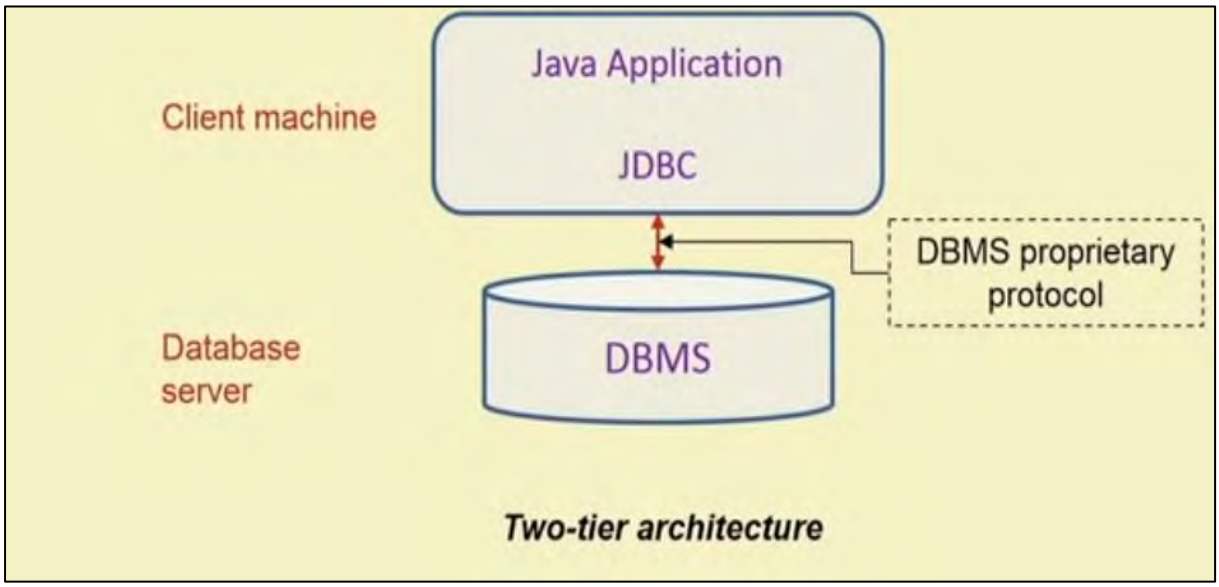
1. Completely implemented in Java to achieve platform independence.
2. These drivers don't translate the requests into an intermediary format (such as ODBC).
3. Better performance than all other drivers.
4. No software is required at client side or server side.

Disadvantage

1. Drivers are database dependent, as different database vendors use widely different (and usually proprietary) network protocols.

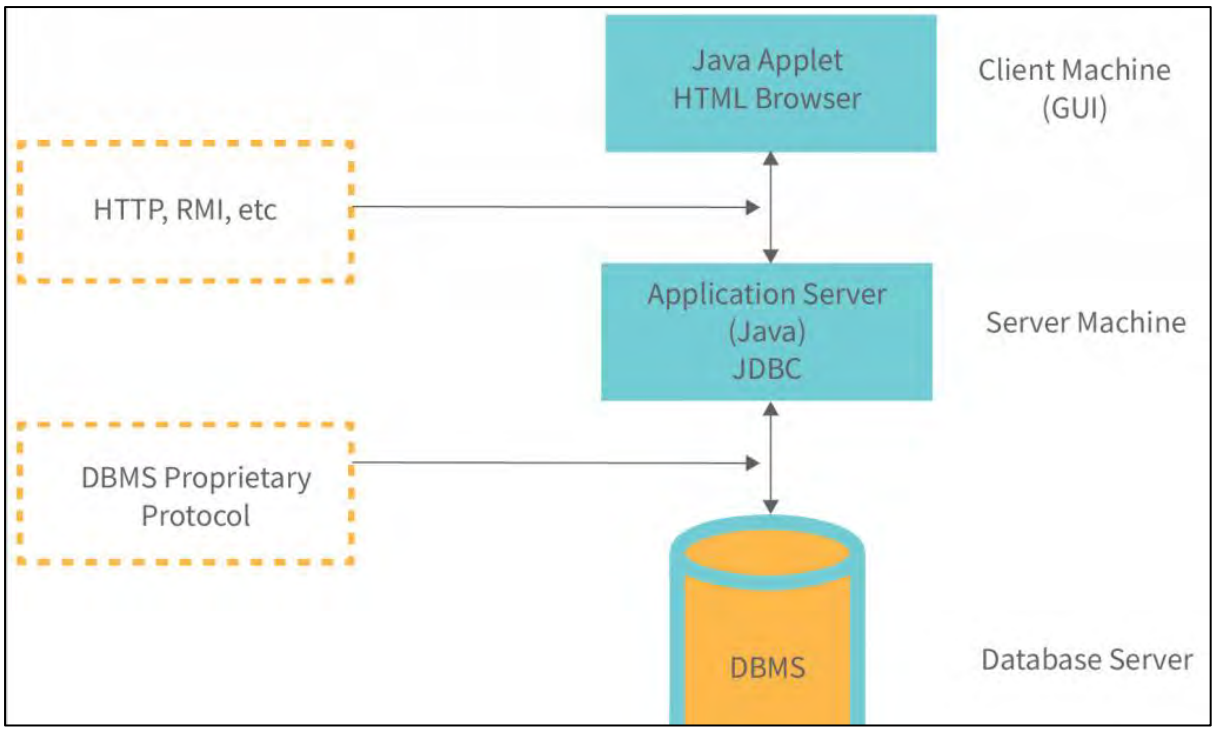
JDBC Architecture: Two-Tier Architecture

- A Java applet or application communicates directly with the data source in the two-tier paradigm. This necessitates the use of a JDBC driver that can interface with the data source in question.
- The user's commands are transmitted to the database or other data source, and the statements' results are returned to the user. The data source could be on another machine to which the user has a network connection.
- A client/server configuration is one in which the user's machine acts as the client and the system that houses the data source acts as the server. An intranet, for example, can connect people within a company, or the Internet can be used as the network.



JDBC Architecture: Three-Tier Architecture

- Commands are sent to a "middle tier" of services in the three-tier paradigm, which subsequently transmits the commands to the data source.
- The data source interprets the commands and provides the results to the middle tier, which ultimately passes them on to the user.
- The three-tier architecture appeals to MIS directors because the intermediate tier allows them to maintain control over access and the types of changes that can be made to company data.
- Another benefit is that it makes application deployment easier. Finally, the three-tier architecture can bring performance benefits in many circumstances.



Common JDBC Components

Class/Interface	Description
DriverManager	This **class** manages a list of database drivers. Matches connection requests from the Java application with the proper database driver using communication sub protocol. The first driver that recognizes a certain sub-protocol under JDBC will be used to establish a database Connection.
Driver	This **interface** handles the communications with the database server. You will interact directly with Driver objects very rarely. In stead, you use DriverManager objects, which manages objects of this type. It also abstracts the details associated with working with Driver objects.
Connection	This **interface** with all methods for contacting a database. The connection object represents communication context, i.e., all communication with database is through connection object only.
Statement	You use objects created from this **interface** to submit the SQL statements to the database. Some derived interfaces accept parameters in addition to executing stored procedures.
ResultSet	This **class** retrieves data from a database after you execute an SQL query using Statement objects. It acts as an iterator to allow you to move through its data.
SQLException	This **class** handles any errors that occur in a database application.

Steps to connect Java program and database

```
// Load the database driver
Class.forName( "sun.jdbc.odbc.JdbcOdbcDriver" ) ;

// Get a connection to the database
Connection conn = DriverManager.getConnection( "jdbc:odbc:Database" ) ;

// Get a statement from the connection
Statement stmt = conn.createStatement() ;

// Execute the query
ResultSet rs = stmt.executeQuery( "SELECT * FROM Cust" ) ;

// Loop through the result set
while( rs.next() )
   System.out.println( rs.getString(1) ) ;

// Close the result set, statement and the connection
rs.close() ;
stmt.close() ;
conn.close() ;
```

1. Loading the Driver

- We first need to load the driver or register it before using it in the program. There should be registration once in your program.
- ***Class.forName():*** In this, we load the driver's class file into memory during runtime. There is no need to use a new operator for the creation of an object.
- To load the Oracle driver:

  Class.forName("oracle.jdbc.driver.OracleDriver");
- To load the MySQL driver:

  Class.forName("com.mysql.cj.jdbc.Driver.");

2. Create the connections

- After loading the driver, we need to establish connections using the following code:

  Connection con = DriverManager.getConnection(url, user, password)
- ***user:*** username from which sql command prompt can be accessed.
- **password:** password from which sql command prompt can be accessed.
- **url:** Uniform Resource Locator. We can create it as follows:
  - *String url = "jdbc:oracle:thin:@localhost:1521:xe"*
  - Where ***oracle*** is the database, ***thin*** is the driver, ***@localhost*** is the IP Address where the database is stored, ***1521*** is the port number, and ***xe*** is the service provider

- Create a `Connection` object
- Use the `DriverManager` to grab a connection with the `getConnection()` method

```
Connection conn = null;
String userName = "guest";
String password = "guest";
String url = "jdbc:mysql://10.14.100.141/test";
conn = DriverManager.getConnection (url, userName, password);
```

3. Create a statement

- Once you establish a connection, you can interact with the database.
- Three types of Statements

Interfaces	Recommended Use
Statement	Use this for general-purpose access to your database. Useful when you are using static SQL statements at runtime. The Statement interface cannot accept parameters.
PreparedStatement	Use this when you plan to use the SQL statements many times. The PreparedStatement interface accepts input parameters at runtime.
CallableStatement	Use this when you want to access the database stored procedures. The CallableStatement interface can also accept runtime input parameters.

PreparedStatement: this represents precompiled sql/my sql statement which allows improved performance. It allows to execute the query multiple times and we can set the values according to our need.

Statement	PreparedStatement	CallableStatement
It is used to execute normal SQL queries.	It is used to execute parameterized or dynamic SQL queries.	It is used to call the stored procedures.
It is preferred when a particular SQL query is to be executed only once.	It is preferred when a particular query is to be executed multiple times.	It is preferred when the stored procedures are to be executed.
You cannot pass the parameters to SQL query using this interface.	You can pass the parameters to SQL query at run time using this interface.	You can pass 3 types of parameters using this interface. They are – IN, OUT and IN OUT.
This interface is mainly used for DDL statements like CREATE, ALTER, DROP etc.	It is used for any kind of SQL queries which are to be executed multiple times.	It is used to execute stored procedures and functions.
The performance of this interface is very low.	The performance of this interface is better than the Statement interface (when used for multiple execution of same query).	The performance of this interface is high.

```
Statement stmt = null;
try {
   stmt = conn.createStatement( );
   . . .
}
catch (SQLException e) {
   . . .
}
finally {
   stmt.close();
}
```

```
PreparedStatement pstmt = null;
try {
   String SQL = "Update Employees SET age = ? WHERE id = ?";
   pstmt = conn.prepareStatement(SQL);
   . . .
}
catch (SQLException e) {
   . . .
}
finally {
   pstmt.close();
}
```

4. Execute the query

- The most crucial part is executing the query. Here, Query is an SQL Query.
  - The query for updating or inserting tables in a database.
  - The query for retrieving data from the database.

```
Statement stmt = null;
stmt = conn.createStatement();

stmt.execute("show tables");
stmt.execute("insert into JavaCourse values (01,'Debasis', 85, 'Ex')");
```

Methods	Description
executeQuery()	Executes SQL query and returns the data in a table (ResultSet) object. This method is used for SQL command that expects a return data from a database.
executeUpdate()	Used to execute INSERT, UPDATE, DELETE, CREATE TABLE, DROP TABLE, ALTER TABLE Returns the number of rows that are affected in the database
execute()	Generic method for executing simple statements, stored procedures, prepared statements. It can be used when the statement is either related to query or update. This method returns true(*if query yields a row*) otherwise returns false.
getMaxRows()	Determines the number of rows a ResultSet can contain.

```
PreparedStatement pstmt = null;

String QryString = "INSERT INTO JavaCourse (Roll,Name,Marks,Grade) VALUES(?,?,?,?)";
pstmt = conn.prepareStatement(QryString);

pstmt.setInt(1, 12);
pstmt.setString(2, "ABC");
pstmt.setInt(3, 64);
pstmt.setString(4, "Ex");
pstmt.executeUpdate();
```

- A `ResultSet` provides access to a table of data generated by executing a Statement.
- Only one `ResultSet` per Statement can be open at once.
- The table rows are retrieved in sequence.
- A `ResultSet` maintains a cursor pointing to its current row of data.
- The `next()` method moves the cursor to the next row.
  - you can't rewind.

Useful ResultSet Methods

Methods	Description
`boolean next()`	– attempts to move to the next row in ResultSet – the first call to next() positions cursor at the first row – returns false if there are no more rows
`Type getType(int columnIndex)`	– returns the given field as the given type – fields indexed starting at 1 (not 0)
`Type getType(String columnName)`	– same, but uses name of field – less efficient
`void close()`	– disposes of the ResultSet – allows you to re-use the Statement that created it
`int findColumn(String columnName)`	– looks up column index given column name

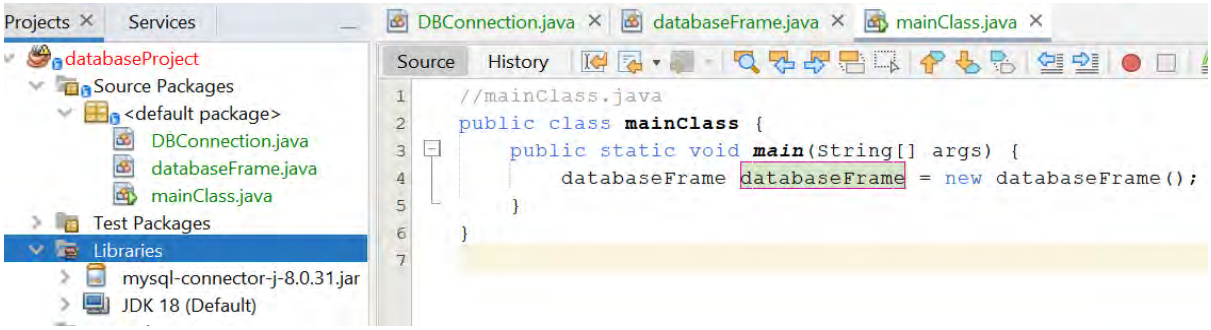
```
ResultSet rs = null;
String NameString, RollString, MarksString, GradeString;

stmt.execute("SELECT * FROM JavaCourse");
rs = stmt.getResultSet();

while (rs.next()) {
    NameString = rs.getString("Name");
    RollString = rs.getString("Roll");
    MarksString = rs.getString("Marks");
    GradeString = rs.getString("Grade");
}
```

```
try {
    String host = "jdbc:derby://localhost:1527/Employees";
    String uName = "Your_Username_Here";
    String uPass= " Your_Password_Here ";

    Connection con = DriverManager.getConnection( host, uName, uPass );

    Statement stmt = con.createStatement();
    String SQL = "SELECT * FROM Workers";
    ResultSet rs = stmt.executeQuery( SQL );

    while ( rs.next() ) {
        int id_col = rs.getInt("ID");
        String first_name = rs.getString("First_Name");
        String last_name = rs.getString("Last_Name");
        String job = rs.getString("Job_Title");

        System.out.println(id_col + " " + first_name + " " + last_name + " " + job);
    }
}
catch ( SQLException err ) {
    System.out.println( err.getMessage( ) );
}
```

5. Close the connection

- Release all the resources that the connection is holding.
    - *stmt.close();*
    - *con.close();*
    - *pstm.close();*

Example: Database CRUD operations Frame

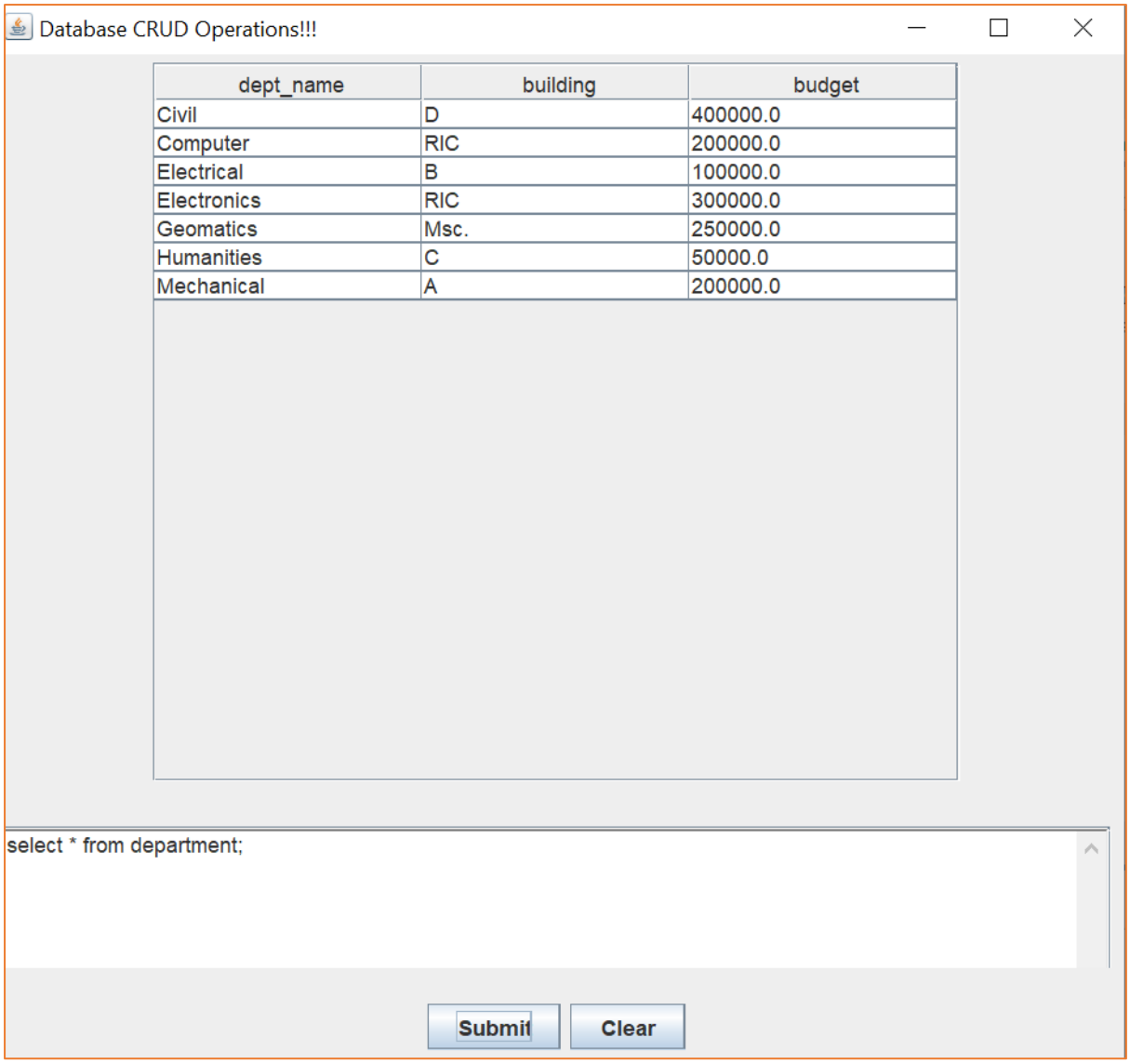


Source History

```java
//DBConnection.java
import java.sql.*;
public class DBConnection {
    public Connection con=null;
    public DBConnection(){
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            con=DriverManager.getConnection("jdbc:mysql://localhost:3306/university","root","");
        } catch (ClassNotFoundException | SQLException e) {
            System.out.println(e);
        }
    }
}
```

```java
//databaseFrame.java
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.sql.ResultSet;
import java.sql.ResultSetMetaData;
import java.sql.SQLException;
import java.sql.Statement;
import javax.swing.table.DefaultTableModel;
public class databaseFrame extends JFrame {
        JButton submitBtn,clearBtn;
        TextArea ta;  JTable resultTable;
        JPanel panel1,panel2,panel3;
        JScrollPane scroll;
        databaseFrame(){
            setTitle("Database CRUD Operations!!!");
            setSize(650,600);
            setLayout(new BorderLayout(15,15));
            setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

            resultTable=new JTable();
            JScrollPane sp = new JScrollPane(resultTable);
            panel1=new JPanel();  panel1.add(sp);
            add(panel1,BorderLayout.NORTH);// adding panel-1 to the frame at

```

```
                    ta=new TextArea(10,60);
                    scroll=new JScrollPane(ta);
                    panel2=new JPanel();  panel2.add(scroll);
                    add(panel2,BorderLayout.CENTER);// adding panel-2 to the frame a

                    submitBtn=new JButton("Submit");
                    clearBtn=new JButton("Clear");
                    panel3=new JPanel(new FlowLayout());
                    panel3.add(submitBtn); panel3.add(clearBtn);

                    myEventHandler eventHandler=new myEventHandler();
                    submitBtn.addActionListener(eventHandler);
                    clearBtn.addActionListener(eventHandler);
                    add(panel3,BorderLayout.SOUTH);// adding panel-3 to the frame at
                    //pack();
                    setVisible(true);
                }// end of constructor

public class myEventHandler implements ActionListener{
    @Override
    public void actionPerformed(ActionEvent e) {
        String str=e.getActionCommand();
        DBConnection db=new DBConnection();
        if(str.equals("Submit")){
            try {
                Statement stmt=db.con.createStatement();
                var query=ta.getText();
                ResultSet rs=stmt.executeQuery(query);

                ResultSetMetaData rsmd=rs.getMetaData();
                DefaultTableModel model= (DefaultTableModel) resultTable.getModel();
                model.setRowCount(0);// to reset the result table

                int columnCount=rsmd.getColumnCount();
                //Get all column names from meta data and add columns to table model
                for (int columnIndex = 1; columnIndex <= columnCount; columnIndex++)
                    model.addColumn(rsmd.getColumnLabel(columnIndex));

                //Create array of Objects with size of column count from metadata
                Object[] row = new Object[columnCount];

                 //Scroll through result set
                 while (rs.next()){
                  //Get object from column with specific index of result
                       for (int i = 0; i < columnCount; i++)
                           row[i] = rs.getObject(i+1);
                  //add row to table model with that array of objects
                      model.addRow(row);
                  }
                 //add that table model to reslut table
                  resultTable.setModel(model);
            }
            catch (SQLException ex) {
                System.out.println(ex);
            }
        } //end of submit if end

        if(str.equals("Clear")){
            ta.setText("");
        }
    }// end of actionPerformed function
}// end of myEventHandler class
}// end of databaseFrame class
```


Database CRUD Operations!!!
dept_name
building
budget
Civil
D
400000.0
Computer
RIC
200000.0
Electrical
B
100000.0
Electronics
RIC
300000.0
Geomatics
Msc.
250000.0
Humanities
C
50000.0
Mechanical
A
200000.0
select * from department;
Submit
Clear

Chapter 8

Introduction to J2EE

Java: Broadest Industry Adoption

9,000,000

JAVA DEVELOPERS

DEPLOYING TO 18 COMPLIANT APPLICATION SERVERS

ORACLE
FUSION MIDDLEWARE
WEBLOGIC SERVER

HITACHI

Cosminexus

GERONIMO

FUJITSU

TmaxSoft
TmaxSoft Co., Ltd.

IBM

CAUCHO

OW2

SAP

redhat

Java SE

- *Java SE:* It stands *Java Standard Edition* which means all concepts related to core java such as I/O, Collections, Threading, Concurrency, etc.
- Java SE's API provides the *core functionality of the Java programming language.*
- It defines everything *from the basic types and objects* of the Java programming language to *high-level classes that* are used for networking, security, database access, graphical user interface (GUI) development, and XML parsing.
- In addition to the core API, the Java SE platform *consists of a virtual machine, development tools, deployment technologies, and other class libraries and toolkits* commonly used in Java technology applications.

Java EE

- The Java EE platform is *built on top of the Java SE platform.*
- Java Platform Enterprise Edition.
- It is a *platform-independent*, *Java-centric environment* from Sun/Oracle *for developing, building and deploying Web-based enterprise applications* online for business solutions.
- The J2EE platform consists of a *set of services, APIs, and protocols* that provide the functionality for *developing multi-tiered, Web-based applications.*
- The Java EE platform provides an API and runtime environment for developing and running large-scale, multi-tiered, scalable, reliable, secure network applications.

J2EE Features

- ✓ J2EE stands for Java 2 Enterprise Edition.
- ✓ J2EE is a platform for building server side components and applications. It provides the infrastructures needed for these applications.
- ✓ J2EE is a platform and not a language.
- ✓ We use J2EE platform to build large scale enterprise level applications in Java that run on Internet.
- ✓ A J2EE platform is also known as J2EE Application Server.
- ✓ A J2EE application server supports all the standard J2EE specifications like JDBC (Java Database Connectivity), Servlet, JSP (Java Server Pages), EJB (Enterprise JavaBeans), JMS (Java Message Service) etc.

Differences between Java EE and Java SE

- Java technology is both a programming language and a platform.
- The Java programming language is a high-level object-oriented language that has a particular syntax and style.
- A Java platform is a particular environment in which Java programming language applications run.
- There are several Java platforms.
- Many developers, even long-time Java programming language developers, do not understand how the different platforms relate to each other.

What is Enterprise Application?

- An enterprise or e-commerce application is an application that an enterprise or an organization uses to do business using Internet.
- The advent of internet has completely changed the world of communication. With the widespread usage of internet, businesses quickly realized a whole new market had opened up to exploit and started taking advantage through e-commerce.
- With almost all the businesses going online, the information assets of a company became more and more valuable.
- In order to sustain the competition, adoption of sophisticated technologies has become the key factor in exploiting the information assets of a business.
- J2EE is one such technology that helps businesses to build better and flexible e-commerce applications by securing the critical business data.

Key challenges of enterprise application

- Performance: Since the application is now exposed to millions of users, its very important that it responds faster enough to user requests. This is one of the most important aspects that play a key role in the success of a business.
- Reliability: The application must be reliable in terms of processing the business transactions successfully and accurately.
- Availability: It's very important that the application be up and running all the times with almost zero downtime. Few seconds of downtime can result in losing thousands of $$.
- Security: The application must be able to provide a secure environment during the exchange of information between customers and businesses.

Core J2EE Technologies

- J2EE technologies are standard helper technologies that we use to build enterprise applications.
- Here is list of important technologies that every J2EE application server supports:

Technology Name	Description
JDBC	Used for easy interaction with various Databases
Servlets/JSP	Used for building dynamic web applications in Java
JNDI	Used for interacting with Naming and Directory Services
JMS	Used for interacting with Messaging Systems
EJB	Component technology for building distributed components
Java mail	Used for sending Emails

Software Architecture

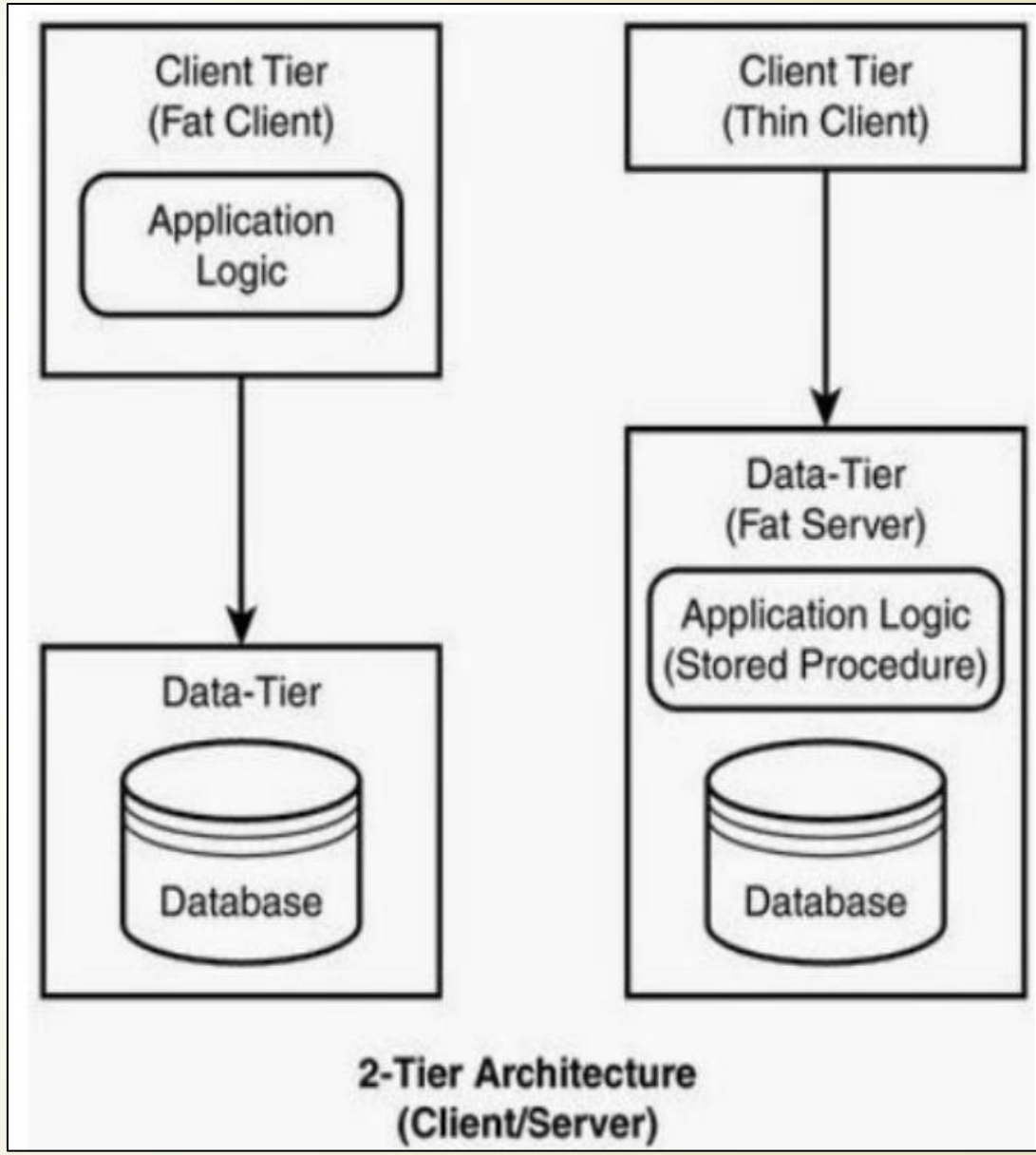
- Software Architecture can be of *One Tier*, *Two Tier*, *Three Tier* and *N-Tier.* A ***"tier"*** can also be referred to as a ***"layer".***
- Generally *three layers* are involved in the application namely *Presentation Layer, Business Layer* and *Data Layer.*
- *Presentation Layer*
  - It is also known as Client layer. Top most layer of an application. By using this layer we can access the webpages.
  - The main functionality of this layer is to communicate with Application layer.
  - This layer passes the information which is given by the user in terms of keyboard actions, mouse clicks to the Application Layer.
  - For example, login page of Gmail where an end user could see text boxes and buttons to enter user id, password and to click on sign-in.

Software Architecture

- *Application Layer*
  - It is also *known as Business Logic Layer* which is also known *as logical layer*.
  - Complete business logic will be written in this layer.
  - As per the Gmail login page example, once user clicks on the login button, Application layer interacts with Database layer and sends required information to the Presentation layer.
  - It controls an application's functionality by performing detailed processing.
  - This layer acts as a mediator between the Presentation and the Database layer.
- *Data Layer:* The *data is stored* in this layer. Application layer communicates with Database *layer to retrieve the data*. It contains methods that connects the database and performs required action *e.g. insert, update, delete etc.*

2-Tier Architecture

- In this architecture, all the business logic and presentation logic of the application is embedded in the clients computer itself.
- Therefore if my application has 100 clients, I *need to install* the application on *all the 100 client computers*.
- However the *database will still reside on a separate computer* that will be shared by all the clients.



2-Tier Architecture

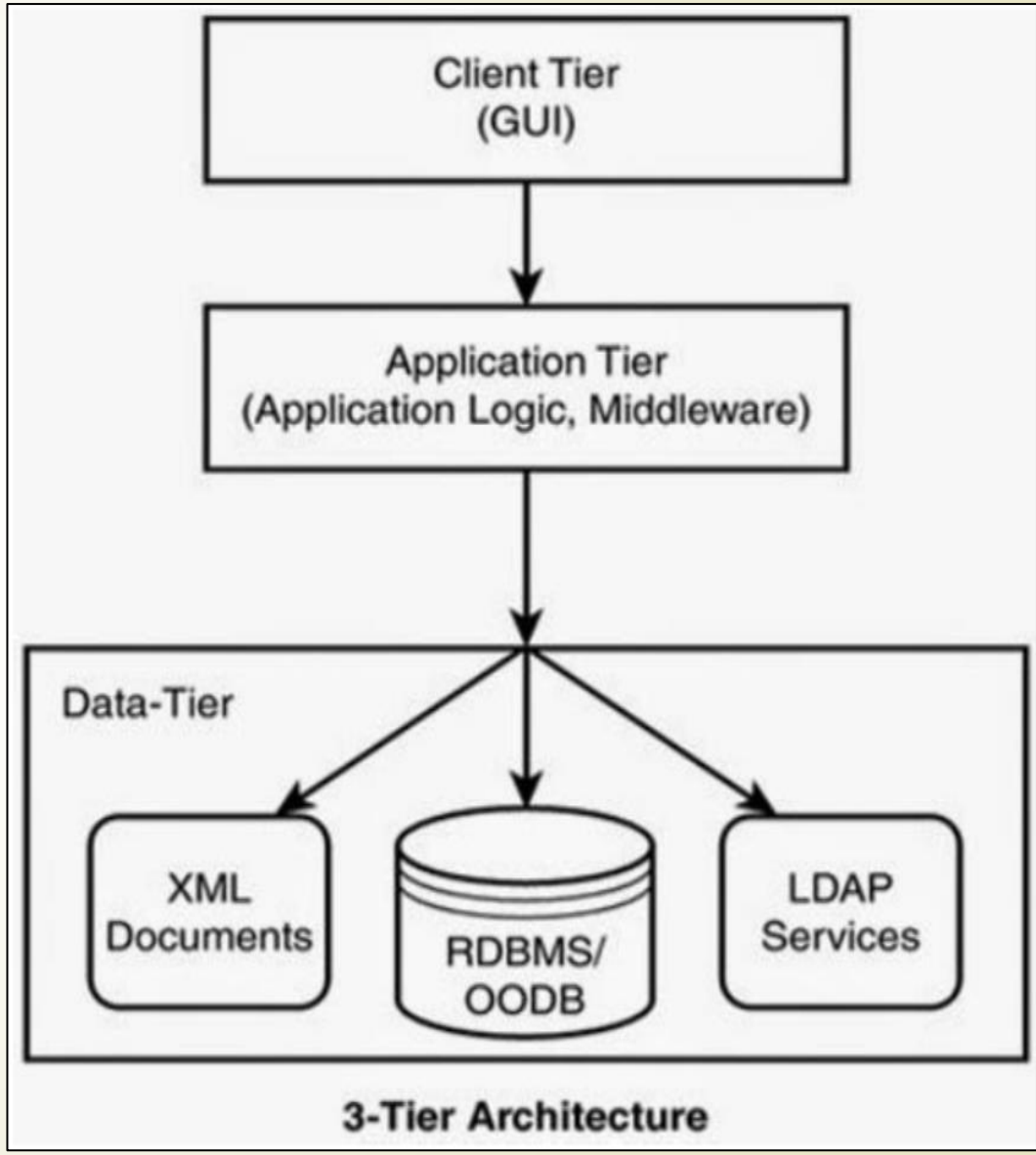
- *Advantages*
- Simple and easy to build, *Low cost*
- *Disadvantages*
- Even a small change to the application requires a reinstall of the application on all the client computers.
- If some clients are not ready to take the changes, then multiple versions of the application will prevail, thereby causing maintenance nightmares.
- The performance of the application is dependent on the performance of client computer.
- Heavy network traffic due to multiple requests to the database from all the client computers.
- Scalability, and Low Security
- *The disadvantages of this architecture outweigh the few advantages making it less useful.*

3-Tier Architecture

- In this architecture, the *presentation logic*, *business logic* and *data* are *logically distributed in three tiers*.
- Client system handles Presentation layer, Application server handles Application layer and Server system handles Database layer.
- The main difference is that the *business logic is permanently isolated from all the client systems* and moved to centralized middle tier.
- The *presentation logic on the client systems* will query the business logic in the middle tier which in turn accesses the data from the database.
- This architecture overcomes all the cons of 2-tier system.

3-Tier Architecture

- The decoupling of application logic from either presentation or data increases the flexibility of the application design.
- Multiple views or a GUI can be added without changing the existing application logic.
- Similarly, multiple applications can be created using the same data model.
- Changing the components of one tier should not impact the other two tiers. For example, any change to the data or GUI will not affect the application logic.



3-Tier Architecture

- *Advantages*
- Centralized business logic will offer more flexibility. Business logic is only required to be changed at one place there by eliminating the installation process of the application on client systems.
- Less network traffic, thereby improving the performance of the application.
- Application performance is no longer dependent on client computer due to the business logic isolation.
- No more maintenance nightmares.
- *Disadvantages*
- Any update to the business logic must be accepted by all the clients even if they are not ready for updates.

N-Tier Architecture

- In this type of architecture, the application logic is divided based on the functionality rather than physically like in 2-tier and 3-tier architectures.
- A typical *n-tier* architecture *contains the following elements:*
  - *User Interface:* This is something like a web browser that handles the client interactions.
  - *Presentation Logic:* This defines format using which the information will be displayed.
  - *Business Logic:* Encapsulates all the business rules by interacting with data sources.
  - *Infrastructure Services:* These are utility services that the presentation and business logic makes use of.
  - *Data tier:* This contains all the enterprise data

N-Tier Architecture

- Breaking the application logic based on functionality offers several benefits like flexibility, better maintenance, improved performance, reusability of code and may more.
- This architecture is also referred to as Model-View-Controller (MVC) architecture.

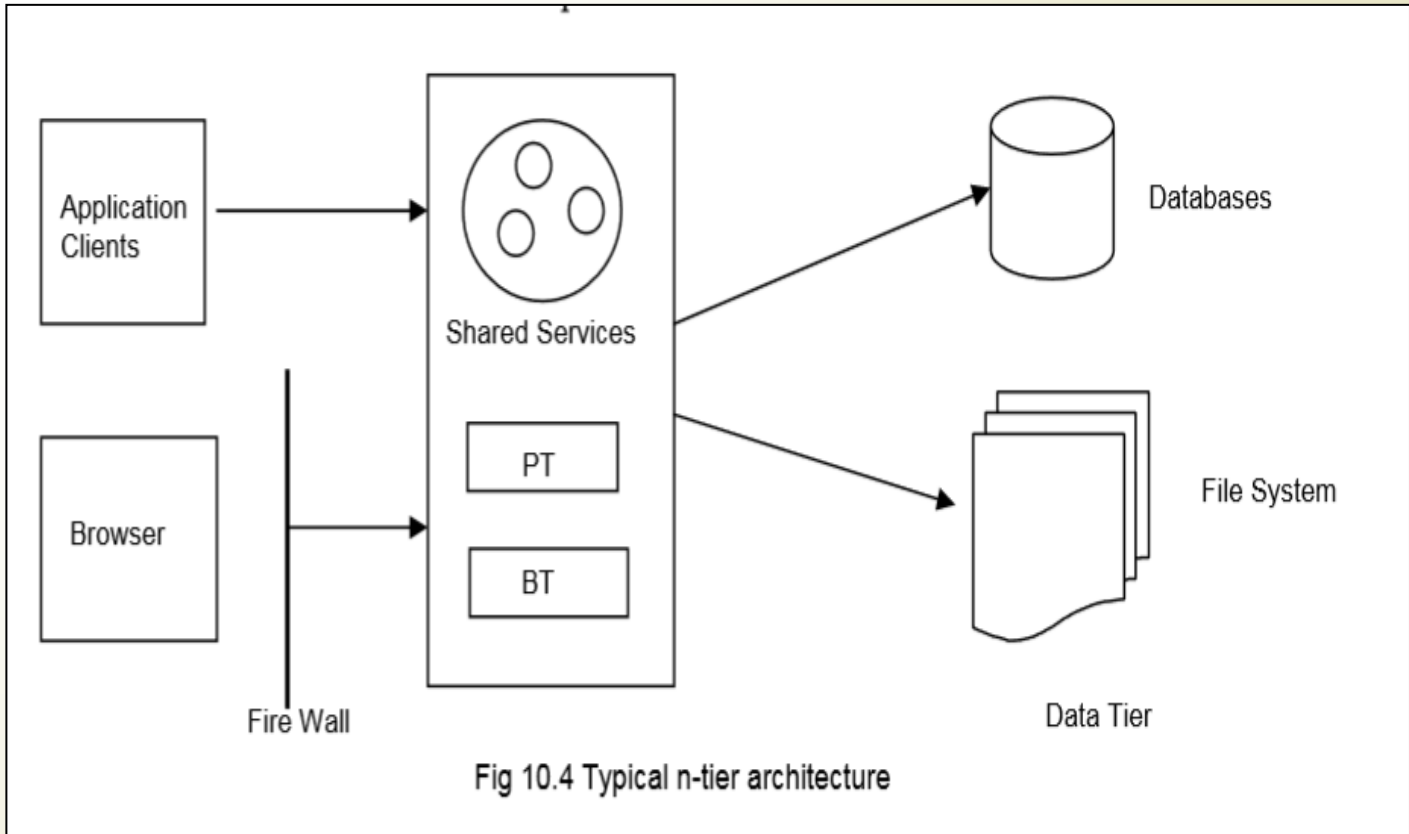


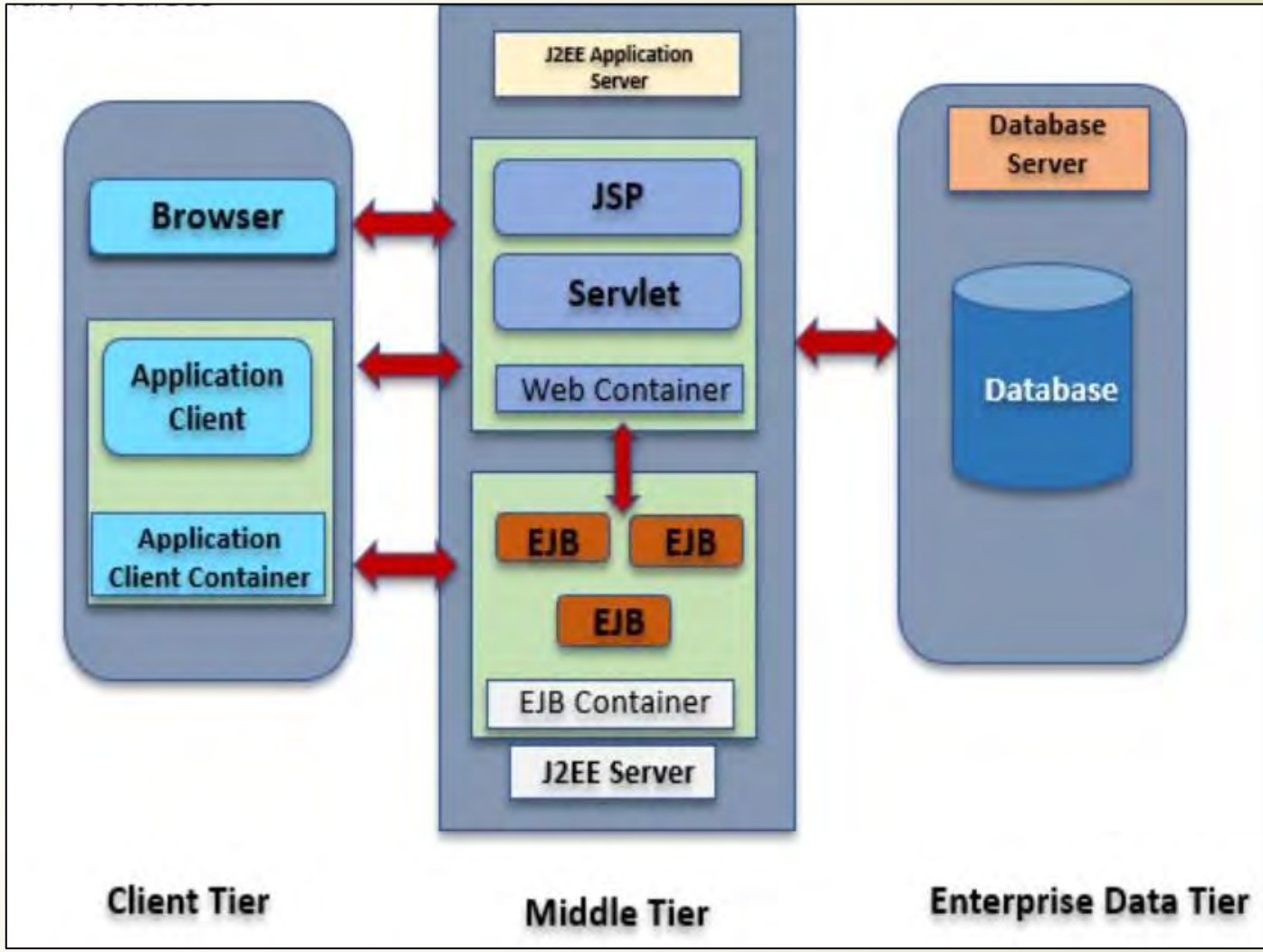
Fig 10.4 Typical n-tier architecture

Enterprise Architecture

- *Simply stating,* enterprise architecture is a *combination of several n-tier architectures.*
- An N-tier architecture is applicable *to a single application in an enterprise.*
- The client/server application architecture, which was a two-tier architecture, evolved over time to a multitier architecture. This natural progression *occurred as additional tiers* were introduced between *the end-user clients* and *back-end systems.*
- Although a multitier architecture *brings greater flexibility* of design. *It also increases the complexity of* building, testing, deploying, administering, and maintaining application components.

Enterprise Architecture

- However, *an enterprise application* is a collection of several applications within the enterprise with all the applications working in tandem and interacting with each other through well defined interfaces a shown in the following figure.



Enterprise Architecture

- An Enterprise JavaBeans (EJB) container manages the execution of all enterprise beans for one J2EE application. Enterprise beans and their container run on the J2EE server.
- EJB is specification for making server side components that enable and simplifies the task of creating distributed objects. EJB is a server-side software component that encapsulates business logic of an application
- *A web container manages* the execution *of all JSP page* and *servlet components* for one J2EE application. Web components and their container run on the J2EE server.
- An *application client container* manages the execution of all application client components for one J2EE application. Application clients and their container run on the client machine.

J2EE Application Servers

- An application server is *a software framework* that provides both facilities to *create web applications* and *a server environment* to run them
- A J2EE application server is *a readymade sophisticated application* that will host and run all the J2EE applications.
- *Once you have a J2EE application server*, you can start building and running enterprise applications.
- A J2EE application *server runs enterprise internet applications* that are built using standard J2EE technologies.
- There are several *free and commercial application servers* available in the market today that are developed by noted companies *like Sun Microsystems Inc, IBM, BEA and many more.*

J2EE Application Servers

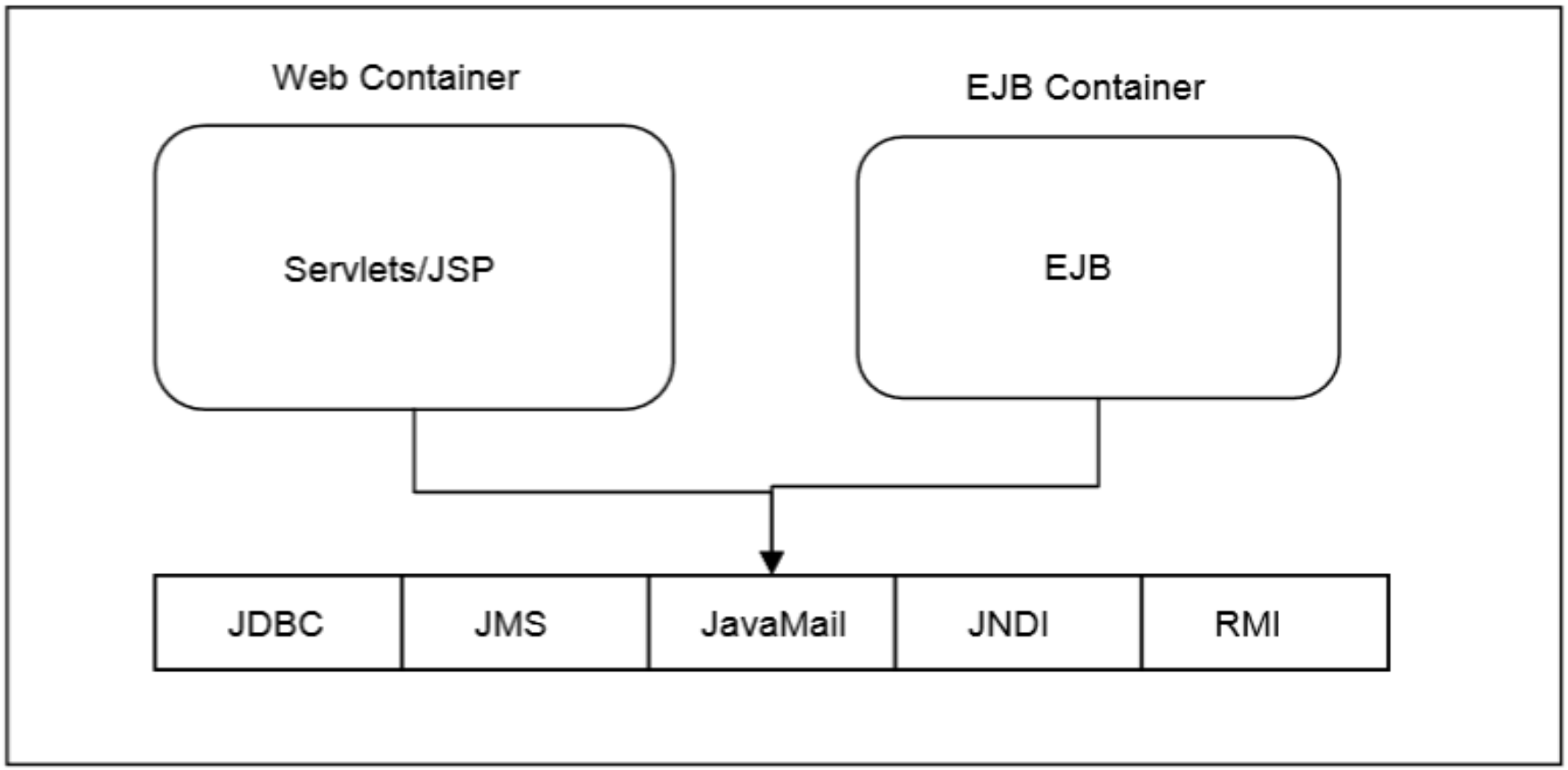


Fig 10.6 J2ee Application Server Components

J2EE Application Servers

- A J2EE application server comprises of *a Web Container* and an *EJB Container.* A *container is nothing but a runtime environment* to manage the application components.
- In J2EE, there are two mainstream technologies like Servlets/JSP/Struts and EJB.
  - *Application components* built using Servlet/JSP/Struts technologies run within Web container, and application components built using EJB technology are run *within EJB container.*
  - *You cannot run EJB components in Web container and Servlets/JSP components in an EJB container.*
  - All the remaining technologies (JDBC, JMS, JNDI(Java Naming and Directory Interface), Spring, Hibernate etc.) are like helper technologies that are used by the above two mainstream technologies in both the containers.

J2EE Application Servers

- An application server has *both Web container* and *EJB container.*
- If our application ***doesn't use EJB technology***, do we need an EJB container? You are right. ***We don't need an EJB container***.
- In such cases, having just a web container is good enough. (Example: Apache Tomcat, Jetty)
- Some examples of fully compliant J2EE application servers are: GlassFish Server , IBM WebSphere Application Server , Wildfly, JBoss Application Server, Apache TomEE etc.

J2EE Application Servers

Company	Server Name
SUN	SunONE Studio Application Server
IBM	IBM WebSphere Application Server
BEA	BEA WebLogic Application Server
JBoss	JBoss Application Server

"A J2EE application server runs enterprise internet applications that are built using standard J2EE technologies".

Thank You

???

Chapter 9
Servlet Programming

Bal Krishna Nyaupane
Assistant Professor
Department of Electronics and Computer Engineering
Institute of Engineering, Tribhuvan University
bkn@wrc.edu.np

Introduction to Web

- Several years ago, client-server applications have become very popular for building enterprise applications.
- In this model, client application is usually installed on the client's personal computer, which then sends requests to the server application via network.
- With the ***advent of internet***, application clients are completely replaced ***with web client.***
- In this model, both the ***application logic*** and ***presentation logic*** are physically ***separated*** from the ***clients*** PC and moved to the ***server side.***

Introduction to Web

- Since the ***application logic is no longer*** present ***on the client*** PC, web clients are referred to ***as thin clients.***
- Since the ***presentation logic*** is also ***moved to the server*** side, the layout of the ***user interface*** can also be ***controlled from the server side.***
- A ***web client*** in this case is nothing but ***a browser application like Internet Explorer, Firefox etc.***
- Applications that use web clients for user interaction are ***called as Web Applications.***

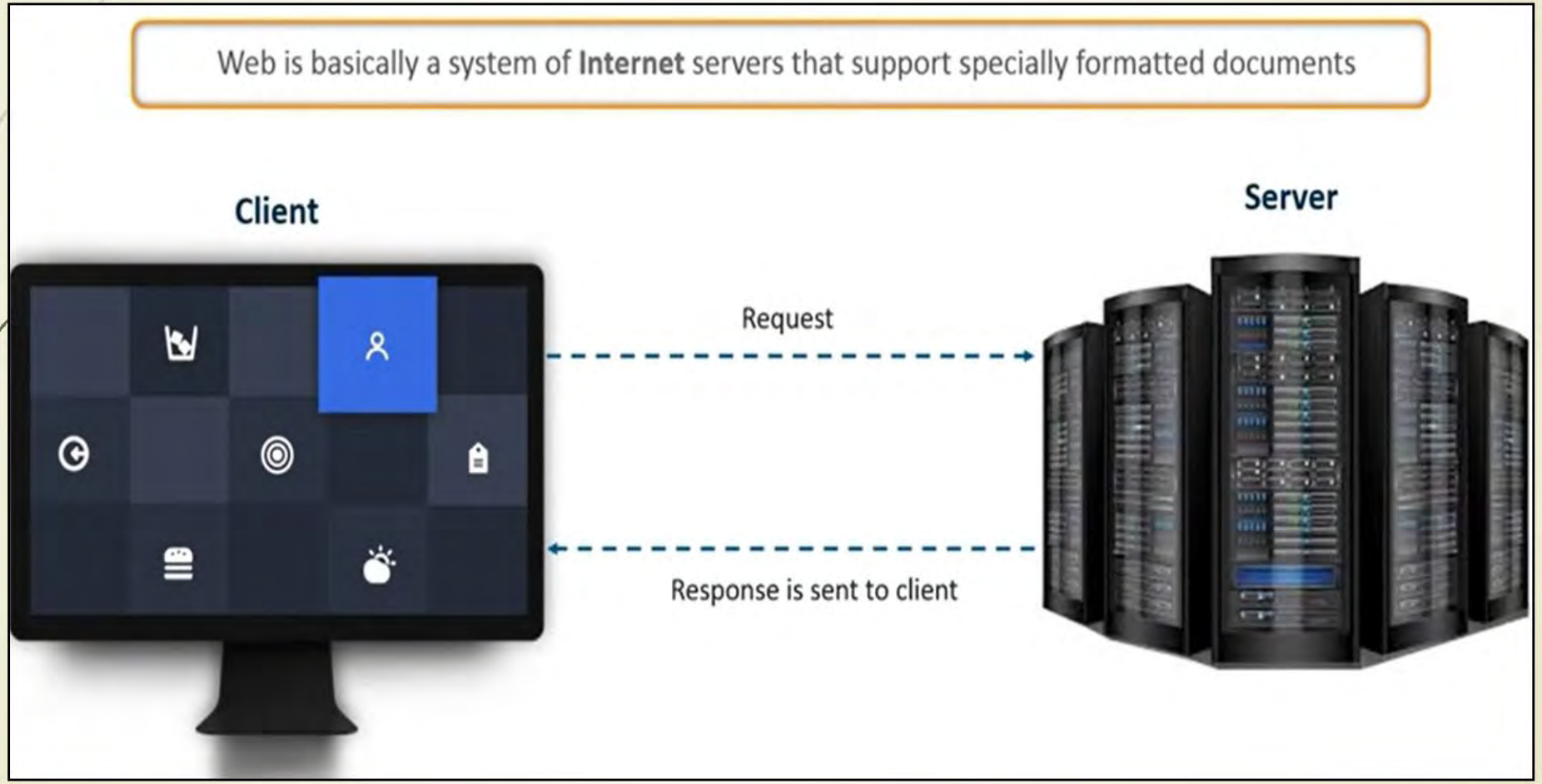
Introduction to Web

- ***Web clients*** to interact with ***server side applications***, use the following:
  1. A general purpose ***browser*** like Internet Explorer to ***send requests*** and ***display the response***.
  2. ***HTML markup*** for defining the ***user interfaces for interaction with the server.***
  3. ***HTTP protocol*** to send and receive requests and responses.

Introduction to Web

- The way the web applications work is,
  1. Web client like Internet Explorer sends a ***request using HTTP protocol.***
  2. The ***server side program*** takes the HTTP request, ***processes it*** and ***sends a HTTP response back to the web client.*** The response includes body content in HTML format.
  3. Web client then reads the HTTP response, formats the HTML and displays it to the user.

Introduction to Web

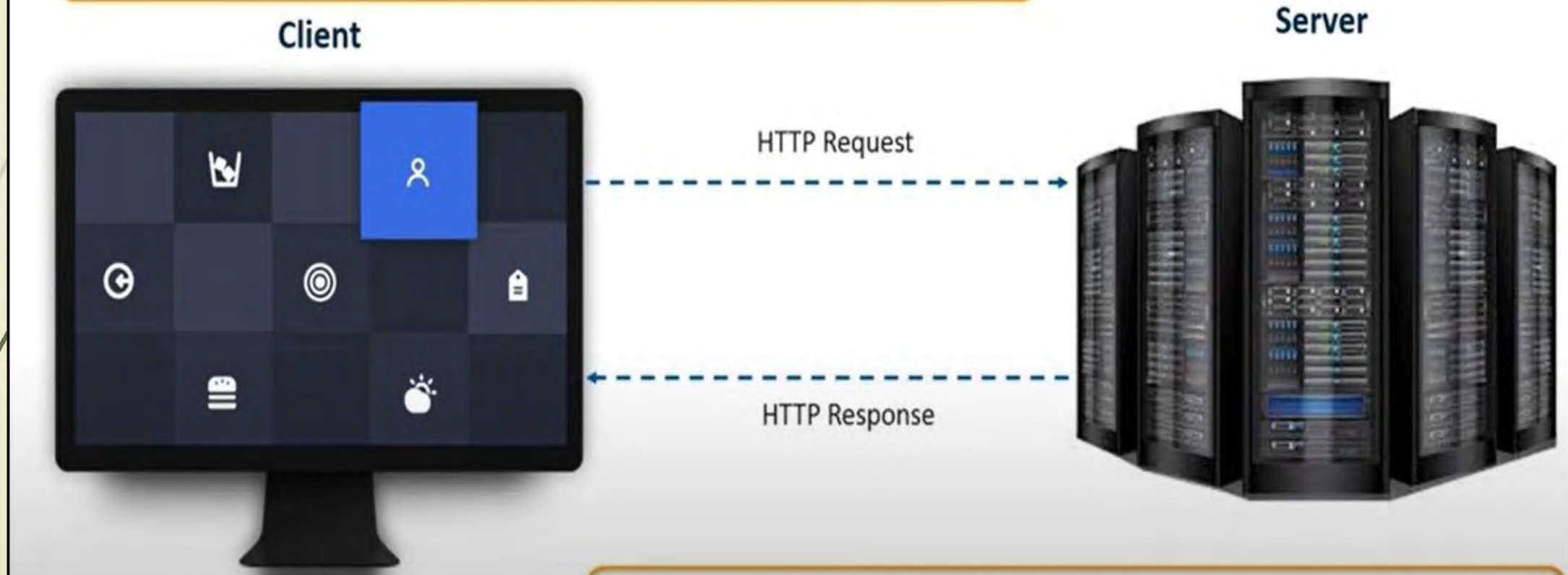


Web and HTTP

MySQL
CSS3
HTML5
PHP
ERP
XHTML
CRM
Web
Applications
WP
http://
HTTP is a protocol that clients and servers use on the web to communicate.

HTTP Request and Response

HTTP Request is a packet of Information that one computer sends to another computer to communicate something



HTTP Response is the packet of information sent by Server to the Client in **response** to an earlier Request made by Client

HTTP

- HTTP stands for ***Hyper Text Transfer Protocol.*** Following are some of the ***important properties of HTTP:***
  1. It is a ***stateless protocol***, meaning that every HTTP request is independent of each other.
  2. It can ***send data in the request***. The server side program reads the data, processes it and sends the response back. This is the most important feature of HTTP, the ability to send data to server side program.
- Based on how the data is sent in the request, ***HTTP requests are categorized into several types***, but the most important and ***widely used are the*** GET requests and POST requests.

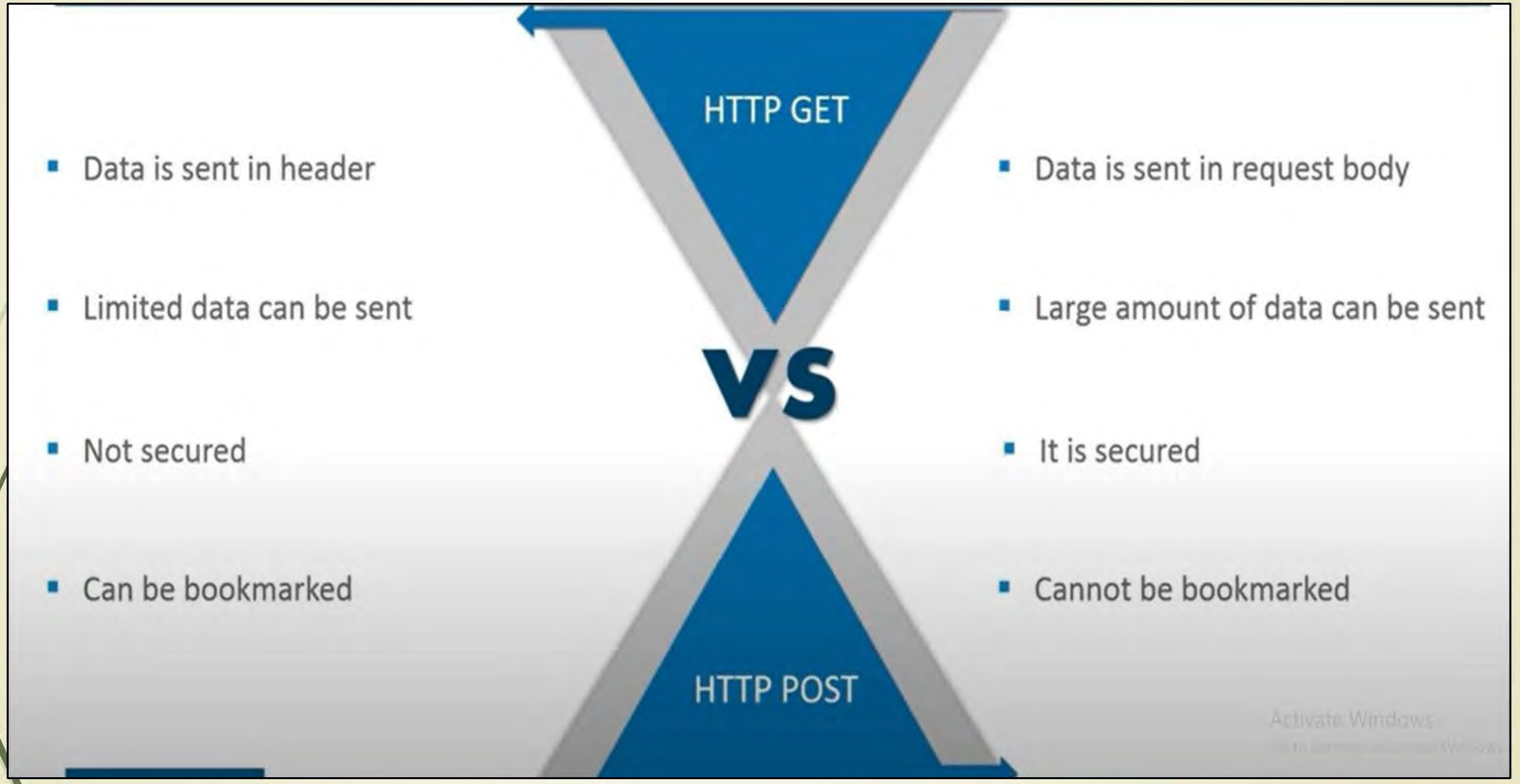
GET request

- GET request is the simplest of all the requests. This type of request is normally used for ***accessing server-side static resources*** such as HTML files, images etc.
- The GET request can send the data by appending it in the URI itself as shown below: ***http://somedomain.com?uid=John&role=admin***
- In the above URI, the HTTP request sends two pieces of data in the form of name value pairs. Each name-value pair must be separated by '&' symbol. The server side program will then read the data, process it, and send a dynamic response.
- Though GET request is simplest of all, it has some limitations:
  1. ***Data is visible to every one***. When secure information need to be sent, GET request is not the solution.
  2. GET requests can ***only send text data*** and ***not the binary data***. Therefore in situations where you need to upload image files to server, GET request cannot be used.

POST request

- ***POST request*** is normally used for ***accessing dynamic resources.***
- POST requests ***are used when*** we need to send ***large amounts of data*** to the server.
- Moreover, the data in the POST request ***is hidden behind the scenes*** therefore making data transmittal ***more secure***.
- In most of the real world applications, ***all the HTTP requests are sent as POST requests.***

Difference between HTTP Get and Post



Activate Windows

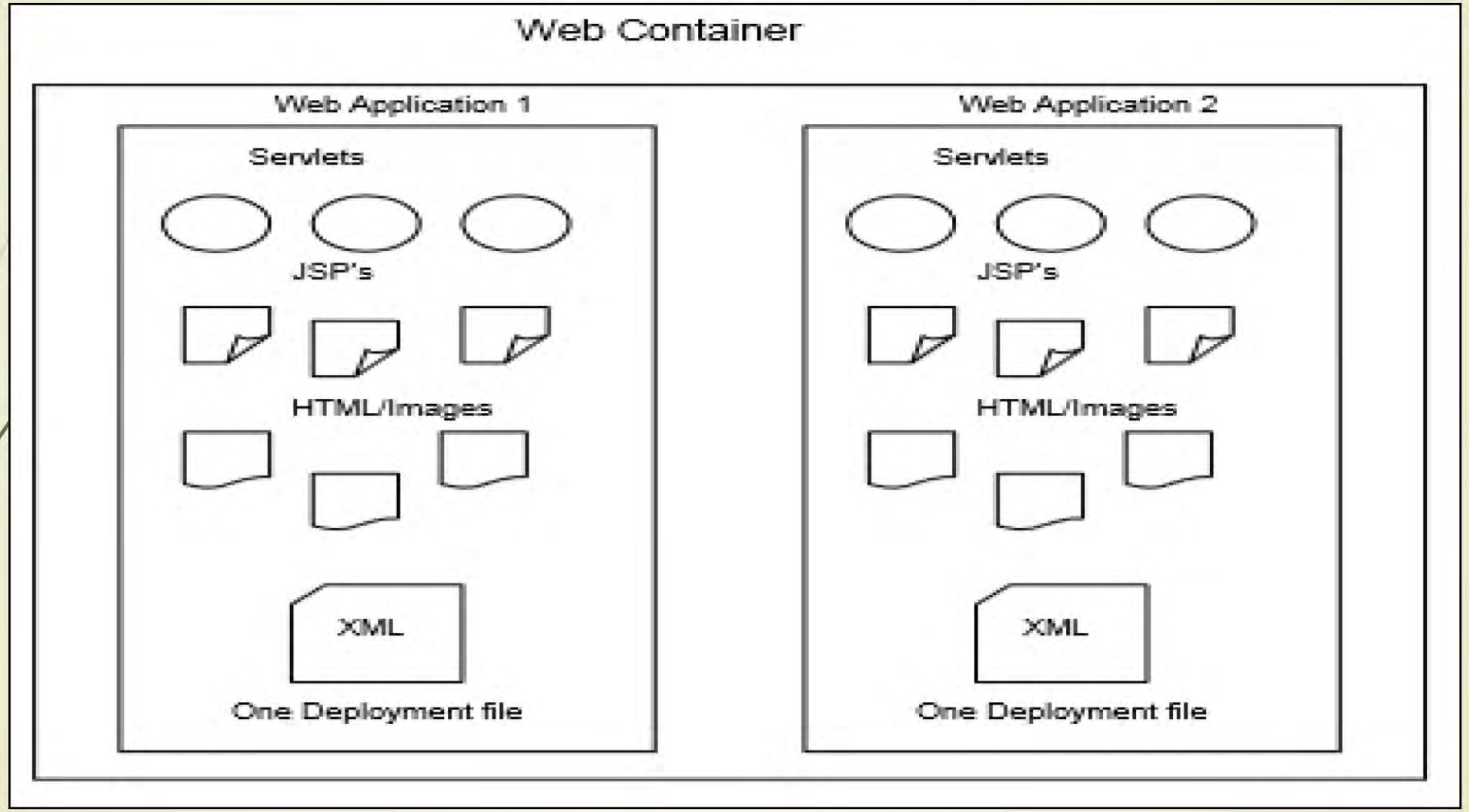
Server Side of the Web Application

- The ***server- side is the heart of any web application*** as it comprises of the following:
  - ***Static resources*** like HTML files, XML files, Images etc.
  - ***Server programs*** for processing the HTTP requests
  - A ***runtime that executes*** the server side programs
  - A ***deployment tool*** for deploying the programs in the server
- In order ***to meet the above requirements***, J2EE offers the following:
  - ***Servlet and JSP technology*** to build server side programs
  - ***Web Container*** for hosting the server side programs.
  - ***Deployment descriptor*** which is an XML file used ***to configure the web application.***

Web Container

- A ***Web container*** is the heart of ***Java based web application*** which is a sophisticated program that hosts the server side programs like Servlets.
- Once ***the Servlet programs are deployed in this container***, the container is ready ***to receive HTTP requests*** and delegate them to the programs ***that run inside the container.***
- These programs then ***process the requests and generate the responses.***
- Also a ***single web container*** can host ***several web applications.***
- ***J2EE supports*** two key ***technologies to build server side*** components in Java. These are ***Servlet*** and ***JSP technologies.***

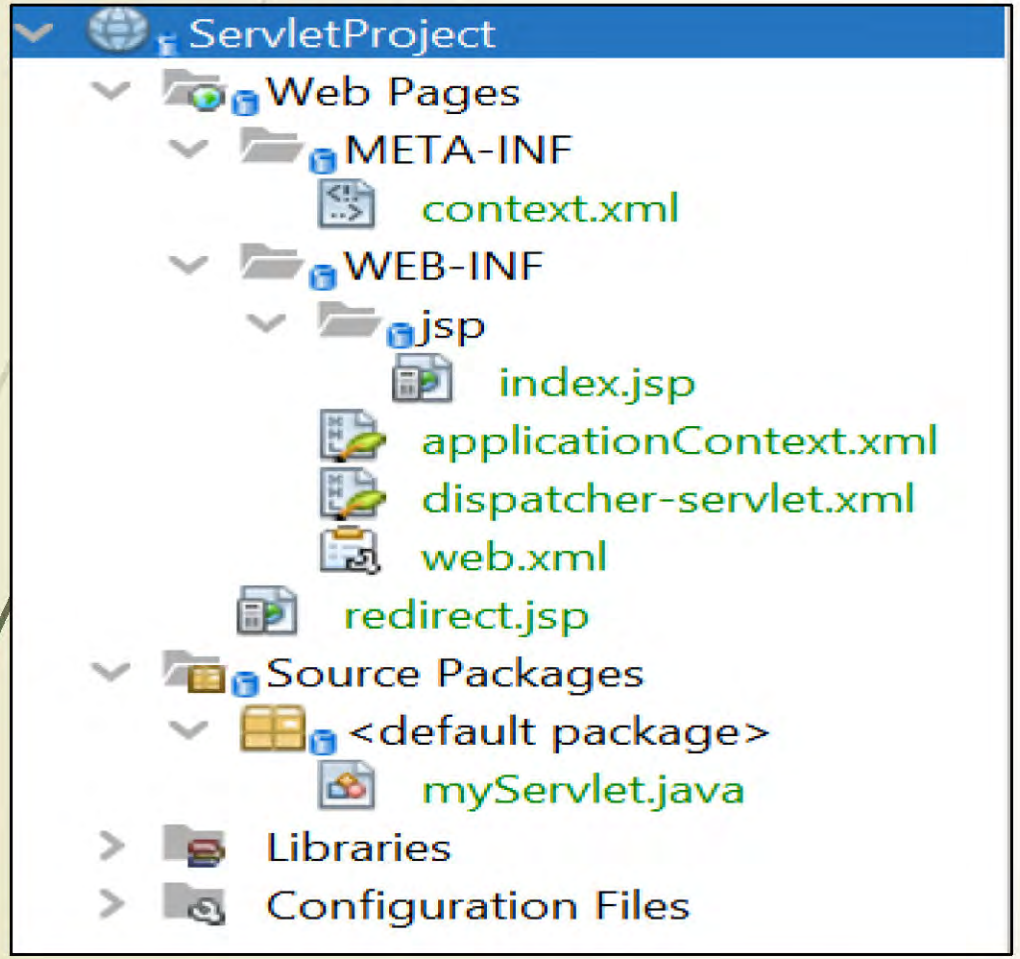
Web Container

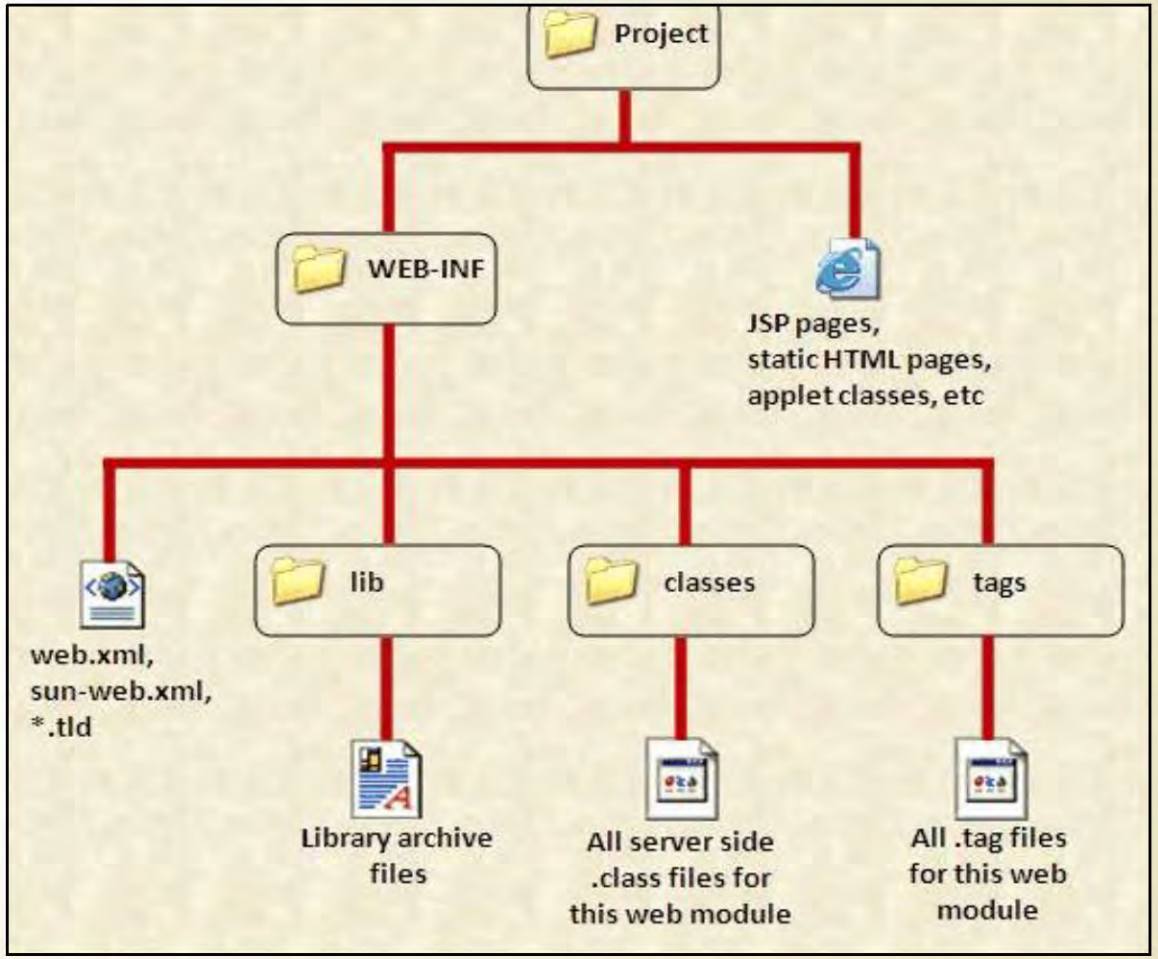


Structure of a Web Application

- For the web container to run the web application, the web application must ***follow a standard directory structure*** within the container.
- Web applications have a directory structure, which is fully accessible from a mapping to the application's document root ***(for example, /ServletProject***). The document root contains JSP files, HTML files, and static files such as image files.
- A ***special directory under the document root***, WEB-INF, contains everything related to the application that is not in the public document tree of the application.
- For the web container to run the web application, the web application must ***follow a standard directory structure*** within the container.

Structure of a Web Application





Structure of a Web Application

- ***WEB-INF/web.xml***
  - Is the web application deployment descriptor.
  - Required in all web application in all platforms
  - Standard way of configuration the web application
  - Contains for example the config info for your servlet
- ***WEB-INF/classes***
  - contains the java classes that are part of the web application
- ***WEB-INF/lib***
  - contains jar files and any other needed libraries for this Web application.

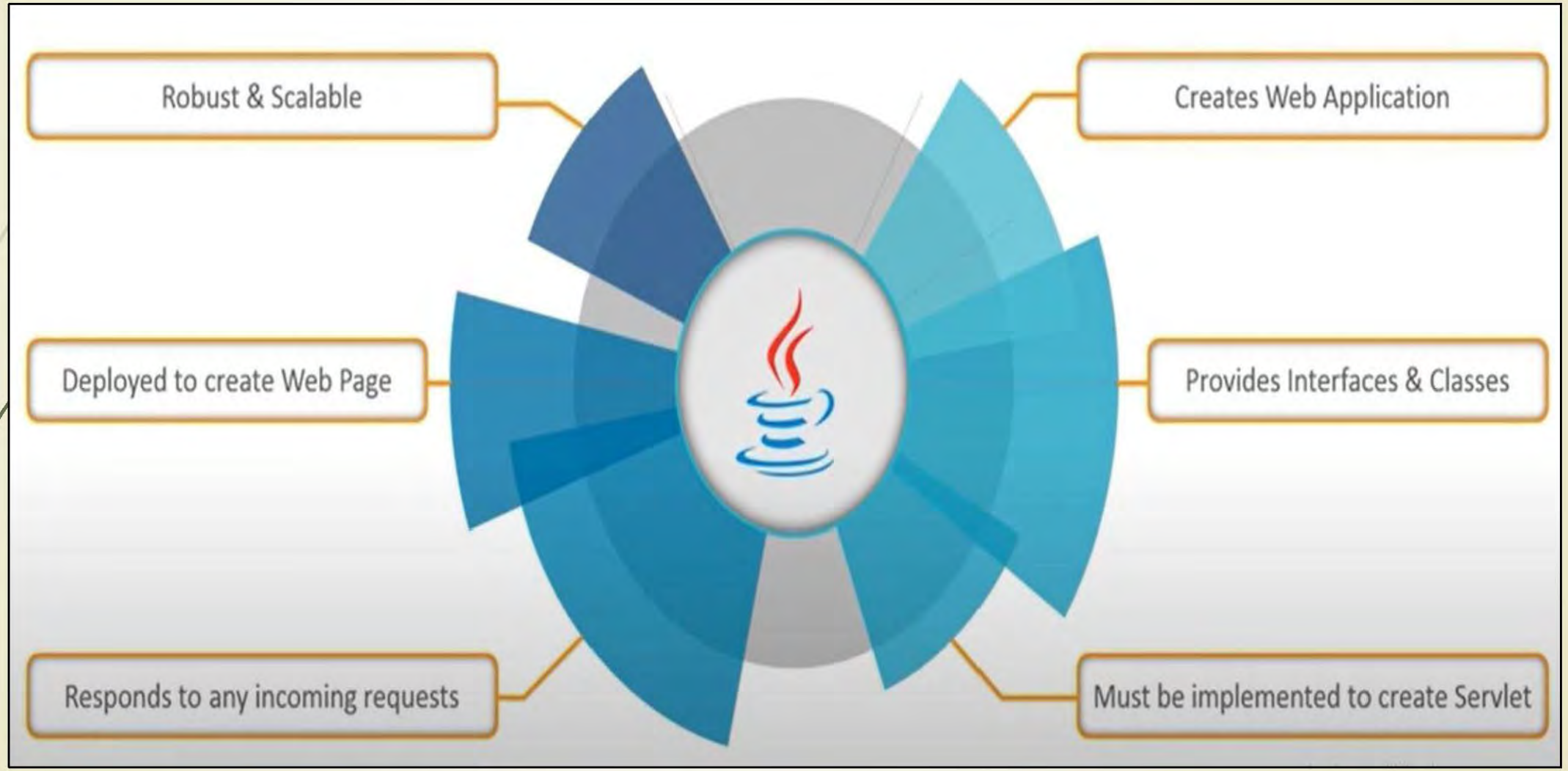
What is Servlet?

- ***Servlet Technology***
  - Servlet technology is a standard J2EE technology ***used for building dynamic web applications*** in Java. Using Servlet technology is again nothing but using the standard classes and interfaces that it comes with. These classes and interfaces form what we call as Servlet API.
- ***Definition of Servlet***
  - A server side Java program that uses the Servlet API is called as Servlet. In simple terms, ***a Servlet is a server side Java program that processes HTTP requests and generates HTTP response.***

What is Servlet?

- **Servlet API**
  - We need to use Servlet API to create servlets.
  - There are two packages that you must remember while using API, the ***javax.servlet*** package that contains the classes to ***support generic servlet*** (protocol-independent servlet) and the ***javax.servlet.http*** package that contains classes to support http servlet.
  - Every Servlet must implement the ***java.servlet.Servlet*** interface, you can do it by extending one of the following two classes: ***javax.servlet.GenericServlet*** or ***javax.servlet.http.HttpServlet***.
  - The first one is for protocol independent Servlet and the second one for http Servlet.

What are Servlets?



Servlets

Package javax.servlet

Some Interfaces	
Interface	**Description**
RequestDispatcher	This interface implements methods to forward a request or include output from another (active) source such as another servlet.
Servlet	This is the interface for all servlets.
ServletRequest	Whenever the server receives a request it creates a ServletRequest object, puts all the request information in it and passes this along with a ServletResponse object to the approriate servlet.

Servlets

Some Classes	
Class	**Description**
GenericServlet	Abstract base class for all servlets.
HttpConstraintElement	An HttpConstraint annotation value.
HttpMethodConstraintElement	An HttpMethodConstraint annotation value.
MultipartConfigElement	A MultipartConfig annotation value.
ServletContextAttributeEvent	an attribute within the ServletContext changed.
ServletContextEvent	something happened to the specified ServletContext.
ServletInputStream	This class serves as a stream where servlets can read data supplied by the client from.
ServletOutputStream	Used to write output from a Servlet to the client.
ServletRequestAttributeEvent	an attribute within the ServletRequest changed.
ServletRequestEvent	something happened to the specified ServletContext.

Servlets

ServletRequestWrapper	This class provides an adapter that makes it easy to wrap a request The default behavior of this class is to pass all method calls in the ServletRequest interface through to the underlying request object
ServletResponseWrapper	A convenience class for developers to adapt to a ServletResponse.

Package javax.servlet.http

Some Interfaces	
Interface	**Description**
HttpServletRequest	Contains all the client's request information.
HttpServletResponse	Object for HttpServlets to return information to the client.
HttpSession	A HttpSession holds session-dependant data on the server side.

Servlets

Some classes	
Class	**Description**
Cookie	A cookie is basically a {String,String} name/value pair that the server tells the client to remember and to send back to him attached to every future request. Using cookies a server can maintain a state in between client requests.
HttpServlet	The mother-of-all-HttpServlets.
HttpServletRequestWrapper	Serves a convenience class for developers to adapt to HttpServletRequests by default, passes all method calls which are part of the HttpServletRequest interface through to the wrapped object
HttpServletResponseWrapper	Provided as a convenience to a developer that wishes to write an adapter to an HttpServletResponse Object.
HttpSessionBindingEvent	Send to an Object that implements HttpSessionBindingListener when bound into a session or unbound from a session.
HttpSessionEvent	A session lifecycle event.

Creating Http Servlet

- If you creating Http Servlet you must extend ***javax.servlet.http.HttpServlet*** class, which is an abstract class.
- It overrides ***one or more of the following methods***. It must override at least one method from the list below:
  - ***doGet():*** This method is called by servlet service method to handle the HTTP GET request from client. The Get method is used for getting information from the server
  - ***doPost():*** Used for posting information to the Server
  - ***doPut():*** This method is similar to ***doPost*** method but unlike ***doPost*** method where we send information to the server, this method ***sends file to the server***, this is similar to the FTP operation from client to server.

Creating Http Servlet

- ***doDelete():*** allows a client to delete a document, webpage or information from the server
- ***init() and destroy():*** Used for managing resources that are held for the life of the servlet
- ***getServletInfo():*** Returns information about the servlet, such as author, version, and copyright.

Deployment Descriptor

- ***What is Deployment?***
  - ***Copying the .class file*** of the Servlet from the current directory ***to the classes folder of Tomcat*** (or any Web server) is known as deployment. When deployed, Tomcat is ready to load and execute the Servlet, at any time, at the client request.
- ***What is Deployment Descriptor?***
  - As the name indicates, the deployment descriptor ***describes the deployment information*** (or Web Information) of a Servlet.
  - The deployment descriptor is an ***XML file known as web.xml.***
  - XML is the easiest way to give the information to a server, just writing in between the tags, instead of writing in a text file or RDBMS file.

Deployment Descriptor

- A deployment descriptor is a standard XML file named web.xml that is used by the web container to run the web applications. Every web application will have one and only one deployment descriptor (web.xml file).
- This file defines the following important information pertaining to a Servlet:
  1. ***Servlet name and Servlet class***
  2. ***The URL mapping used to access the Servlet***

Deployment Descriptor

```
<servlet>
    <servlet-name>myServlet</servlet-name>
    <servlet-class>myServlet</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>myServlet</servlet-name>
    <url-pattern>/myServlet</url-pattern>
</servlet-mapping>
```

Deployment Descriptor

- Let's assume we wrote a servlet named ***LoginServlet*** in a package named ***myservlets.***
- The definition for this servlet in the web.xml will be as shown below:

```
<servlet>
        <servlet-name>LoginServlet </servlet-name>
        <servlet-class> LoginServlet </servlet-class>
</servlet>
```

- The servlet name can be any arbitrary name, but it's ***a good practice*** to have the ***class name*** as the ***servlet name***.

Deployment Descriptor

- The next thing we need to define is the URL mapping which identifies how the servlet is accessed from the browser. For the above servlet, the url mapping will be as shown below:

```
<servlet-mapping>
    <servlet-name> LoginServlet </servlet-name>
    <url-pattern> /LoginServlet </url-pattern>
</servlet-mapping>
```

- If you noticed carefully, the servlet name in both the XML snippets is the same. This is how we bind the ***url mapping*** with a servlet.
- The ***url pattern defines how the servlet will be accessed.***

Deployment Descriptor

- ***How web.xml works ?***
  - When a request comes ***it is matched with url pattern*** in servlet mapping attribute.
  - When ***url matched with url pattern*** web server try to find the ***servlet name in servlet attributes*** same as in servlet mapping attribute.
  - When match found control is ***goes to the associated servlet class.***

Steps for Writing a Servlet

- Once Tomcat is installed and configured, you can put it to work.
- Six steps take you from writing your servlet to running it. ***These steps are as follows:***

1. Create ***a directory structure under*** Tomcat for your application.
2. Write the ***servlet source code***. You need to import the javax.servlet package and the javax.servlet.http package in your source file.
3. ***Compile your source code.***
4. Create ***a deployment descriptor.***
5. ***Run Tomcat.***
6. Call your ***servlet from a web browser.***

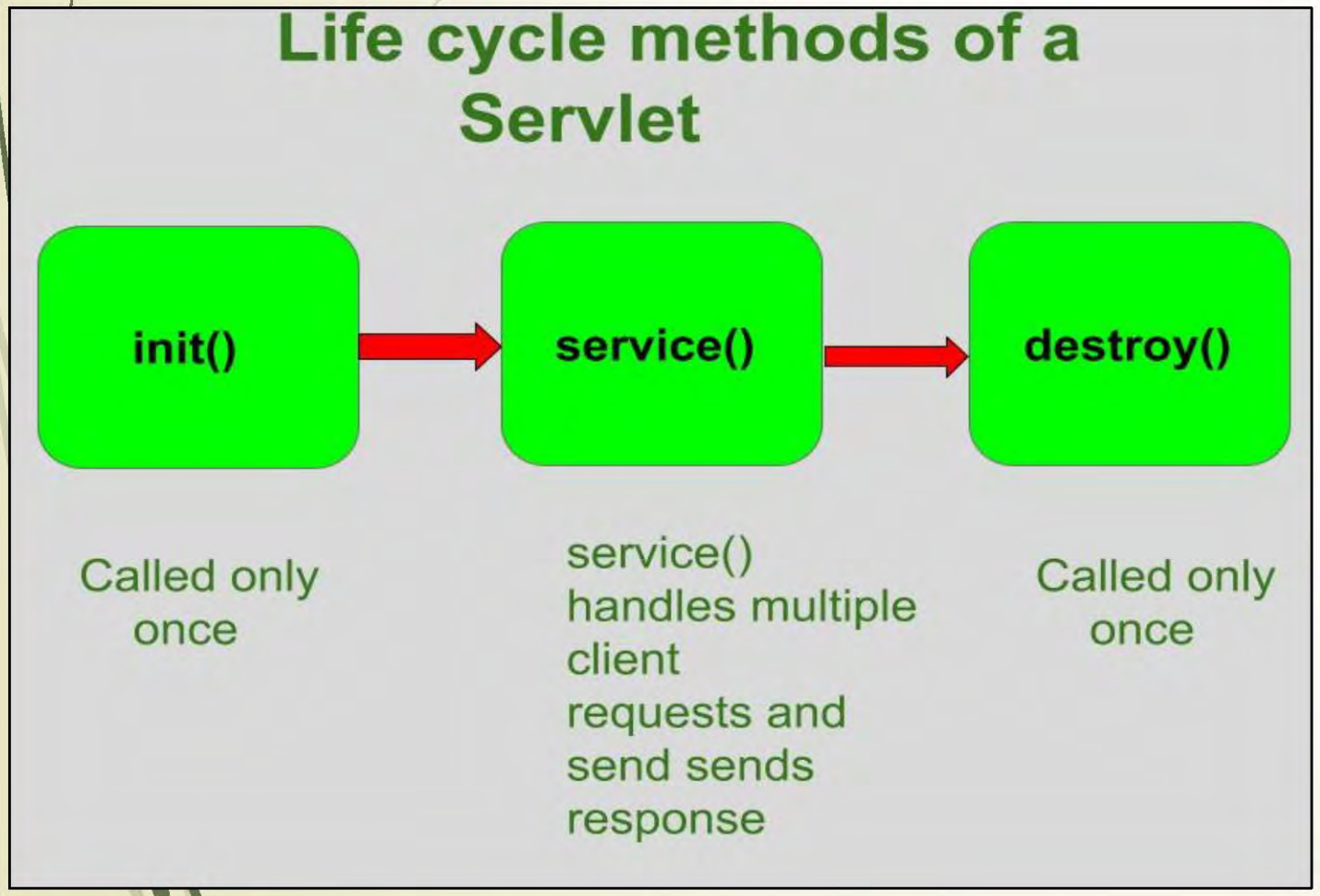
Steps for Writing a Servlet

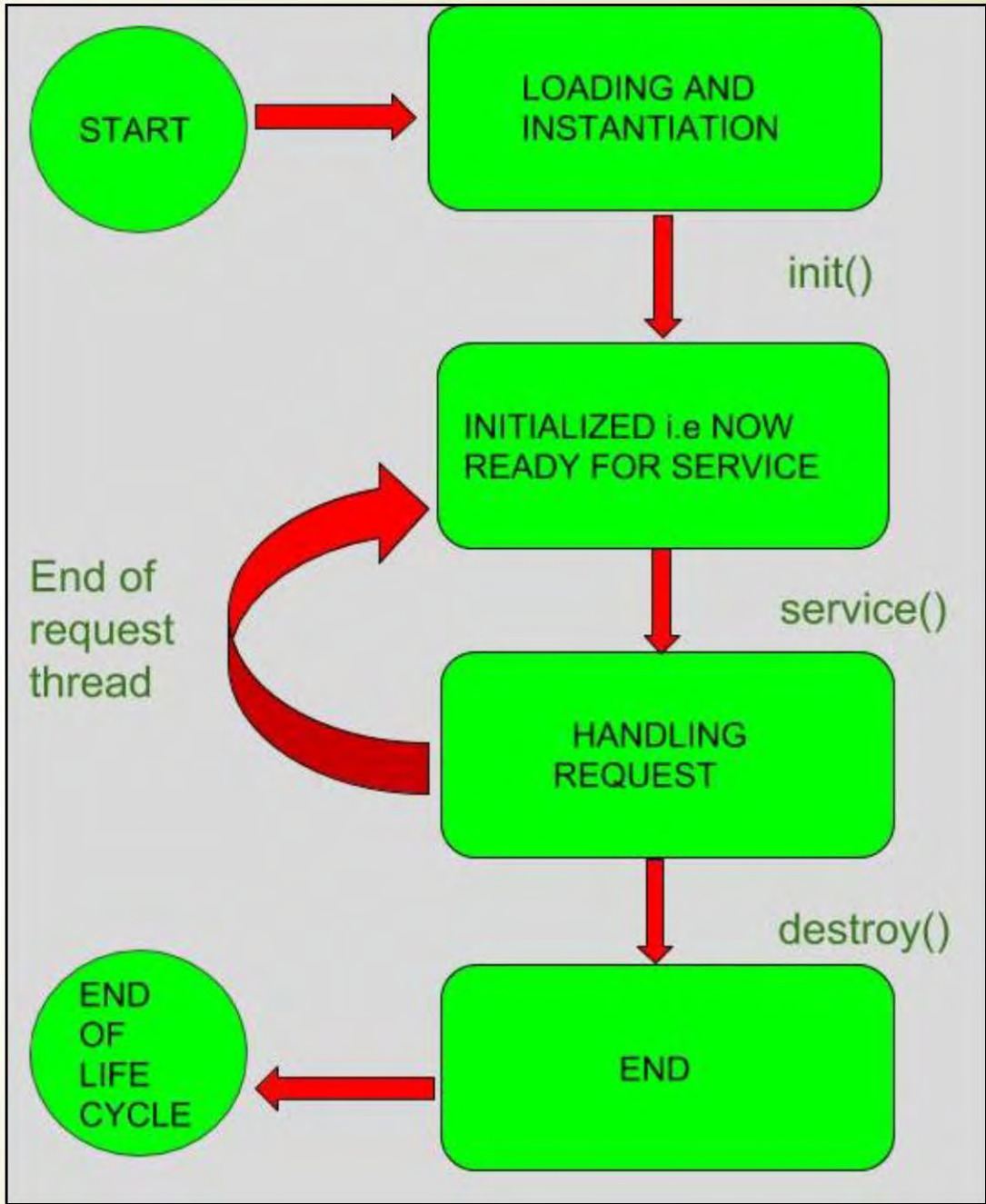
- ***The servlet example can be created by three ways:***
  1. *By implementing Servlet interface,*
  2. *By inheriting GenericServlet class, (or)*
  3. *By inheriting HttpServlet class*
- The mostly used approach is by ***extending HttpServlet*** because it provides ***http request specific*** method such as ***doGet(), doPost(), doHead() etc.***
- So, writing a servlet is very simple. You just have to follow a standard process as shown below:
  1. ***Create a class that extends HttpServlet***
  2. ***Define methods like init(), doGet() ,doPost() tec.***
- These three methods are the standard methods for any servlet. These are called as callback methods that the web container invokes automatically when a HTTP request comes to this servlet. For all the GET requests, the web container invokes the doGet() method, and for POST requests it invokes the doPost() method.

Life Cycle of a Servlet

- The entire life cycle of a Servlet is managed by the ***Servlet container*** which uses the ***javax.servlet.Servlet*** interface to understand the Servlet object and manage it.
- Servlet life cycle steps:
  1. ***Loading a Servlet***
  2. ***Create Servlet instance.***
  3. Initializes the servlet by executing ***the init() method.***
  4. ***Call service() method:*** Invokes service() / doGet()/ doPost /... method repeatedly for each client request.
  5. ***Call destroy() method:*** Destroys the servlet by invoking destroy() method.

Life Cycle of a Servlet





Life Cycle of a Servlet

- ***Loading a Servlet:*** Web container loads the servlet when the first request is received. This step is ***executed only once*** at the time of first request.
- ***Create Servlet instance:*** After loading the servlet class web container creates the servlet instance. ***Only one instance is created for a servlet*** and all concurrent requests are executed on the same servlet instance.
- ***Call init() method:*** After creating the servlet instance web container calls the servlet's init method.
  - This method is used ***to initialize the servlet*** before processing first request.
  - It is called ***only once by the web container.***
  - This method receives **only one parameter, *i.e. ServletConfig object.***

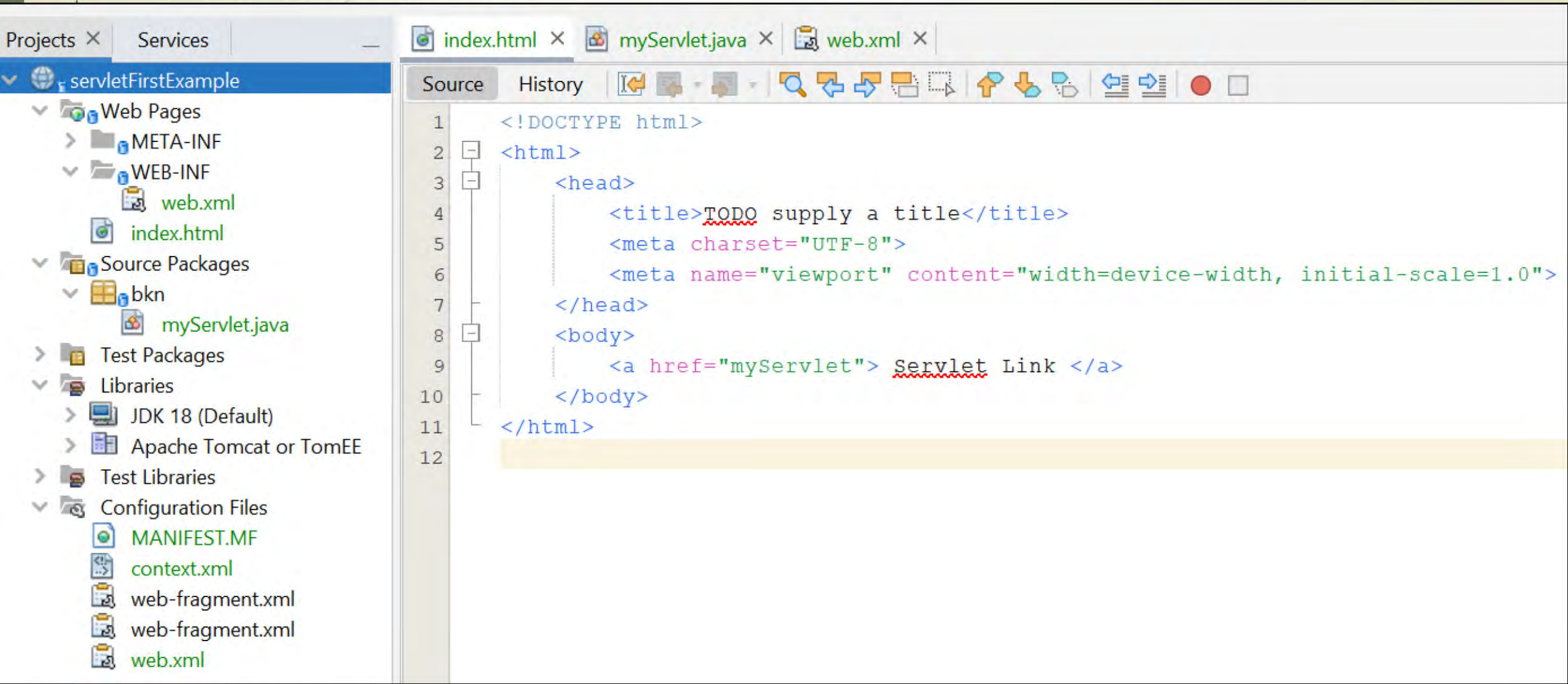
Life Cycle of a Servlet

- ***Call service() method:*** After initialization process web container calls service method. ***Service method is called for every request***. For every request servlet creates a separate thread.
  - The service() method is the most important method to perform that provides the connection between client and server.
  - The web server calls the service() method to handle requests coming from the client(web browsers) and to send response back to the client.
  - This method determines the type of Http request (GET, POST, PUT, DELETE, etc.) .
  - This method also calls various other methods such as doGet(), doPost(), doPut(), doDelete(), etc. as required.
  - This method accepts two parameters.

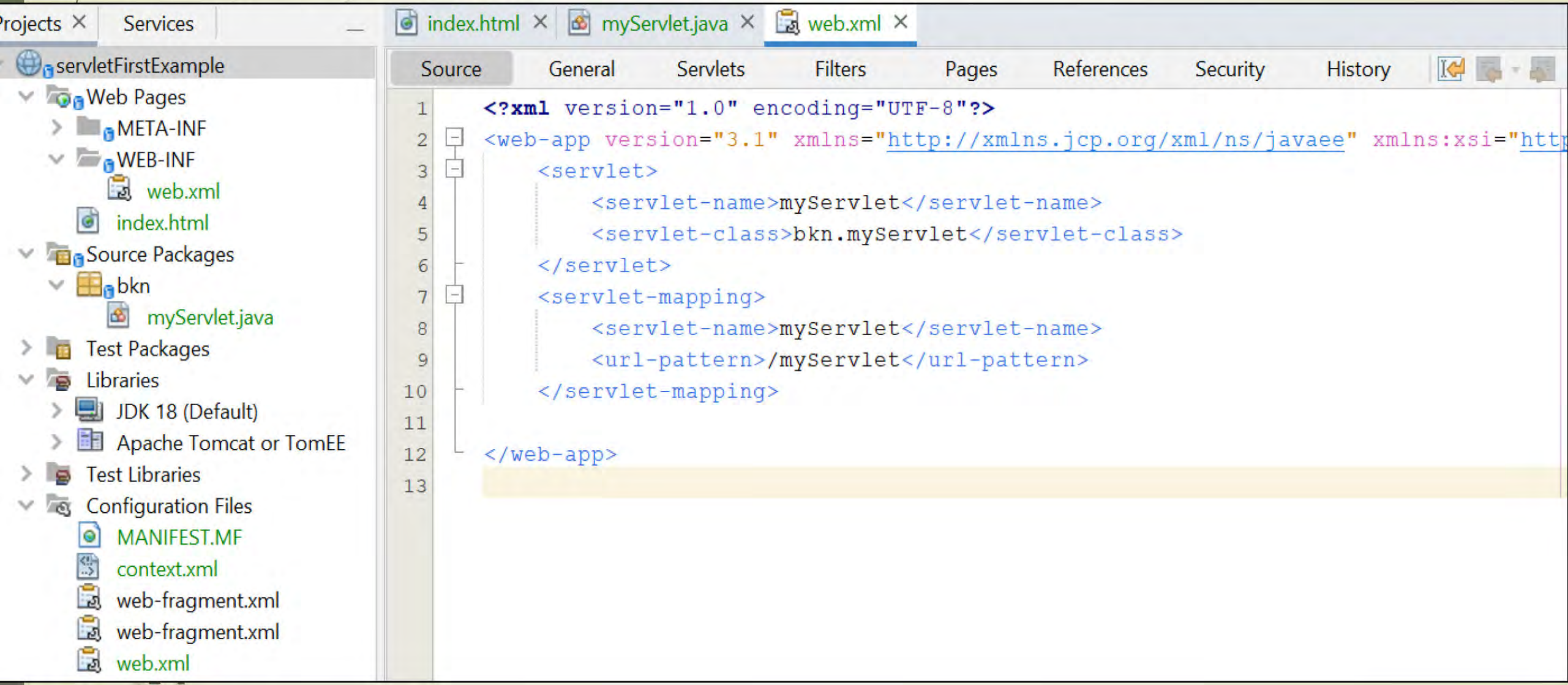
Life Cycle of a Servlet

- **Call destroy() method**
  - The destroy() method is ***called only once.***
  - It is called at the end of the life cycle of the servlet.
  - This method performs various tasks such as closing connection with the database, releasing memory allocated to the servlet, releasing resources that are allocated to the servlet and other cleanup activities.
  - When this method is called, the garbage collector comes into action.
  - The prototype for this method is:
  - public void destroy() { *// Finalization code...}*

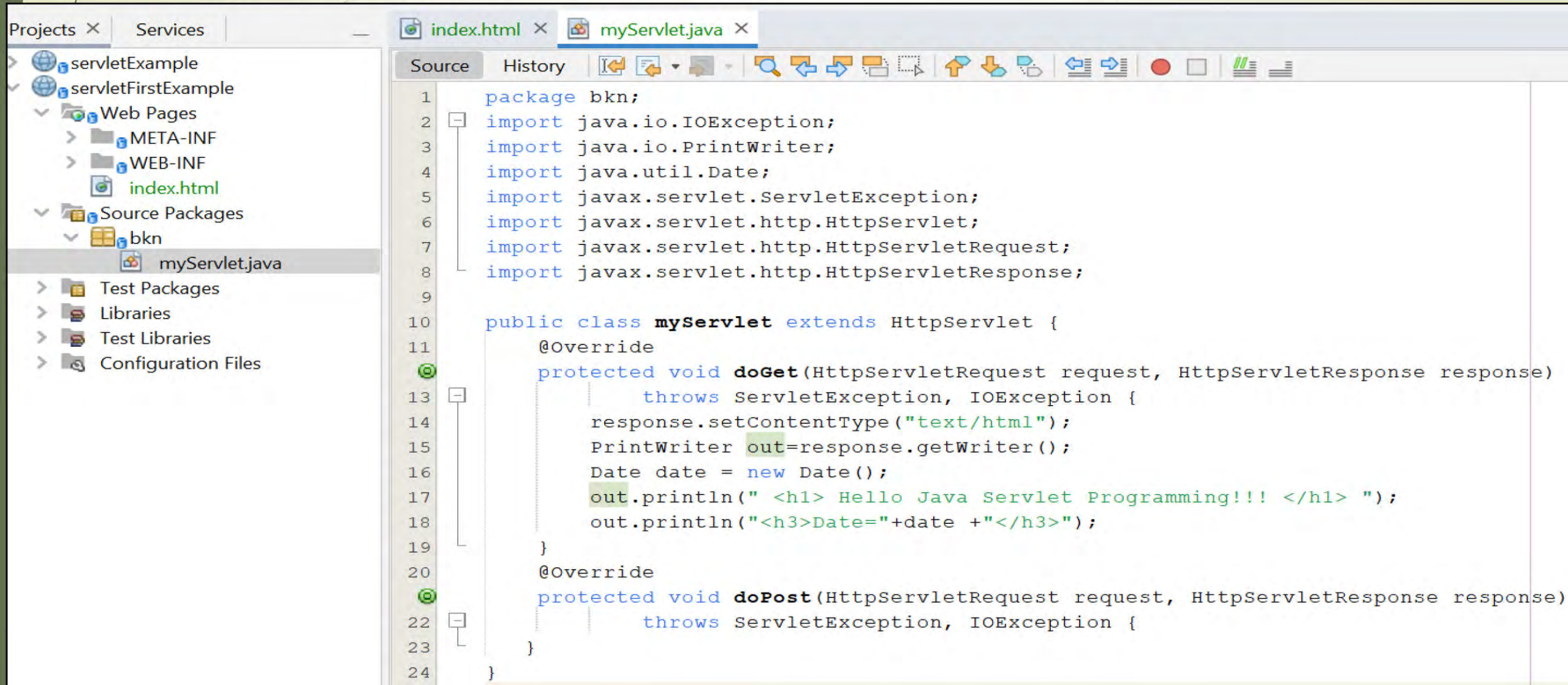
Simple Servlet to display "***Hello Java Servlet Programming !!!!***" in browser.



Simple Servlet to display "***Hello Java Servlet Programming !!!!***" in browser.



Simple Servlet to display "***Hello Java Servlet Programming !!!!***" in browser.



Servlet Initialization

- Initializing a servlet is one of the most common practices. Servlet ***initialization is done in the init() method*** of the servlet.
- Defining ***initialization parameters*** for servlets in ***web.xml*** is a good practice as it eliminates hard coding in the servlet.
- The web container while loading the servlet, ***invokes the init() method of the servlet*** as part of the life cycle and passes the parameters defined in the web.xml to it. The other ***advantage with this is, in the future if the initialization values need to be changed***, you only have to change the web.xml without having to recompile the entire web application.
- Initialization parameters are stored ***as key value pairs.*** They are included in web.xml file inside init-param tags. The **key is specified using the param-name tags** and **value is specified** using ***the param-value tags.***

Servlet Initialization

- Initialization parameters to a servlet are defined in the web.xml ***using the <init-param>*** element as shown below:

```
<servlet>
        <servlet-name>Name of servlet</servlet-name>
        <servlet-class>Servlet class</servlet-class>
        <init-param>
            <param-name> Mail </param-name>
            <param-value> admin@abc.com </param-value>
        </init-param>
</servlet>
```

Servlet Initialization

- A servlet can access the parameter in the init() method as shown below:

```
public void init(ServletConfig config) {
    String mail = config.getInitParameter("Mail");
}
```

- As you can see from the above code, the ServletConfig class defines a method named getInitParameter() that takes the name of the initialization parameter defined in the web.xml for that servlet, and returns the value. Therefore Mail will be initialized with admin@abc.com.

Servlet Initialization

index.html | servletInitialization.java | web.xml

Source | General | Servlets | Filters | Pages | References | Security | History

```
1   <?xml version="1.0" encoding="UTF-8"?>
2   <web-app version="3.1" xmlns="http://xmlns.jcp.org/xml/ns/javaee"
3           xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4           xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee http://xmlns.jcp.
5       <servlet>
6
7           <servlet-name>servletInitialization</servlet-name>
8           <servlet-class>bkn.servletInitialization</servlet-class>
9
10          <init-param>
11              <param-name> userName </param-name>
12              <param-value> Ram Prasad Sharma </param-value>
13          </init-param>
14
15          <init-param>
16              <param-name> Address </param-name>
17              <param-value> Pokhara - 16, Lamachaur </param-value>
18          </init-param>
19
20      </servlet>
21
22      <servlet-mapping>
23          <servlet-name>servletInitialization</servlet-name>
24          <url-pattern>/servletInitialization</url-pattern>
25      </servlet-mapping>
26  </web-app>
```

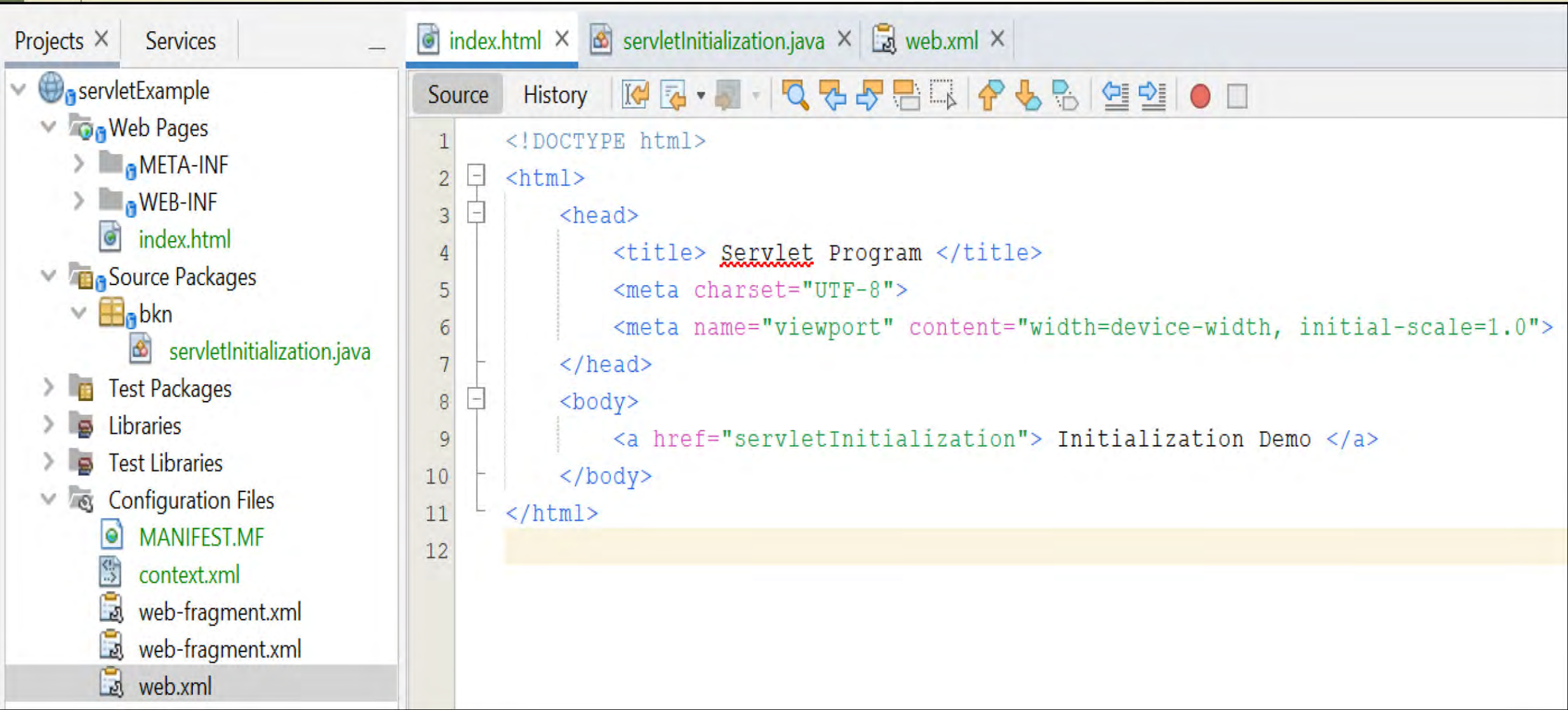
Servlet Initialization

index.html | servletInitialization.java | web.xml

Source | History

```
4   import javax.servlet.ServletConfig;
5   import javax.servlet.ServletException;
6   import javax.servlet.http.HttpServlet;
7   import javax.servlet.http.HttpServletRequest;
8   import javax.servlet.http.HttpServletResponse;
9   public class servletInitialization extends HttpServlet {
10      String name;  String address;
11      @Override
        public void init(ServletConfig config) throws ServletException {
13          name = config.getInitParameter("userName");
14          address = config.getInitParameter("Address");  }
15
16      @Override
        protected void doGet(HttpServletRequest request, HttpServletResponse response)
18              throws ServletException, IOException {
19          response.setContentType("text/html");
20          PrintWriter out = response.getWriter();
21          //Send the response
22          out.print("<h1>User Name is : </h1> " + "<h3>" + name + "</h3>");
23          out.print("<h1>User Address is : </h1>" + "<h3>" + address+ "</h3>");
24      }
25      @Override
        protected void doPost(HttpServletRequest request, HttpServletResponse response)
27              throws ServletException, IOException { }
28  }
```

Servlet Initialization



Servlet Initialization

Reading HTML Form Data

- Following are the ***steps for processing a HTML form using a Servlet:***
  1. *User fills the data in the html page and submits it.*
  2. *The form data will then be sent to Servlet.*
  3. *Servlet reads the form data, processes it and send a confirmation.*
- The form data will be sent in the HTTP request object as name value pairs as shown below:
  - ***http://localhost:8080/myweb/SomeServlet?name=John&age=20***
- The servlet then reads the above request data using the HttpServletRequest object as shown below:
  - *String fn = request.getParameter("name");*
  - *String age = request.getParameter("age");*

Reading HTML Form Data

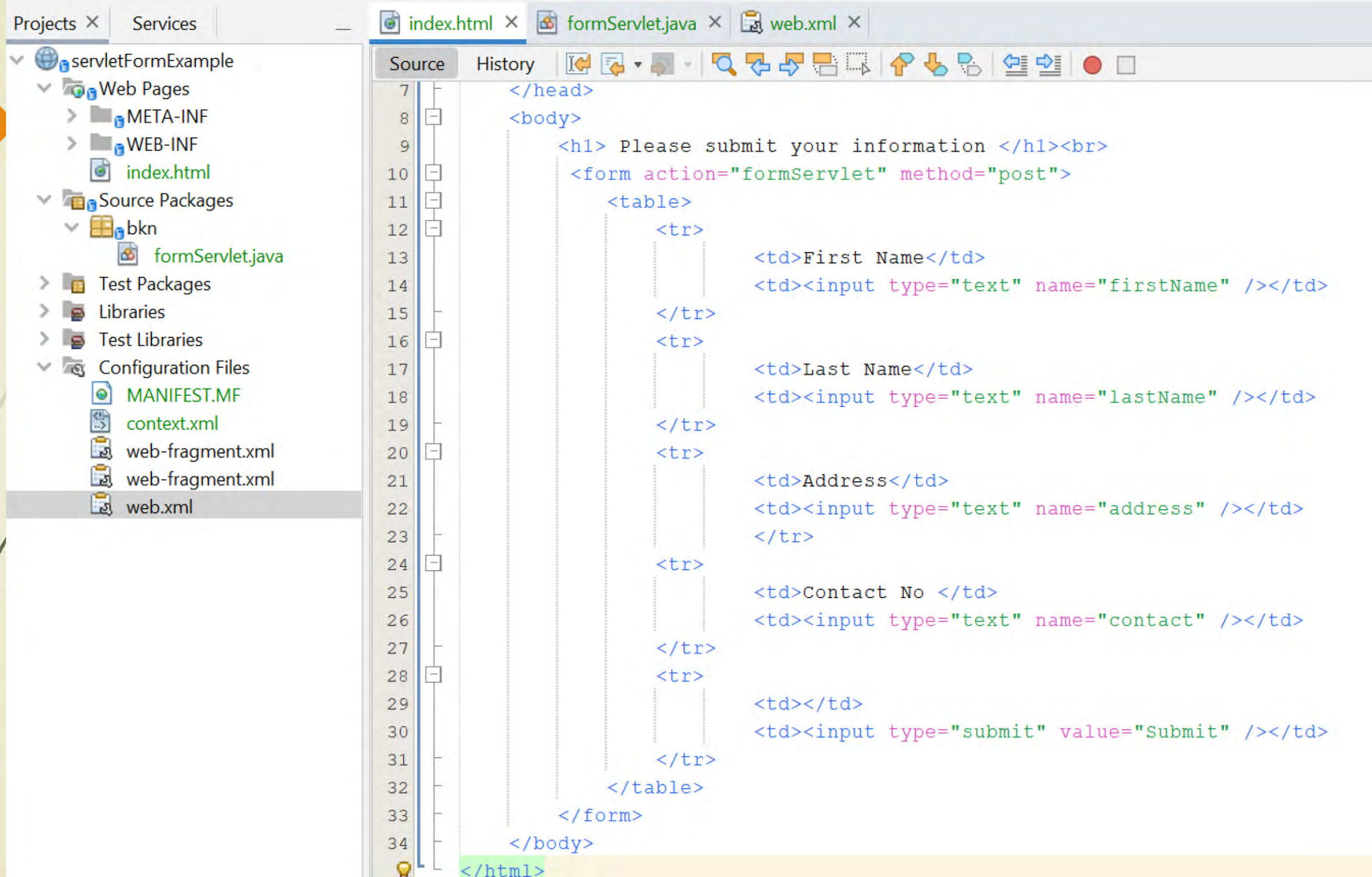
Reading Form Data using Servlet

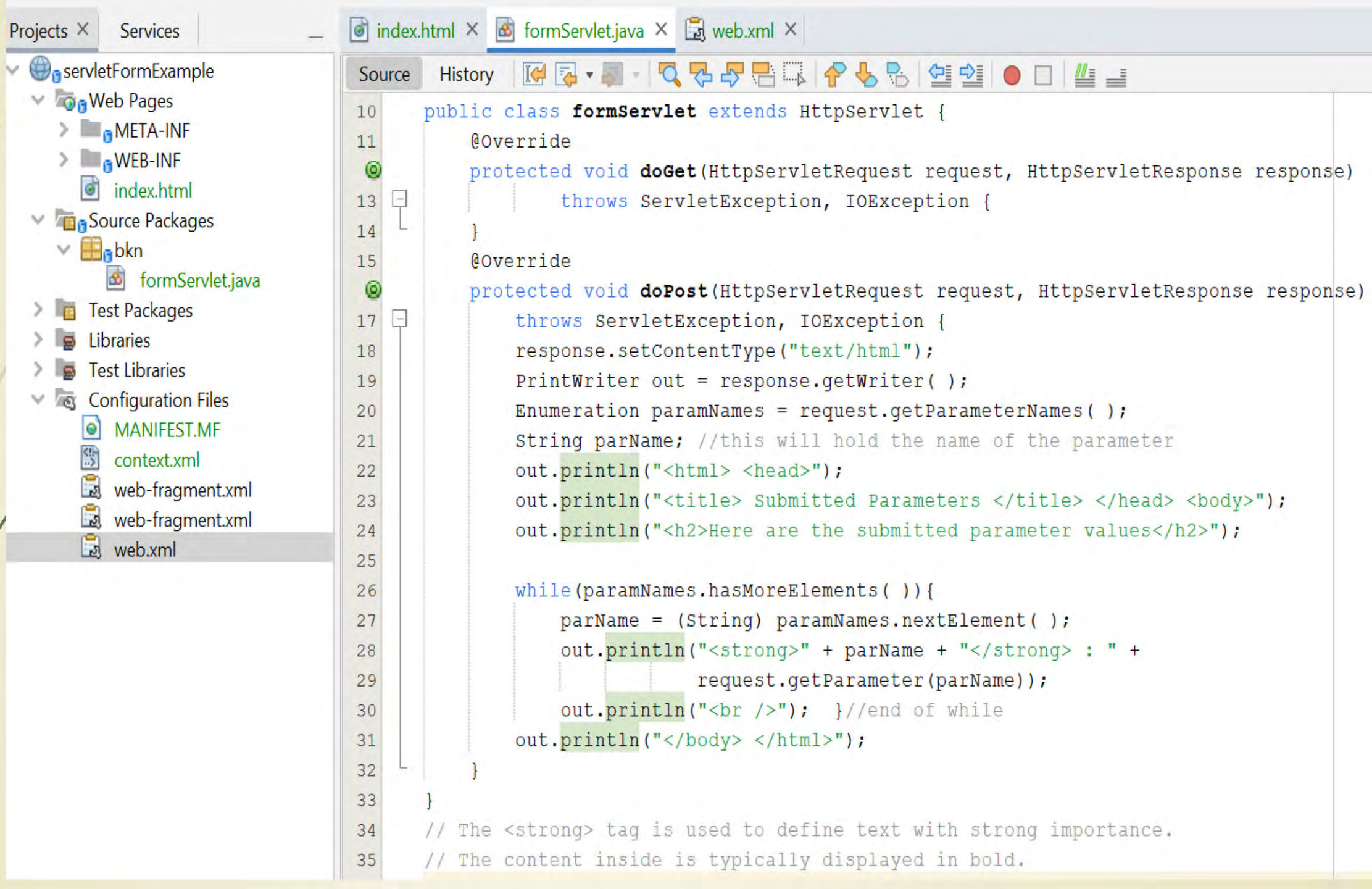
Servlets handles form data parsing automatically using the following methods depending on the situation −

getParameter() − You call **request.getParameter()** method to get the value of a form parameter.

getParameterValues() − Call this method if the parameter appears more than once and returns multiple values, for example checkbox.

getParameterNames() − Call this method if you want a complete list of all parameters in the current request.


Projects
Services
servletFormExample
Web Pages
META-INF
WEB-INF
index.html
Source Packages
bkn
formServlet.java
Test Packages
Libraries
Test Libraries
Configuration Files
MANIFEST.MF
context.xml
web-fragment.xml
web-fragment.xml
web.xml
index.html
formServlet.java
web.xml
Source
History
</head>
<body>
<h1> Please submit your information </h1>
<form action="formServlet" method="post">
<table>
<tr>
<td>First Name</td>
<td><input type="text" name="firstName" /></td>
</tr>
<tr>
<td>Last Name</td>
<td><input type="text" name="lastName" /></td>
</tr>
<tr>
<td>Address</td>
<td><input type="text" name="address" /></td>
</tr>
<tr>
<td>Contact No </td>
<td><input type="text" name="contact" /></td>
</tr>
<tr>
<td></td>
<td><input type="submit" value="Submit" /></td>
</tr>
</table>
</form>
</body>
</html>


Projects
Services
servletFormExample
Web Pages
META-INF
WEB-INF
index.html
Source Packages
bkn
formServlet.java
Test Packages
Libraries
Test Libraries
Configuration Files
MANIFEST.MF
context.xml
web-fragment.xml
web-fragment.xml
web.xml
index.html
formServlet.java
web.xml
Source
History
public class formServlet extends HttpServlet {
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {
}
@Override
protected void doPost(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {
response.setContentType("text/html");
PrintWriter out = response.getWriter( );
Enumeration paramNames = request.getParameterNames( );
String parName; //this will hold the name of the parameter
out.println("<html> <head>");
out.println("<title> Submitted Parameters </title> </head> <body>");
out.println("<h2>Here are the submitted parameter values</h2>");
while(paramNames.hasMoreElements( )){
parName = (String) paramNames.nextElement( );
out.println("" + parName + " : " +
request.getParameter(parName));
out.println("<br />"); }//end of while
out.println("</body> </html>");
}
}
// The tag is used to define text with strong importance.
// The content inside is typically displayed in bold.

Session Management

- A typical web application comprises of several servlets that require collaborating with each other to give a complete response. For instance, if you go online to purchase a book, you need to go through multiple pages like search page, shopping page, billing page etc. before you can complete the transaction. ***In situations like this***, it is important that ***one servlet shares information or data with other servlet.***
- In web application terminology we call the ***shared data or information as state***. Having state is just not sufficient. Someone must be able to pass this state from one servlet to other servlet so that they can share the data.
- So, ***who does this state propagation?*** Can HTTP do this? No, because HTTP is a stateless protocol which means it cannot propagate the state.
- So, is there anyone to help us out here to make the state available to all the servlets? ***Yes, there is one guy who is always there for our rescue and it's none other than web container (Tomcat).***

Session Management

- A web container provides a common storage area called as session store the state and provides access to this session to all the servlets within the web application.
- For instance, ***servlet A can create some state*** (information) and store it in the session. ***Servlet B can then get hold of the session and read the state.***
  - *Session simply means a particular interval of time.*
  - *Session tracking is a way to maintain state (data) of an user. It is also known as session.*
  - *Management in servlet.*

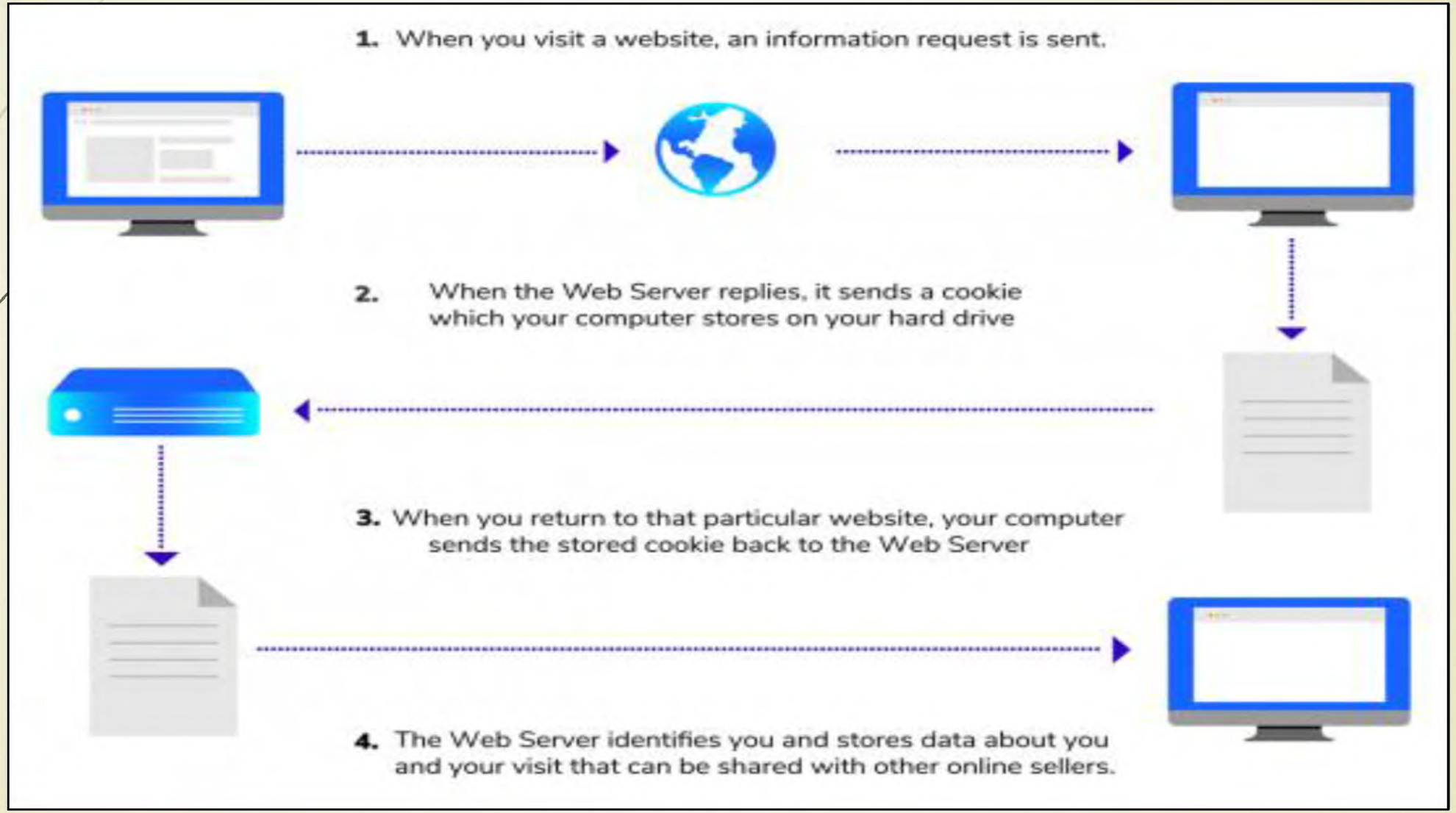
Session Management

- There are following ***ways to maintain session between web client and web server*** (i.e. State can be maintained indirectly) using:
  1. *Cookies*
  2. *Hidden Form Fields*
  3. *URL Rewriting*
  4. *HttpSession Object*

Session Management by Cookie

- **Cookie:** A cookie is ***a small piece of information as a text file*** stored on client's machine by a web application.
- ***Why use Cookies?***
  - As ***HTTP is a stateless protocol*** so there is ***no way to identify*** that it is a new user or previous user for every new request. *For this purpose,* ***we can use Cookies. It allows us to store the information on the user's computer*** *and track the state of applications.*
  - They are ***stored on client's machine***. Now when a new request ***comes cookie is by default added*** with the request.
  - ***With this information we can identify*** that it is a new user or a previous user.

How cookie works?



Session Management by Cookie

- ***Types of cookies***

1. ***Session cookies/Non-persistent cookies:*** These types of cookies are session dependent i.e. they are accessible as long as session is open and ***they are lost when session is closed*** by exiting from the web application.
2. ***Permanent cookies/Persistent cookies:*** These types of cookies are session independent i.e. ***they are not lost when session is closed*** by exiting from the web application. They are lost when they expire.

- ***Advantages of cookies***
  - They are stored on client side so don't need any server resource.
  - Easy technique for session management.
- ***Disadvantages of cookies***
  - Cookies can be disabled from the browser.
  - Security risk is there because cookies exist as a text file so any one can open and read user's information.

Session Vs. Cookies

- A session stores the variables and their values ***within a file in a temporary directory on the server***. ***Cookies are stored on the user's computer as a text file.***
- The ***session ends when the user logout*** from the application or closes his web browser. ***Cookies end on the lifetime set by the user.***
- We can store ***as much data as we want within a session***, but there is a maximum memory limit, which a script can use at one time, ***and it is 128 MB***. ***The maximum size of the browser's cookies is 4 KB.***
- We ***need to call the session start() function*** to start the session. We ***don't need to call a function to start a cookie*** as it is stored within the local computer.
- ***Sessions are more secured*** compared to cookies, as they save data in encrypted form. ***Cookies are not secure, as data is stored in a text file***, and if any unauthorized user gets access to our system, he can temper the data.

How to create cookie?

Creating a new Cookie

```
Cookie ck = new Cookie("username",name);
```

creating a new cookie object

Setting up lifespan for a cookie

```
ck.setMaxAge(30*60);
```

setting maximum age of cookie

Sending the cookie to the client

```
response.addCookie(ck);
```

adding cookie to **response** object

Getting cookies from client request

```
Cookie[] cks = request.getCookies();
```

getting the cookie for **request** object

Commonly used methods of cookie

- ***setMaxAge(int expiry):*** Sets the maximum age of the cookie.
- ***germanane():*** Returns the maximum age of the cookie. Default value is -1.
- ***setValue(String newValue):*** Change the value of the cookie with new value.
- ***getValue():*** Returns the value of the cookie.
- ***getName():*** Returns the name of the cookie.
- ***getPath():*** Returns the path on the server to which the browser returns this cookie.
- ***getDomain():*** Gets the domain name of this Cookie

Cookies

```
<h1>Java Cookies Example using Servlet!!!</h1>
<form method="post" action="loginServlet">
    <table>
        <tr>
            <td> User Name: </td>
            <td> <input type="text" name="userName" ></td>
        </tr>
        <tr>
            <td>Password: </td>
            <td> <input type="text" name="passWord" > </td>
        </tr>
        <tr>
            <td>  </td>
            <td> <input type="submit" value="submit"> </td>
        </tr>
    </table>
</form>
```

Cookies

```
8   public class loginServlet extends HttpServlet {
9       @Override
10      protected void doGet(HttpServletRequest request, HttpServletResponse response)
11      throws ServletException, IOException {}
12      @Override
13      protected void doPost(HttpServletRequest request, HttpServletResponse response)
14      throws ServletException, IOException {
15          String username = request.getParameter("userName");
16          String password = request.getParameter("passWord");
17          Cookie ckUser = new Cookie("username", username);
18          Cookie ckPass = new Cookie("password", password);
19          response.addCookie(ckUser);
20          response.addCookie(ckPass);
21          response.sendRedirect("welcomeServlet");
22      }
23  }
```

Cookies

```
10  public class welcomeServlet extends HttpServlet {
11      @Override
        protected void doGet(HttpServletRequest request, HttpServletResponse response)
13      throws ServletException, IOException {
14          response.setContentType("text/html");
15          PrintWriter out = response.getWriter();
16
17          Cookie[] cks = request.getCookies();
18          out.println("<h1>Example of Servlet Cookies !!!</h1>");
19          out.println("<h2> Cookies Parameters </h2> ");
20          out.println("<h2> Welcome " + cks[0].getValue()+ "</h2>");
21          out.println("<h3>Your Password is  "+ cks[1].getValue()+ "</h3>");
22      }
23      @Override
        protected void doPost(HttpServletRequest request, HttpServletResponse response)
25      throws ServletException, IOException {}
26  }
```

Session management by Hidden Fields

- Hidden field ***is a field in HTML form*** which is used ***to store a user information or a session data***, to maintain its session and differentiate it from other users.
- This hidden field ***is not displayed to the user.***
- Through hidden field we ***add a session data*** such as to the request going to be made ***to the next Servlet***. The next Servlet ***reads the hidden field passed to it associated*** with the request and ***send back an appropriate response to the client.***
- ***Advantages***
  - Does not have to depend on browser whether the cookie is disabled or not.
  - Inserting a simple HTML Input field of type hidden is required. Hence, its easier to implement.

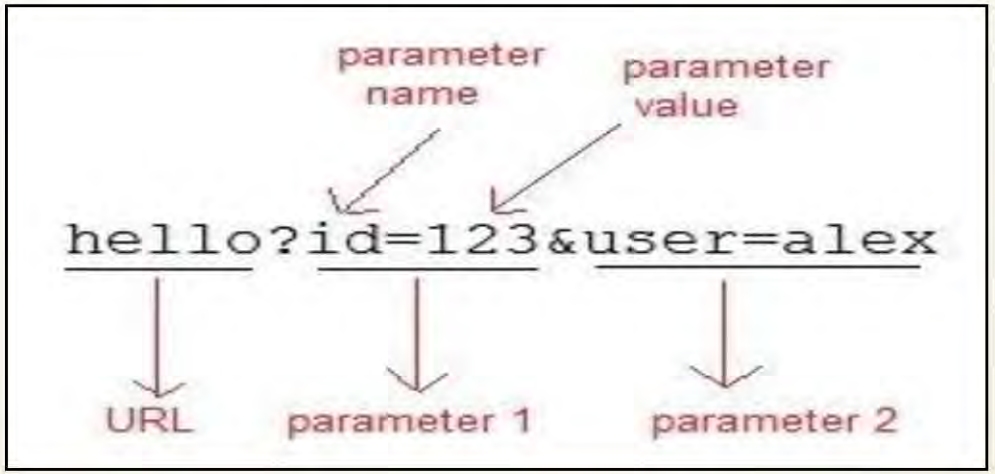
Session management by Hidden Fields

- **Disadvantage**
  - Extra form submission is required on every page. ***This is a big overhead.***
- **Syntax of Hidden field**

  <input type = "hidden" name = "user" value = "max" >
- We have created a hidden form field, with ***a variable name user*** and its ***value max*** and have associated it with form data within the request passed to the calling Servlet.
- This hidden field ***is used to identify a user***, maintain a session and send back an appropriate response.

Session management by URL Rewriting

- If the client ***has disabled cookies*** in the browser then session management using cookie ***won't work.*** In that case ***URL Rewriting can be used as a backup.*** URL rewriting will always work.
- In URL rewriting, ***a token(parameter) is*** added at the end of the URL. The token consist of ***name/value pair separated by an equal(=) sign.***



Session management by URL Rewriting

- When the ***User clicks on the URL having parameters***, the request goes to the Web Container with extra bit of information at the end of URL.
- The ***Web Container will fetch the extra part of the requested*** URL and use it for session management.
- The ***getParameter() method*** is used to get the parameter value at the server side.

```
<h2> Example for Hidden Field </h2>
<form action = "hiddenFieldServlet1"  method = "post" >
    Name : <input  type = "text" name = "username"/>
    <input  type = "submit" name = "submit"/>
</form>
```

```
9   public class hiddenFieldServlet1 extends HttpServlet {
10      @Override
11      protected void doGet(HttpServletRequest request, HttpServletResponse response)
12              throws ServletException, IOException {}
13      @Override
14      protected void doPost(HttpServletRequest request, HttpServletResponse response)
15      throws ServletException, IOException {
16          response.setContentType("text/html");
17          PrintWriter out = response.getWriter();
18          String name = request.getParameter("username");
19          out.println("<b> Welcome "+ name + "!!!"+ "</b>" );
20          out.println("<form action = 'hiddenFieldServlet2' method = 'post' >");
21          out.println("<input type = 'hidden'  name = 'userName' value='"+name+"'>");
22          out.println("<input type = 'submit' value = 'Submit'  />");
23          out.println("</form>");
24      }
25  }
```

```
9   public class hiddenFieldServlet2 extends HttpServlet {
10      @Override
11      protected void doGet(HttpServletRequest request, HttpServletResponse response)
12      throws ServletException, IOException {}
13      @Override
14      protected void doPost(HttpServletRequest request, HttpServletResponse response)
15      throws ServletException, IOException {
16          response.setContentType("text/html");
17          PrintWriter out = response.getWriter();
18          String name = request.getParameter("userName");
19          out.println("<b> Welcome from hiddenFieldServlet2 for ---"+ name + "!!!");
20      }
21  }
```

```
<h2> Example for URL Rewriting </h2>
<form method="post" action="urlRewritingServlet1">
    <table>
        <tr>
            <td>Name: </td>
            <td> <input type="text" name="user" /> </td>
        </tr>
        <tr>
            <td>Password: </td>
            <td> <input type="text" name="pass" > </td>
        </tr>
        <tr>
            <td> </td>
            <td> <input type="submit" value="submit"> </td>
        </tr>
     </table>
</form>
```

```java
public class urlRewritingServlet1 extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException { }
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
        response.setContentType("text/html");
        String name = request.getParameter("user");
        String pass = request.getParameter("pass");
        response.sendRedirect("urlRewritingServlet2?userName="+ name+ " & "+ "passWord="+pass);
    }
}
```

```java
public class urlRewritingServlet2 extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        String user = request.getParameter("userName");
        out.println("<b> Welcome from URL Rewriting-2 for ---"+ user + "!!!");
    }
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {}
}
```

Session management by The HttpSession Object

- The ***HttpSession*** object is ***used for session management***.
- A ***session contains information*** specific to a particular ***user across the whole application.*** When a user enters into a website for t***he first time*** HttpSession is obtained via ***request.getSession(),*** the ***user is given a unique ID*** to identify his session. ***This unique ID can be stored into a cookie or in a request parameter.***
- A servlet provides ***HttpSession Interface*** which provides ***a way to identify a user*** across more than one page request or visit to a Web site and to ***store information about that user.***
- The servlet container uses this interface ***to create a session between an HTTP client and an HTTP server***. The session ***persists for a specified time period***, across more than one connection or page request from the user.

Session management by The HttpSession Object

- The ***HttpSession*** stays alive until it has not been used for more than the timeout value specified in tag in ***deployment descriptor file(web.xml)***. The ***default timeout value is 30 minutes.***
- ***Advantages***
  - Any kind of object can be stored into a session such as a text, database, dataset etc.
  - Usage of sessions is not dependent on the client's browser.
  - Sessions are secure and transparent.
- ***Disadvantages***
  - Performance overhead due to session object being stored on server.
  - Overhead due to serialization and de-serialization of data.

Session management by The HttpSession Object

Creating a new session

```
HttpSession session = request.getSession();
```

getSession() method returns a session. If the session already exist, it return the esisting session else create a new sesion

```
HttpSession session = request.getSession(true);
```

getSession(true) always return a new session

Getting a pre-existing session

```
HttpSession session = request.getSession(false);
```

return a pre-existing session

Destroying a session

```
session.invalidate();
```

destroy a session

Session management by The HttpSession Object

Creating a Session

The servlet API provides us with a class called **HttpSession** to work with sessions. To create a session, we do the following:

```
HttpSession session = request.getSession(true);
```

The above method returns a new session object if one is not present, otherwise it returns the old session object that is already created before.

Storing the data in Session

Data in session is stored as key-value pair just like in HashMap or Hashtable. The value can be any Java object and the key is usually a String.

To store the data we use the **`setAttribute()`** method as shown below:

```
session.setAttribute("price",new Double("12.45"));
```

Session management by The HttpSession Object

Reading the data from the Session

To read the data, we need to use the **`getAttribute()`** method by passing in the key as shown below which then returns the value object:

```
Double d = (Double)session.getAttribute("price");
```

Notice that we need to do proper casting here. Since we stored Double object, we need to cast it again as Double while reading it.

Destroying the Session

A session is usually destroyed by the last page or servlet in the web application. A session is destroyed by invoking the **`invalidate()`** method as shown below:

```
session.invaliadte()
```

Methods of HttpSession

- ***public void setAttribute(String name, Object value):*** Binds the object with a name and stores ***the name/value pair as an attribute of the HttpSession object***. If an attribute already exists, then this method replaces the existing attributes.
- ***public Object getAttribute(String name):*** Returns the String object specified in the parameter, from the session object. If no object is found for the specified attribute, then the getAttribute() method returns null.
- ***public Enumeration getAttributeNames():*** Returns an Enumeration that contains the name of all the objects that are bound as attributes to the session object.
- ***public void removeAttribute(String name):*** Removes the given attribute from session.

Methods of HttpSession

- ***setMaxInactiveInterval(int interval):*** Sets the session inactivity time in seconds. This is the time in seconds that specifies how long a sessions remains active since last request received from client.
- ***public String getId():*** Returns a string containing the unique identifier value.
- ***public long getCreationTime():*** Returns the time when this session was created, measured in milliseconds since midnight January 1, 1970 GMT.
- ***public long getLastAccessedTime():*** Returns the last time the client sent a request associated with this session as the number of milliseconds since midnight January 1, 1970 GMT.

HttpSession Example

```
<body>
    <h2> Example for HttpServlet  </h2>
    <form method="post" action="loginServlet">
        <table>
            <tr>
                <td>User Name: </td>
                <td> <input type="text" name="username" /> </td>
            </tr>
            <tr>
                <td>Password: </td>
                <td> <input type="text" name="password" > </td>
            </tr>
            <tr>
                <td> </td>
                <td> <input type="submit" value="Login"> </td>
            </tr>
        </table>
    </form>
</body>
```

HttpSession Example

```
9   public class loginServlet extends HttpServlet {
10      @Override
        protected void doPost(HttpServletRequest request, HttpServletResponse response)
12      throws ServletException, IOException {
13          response.setContentType("text/html");
14          String name = request.getParameter("username");
15          String pass = request.getParameter("password");
16          HttpSession session = request.getSession();
17          if(pass.equals("12345")){
18              //creating a session
19              session.setAttribute("userName", name);
20              response.sendRedirect("welcomeServlet");
21          }
22      }
23      @Override
        protected void doGet(HttpServletRequest request, HttpServletResponse response)
25      throws ServletException, IOException {}
26  }
```

HttpSession Example

```
6   public class welcomeServlet extends HttpServlet {
7       @Override
        protected void doPost(HttpServletRequest request, HttpServletResponse response)
9       throws ServletException, IOException {}
10      @Override
        protected void doGet(HttpServletRequest request, HttpServletResponse response)
12      throws ServletException, IOException {
13          response.setContentType("text/html");
14          PrintWriter out = response.getWriter();
15          HttpSession session = request.getSession();
16          String user = (String)session.getAttribute("userName");
17          out.println("<title> Session Parameters </title> ");
18          out.println("<h2>Example of Http Session !!!</h2>");
19          out.println("Hello "+ user);
20      }
21  }
```

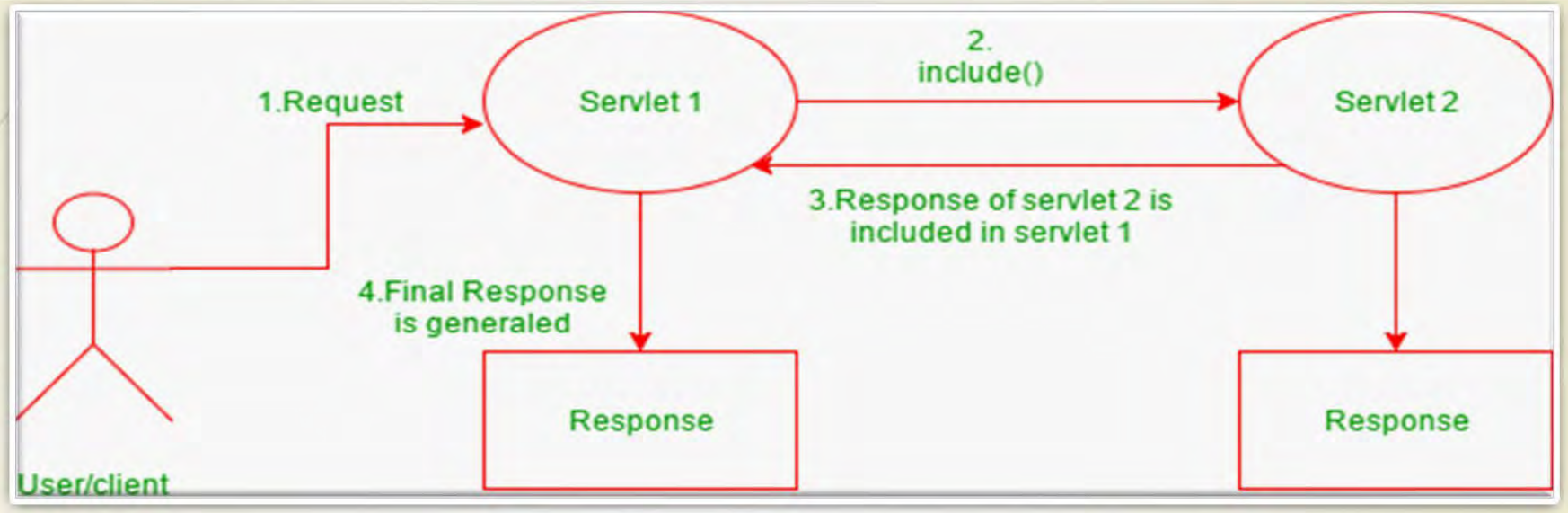
Request Dispatching

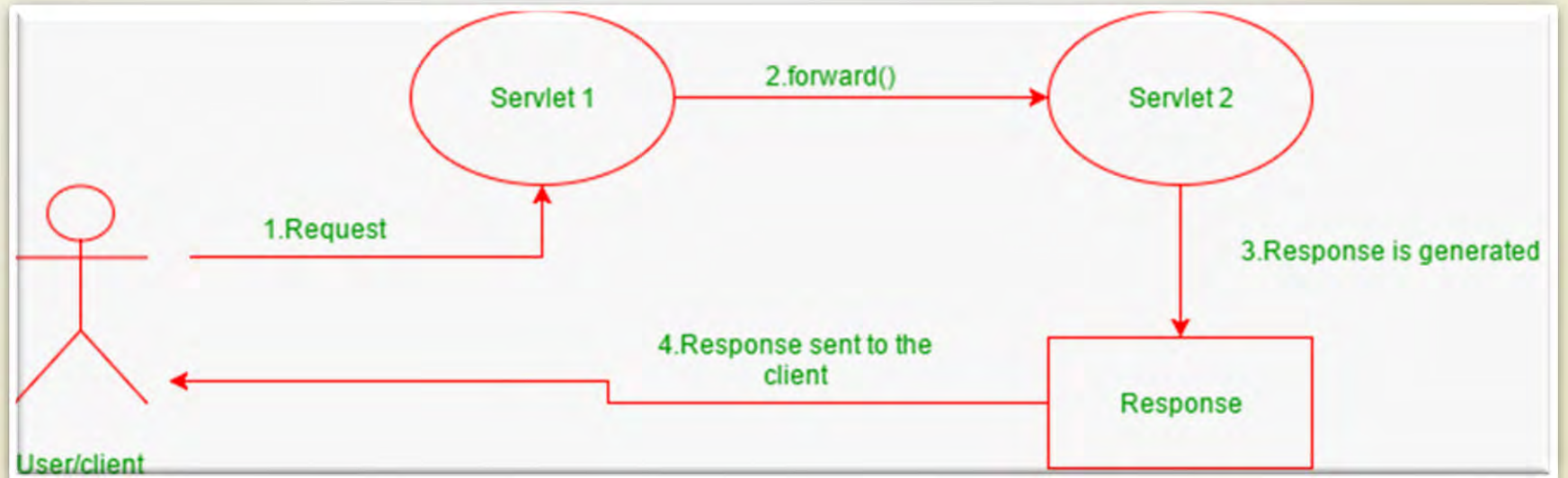
- ***Request dispatching*** is the ability of one servlet to dispatch or delegate the request to another servlet for processing.
- In simple words, let's say we have ***a servlet A which doesn't know how to completely process the request***. Therefore, after partially processing the request, it should ***forward the request to another servlet B.*** This servlet then ***does the rest of the processing*** and sends the final ***response back to the browser***.
- The class used for dispatching requests is the ***RequestDispatcher*** interface in ***Servlet API.***
- This interface has two methods namely forward() and include().

Request Dispatching

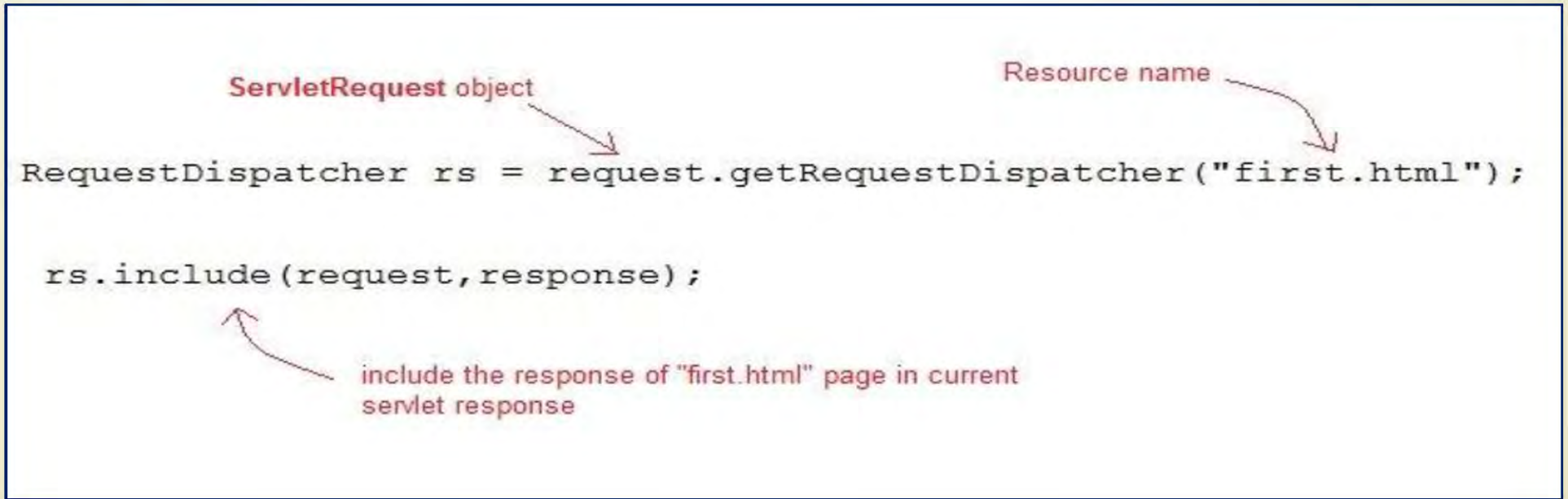
- ***The forward() method***
  - This method is used for ***forwarding request*** from one ***servlet to another.*** Consider two servlets A and B. Using this method, A gets the request, which then forwards the request to B, B processes the request and sends the response to the browser. This method takes ***HttpServletRequest*** and ***HttpServletResponse*** as parameters.
- ***The include() method***
  - With this method, ***one servlet can include the response of other servlet.*** The first servlet will then ***send the combined response back to the browser.*** This method also takes ***HttpServletRequest*** and ***HttpServletResponse*** as parameters.

Request Dispatching





Request Dispatching



Request Dispatching

```
<body>
    <h1>Request Dispatching Servlet Example!!!</h1>
    <form method="post" action="loginServlet">
        <table>
            <tr>
                <td> User Name: </td>
                <td> <input type="text" name="userName" ></td>
            </tr>
            <tr>
                <td>Password: </td>
                <td> <input type="text" name="passWord" > </td>
            </tr>
            <tr>
                <td>  </td>
                <td> <input type="submit" value="login"> </td>
            </tr>
        </table>
    </form>
</body>
```

Request Dispatching

```
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
15  throws ServletException, IOException {
        PrintWriter out=response.getWriter();
17      try {
18          String uname = request.getParameter("userName");
19          String pword = request.getParameter("passWord");
20          if(uname.equals("Ramu123")&& pword.equals("12345")){
21              RequestDispatcher rd=request.getRequestDispatcher("welcomServlet");
22              rd.forward(request, response);
23          }else{
24              out.println("User Name or Password Incorrect!!!");
25              RequestDispatcher rd=request.getRequestDispatcher("/index.html");
26              rd.include(request, response);  }
27      }
28      finally {
29          out.close();}
30  }
31  }
```

Request Dispatching

```
9   public class welcomServlet extends HttpServlet {
10      @Override
        protected void doGet(HttpServletRequest request, HttpServletResponse response)
12      throws ServletException, IOException {}
13      @Override
        protected void doPost(HttpServletRequest request, HttpServletResponse response)
15      throws ServletException, IOException {
16          response.setContentType("text/html");
17          PrintWriter out = response.getWriter();
18          String name=request.getParameter("userName");
19          out.print("<h1> Example: forward Request Dispatcher </h1>");
20          out.print("<h1>Welcome " + name + "</h1>");
21      }
22  }
```

Thank You
???

Chapter 10

JSP Programming

Bal Krishna Nyaupane
Assistant Professor
Department of Electronics and Computer Engineering
Institute of Engineering, Tribhuvan University
bkn@wrc.edu.np

Introduction to JSP

- JSP is another J2EE technology ***for building web applications*** using Java.
- JSP technology is used to create web application just like Servlet technology. It can bethought of as ***an extension to Servlet*** because it provides more functionality than servlet such as expression language, JSTL, etc.
- ***Java Server Pages (JSP)*** is a technology which is used to develop ***web pages by inserting Java code into the HTML pages*** by making special JSP tags.

Introduction to JSP

- JSP technology ***is built on top of servlet technology***. This is why we can call JSP as an abstraction over servlet.
- ***What does abstraction mean?*** In simple terms, abstraction is as ***simplified fine grained layer over a slightly more complex layer*** that makes the development faster and easier. More the abstraction, more the simplicity.
- JSP technology is used ***to create dynamic web*** applications. JSP pages are ***easier to maintain than a Servlet***.
- JSP pages are ***opposite of Servlets*** as a ***servlet adds HTML code inside Java code***, while ***JSP adds Java code inside HTML*** using JSP tags. Everything a Servlet can do, a JSP page can also do it.

Introduction to JSP

- A typical JSP page ***very much looks like html page*** with all the html markup except that you also ***see Java code in it***. So, in essence,

HTML + Java = JSP

- ***To summarize***, a html file will only have html markup, and a ***jsp file will have both html markup*** and ***Java code***.

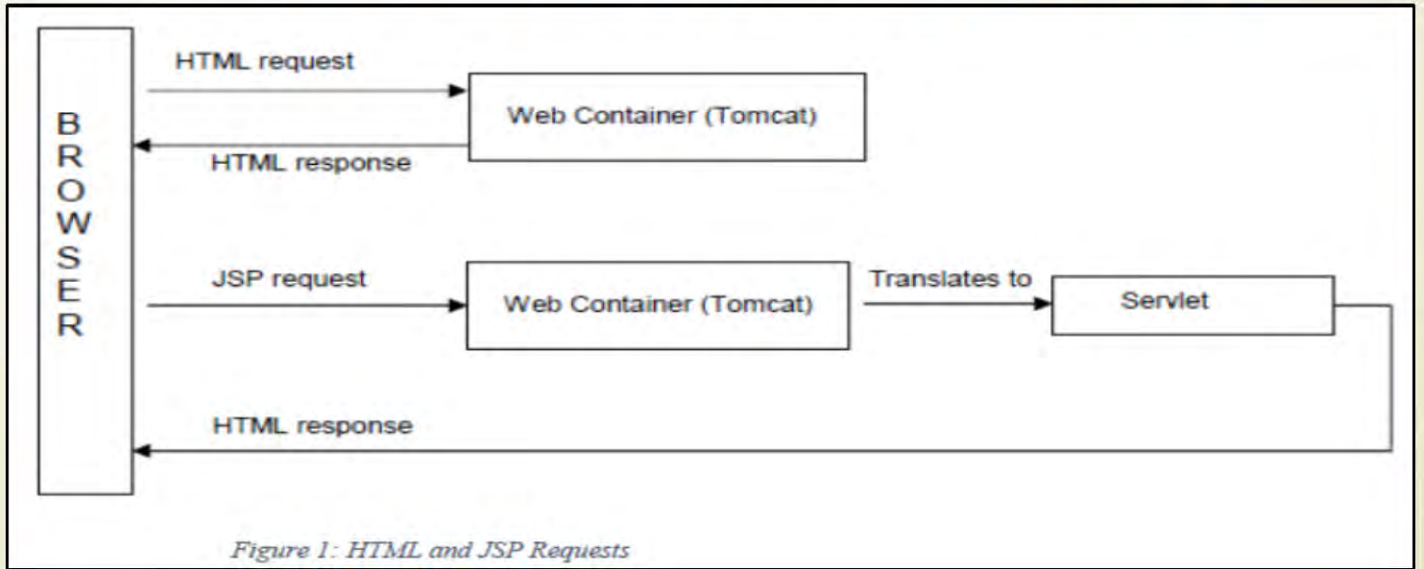


Figure 1: HTML and JSP Requests

Introduction to JSP

- ***When the browser requests*** for html file, the ***web container*** simply ***responds with a html response*** without any processing.
- When the ***browser sends a JSP page request***, the web container ***assumes that the JSP page might include Java code***, and translates the page into a Servlet. The ***servlet then processes the Java code***, and ***returns the complete html response including the dynamic content*** generated by the Java code.
- For a ***web container to translate JSP***, it ***needs to identify*** from the JSP page, as to ***which is HTML markup*** and ***which is Java code.***
- According to J2EE specification, ***a JSP page must use special symbols as placeholders for Java code***. The web container instead of scratching its head to identify Java code, ***simply uses the special symbols to identify it***.

Features of JSP

- ***Coding in JSP is easy:*** As it is just adding JAVA code to HTML/XML.
- ***Reduction in the length of Code:*** In JSP we use action tags, custom tags etc.
- ***Connection to Database is easier*** : It is easier to connect website to database and allows to read or write data easily to the database.
- ***Make Interactive websites -*** In this we can create dynamic web pages which helps user to interact in real time environment.
- ***Portable, Powerful, flexible and easy to maintain -*** as these are browser and server independent.
- ***No Redeployment and No Re-Compilation -*** It is dynamic, secure and platform independent so no need to re-compilation.
- ***Extension to Servlet -*** as it has all features of servlets, implicit objects and custom tags.

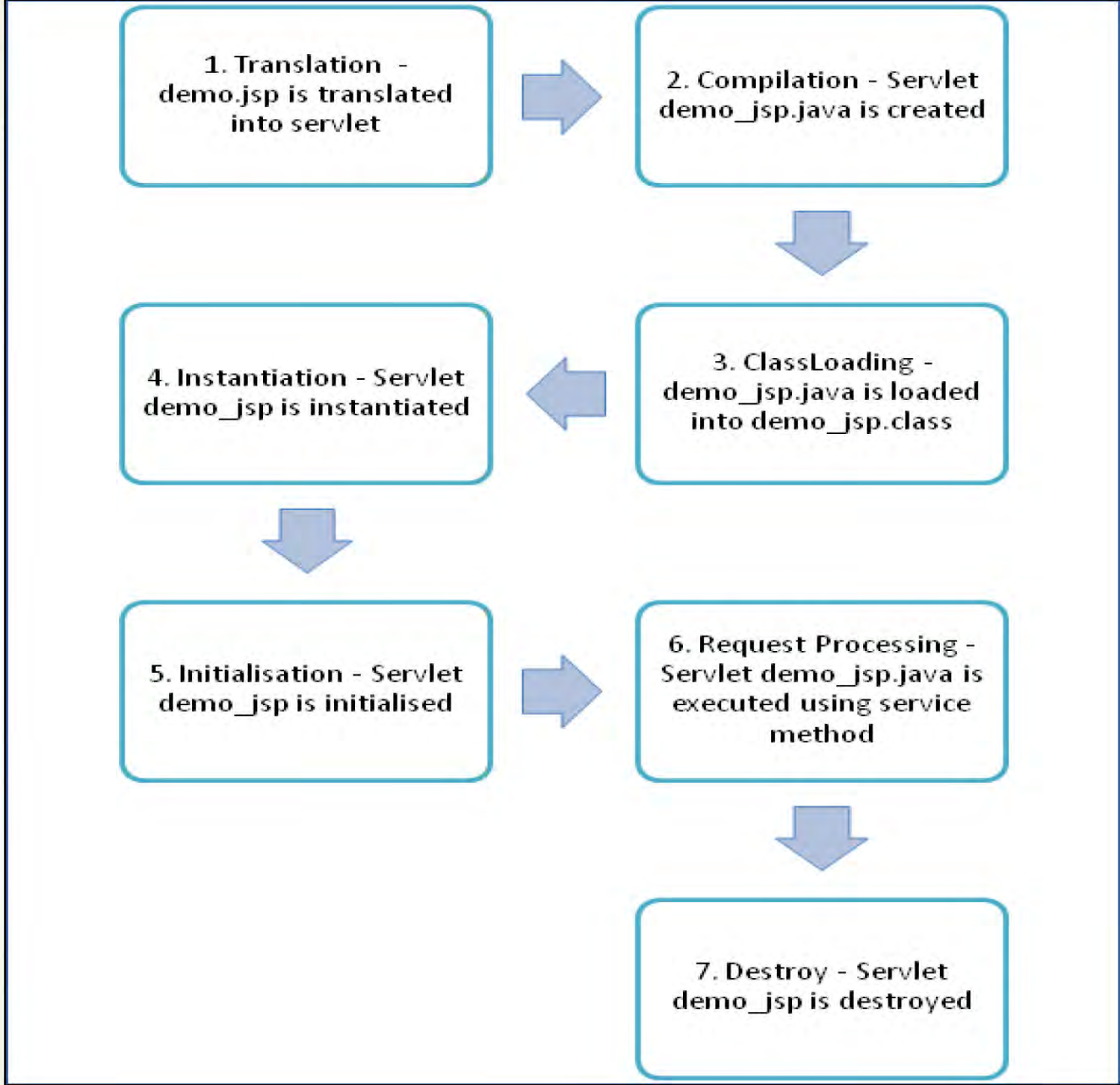
Introduction to JSP

JSP	Servlets
Extension to Servlet	Not an extension to servlet
Easy to Maintain	Bit complicated
No need to recompile or redeploy	The code needs to be recompiled
Less code than a servlet	More code compared to JSP

Lifecycle of JSP

- A ***JSP page is converted into Servlet*** in order to service requests. The translation of a JSP page to a Servlet is called Lifecycle of JSP.
- JSP Lifecycle ***is exactly same as the Servlet Lifecycle***, with one additional first step, which is, translation of JSP code to Servlet code.
- ***Following are the JSP Lifecycle steps:***
  1) Translation of JSP to Servlet code.
  2) Compilation of Servlet to bytecode.
  3) Loading Servlet class.
  4) Creating servlet instance.
  5) Initialization by calling ***jspInit()*** method
  6) Request Processing by calling ***_jspService()*** method
  7) Destroying by calling ***jspDestroy()*** method

Lifecycle of JSP



JSP Scripting Element

Scripting Element	Example
Comment	<%-- comment --%>
Directive	<%@ directive %>
Declaration	<%! declarations %>
Scriptlet	<% scriplets %>
Expression	<%= expression %>

JSP Directives

- Directives are used for declaring classes, session usage etc., and does not produce any response to the client. A directive uses attributes for declarations.
- Directives can have a list of attributes ***defined in key-value pairs*** and separated by commas.
- There are 3 types of directives as listed below:
  1. ***page directive***
  2. ***include directive***
  3. ***taglib directive***

Directive	Description
<%@ page ... %>	Defines page-dependent attributes, such as scripting language, error page, and buffering requirements.
<%@ include ... %>	Includes another file.
<%@ taglib ... %>	Declares a tag library containing custom actions which can be used in the page.

JSP Directives

- ***The page directive:*** This directive is used to declare things that are important to the entire JSP page. The syntax for this directive is shown below

<%@ page attribute="value", attribute="value",...%>

Name	Description	Default value
import	Used to import Java classes and packages you want	
session	Used to have HTTP sessions	true
errorPage	Specify the URL to another JSP page that can handle errors	null
isErrorPage	Boolean literal to check the page can handle errors	false
language	Any scripting language can be used with JSP	java
extends	Valid Java class that implements	
buffer	Specify the size of output buffer	
autoflush	Boolean literal checks the buffer should be flushed or not	true
info	Any information about JSP page	
contentPage	MIME type and encoding of output	txt/html; char set=ISO-8859-1
pageEncoding	Character encoding of the JSP page	ISO-8859-1

JSP Directives

- ***JSP session: <%@ page session= "true/false" %>***
  - If the value of ***session="true"***, this value specifies that the page needs ***an HTTP session.*** The ***default value*** of this attribute is ***true.***
  - If the value of ***session="false",*** this value shows that the ***page doesn't require an HTTP session.*** The implicit variable of the ***session generation can't be used*** if the value of the session is false. It causes errors at the time of translation.
- ***JSP language: <%@ page language="javascript" %>.***
  - This indicates language is JavaScript.
- ***JSP import: <%@ page import="java.io.*,java.sql.*,java.util.*" %>***
  - If the ***JSP page needs to import the core library classes***, it uses the page directive as:

JSP Directives

Ex 3: If a JSP page should forward to a custom error page for any exceptions within the page, the page directive will be as shown below:

```
<%@ page errorPage="Error.jsp" %>
```

The web container will then forward the request to **Error.jsp** if it encounters exceptions within the page.

Ex 4: If a JSP page should allow itself as an error page for other JSP pages, it should use the following page attribute:

```
<%@ page isErrorPage="true" %>
```

Instead of defining one attribute per page directive, we can define all the attributes at a time as shown below:

```
<%@ page import="java.util.*" session="true"
errorPage="Error.jsp" %>
```

JSP Directives

- ***JSP pageEncoding***
  - It is a character encoding in which JSP files are encoded. The ***default value*** for ***page encoding is ISO-8859-1***. It is a character 8-bit (single-byte) coded graphic character set.
- ***JSP isELignored: <%@ isELignored = "true/false"%>.***
  - The attribute stands for ***Expression Language ignored***. This attribute specifies the inability of an expression to be evaluated saying that expression is static and will not be evaluated. However, the ***default value of the attribute is false*** and expressions are evaluated as specified.
- ***JSP Include Directive: <%@ include file="login.jsp" %>***
  - JSP include directive is used to ***include other files into the current JSP page.*** These files can be html files, other jsp files etc.

JSP Directives

- ***JSP contentType:*** ***<%@page contentType = "text/html" %>***
  - This defines the ***format of data*** that is being ***exchanged between the client and the server.*** It does the same thing as the setContentType method in servlet used to.
- ***JSP buffer:*** ***<%@page buffer = "size in kb"%>***
  - Defines the ***size of the buffer*** that is allocated ***for handling the JSP page.*** The size is defined in Kilo Bytes.
- ***JSP isThreadSafe:*** ***<%@ page isThreadSafe="true/false" %>***
  - If you specify ***isThreadSafe="true"***, it implies that you take care of any possible thread conflicts. A defined JSP container safely dispatches multiple requests.
  - If the value is "false", the JSP container implements ***SingleThreadModel.*** By default value is taken as "true".

JSP Scriptlet Tag

- ***JSP Scriptlet Tag: <% JAVA CODE %>***
- *Scriptlet Tag allows **to write java code inside JSP page.***
- *Scriptlet tag implements the **_jspService** method functionality by writing script/java code.*

```
<body>
    <h1>My First JSP Page</h1>
    <% int count = 0;   %>
    Page Count is <% out.println(++count); %>
    <% String user = request.getParameter("user"); %>
</body>
```

JSP Scriptlet Tag

```
<%@ page import = "java.util.Date" %>
<% Date date = new Date(); %>
<h3>Current date is:  <% out.print(date); %> </h3>

<h3>The multiplication table of 10 </h3>
<% for(int i =1; i<=10;i++){
    out.print("10 * "+ i + "= "+10*i+"<br/>");
    }
%>
<table border = 1>
    <%
        for ( int i = 1; i < 10; i++ ) {
    %>
        <tr>
            <td>Number</td>
            <td><%= i*10 %></td>
        </tr>
    <%
        }
    %>
</table>
```

JSP Declaration Tag

- ***JSP Declaration Tag: <%! declaration %>***
- The JSP declaration tag is ***used to declare fields and methods.***
- JSP declarations are used to ***declare global variables and methods*** that can be used in the entire JSP page.

```
<%!
    int count = 0;
    int getCount() {
        System.out.println( "In getCount() method" );
        return count;
    }
%>
```

JSP Declaration Tag

```
<body>
    <h1>Use of Declaration in JSP </h1>
    <%! int num1=20, num2=33, sum=0; %>
    <%      sum=num1+num2;
            out.println("The n is " + sum);
    %>
    <h1>Calculation of Factorial in JSP </h1>
    <%!
        int n=5;
        int fact(int n){
            if(n==0)
                return 1;
            else
                return (n*fact(n-1)); }
    %>
    <% out.println("Factorial of "+ n +":"+ fact(n));   %>
</body>
```

JSP Expression Tag

- **JSP Expression Tag: <%= Java Expression %>**
  - Expression Tag is ***used to print out java language expression*** that is put between the tags.
  - An expression tag can hold any java language expression that can be used as an argument to the ***out.print()*** method.
  - When the Container sees this **<%= (2*5) %>** It turns it into this ***out.print((2*5));***
  - ***Note:*** Never end an expression with semicolon inside Expression Tag. Like this: **<%= (2*5); %>**

Implicit Objects

- Implicit objects are ***a set of Java objects*** that the JSP Container makes ***available to on each page***.
- These objects may be accessed ***as built-in variables*** via scripting elements and can also be accessed programmatically by JavaBeans and Servlets.
- JSP implicit objects ***are created during the translation phase of JSP to the servlet***.
- These objects can be directly used in scriplets that goes in the service method. ***They are created by the container automatically***, and they can be accessed using objects.
- There ***are 9 implicit objects*** available in the web container.

Implicit Objects

S.No.	Object & Description
1	**request** This is the **HttpServletRequest** object associated with the request.
2	**response** This is the **HttpServletResponse** object associated with the response to the client.
3	**out** This is the **PrintWriter** object used to send output to the client.
4	**session** This is the **HttpSession** object associated with the request.

Implicit Objects

5	**application** This is the **ServletContext** object associated with the application context.
6	**config** This is the **ServletConfig** object associated with the page.
7	**pageContext** This encapsulates use of server-specific features like higher performance **JspWriters**.
8	**page** This is simply a synonym for **this**, and is used to call the methods defined by the translated servlet class.
9	**Exception** The **Exception** object allows the exception data to be accessed by designated JSP.

Implicit Objects

- ***JSP out implicit object:*** For writing any data to the buffer, JSP provides an implicit object named out. ***It is the object of JspWriter.***
- ***In case of servlet*** you need to write:

  PrintWriter out=response.getWriter();

- But in JSP, ***you don't need to write this code.***

```
<h1><% out.print(exp); %> </h1>
<h2> <% out.print(exp.toUpperCase()); %> </h2>
```

```
<% for(int i =1; i<=10;i++){
    out.print("10 * "+ i + "= "+10*i+"<br/>");
    }
%>
```

Implicit Objects

- ***JSP request implicit object***
  - The JSP request is an implicit object of type HttpServletRequest i.e. created for each jsp request by the web container.
  - It can be used ***to get request information such as parameter***, header information, remote address, server name, server port, content type, character encoding etc.

```
<form method="post" action="welcome.jsp">
    Name <input type="text" name="user" >
    <input type="submit" value="Submit">
</form>
```

```
<body>
    <%  String user = request.getParameter("user");    %>
    <h2> Hello <% out.println(user); %> !!!</h2>
</body>
```

Implicit Objects

- ***JSP response implicit object***
  - In JSP, response is an implicit object of type HttpServletResponse.
  - The instance of HttpServletResponse is created by the web container for each ***jsp request.***

```
<body>
    <%  String user = request.getParameter("user");    %>
        <h2> Hello <% out.println(user); %> !!!</h2>
    <%  response.sendRedirect("jspScriptletTag.jsp"); %>
</body>
```

JSP Config – Implicit Objects

- ***JSP Config*** is an implicit object which is ***used to transmit the configuration details*** to the JSP page.
- In JSP, Config is an ***instance of type ServletConfig***. This implicit object is ***used to acquire an initialization parameter*** for a certain JSP page.
- For each JSP page, the config object is ***generated through the web container.***
- JSP's config object carries the configuration pieces of information like the username, password, driver name, servlet name, servlet context, specification names, and their values settle in the web.xml (configuration file).

JSP Config – Implicit Objects

web.xml

```
<init-param>
    <param-name>dname</param-name>
    <param-value>com.mysql.jdbc.Driver</param-value>
</init-param>
```

welcome.jsp

```
<%
    String driver=config.getInitParameter("dname");
    out.print("<br/> driver name is= "+ driver);
%>
```

JSP Application – Implicit Objects

- In JSP, ***application is an implicit object of type ServletContext***. This is an instance of javax.servlet.ServletContext.
- It is generated ***onetime by the web container*** when web application or web project is deployed on server.
- This object is ***used to acquire the initialization parameter*** from the configuration ***file (web.xml).***
- It is used for ***allotting the attributes and the values of the attributes over all JSP page.*** This indicates that any attribute which is set by application object will be accessible to all the JSP pages.
- In the application scope, it is also used to get, set, and remove attribute.

JSP Application – Implicit Objects

web.xml

```
<init-param>
    <param-name>dname</param-name>
    <param-value>com.mysql.jdbc.Driver</param-value>
</init-param>
```

welcome.jsp

```
<%
    String driver=application.getInitParameter("dname");
    out.print("<br/> driver name is= "+ driver);
%>
```

JSP pageContext – Implicit Objects

- In JSP, pageContext is ***an implicit object*** of type ***PageContext class.***
- The pageContext object can be used to ***set attributes, get attributes and remove attributes*** that are present in the different scopes like as page, request, session, and application scopes which are given below as follows:
  1. ***page – Scope:*** PAGE_CONTEXT
  2. ***request – Scope:*** REQUEST_CONTEXT
  3. ***session – Scope:*** SESSION_CONTEXT
  4. ***application – Scope:*** APPLICATION_CONTEXT
- In JSP, page scope is the default scope.

JSP pageContext – Implicit Objects

welcome.jsp

```
<%
    pageContext.setAttribute("user",name,PageContext.SESSION_SCOPE);
    <a href="second.jsp">second jsp page</a>
%>
```

second.jsp

```
<%
    String name=(String)pageContext.getAttribute("user",PageContext.SESSION_SCOPE);
    out.print("Hello "+ name);
%>
```

JSP Implicit Objects

The exception Implicit Object

- An exception implicit object is implemented to handle exceptions to display error messages.
- The exception implicit object is an instance of the **java.lang.Throwable class**.
- It is only available for JSP pages, with the isErrorPage value set as "True". This means Exception objects can only be used in error pages.

The page Implicit Object

A page object is an implicit object that is referenced to the current instance of the servlet. You can use it instead. Covering it specifically is hardly ever used and not a valuable implicit object while building a JSP application.

JSP Objects Scope

The availability of a JSP object for use from a particular place of the application is defined as the scope of that JSP object. Every object created in a JSP page will have a scope. Object scope in JSP is segregated into four parts and they are page, request, session and application.

Page Scope

'page' scope means, the JSP object can be accessed only from within the same page where it was created. The default scope for JSP objects created using <jsp:useBean> tag is page. JSP implicit objects out, exception, response, pageContext, config and page have 'page' scope.

Request Scope

A JSP object created using the 'request' scope can be accessed from any pages that serves that request. More than one page can serve a single request. The JSP object will be bound to the request object. Implicit object request has the 'request' scope.

Session Scope

'session' scope means, the JSP object is accessible from pages that belong to the same session from where it was created. The JSP object that is created using the session scope is bound to the session object. Implicit object session has the 'session' scope.

Application Scope

A JSP object created using the 'application' scope can be accessed from any pages across the application. The JSP object is bound to the application object. Implicit object application has the 'application' scope.

Java Beans

- JavaBeans are introduced in ***1996 by Sun Microsystem.***
- ***What is JavaBeans?***
  - JavaBeans is a ***reusable, portable, platform-independent*** model written in Java Programming Language. ***Its components are referred to as beans.***
- In simple terms, ***JavaBeans are classes which encapsulate several objects into a single object.*** It helps in accessing these object from multiple places. JavaBeans contains several elements like Constructors, Getter/Setter Methods and much more.
- ***In simple words JavaBean is nothing but a Java class.*** When these JavaBeans are used in other applications, the internal working of such components are hidden from the application developer.

Java Beans

- ***JavaBeans has several conventions*** that should be followed:
  - Beans ***should have a default constructor***
  - Beans ***should provide getter and setter methods***
    - A ***getter method is used to read the value*** of a readable property.
    - To ***update the value***, a setter method should be called.
  - Beans should implement ***java.io.serializable***, as it allows to save, store and restore the state of a JavaBean you are working on.

Java Beans

- ***Why use JavaBean?***
  - According to Java white paper, it is ***a reusable software component***.
  - A bean ***encapsulates many objects into one object*** so that we can access this object from multiple places.
  - Moreover, ***it provides easy maintenance.***
- ***Keep in mind:***
  - Java bean ***class name ends with Bean*** at last eg: NewBean, MyBean, TestBean, CircleBean ,etc (not hard and fast rule)
  - Bean Class ***should be public***
  - Bean Class should implement java.io.Serializable Interface (good practice)
  - Every ***variable of bean class are private***
  - Should ***Contain getter and setter methods*** and they are public
  - Should ***contain no argument constructor***

Java Beans

- ***What are JavaBean Properties?***
  - A JavaBean property can be accessed by the user of the object. The feature can be of any Java data type, containing the classes that you define.
  - It may be of the following mode: read, write, read-only, or write-only. JavaBean features are accessed through two methods:
- ***getPropertyName()***
  - For example, if the employee ***name is firstName***, the method name ***would be getFirstName()*** to read that employee name. This method is ***known as an accessor***. ***Properties of getter methods are as follows:***
    - Must be public in nature,
    - Return-type should not be void,
    - The getter method should be prefixed with the word get,
    - It should not take any argument

Java Beans

- ***setPropertyName ()***
  - For example, if the employee ***name is firstName***, the method name ***would be setFirstName()*** to write that employee name. This method is ***known as a mutator.***
  - ***Properties of setter methods:***
    - Must be public in nature,
    - Return-type should be void,
    - The setter method has to be prefixed with the word set,
    - It should take some argument.

Java Beans

- ***Benefits of JavaBeans in JSP***
  - Another programme can access the JavaBean attributes and methods.
  - It makes reusing software components easier.
- ***Drawbacks of JavaBeans in JSP***
  - JavaBeans are mutable objects so it cannot take advantage of immutable objects.
  - JavaBeans will be in an inconsistent state because of creating the setter and getter method for each property separately.

How to access JavaBeans in JSP Application?

- Using Java beans in JSP ***offers whole lot of flexibility*** and ***avoids duplicate business logic***.
- JSP technology uses standard actions for working with beans.
- Following are the three standard actions for working with Java beans:
  - ***<jsp: useBean/>*** – find or instantiate a JavaBean.
  - ***<jsp: setProperty/>*** – sets the property of a JavaBean.
  - ***<jsp: getProperty/>*** – gets the value of a JavaBean property.

How to access JavaBeans in JSP Application?

- **Syntax:**

```
<jsp:useBean id= "instanceName"
    scope= "page | request | session | application"
    class= "packageName.className" type= "packageName.className"
    beanName="packageName.className | <%= expression >"
/>
```

- **Id:** It will take a variable to manage generated Bean object reference.
- **Class:** This attribute will take the fully qualified name of the Bean class.

How to access JavaBeans in JSP Application?

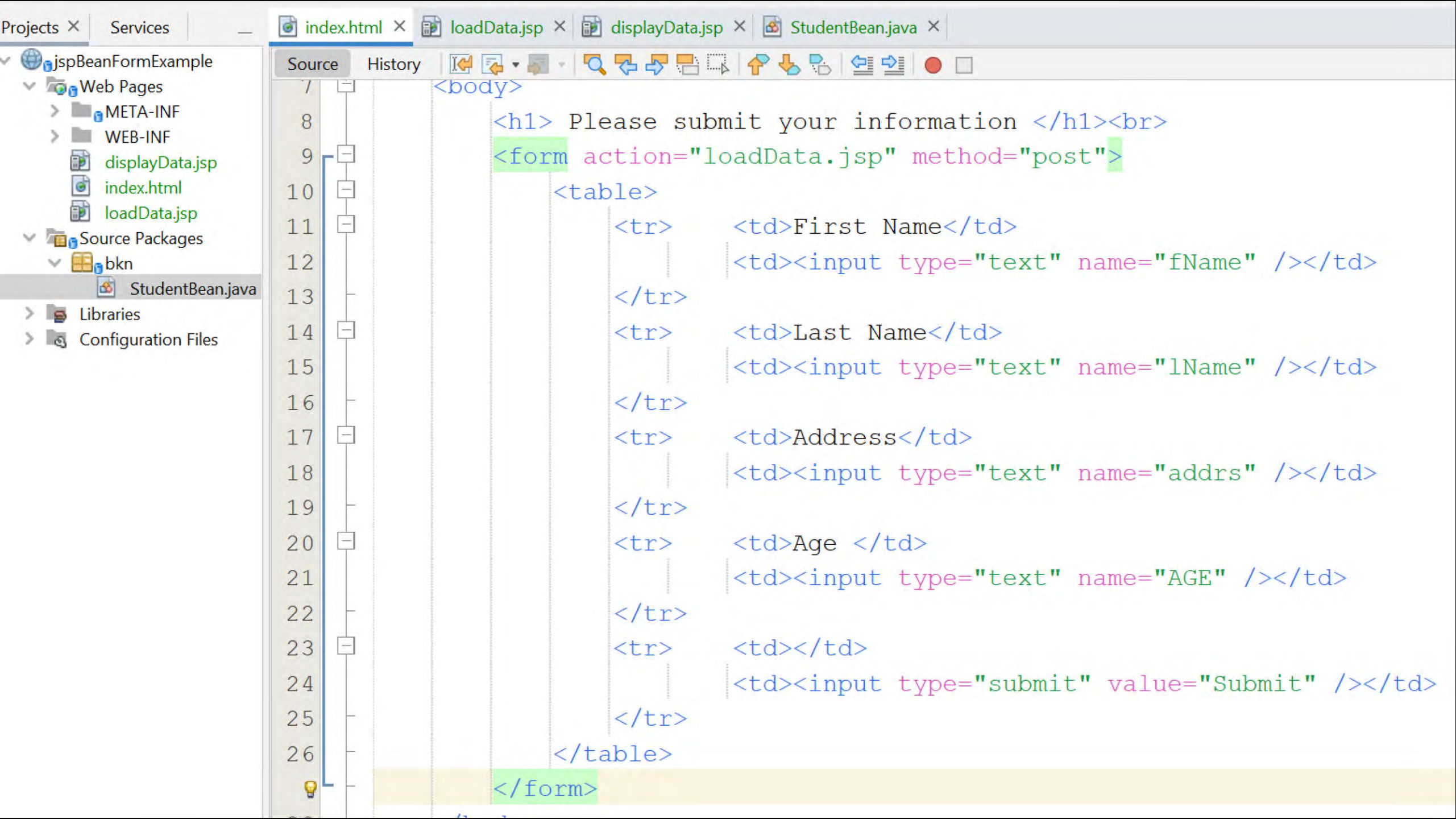
- ***Type:*** It will take the fully qualified name of the Bean class to define the type of variable in order to manage the Bean object reference.
- ***Scope:*** It will take ***either of the JSP scopes*** to the Bean object.
  - ***page:*** It lasts until the page completes and there is no storage of state.
  - ***request:*** The JavaBean instance lasts for the client request and so will remain if the request is forwarded.
  - ***session:*** The JavaBean lasts as long as the client session.
  - ***application:*** The JavaBean is instantiated with the application and remains in use until the application ends.

How to access JavaBeans in JSP Application?

- ***jsp:setProperty***
  - This action as the name suggests is used to populate the bean properties in the specified scope.
  - Following is the ***syntax for this action.***
    - <***jsp:setProperty*** ***name*** ="bean name" ***property*** ="property name" ***value***= "data" />
    - For instance, if we need to populate a bean whose property is ***firstName*** with a value ***Ram*** we use this action as shown below:
    - **<**jsp:setProperty ***name*** =“student" ***property*** ="firstName" ***value***= “Ram" />

How to access JavaBeans in JSP Application?

- ***jsp:getProperty***
  - This standard action is used to retrieve a specified property from a bean in a specified scope.
  - Following is how we can use this action to retrieve the value of ***firstName*** property in a bean ***identified by "student"*** in the session scope.
  - <***jsp:getProperty*** ***name***="student" ***property***="firstName" ***scope***="session" />


Projects
Services
index.html
loadData.jsp
displayData.jsp
StudentBean.java
jspBeanFormExample
Web Pages
META-INF
WEB-INF
displayData.jsp
index.html
loadData.jsp
Source Packages
bkn
StudentBean.java
Libraries
Configuration Files
Source
History
<body>
<h1> Please submit your information </h1>
<form action="loadData.jsp" method="post">
<table>
<tr> <td>First Name</td>
<td><input type="text" name="fName" /></td>
</tr>
<tr> <td>Last Name</td>
<td><input type="text" name="lName" /></td>
</tr>
<tr> <td>Address</td>
<td><input type="text" name="addrs" /></td>
</tr>
<tr> <td>Age </td>
<td><input type="text" name="AGE" /></td>
</tr>
<tr> <td></td>
<td><input type="submit" value="Submit" /></td>
</tr>
</table>
</form>

index.html | loadData.jsp | displayData.jsp | StudentBean.java

Source | History

```
<html>
    <body>
        <jsp:useBean id = "student" class = "bkn.StudentBean" scope= "session">
        </jsp:useBean>
        <jsp:setProperty name = "student" property = "firstName"
                         value = '<%= request.getParameter("fName")%>'/>
         <jsp:setProperty name = "student" property = "lastName"
                          value = '<%= request.getParameter("lName")%>'/>
        <jsp:setProperty name = "student" property = "address"
                         value = '<%= request.getParameter("addrs")%>'/>
        <jsp:setProperty name = "student" property = "age"
                  value = '<%=Integer.parseInt(request.getParameter("AGE"))%>'/>
        <%
        RequestDispatcher rd = request.getRequestDispatcher("displayData.jsp");
        rd.forward(request, response);
        %>
    </body>
</html>
```

index.html | loadData.jsp | displayData.jsp | StudentBean.java

Source | History

```
<html>
    <head>
        <title>JSP using Bean</title>
    </head>
    <body>
        <h1>Student Information using Bean</h1>
        <jsp:useBean id = "student" class = "bkn.StudentBean" scope= "session">
        </jsp:useBean>
        <h3> First Name:
        <jsp:getProperty name = "student" property = "firstName" /> </h3>
        <h3> Last Name:
        <jsp:getProperty name = "student" property = "lastName" /> </h3>
        <h3> Address:
        <jsp:getProperty name = "student" property = "address" /> </h3>
        <h3> Age:
        <jsp:getProperty name = "student" property = "age" /> </h3>
    </body>
</html>
```

index.html | loadData.jsp | displayData.jsp | StudentBean.java

Source | History

```
1   package bkn;
2   import java.io.Serializable;
3   public class StudentBean implements Serializable{
4       private String firstName;
5       private String lastName;
6       private String address;
7       private int age;
8       public StudentBean() {}
9       public String getFirstName() {
10          return firstName; }
11      public void setFirstName(String firstName){this.firstName = firstName;}
12      public String getLastName() {
13          return lastName;}
14      public void setLastName(String lastName) { this.lastName = lastName;}
15      public String getAddress() {
16          return address;}
17      public void setAddress(String address) {this.address = address;}
18      public int getAge() {
19          return age; }
20      public void setAge(int age) {  this.age = age; }
21  }
```

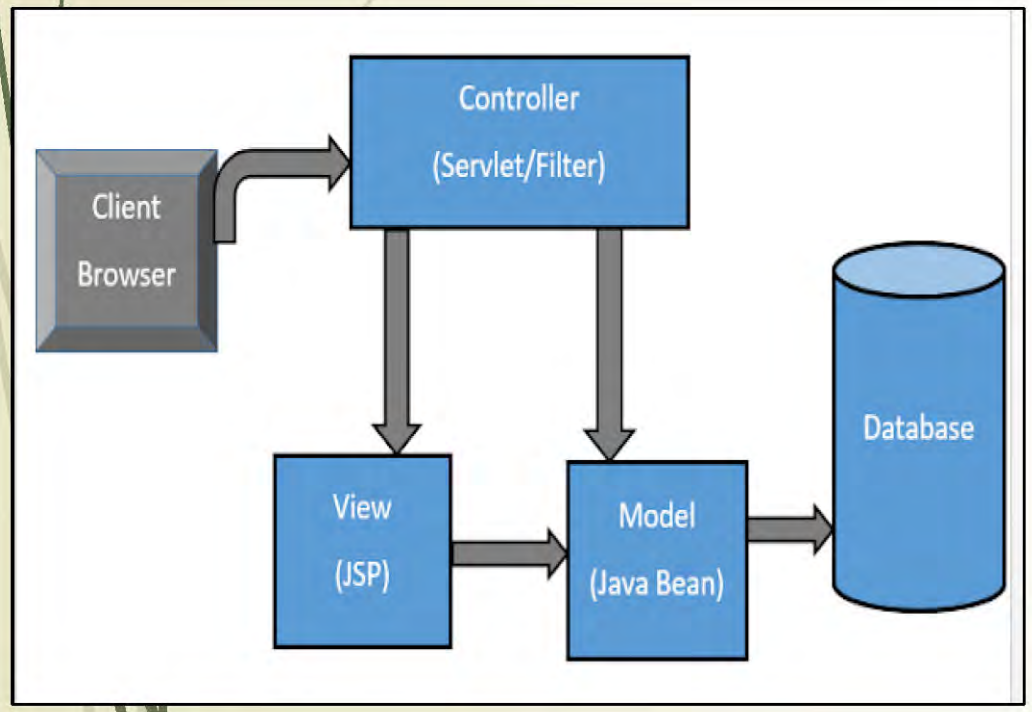
MVC Architecture in JSP

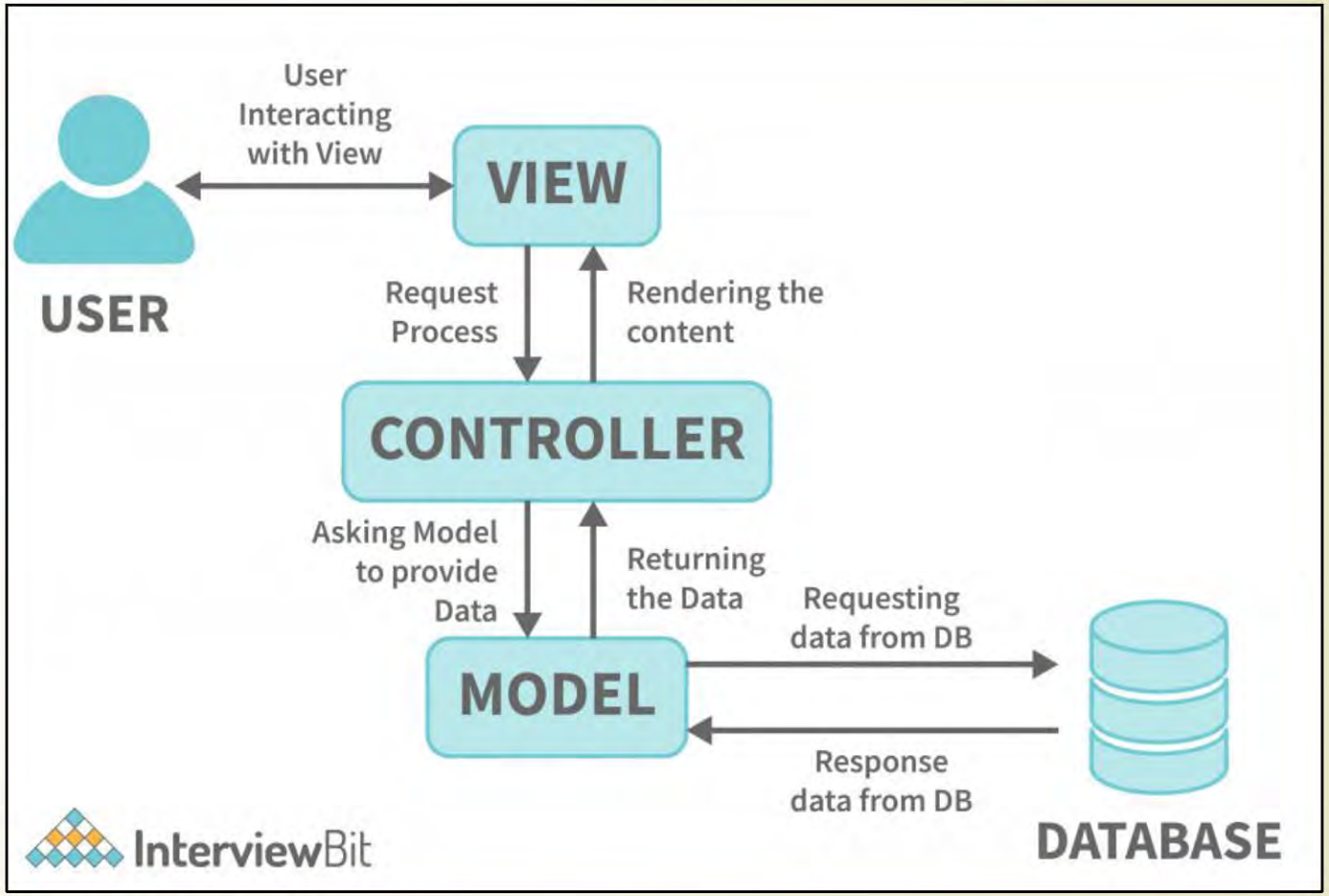
- MVC stands for ***Model View Controller***.
- It is a design pattern which is used to ***separate the business logic, presentation logic and data***. It can be used to design a web application in a standard manner. As per MVC, a application will be ***divided into three layers***
- ***Model Layer:*** Represents the business layer of the application.
  - This is the data layer which consists of the business logic of the system.
  - It consists of all the data of the application
  - It also represents the state of the application.
  - It consists of classes which have the connection to the database.
  - The controller connects with model and fetches the data and sends to the view layer.
  - The model connects with the database as well and stores the data into a database which is connected to it.

MVC Architecture in JSP

- ***View Layer:***
  - It represents the presentation layer.
  - It normally represents the User Interface (UI) of the application.
  - We can use HTML,CSS,JS etc. to create presentation layer.
- ***Controller Layer:***
  - It is an interface between the view layer and the model layer.
  - It receives the requests from the view layer and processes the requests and does the necessary validation for the request. This requests is further sent to model layer for data processing, and once the request is processed, it sends back to the controller with required information and displayed accordingly by the view.
  - To read data from the presentation layer, can use Servlet, JSP, Filter, etc.

MVC Architecture in JSP

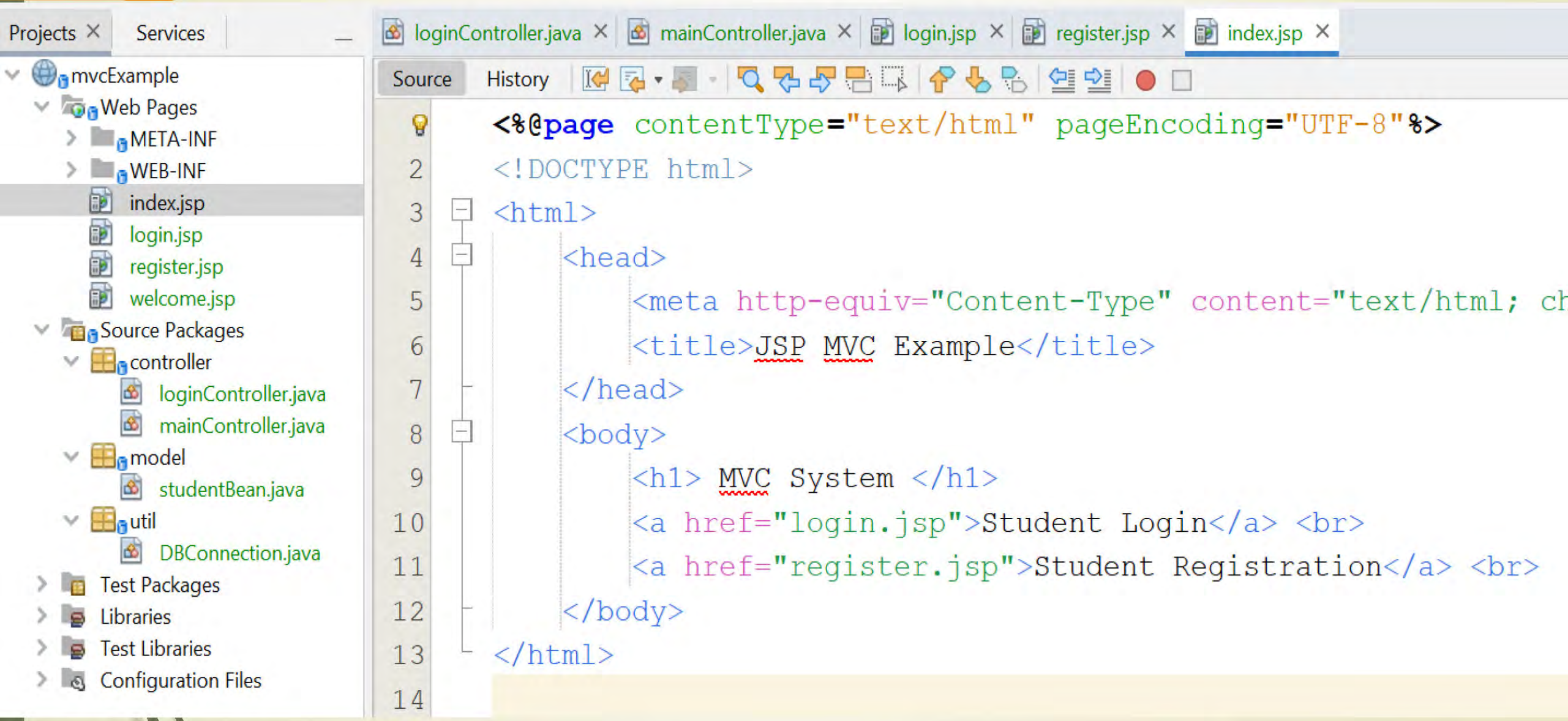




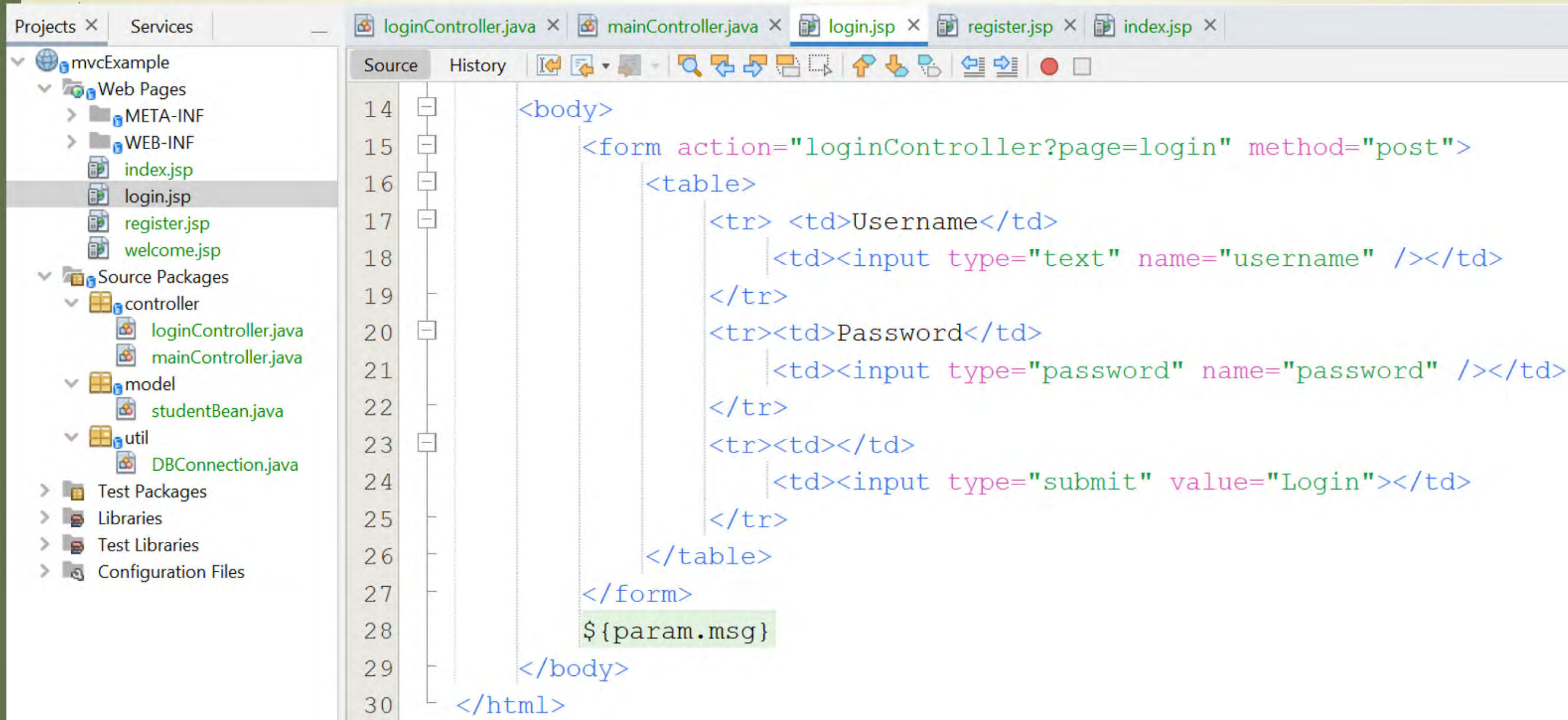
MVC Architecture in JSP

- ***Advantages of MVC Architecture in Java***
  - Multiple developers can work with the three layers (Model, View, and Controller) simultaneously
  - Offers improved scalability, that supplements the ability of the application to grow
  - As components have a low dependency on each other, they are easy to maintain
  - A model can be reused by multiple views which provides reusability of code
  - Adoption of MVC makes an application more expressive and easy to understand
  - Extending and testing of the application becomes easy

MVC Architecture in JSP



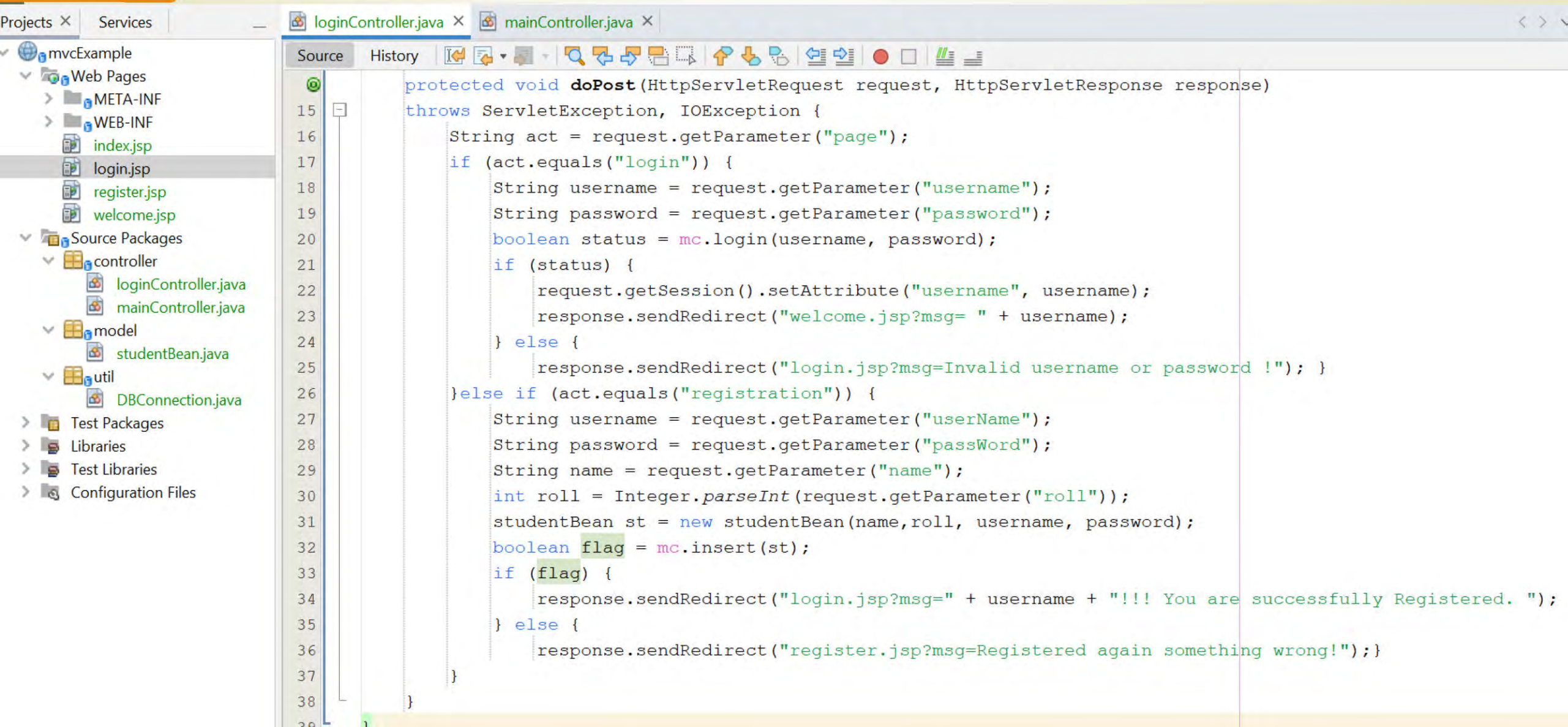
MVC Architecture in JSP



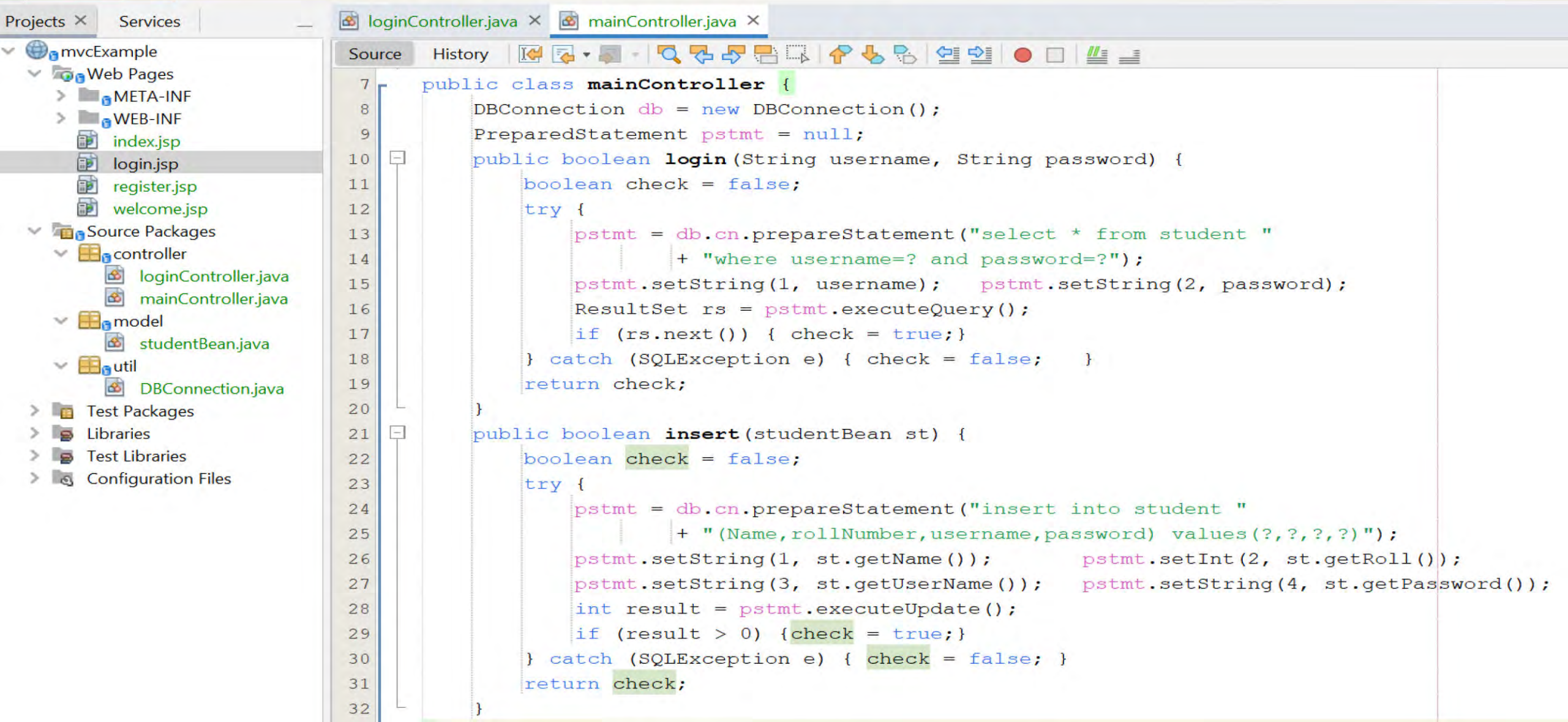
MVC Architecture in JSP

```
<body>
        <h1> Student Record Registration System </h1>
        <form action="loginController?page=registration" method="post">
            <table>
                <tr>
                    <td>Name</td>
                    <td><input type="text" name="name" /></td>
                </tr>
                <tr>
                    <td>Roll Number</td>
                    <td><input type="text" name="roll" /></td>
                </tr>
                <tr>
                    <td>User Name</td>
                    <td><input type="text" name="userName" /></td>
                </tr>
                <tr>
                    <td>Password</td>
                    <td><input type="text" name="passWord" /></td>
                </tr>
                <tr>
                    <td></td>
                    <td><input type="submit" value="Submit" /></td>
                </tr>
            </table>
        </form>
        ${param.msg}
</body>
```

MVC Architecture in JSP



MVC Architecture in JSP



Thank You
???